

Algoritmy a údajové štruktúry 1

Prednáška č. 1

- Predstavenie predmetu
- Spracovanie údajov
- Zložitosť algoritmov
- Kódovanie údajov



Predstavenie predmetu *6UI0004 algoritmy a údajové štruktúry 1*

- Štúdiom predmetu získa študent základné znalosti z teórie údajových štruktúr a naučí sa ich efektívne implementovať.
- Po absolvovaní predmetu študent:
 - Pozná základné údajové štruktúry a vie ich využiť pri riešení praktických problémov.
 - Ovláda postup efektívnej implementácie základných údajových štruktúr.
 - Dokáže posúdiť navrhnuté algoritmy z hľadiska výpočtovej a pamäťovej zložitosti.

Predstavenie predmetu *6UI0004 algoritmy a údajové štruktúry 1*

- **Prednášať budú:**

- Ing. Michal Varga, PhD.
- doc. Ing. Miroslav Kvaššay, PhD.

- **Cvičenia budú viesť:**

- doc. Ing. Miroslav Kvaššay, PhD. (miroslav.kvassay@fri.uniza.sk, RA302)
- Ing. Michal Mrena, PhD. (michal.mrena@fri.uniza.sk, RA302)
- Ing. Michal Varga, PhD. (michal.varga@fri.uniza.sk, RA324)

Náplň prednášok pokrýva nasledujúce okruhy

- Úvod do údajových štruktúr.
- Správa pamäte na úrovni aplikácie a operačného systému. Kódovanie údajov v pamäti.
- Lineárna, hierarchická a sieťová organizácia údajov v pamäti.
- Polia (jednorozmerné, viacrozmerné, prešité).
- Zoznamy (implicitné, jednostranne zreťazené, obojstranne zreťazené, cyklické).
- Stromy (implicitné, K-cestné, viaccestné).
- Grafy (tabuľka hrán, dvojúrovňová reprezentácia, krížové reprezentácie, implementácia neorientovaných a zmiešaných grafov)
- Prioritné fronty (FIFO, LIFO, sekvenčný prioritný front, dvojzoznam, ľavostranná halda).
- Tabuľky (sekvenčné tabuľky, tabuľka s rozptýlenými záznamami, binárny vyhľadávací strom, treap).
- Triedenia tabuliek (priame metódy – InsertSort, SelectSort, BubbleSort; pokročilé metódy – QuickSort, HeapSort, ShellSort, RadixSort a MergeSort; metódy využívajúce paralelné prostriedky - BitonicSort).

Hodnotenie predmetu

Semester

- Priebežne je hodnotená práca na cvičeniach, ktorá je posudzovaná podľa počtu bodov získaných za testy (1 test v polovici semestra a 1 test na konci semestra) a za vypracovanie a obhájenie semestrálnej práce (1 semestrálna práca, ktorá je vypracovávaná a hodnotená priebežne počas semestra).
 - **1. test** 10 bodov
 - **2. test** 10 bodov
 - **Semestrálna práca** 80 bodov

Skúška

- Podmienkou pre absolvovanie predmetu a pridelenie kreditov je úspešné absolvovanie skúšky z predmetu. Študent sa môže prihlásiť na záverečnú skúšku iba ak
 - za priebežné hodnotenie získa **minimálne 61 bodov** a
 - **vypracuje a úspešne obháji semestrálnu prácu.**
- Skúška sa skladá z písomnej časti (max. 100 bodov) a z ústnej časti. **Pre absolvovanie skúšky je potrebné získať z písomnej časti minimálne 61 bodov.**

Hodnotenie predmetu

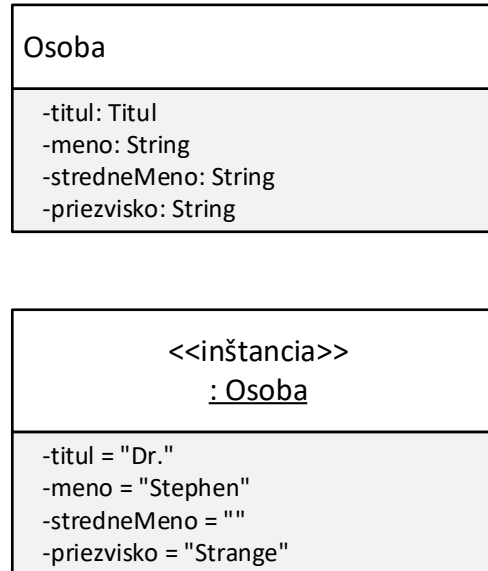
- Záverečné hodnotenie sa vykonáva na základe súčtu bodov získaných za priebežné hodnotenie a za skúšku:
 - **A:** aspoň 186 bodov
 - **B:** < 170, 185) bodov
 - **C:** < 154, 170) bodov
 - **D:** < 138, 154) bodov
 - **E:** < 122, 138) bodov
 - **Fx:** menej ako 122 bodov

Spracovanie údajov

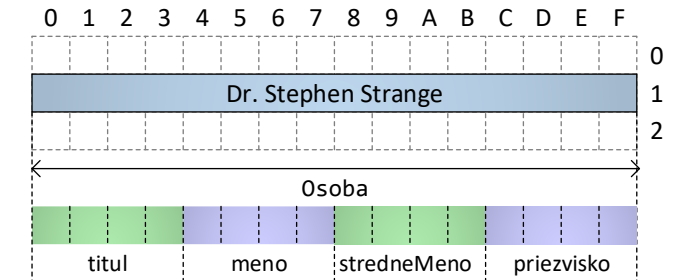
Aplikačná úroveň



Abstraktná úroveň



Fyzická úroveň



A čo, keby som ich chcel **evidovať viac** postáv? A keby som **toho istého** musel **evidovať viackrát**?

Ako mám evidovať poradie **podľa obľúbenosti**? A keby som chcel evidovať poradie **podľa výskytu vo filmoch**?

Chcem rýchlo **nájsť** postavu z MCU **podľa** schopností! Ako ich mám uložiť? A keby som chcel **hľadať podľa** mena?

Dôležité otázky

Ako mám implementovať tabuľku osôb, v ktorej ich budem môcť efektívne vyhľadávať?

Tabuľka osôb				
ID	Titul	Meno	Stredné meno	Priezvisko
1		Tony	Edward	Stark
2		Thor		Odinson
3		Janet		Van Dyne
4	Dr.	Henry	Jonathan	Pym
5	Dr.	Robert	Bruce	Banner
6		Steven	Grant	Rogers
7		Clint	Francis	Barton
8		Pietro		Maximoff
9		Wanda		Maximoff
10		Victor		Shade
11		Natasha	Alianova	Romanoff
12	Dr.	Stephen		Strange

Ako implementovať údajovú štruktúru, ktorá umožní efektívne spracovávať údaje v danej situácii?

Údajové štruktúry

- **Údajová štruktúra agreguje údaje a poskytuje operácie na prácu s nimi.**
- Údajovú štruktúru považujeme za objekt, ktorý má svoje vlastnosti a operácie.
- Pre efektívnu prácu s údajmi v konkrétnej situácii má používateľ (programátor) dve úlohy:
 1. Na základe požadovaných operácií s údajmi zvoliť vhodný **typ štruktúry** (napríklad rozhodnúť, či sa jedná o tabuľku, zoznam, prioritný front, atď.).
 2. Na základe očakávanej početnosti operácií zvoliť vhodnú **implementáciu** zvolenej **údajovej štruktúry**, ktorá minimalizuje časové a pamäťové nároky v konkrétnom prípade jej použitia.

Cieľom je kvalitná realizácia cieľového modelu na počítači vzhľadom k úspornému využívaniu jeho pamäte a efektivite (rýchlosti) často využívaných výpočtov algoritmov.

Operácie údajových štruktúr

- Operácie údajových štruktúr môžeme kategorizovať na nasledujúce:
 - **Základné operácie** – konštruktor, deštruktor, priradenie, porovnanie, ..
 - **Selektory** – sprístupňujú (vyhľadávajú) prvok údajovej štruktúry.
 - **Dotazy** – umožňujú získať informácie o prvkoch resp. o štruktúre samotnej.
 - **Modifikátory** – operácie určené na pridávanie a odoberanie prvkov (modifikujú obsah údajovej štruktúry).
 - **Prehliadky** – sprístupňujú všetky prvky v určitom poradí. Prehliadku štruktúry je možné zapúzdriť do objektov, ktoré sú zodpovedné za efektívny prístup do štruktúry bez odhalenia jej implementačných detailov. Takýmto objektom budeme hovoriť **iterátory**.

Druhá úloha

- Pre efektívnu prácu s údajmi v konkrétnej situácii má používateľ (programátor) dve úlohy:
 1. Na základe požadovaných operácií s údajmi zvoliť vhodný **typ štruktúry** (napríklad rozhodnúť, či sa jedná o tabuľku, zoznam, prioritný front, atď.).
 2. Na základe očakávanej početnosti operácií zvoliť vhodnú **implementáciu** zvolenej **údajovej štruktúry**, ktorá **minimalizuje časové a pamäťové nároky** v konkrétnom prípade jej použitia.

Zložitosť algoritmov a údajových štruktúr

- **Algoritmus** je postupnosť krokov, realizovateľných na počítači, vedúcich k očakávanému výsledku.
- Pre vyhodnotenie zložitosti je potrebné definovať objektívne kritériá:
 1. **Časová zložitosť algoritmu** – kritérium je čas potrebný k vykonaniu algoritmu.
 2. **Pamäťová zložitosť algoritmu** – kritérium je veľkosť pamäťového priestoru potrebného k realizácii algoritmu.

Zložitosť (časová aj pamäťová) sa vyjadruje pomocou funkcií, ktorých argumentom je množstvo dát.

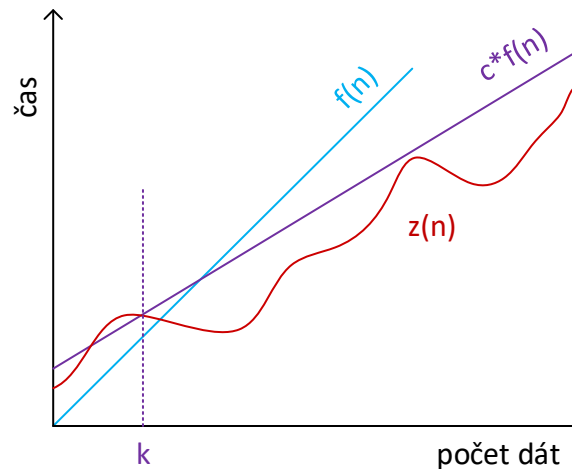
Časová zložitosť algoritmu

- Vyjadruje počet operácií potrebných na realizáciu algoritmu bez rozlišovania typu operácie.
- Počet operácií sa dá vyjadriť pomocou vzťahu $q \cdot z(n)$, kde:
 - n je počet (množstvo) spracovávaných dát.
 - q je konštanta vyjadrujúca kvalitu implementácie (použitý jazyk, prekladač, ..).
 - $z(n)$ je funkcia závislosti počtu operácií od množstva dát.

Pre hrubý odhad zložitosti algoritmu nie je podstatná veľkosť konštanty q , dôležitejší je typ funkcie $z(n)$.

Odhady funkcie zložitosti – horný asymptotický odhad

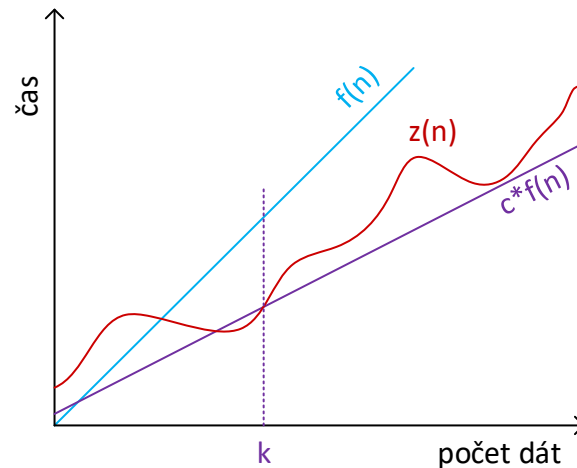
- Ak $\exists$ konštanty $c > 0$, $k > 0$ také, že $\forall n \geq k$, potom **horný asymptotický odhad** funkcie $z(n)$ označujeme:
 - $O(f(n))$, ak $0 \leq z(n) \leq c \cdot f(n)$: $z(n)$ je asymptoticky menšia alebo rovná ako $c \cdot f(n)$.
 - $o(f(n))$, ak $0 \leq z(n) < c \cdot f(n)$: $z(n)$ je asymptoticky ostro menšia ako $c \cdot f(n)$.



Cieľom je minimalizovať hornú asymptotickú zložitosť algoritmu.

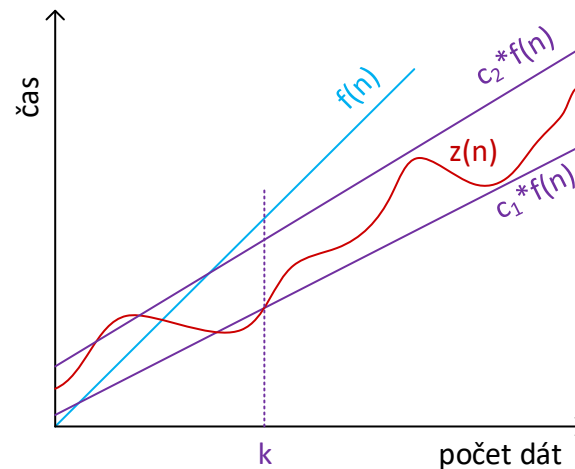
Odhady funkcie zložitosti – dolný asymptotický odhad

- Ak $\exists$ konštanty $c > 0$, $k > 0$ také, že $\forall n \geq k$, potom **dolný asymptotický odhad** funkcie $z(n)$ označujeme:
 - $\Omega(f(n))$, ak $0 \leq c \cdot f(n) \leq z(n)$: $z(n)$ je asymptoticky väčšia alebo rovná ako $c \cdot f(n)$.
 - $\omega(f(n))$, ak $0 \leq c \cdot f(n) < z(n)$: $z(n)$ je asymptoticky ostro väčšia ako $c \cdot f(n)$.



Odhady funkcie zložitosti – asymptotický odhad

- Ak $\exists$ konštanty $c_1 > 0$, $c_2 > 0$, $k > 0$ také, že $\forall n \geq k$, potom **asymptotický odhad** funkcie **$z(n)$** označujeme:
 - $\Theta(f(n))$, ak $0 \leq c_1 * f(n) \leq z(n) \leq c_2 * f(n)$: **$z(n)$** je asymptoticky rovnaká ako **$f(n)$** .



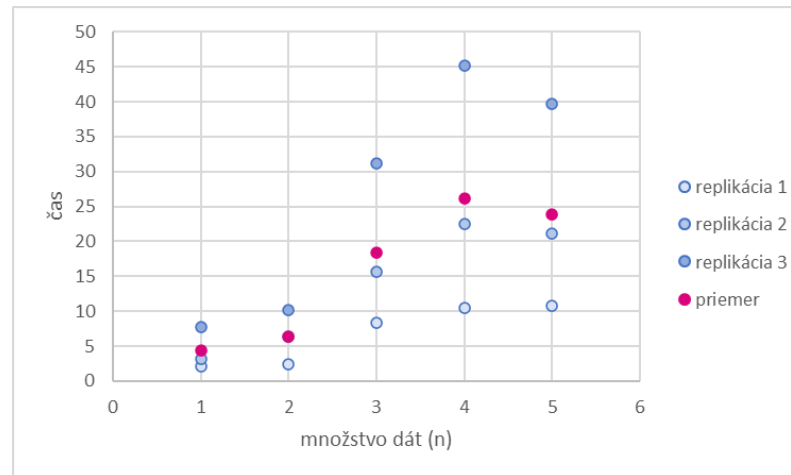
Triedy zložitostí algoritmov

- **Trieda zložitosti algoritmu** je množina vypočítateľných problémov s rovnakou asymptotickou zložitosťou (pamäťovou alebo časovou) – teda s rovnakou $f(n)$.

$f(n)$	Asymptotický odhad					Názov triedy zložitosti
	horný		dolný			
	O	o	Ω	ω	Θ	
1	$O(1)$	$o(1)$	$\Omega(1)$	$\omega(1)$	$\Theta(1)$	Konštantná
N	$O(n)$	$o(n)$	$\Omega(n)$	$\omega(n)$	$\Theta(n)$	Lineárna
n^2	$O(n^2)$	$o(n^2)$	$\Omega(n^2)$	$\omega(n^2)$	$\Theta(n^2)$	Kvadratická
n^k	$O(n^k)$	$o(n^k)$	$\Omega(n^k)$	$\omega(n^k)$	$\Theta(n^k)$	Polynomiálna
$\log(n)$	$O(\log(n))$	$o(\log(n))$	$\Omega(\log(n))$	$\omega(\log(n))$	$\Theta(\log(n))$	Logaritmická
$n \cdot \log(n)$	$O(n \cdot \log(n))$	$o(n \cdot \log(n))$	$\Omega(n \cdot \log(n))$	$\omega(n \cdot \log(n))$	$\Theta(n \cdot \log(n))$	„Linearitmická“
a^n	$O(a^n)$	$o(a^n)$	$\Omega(a^n)$	$\omega(a^n)$	$\Theta(a^n)$	Exponenciálna

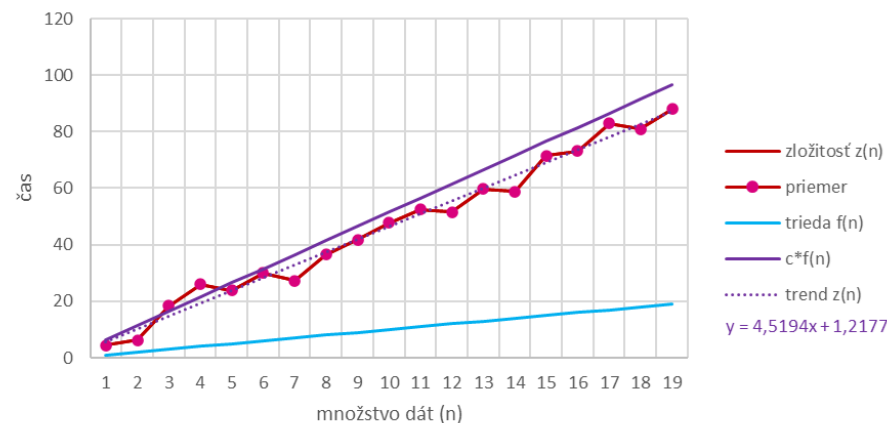
Ako zistiť (časovú) zložitosť algoritmu?

- Zložitosť (**časová**) sa vyjadruje pomocou funkcie, ktorej argumentom je **množstvo dát** → pre (každé) množstvo dát potrebujeme odmerať čas.
- Pre dané množstvo dát (n) nestačí odmerať jeden čas, je potrebné získať viac vzoriek.
- Je potrebné vykonať niekoľko **replikácií** (opakovaní) operácie s daným množstvom dát a získané časy spracovať (vziať do úvahy napr. **priemer**, maximum).

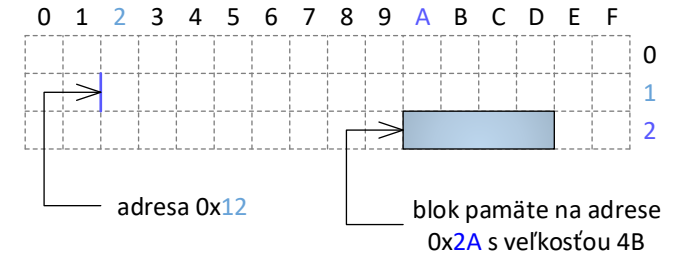


Ako zistiť (časovú) zložitosť algoritmu?

- Zložitosť (časová) sa vyjadruje pomocou funkcie, ktorej argumentom je množstvo dát.
- (v našom prípade) Priemery pre jednotlivé množstvá dát (n) vynesieme do grafu a potom:
 - získame funkciu zložitosti $z(n)$.
 - vieme získať trend $z(n)$: $y = 4,827x - 1,6372 \rightarrow$ lineárna trieda zložitosti $f(n)$.
 - vieme odhadnúť $c \cdot f(n)$ a určiť horný asymptotický odhad ako $O(n)$.



Údaje



- Údaje môžu mať charakter **premenných** alebo **konštánt**.
- Sú uložené v pamäti počítača – **bity** (každý bit má hodnotu 0 alebo 1).
- Bity sú zoskupené – minimálna skupina bitov sa nazýva **bajt** (byte).
- Na väčšine súčasných architektúr je **bajt najmenší adresovateľný blok operačnej pamäte**.
- Každý bajt v operačnej pamäti má svoju **adresu**, typicky sa udáva v hexadecimálnom tvare.
- Bajt tvorí 8 bitov – je schopný uchovať jednu z $2^8 = 256$ sekvencií bitov od 0000 0000, 0000 0001, 0000 0010, .., 1111 1111.
- 256 hodnôt je pomerne málo – pre viac hodnôt potrebujeme viac bajtov – **n**.
- **Kódovanie** údajového typu zaberajúceho n bajtov určuje, ako sa jednotlivé hodnoty tohto typu premapujú na jednu z 2^{8n} možných sekvencií 8n bitov.

Primitívne typy

- Priamo vstavané do programovacích jazykov, typicky sú to tieto:
 - **celé čísla** – predstavujú určitú podmnožinu množiny celých čísel,
 - **reálne čísla** – predstavujú podmnožinu racionálnych čísel,
 - **typ znak** – predstavuje množinu riadiacich znakov a znakov vytlačiteľných na obrazovke počítača,
 - **logický typ** – slúži na reprezentáciu dvoch logických hodnôt, označovaných ako pravda (**true**) a nepravda (**false**),
 - **referencia** – slúži na nepriamy prístup k údajom.
- Primitívne typy sú **skalárne typy**: hodnoty je možné navzájom porovnávať (**false** < **true**).
- Ak je typ aj **ordinálny**, potom pre každú hodnotu vieme jednoznačne určiť jej predchodcu a nasledovníka (z vyššie uvedených všetko okrem reálnych čísiel).

Architektúry operačných systémov

- Pamäťová reprezentácia typu závisí ako od kompilátora, tak od operačného systému a hardvérovej architektúry.
- Označenie modelu architektúry sa skladá z 2 častí – špecifikuje údajové typy a ich veľkosť v bitoch na danej architektúre.
 - I = int
 - L = long
 - P = pointer (ukazovateľ)
- My budeme predpokladať **LLP64**.

Model architektúry	Príklad architektúry	Príklad kompilátora
LP32	16-bitové operačné systémy od spoločnosti Microsoft (Windows 3.1, prvé verzie operačného systému pre počítače Macintosh od spoločnosti Apple)	
ILP32	32-bitové verzie operačných systémov Microsoft Windows (napr. Windows 95, 97, XP), Unix a operačných systémov inšpirovaných Unixom (napr. Linux, macOS)	Visual C++ pre Microsoft Windows; g++ pre Linux
LLP64	64-bitové verzie operačných systémov Microsoft Windows (napr. Windows 7, 8, 8.1, 10)	Visual C++ pre Microsoft Windows
LP64	64-bitové verzie operačného systému Unixu a operačných systémov inšpirovaných Unixom (napr. Linux, macOS), ako aj 64-bitová verzia prostredia Cygwin pre Microsoft Windows	g++ pre Linux; g++ pre Cygwin

Typy uchovávajúce celé čísla

- Znamienkové (signed) a neznamienkové (unsigned).
- Pozor na pretečenie.
- C++ Na LLP64:
 - [unsigned] char: 1B
 - [unsigned] short: 2B
 - [unsigned] int: 4B
 - [unsigned] long: 4B
 - [unsigned] long long: 8B

Údajový typ	Veľkosť údajového typu v bitoch				
	Štandard	LP32	ILP32	LLP64	LP64
signed char unsigned char	CHAR_BIT	8	8	8	8
short unsigned short	aspoň 16	16	16	16	16
int unsigned int	aspoň 16	16	32	32	32
long unsigned long	aspoň 32	32	32	32	64
long long unsigned long long	aspoň 64	64	64	64	64

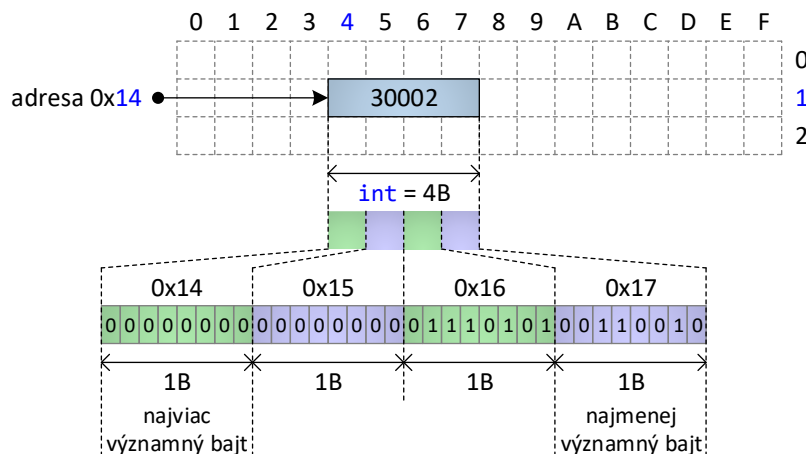
Údajový typ	Rozsah možných hodnôt údajového typu
unsigned char	0..255
unsigned short	0..65 535
unsigned int	0..4 294 967 295
unsigned long	0..4 294 967 295
unsigned long long	0..18 446 744 073 709 551 615

Endianita

Definuje poradie, v akom sa bajty viacbajtového údajového typu uchovávajú v pamäti

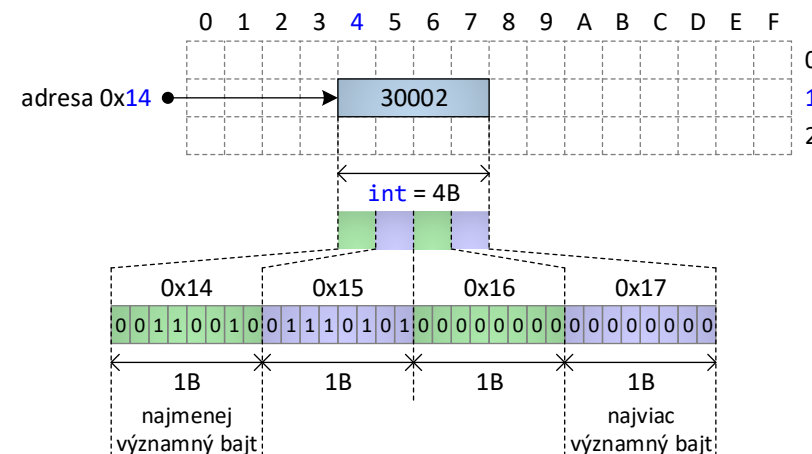
big-endian

- **najviac** významný bajt sa ukladá na **nižšiu** adresu.
- **najmenej** významný bajt sa ukladá na **vyššiu** adresu.
- „sieťové poradie bajtov“ – v tomto poradí sa posielajú údaje vo väčšine sieťových protokolov.



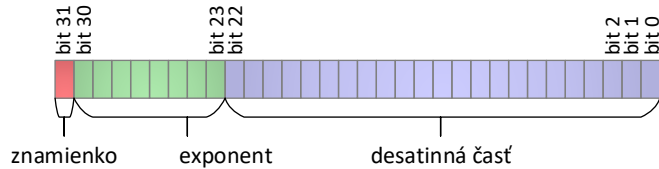
little-endian

- **najmenej** významný bajt sa ukladá na **nižšiu** adresu.
- **najviac** významný bajt sa ukladá na **vyššiu** adresu.
- PC



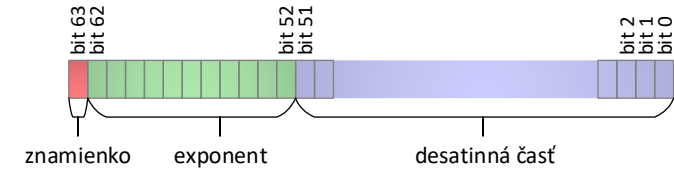
Typy uchovávajúce reálne čísla (podľa štandardu IEEE-754)

float

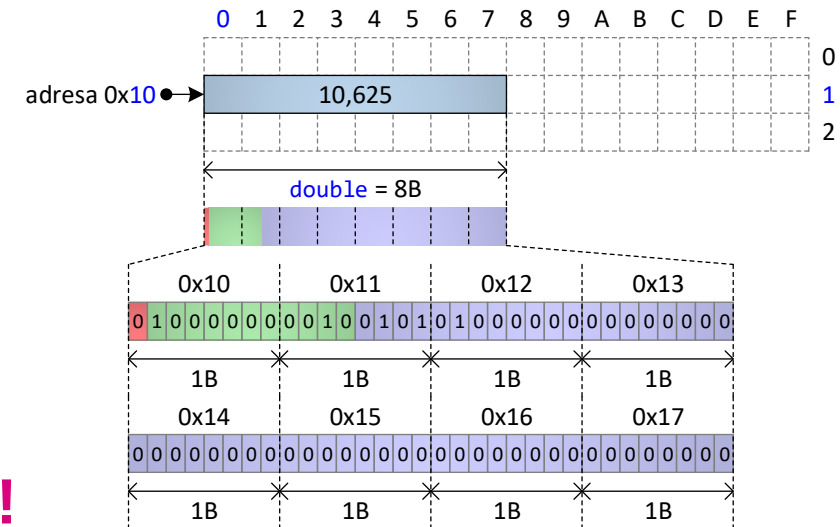
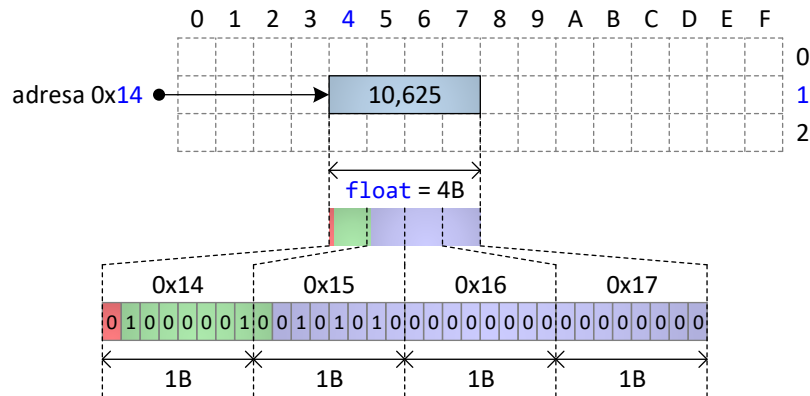


$$(-1)^{b_{31}} \times (1, b_{22}b_{21} \dots b_0)_2 * 2^{(b_{30}b_{29} \dots b_{23})_2 - 127},$$

double



$$(-1)^{b_{63}} \times (1, b_{51}b_{50} \dots b_0)_2 * 2^{(b_{62}b_{61} \dots b_{52})_2 - 1023},$$



Pozor na test rovnosti/nerovnosti!!!

$$|a - b| \leq \varepsilon$$

a, b – porovnávané reálne čísla; ε – tolerancia (napr. 0,001)

Typy uchovávajúce znaky – kódovanie znakov

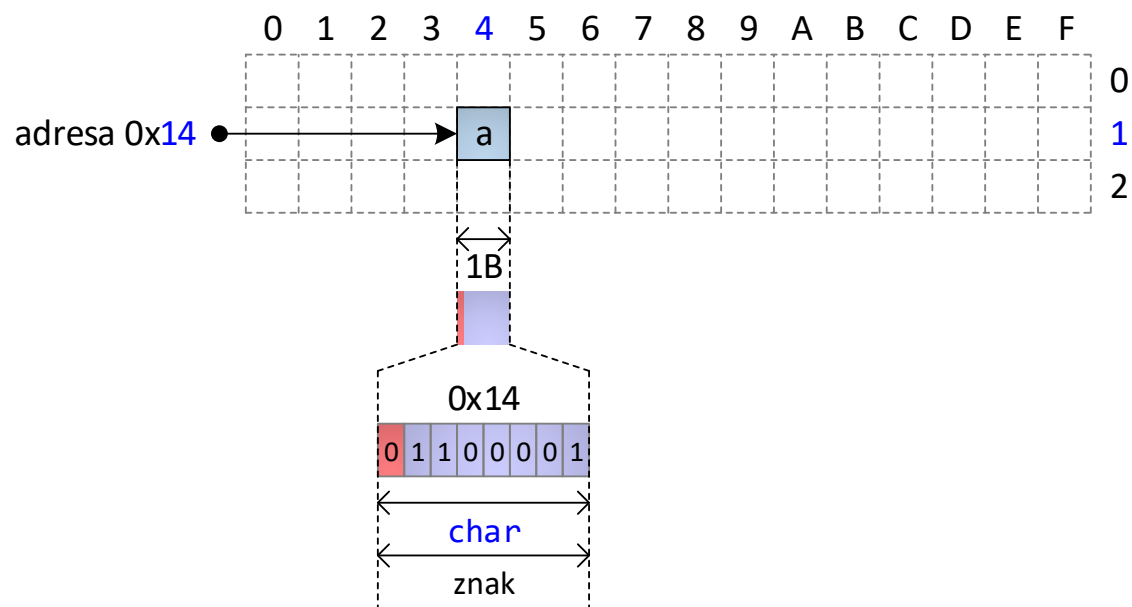
- Kódovanie znakov:
 - priradenie unikátnej celočíselnej hodnoty každému znaku, ktorý má byť spracovávaný číslíkovým systémom. Táto hodnota sa označuje ako „**code point**“.
- Najčastejšie používané kódovania:
 - **ASCII** („American Standard Code for Information Interchange“)
 - **rozšírenia ASCII**
 - **Unicode**

Typy uchovávajúce znaky – štandard ASCII

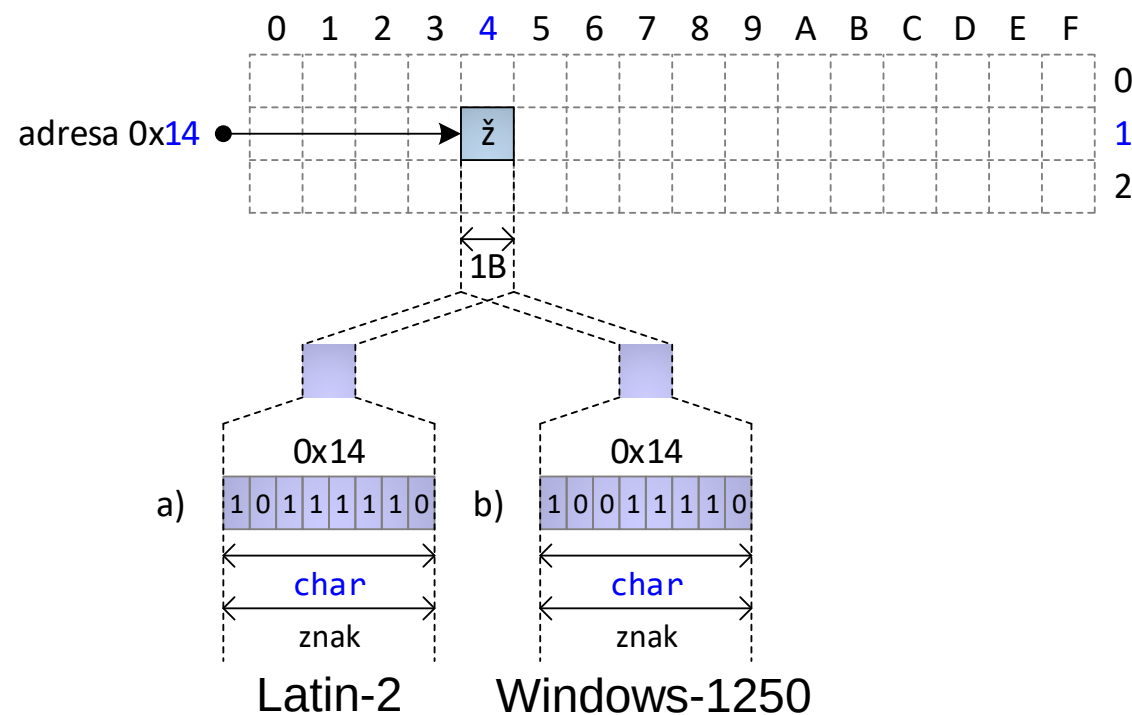
- **ASCII** („American Standard Code for Information Interchange“):
 - **7-bitový** štandard kódovania znakov vyvinutý pre účely elektronickej komunikácie;
 - definuje kódovanie pre 95 vytlačiteľných znakov (vrátane malých a veľkých písmen anglickej abecedy) a 33 kontrolných znakov;
- **rozšírenia ASCII**:
 - **8-bitové** (1-bajtové) štandardy kódovania znakov;
 - bajty, ktoré reprezentujú čísla 0 až 127 (t. j. bajty, v ktorých má prvý bit hodnotu 0), slúžia na uchovávanie znakov kódovaných v súlade so štandardom ASCII;
 - bajty, ktoré majú prvý bit rovný hodnote 1 sa používajú na kódovanie znakov národných abecied, ktoré nie sú definované v štandarde ASCII;
 - príklady:
 - ISO/IEC 8859-2 (Latin-2) – stredoeurópske znaky;
 - Windows-1250, ktorý z veľkej časti korešponduje s ISO/IEC 8859-2 (Latin-2);

Porovnanie ASCII

Štandard ASCII



Rozšírenie ASCII



Typy uchovávajúce znaky – štandard Unicode

- 21-bitový štandard (**na reprezentáciu jedného znaku využíva viacero bajtov**).
- Celková kapacita štandardu – 1 112 064 rôznych znakov, ktorých hexadecimálne kódy sa označujú ako **U+0000** až **U+0010 FFFF**.
- Verzia 16.0.0 (schválená 10. septembra 2024) definuje kódy pre 154 998 znakov.
- **UTF („Unicode Transformation Format“):**
 - definuje spôsob, akým sa 21-bitový kód znaku v štandarde Unicode uloží do jedného alebo viacerých bajtov;
 - najznámejšie formáty (<https://www.branah.com/unicode-converter>):
 - **UTF-8**
 - **UTF-16**
 - **UTF-32**

Typy uchovávajúce znaky – štandard Unicode

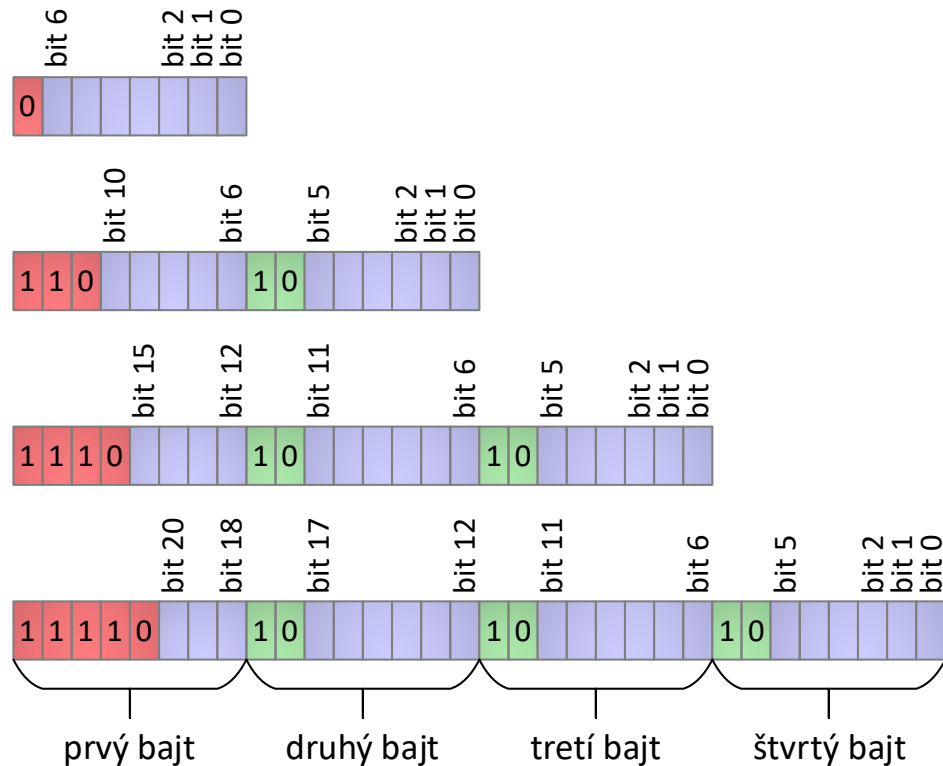
- **UTF-8:**
 - spätne kompatibilný s kódovaním ASCII;
 - znak je reprezentovaný 1 až 4 **bajtmi** (znaky anglickej abecedy – 1 bajt, ďalšie európske znaky – 2 bajty, východoázijské znaky – obyčajne 3 bajty);
 - **bajtovo orientované kódovanie**;
 - najpoužívanejší formát kódovania textov na internete; používaný v operačnom systéme Linux (údajový typ char);
- **UTF-16:**
 - znak je reprezentovaný 1 alebo 2 **dvojbajtmi**;
 - **blokovo orientované kódovanie**;
 - používaný v jazyku Java od verzie 5.0 (údajový typ char) a v operačnom systéme Windows od verzie 2000 (údajový typ wchar_t);
- **UTF-32:**
 - znak je reprezentovaný 1 **štvorbajtom**;
 - pamäťovo neefektívny;
 - nie je veľmi používaný (napr. wchar_t v operačnom systéme Linux).

Transformácia 21-bitového kódu znaku v Unicode do UTF-8

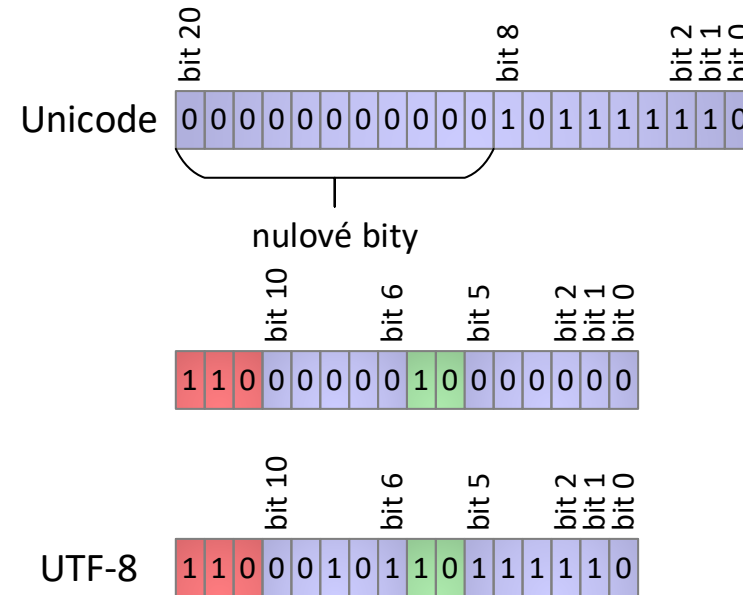
- Z 21-bitového kódu Unicode sa zľava odstránia najskôr všetky bity s hodnotou 0, ktoré predchádzajú prvému nenulovému bitu.
- Podľa počtu zvyšných bitov sa vyberie minimálny počet bajtov, ktoré sú potrebné na uloženie týchto bitov vrátane (červených) riadiacich bitov a (zelených) bitov definujúcich začiatok nasledujúceho bajtu.
 - Riadiace bity: sekvencia začína bitom 1 a nasleduje toľko na 1 nastavených bitov, koľko bajtov sa použije. Sekvencia riadiacich bitov končí 0.
 - Bity definujúce začiatok ďalšieho bajtu: sekvencia bitov 1 a 0.
- V týchto bajtoch sa inicializujú riadiace (červené) bity a (zelené) bity definujúce začiatky ostatných bajtov, zatiaľ čo všetky (modré) bity reprezentujúce kód znaku v 21-bitovom štandarde Unicode sa nastaví na hodnotu 0.
- Bity, ktoré boli vybrané na začiatku, sa skopírujú na základe svojho indexu do jednotlivých bitov formátu UTF-8.

UTF-8

Bitové reprezentácie znaku pri použití formátu kódovania UTF-8



Ukážka transformácie 21-bitového kódu znaku „ž“ definovaného štandardom Unicode do formátu UTF-8



Identifikácia nulových bitov v 21-bitovom kóde Unicode

Na reprezentáciu znaku sú potrebné 2 bajty

Inicializácia bitov vo formáte UTF-8

Skopírovanie bitov z 21-bitového kódu Unicode

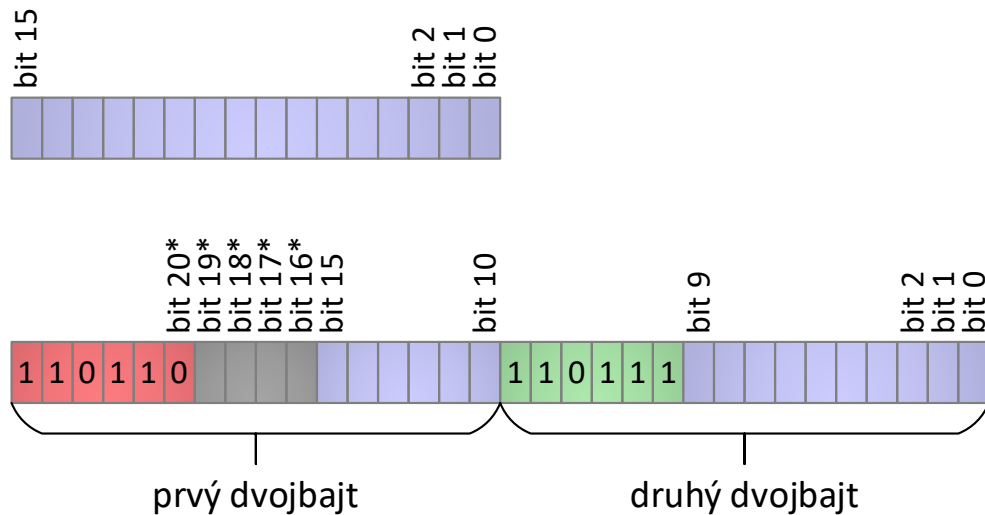
Kód Unicode vo formáte UTF-8

Transformácia 21-bitového kódu znaku v Unicode do UTF-16

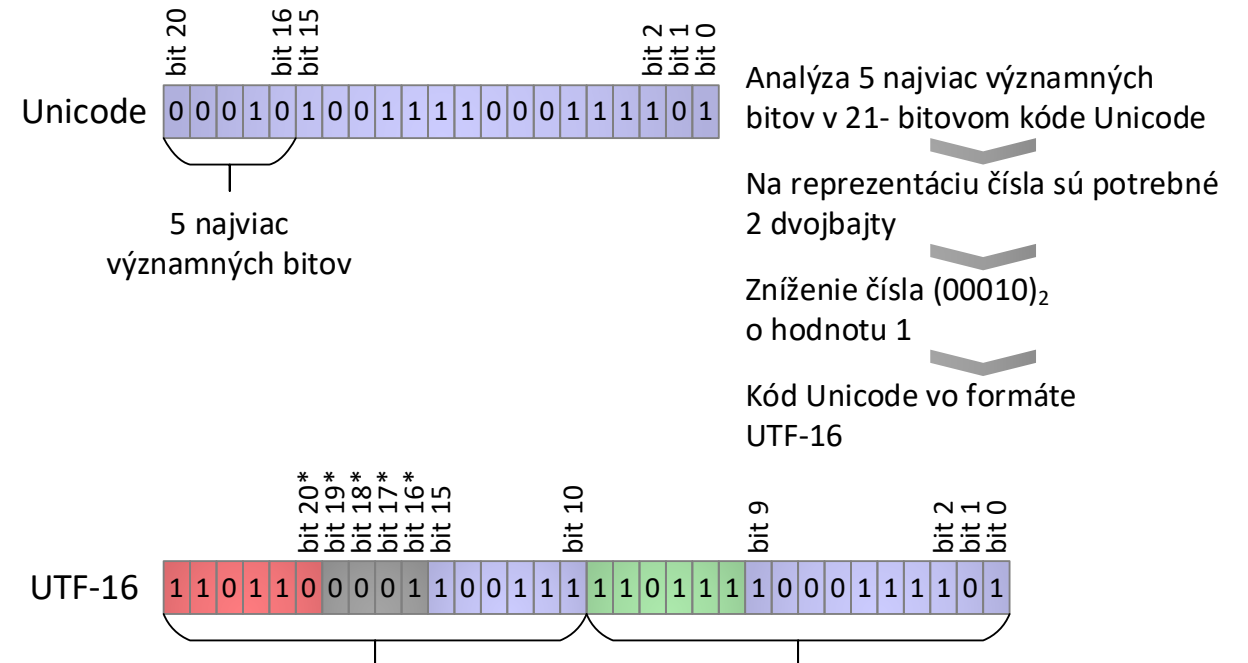
- Ak v 21-bitovom kóde Unicode majú všetky bity s indexami 16 až 20 hodnotu 0, na reprezentáciu znaku vo formáte UTF-16 sa použije jeden dvojbajt, ktorého bity sa nastavlia podľa hodnôt bitov 0 až 15 z kódu Unicode.
- V prípade, že niektorý z bitov s indexami 16 až 20 v 21-bitovom kóde Unicode má hodnotu 1, na kódovanie znaku vo formáte UTF-16 sa využijú 2 dvojbajty:
 - Bity definujúce začiatok:
 - prvého dvojbajtu: (červená) sekvencia bitov 110110.
 - druhého dvojbajtu: (zelená) sekvencia bitov 110111.
 - Bity s indexami 0 až 15 z kódu Unicode sa skopírujú na korešpondujúce bity dvojbajtu.
 - Na zvyšných 5 bitov s indexami 16 až 20 v kóde Unicode sa následne pozerá ako na celé číslo vyjadrené v dvojkovej sústave.
 - Od tohto čísla sa odráta hodnota 1 a výsledok, ktorý dostaneme, sa prekopíruje do bitov 16 - 20 (označených hviezdikami).
 - Rozsah platných kódov v štandarde Unicode zaistí, že najviac významný bit v 5-bitovom čísle, ktoré sme dostali po odrátaní čísla 1, bude mať vždy hodnotu 0, preto červený bit označený ako „bit 20“ nie je nutné nastavovať a môže byť vždy fixovaný na hodnotu 0.

UTF-16

Bitové reprezentácie znaku pri použití formátu kódovania UTF-16



Ukážka transformácie 21-bitového kódu znaku „魚花“ definovaného štandardom Unicode do formátu UTF-16



Typy uchovávajúce znaky

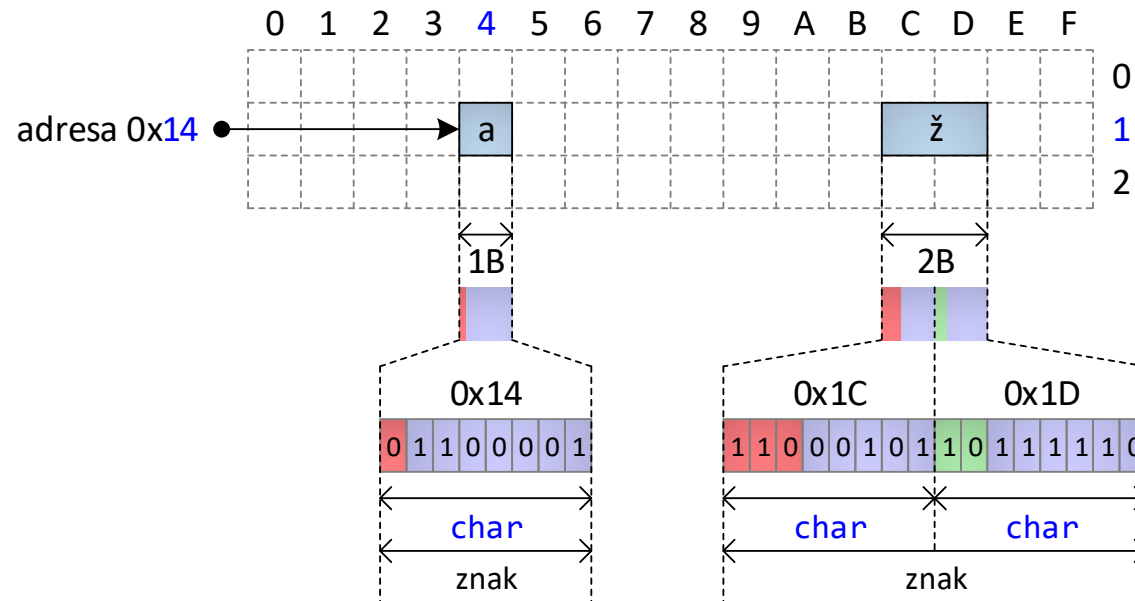
Údajový typ char

- Na väčšine bežných architektúr zaberá **1 bajt**, preto dokáže reprezentovať len 256 rôznych znakov.
- Spôsob kódovania znakov je definovaný cieľovou platformou a nie jazykom C++ (na rozdiel od jazyka Java):
 - ak cieľová platforma používa ASCII, tak:
 - kódy z rozsahu 0 až 127 slúžia na reprezentáciu znakov z ASCII;
 - kódy z rozsahu 128 až 255 (alebo -128 až -1) slúžia na reprezentáciu národných znakov v závislosti od použitého rozšírenia ASCII;
 - **ak cieľová platforma používa UTF-8, tak 1 znaku môže zodpovedať sekvencia 1 až 4 premenných typu char.**

Údajový typ wchar_t

- **Viacbajtový** údajový typ.
- Jeho pamäťovú reprezentáciu ovplyvňuje endianita architektúry.
- Na operačnom systéme Linux je implementovaný ako 4-bajtový údajový typ (formát UTF-32).
- Na operačnom systéme Windows je implementovaný ako 2-bajtový údajový typ (formát UTF-16):
 - ak máme znak, ktorý potrebuje na 2 dvojbyty, tak tento je reprezentovaný sekvenciou 2 premenných typu wchar_t;
- Vo všeobecnosti **nie je prenositeľný**, a preto ho nie je vhodné používať na reprezentáciu znakov v aplikáciách, ktoré majú byť spustiteľné na rôznych architektúrach.

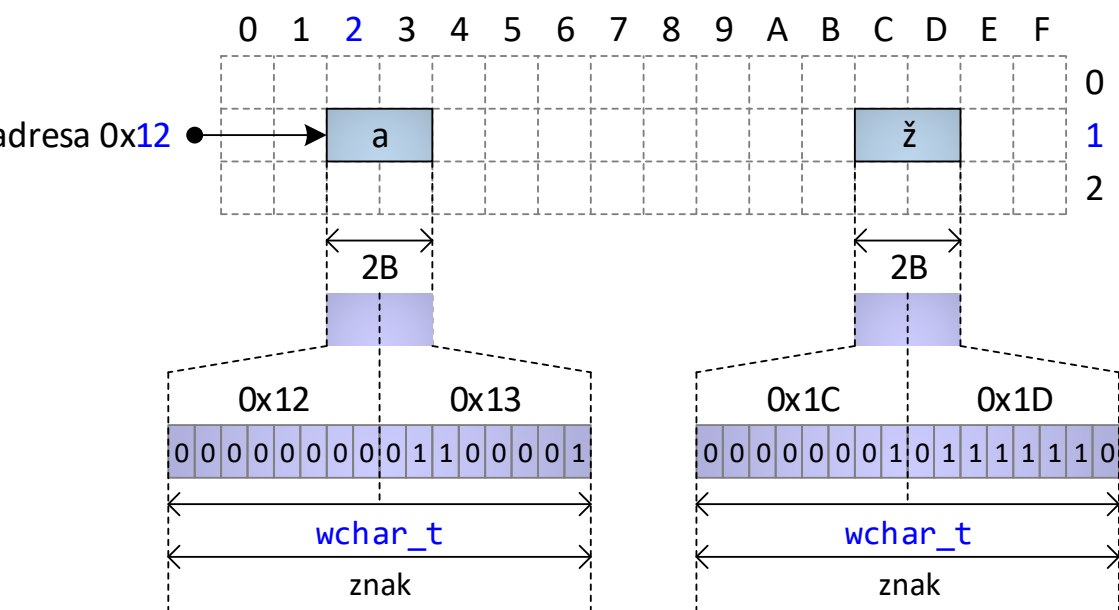
Rozdielne kódovanie



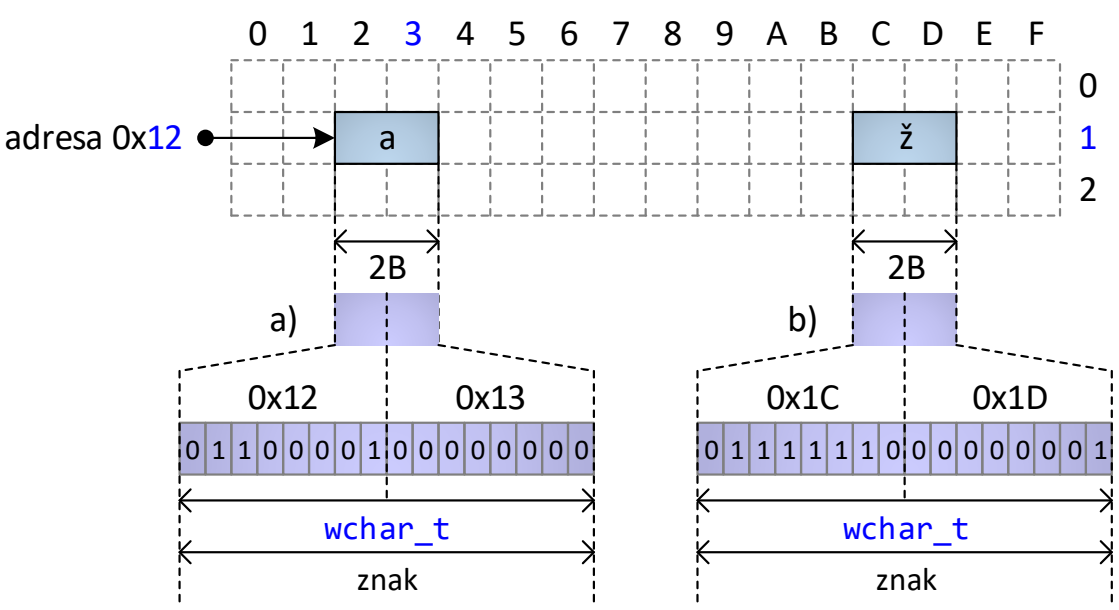
- Pri použití (nesprávneho) kódovania Windows-1250 by bol kód znaku 'ž' vyhodnotený ako sekvencia znakov 'Ĺ' a 'I'.

Údajový typ wchar_t realizovaný ako dvojбайт

Big-endian architektúra



Little-endian architektúra



Operácie so znakmi

- Medzi štandardné operácie definované nad údajovými typmi `char` a `wchar_t` patria:
 - rozdiel 2 znakov ('5' – '0');
 - pripočítanie ('0' + 2) alebo odpočítanie celého čísla od znaku;
 - relačné operácie:
 - `==`, `!=`, `<`, `<=`, `>`, `>=` .
- Tieto základné operácie pracujú s **celočíselnou reprezentáciou** znaku.
- Napr. v kódovaniach ASCII, UTF-8 alebo UTF-16 platí, že kódy znakov číslíc '0' až '9', veľkých písmen anglickej abecedy 'A' až 'Z', ako aj malých písmen anglickej abecedy 'a' až 'z' tvoria sekvenciu postupne rastúcich čísel, vďaka čomu je porovnanie znakov ľubovoľných dvoch číslíc ekvivalentné porovnaniu hodnoty týchto číslíc a porovnanie ľubovoľných dvoch písmen veľkej (alebo malej) anglickej abecedy porovnaniu pozície týchto písmen v anglickej abecede.

Typy uchovávajúce logické hodnoty

- `bool`
- Uchováva iba 2 hodnoty, ale musí byť uložený v aspoň 1B (najmenšia adresovateľná jednotka pamäte).
- operácie `&&` a `||` je možné vyhodnotiť dvomi spôsobmi:
 - **úplné vyhodnocovanie** – vždy sa vyhodnotí ľavý aj pravý operand príslušnej operácie a až následne sa vyhodnotí celá operácia.
 - **skrátené vyhodnocovanie** (angl. „short-circuit evaluation“) – vždy sa vyhodnotí ľavý operand a v závislosti od neho sa vyhodnotí celý výraz alebo druhý operand a potom celý výraz:
 - `&&`: ak je hodnota ľavého operandu **false**, celý výraz bude **false** bez ohľadu na hodnotu pravého operandu.
 - `||`: ak je hodnota ľavého operandu **true**, celý výraz bude **true** bez ohľadu na hodnotu pravého operandu.

Referenčné údajové typy

Ukazovateľ (angl. „pointer“)

- Špecifický údajový typ schopný uchovávať adresu objektu v pamäti.
- Veľkosť závisí od cieľovej architektúry (4B/8B).
- Zoznam podporovaných operácií:
 - dereferencovanie (*),
 - aritmetika ukazovateľov (++; --; pričítanie/odčítanie konštanty; rozdiel dvoch ukazovateľov za predpokladu, že ukazujú na položky v rovnakom poli),
 - relačné (==, !=),
 - relačné (<, <=, >, >=) za predpokladu, že ukazujú na položky v rovnakom poli.
- Príklady využitia:
 - **uchovávanie adres dynamicky alokovaných premenných,**
 - **odovzdávanie argumentov do funkcie adresou,**
 - realizácia viacrozmerých polí a callback-u,
 - preskúmanie pamäťovej reprezentácie údajového typu.
- **V C++ používame len, ak je to nutné.**

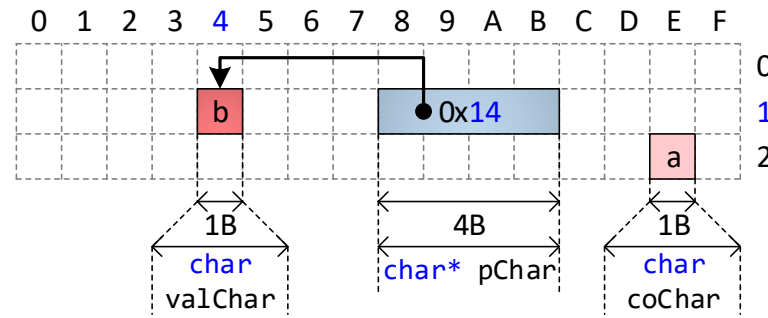
Odkaz (angl. „reference“)

- Alternatívny názov pre už existujúci objekt v pamäti.
- Definovaním odkazu nevzniká v pamäti z pohľadu programátora nová premenná.
- Od svojho vzniku až do svojho zániku musí odkazovať vždy len na pamäťové miesto (premennú), na ktorú bol pri svojom vzniku nastavený (inicializovaný).
- Podporuje rovnaké operácie ako údajový typ, ktorý referencuje.
- Hlavné využitie – **odovzdávanie argumentov do funkcie odkazom.**
- **V C++ používame vždy, ak je to možné.**

Údajový ukazovateľ – ukážky na 32-bitovej architektúre

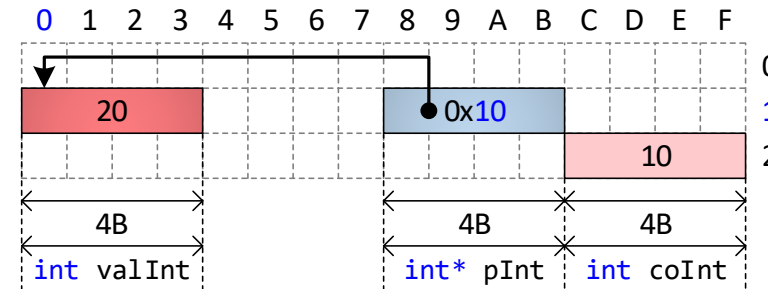
Definícia premennej typu **char**, ukazovateľa na ňu a ukážka dereferencovania v jazyku C++

```
1. char valChar = 'a';  
2. char* pChar = &valChar;  
3. char coChar = *pChar;  
4. *pChar = 'b';
```



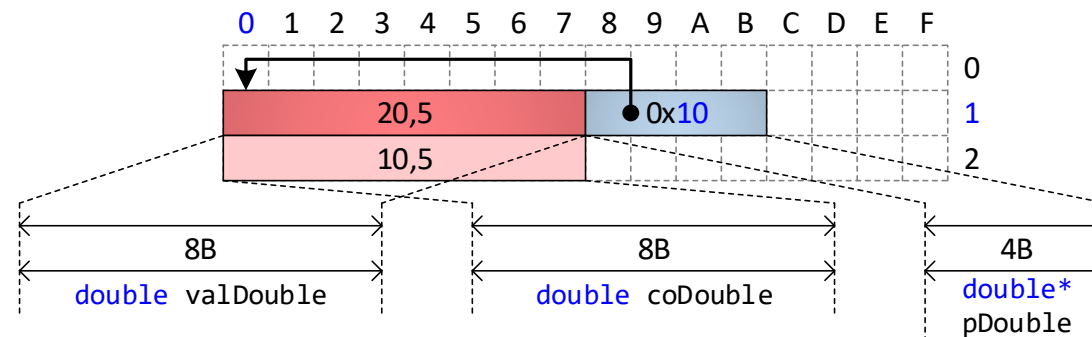
Definícia premennej typu **int**, ukazovateľa na ňu a ukážka dereferencovania v jazyku C++

```
1. int valInt = 10;  
2. int* pInt = &valInt;  
3. int coInt = *pInt;  
4. *pInt = 20;
```



Definícia premennej typu **double**, ukazovateľa na ňu a ukážka dereferencovania v jazyku C++

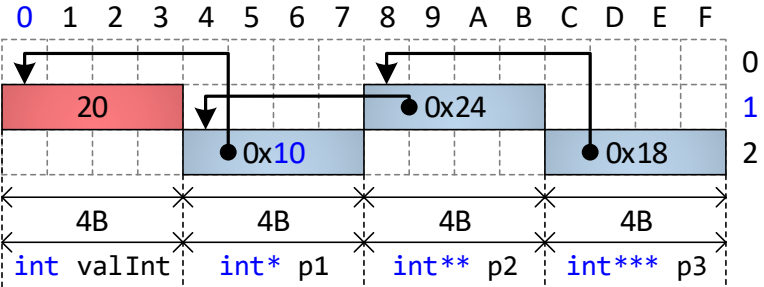
```
1. double valDouble = 10.5;  
2. double* pDouble;  
3. pDouble = &valDouble;  
4. double coDouble = *pDouble;  
5. *pDouble = 20.5;
```



Reťazenie údajových ukazovateľov – ukážka na 32-bitovej architektúre

Definícia premennej typu `int` a postupnosti ukazovateľov na ňu v jazyku C++

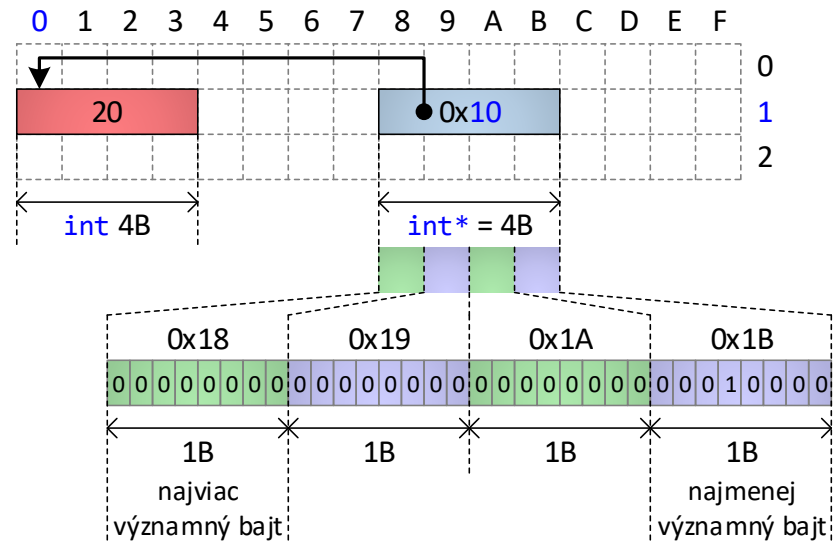
1. `int valInt = 20;`
2. `int* p1 = &valInt;`
3. `int** p2 = &p1;`
4. `int*** p3 = &p2;`



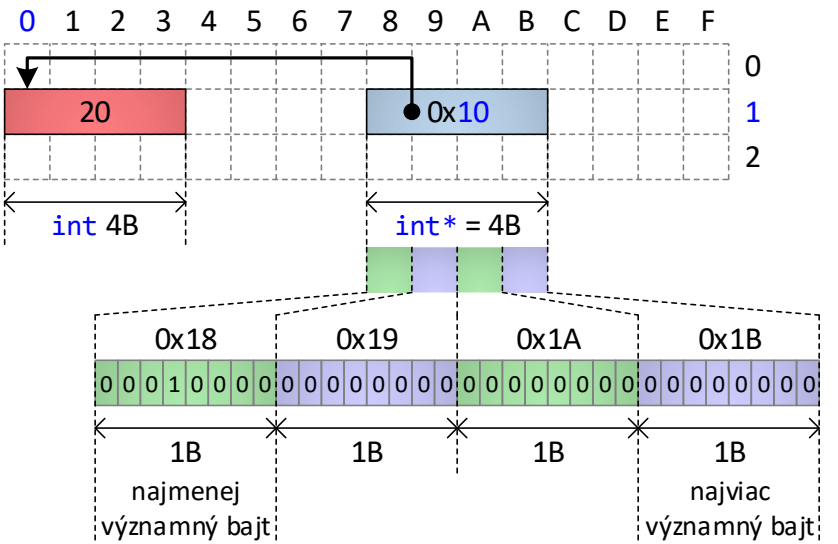
`p3 == &p2`
`*p3 == p2 == &p1`
`**p3 == *p2 == p1 == &valInt`
`***p3 == **p2 == *p1 == valInt`

Pamäťová reprezentácia ukazovateľa – ukážky na 32-bitovej architektúre

Big-endian architektúra



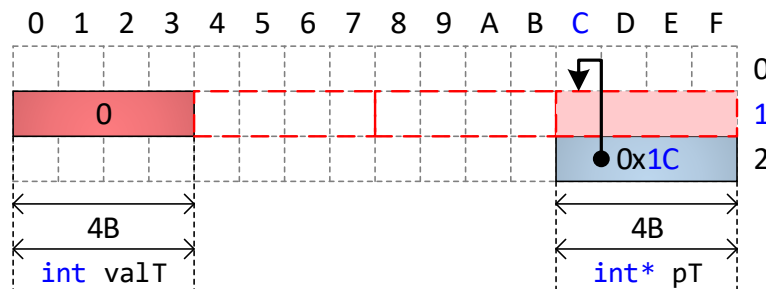
Little-endian architektúra



Aritmetika ukazovateľov – ukážka na 32-bitovej architektúre

Ukážka definície premennej typu *T*, ukazovateľa na ňu a následnej zmeny ukazovateľa pripočítaním celočíselnej hodnoty v jazyku C++

```
1. T valT();  
2. T* pT = &valT;  
3. pT = pT + 3;
```



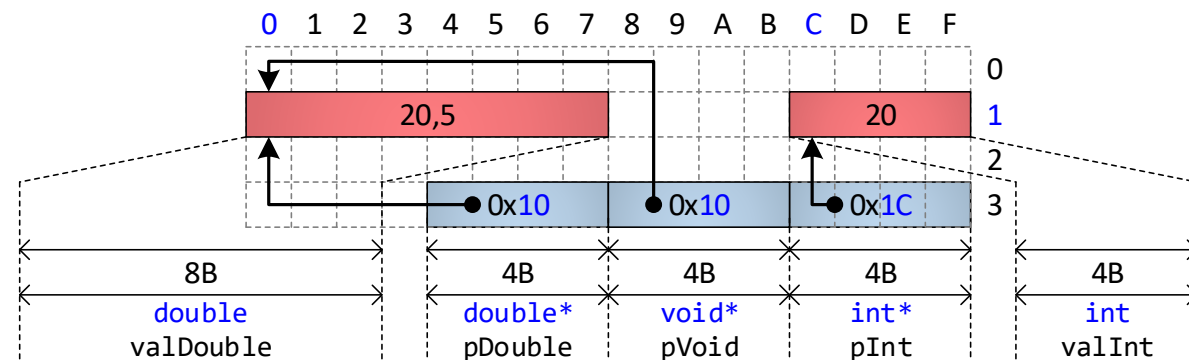
Ilustračný príklad. Pozor na nedefinované správanie pri dereferencovaní!!!

Generický ukazovateľ (void*) – ukážky na 32-bitovej architektúre

- Schopný uchovávať adresu ľubovoľného objektu.
- Nepodporuje aritmetiku ukazovateľov.
- Nie je možné ho dereferencovať.
- Je naň možné pretypovať ľubovoľný iný ukazovateľ.

Ukážka jednoduchkej práce s generickým ukazovateľom v jazyku C++

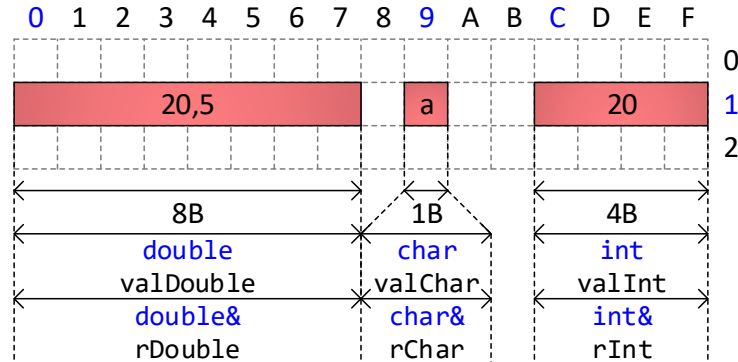
```
1. double valDouble = 20.5;  
2. int valInt = 20;  
3. double* pDouble = &valDouble;  
4. void* pVoid = &valInt;  
5. int* pInt = (int*)pVoid; //explicitné pretypovanie  
6. pVoid = pDouble; //implicitné pretypovanie
```



Odkaz – ukážky na 32-bitovej architektúre

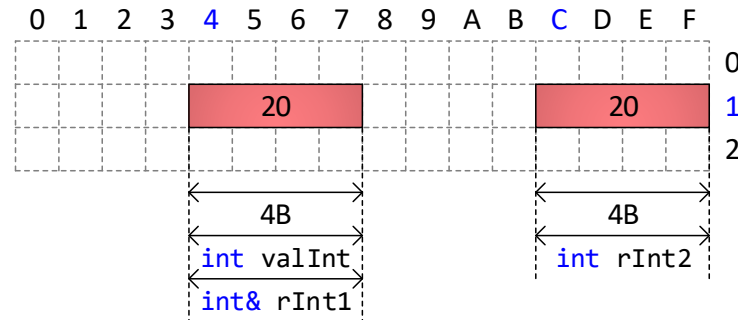
Vytvorenie 3 premenných odlišných typov
a následné vytvorenie odkazov na ne v jazyku C++

```
1. double valDouble = 20.5;
2. char valChar = 'a';
3. int valInt = 20;
4. double& rDouble = valDouble;
5. int& rInt = valInt;
6. char& rChar = valChar;
```



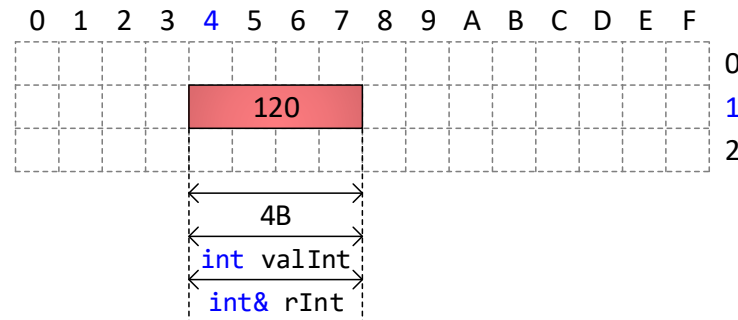
Rozdiel medzi definovaním odkazu a kópie
premennej v jazyku C++

```
1. int valInt = 20;
2. int& rInt1 = valInt;
3. int rInt2 = valInt;
```



Práca s premennou prostredníctvom odkazu
v jazyku C++

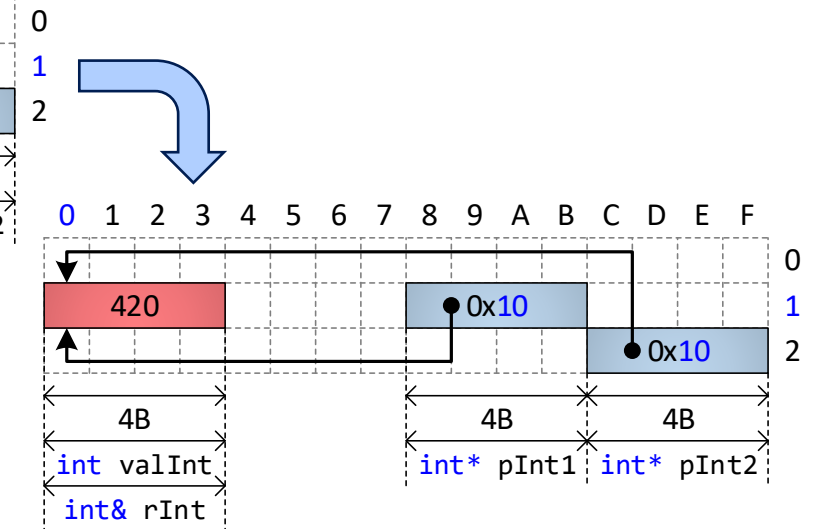
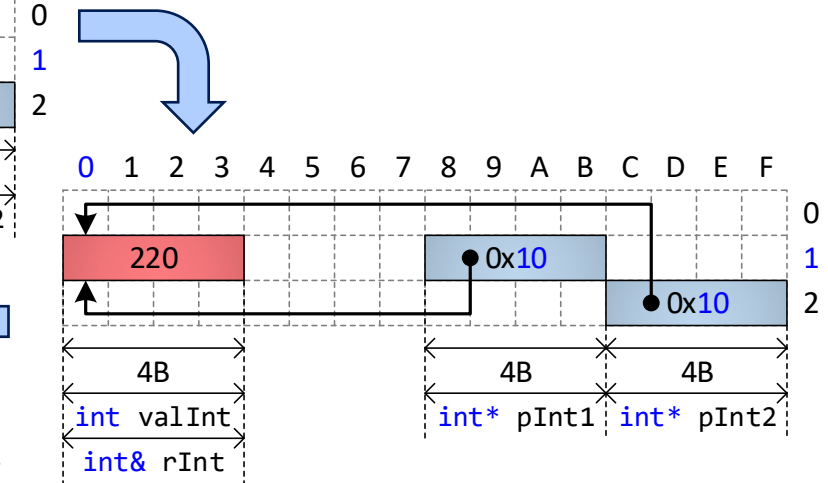
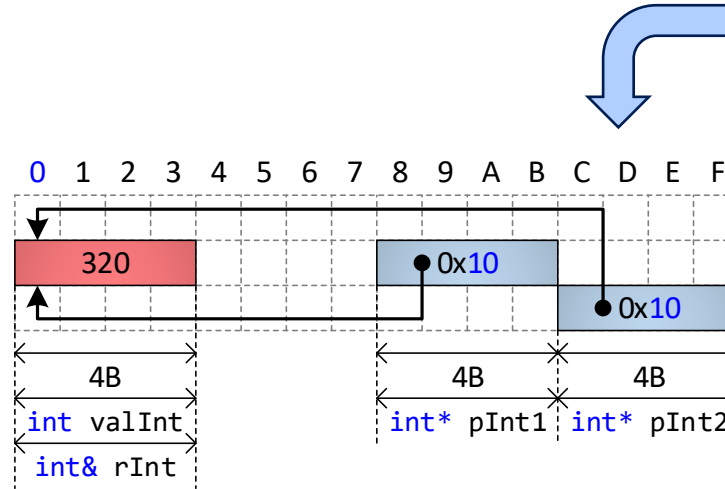
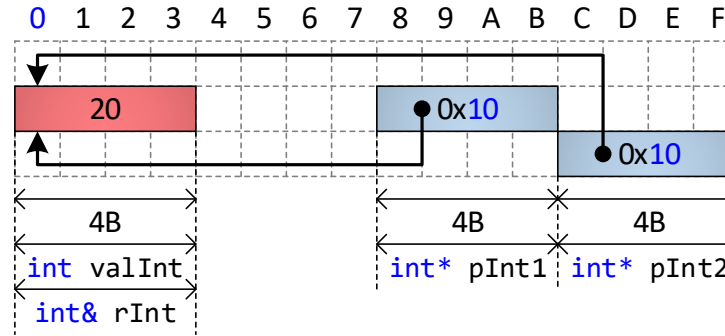
```
1. int valInt = 20;
2. int& rInt = valInt;
3. rInt = rInt + 100;
```



Odkaz – ukážky aplikácie operátora & na odkaz v jazyku C++

Aplikácia operátora adresa (&) na odkaz v jazyku C++

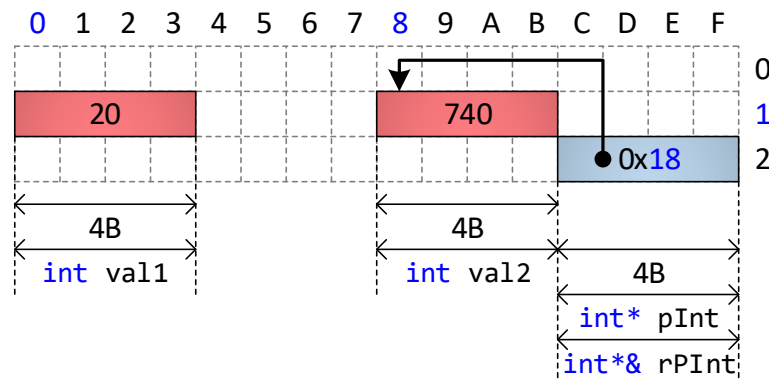
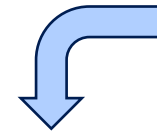
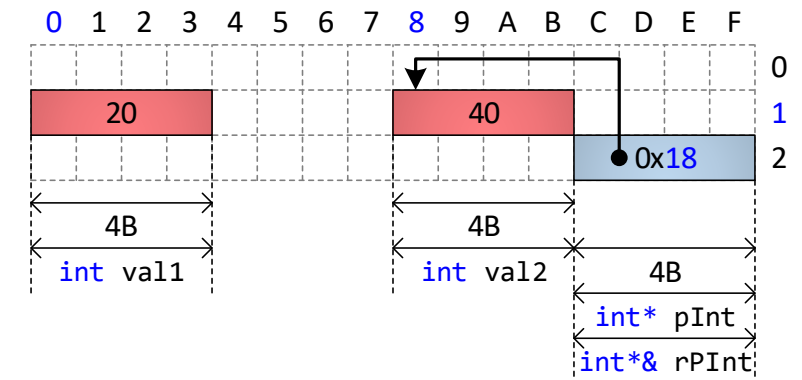
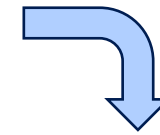
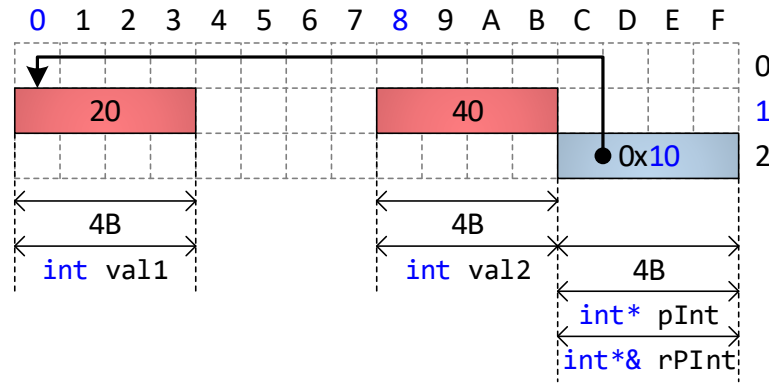
```
1.  int valInt = 20;
2.  int& rInt = valInt;
3.  int* pInt1 = &valInt;
4.  int* pInt2 = &rInt;
5.  //porovnanie ukazovateľov, t. j. adres
6.  if (pInt1 == pInt2) {
7.      *pInt1 = 220;
8.  }
9.  //porovnanie premennej a odkazu na ňu, t. j. hodnôt
10. if (valInt == rInt) {
11.     rInt = 320;
12. }
13. /* porovnanie adres premennej a odkazu na ňu, toto
    porovnanie je identické s porovnaním na riadku 6 */
14. if (&valInt == &rInt) {
15.     valInt = 420;
16. }
```



Odkaz na ukazovateľ – ukážky na 32-bitovej architektúre

Ukážka práce s odkazom na ukazovateľ v jazyku C++

```
1. int val1 = 20;
2. int val2 = 40;
3. int* pInt = &val1;
4. int*& rPInt = pInt;
   /* zmena adresy uloženej v ukazovateli pInt
   prostredníctvom odkazu rPInt na tento
   ukazovateľ */
5. rPInt = &val2;
   /* zmena hodnoty premennej, na ktorú ukazuje
   ukazovateľ pInt prostredníctvom odkazu
   rPInt na tento ukazovateľ */
6. *rPInt = 740;
```

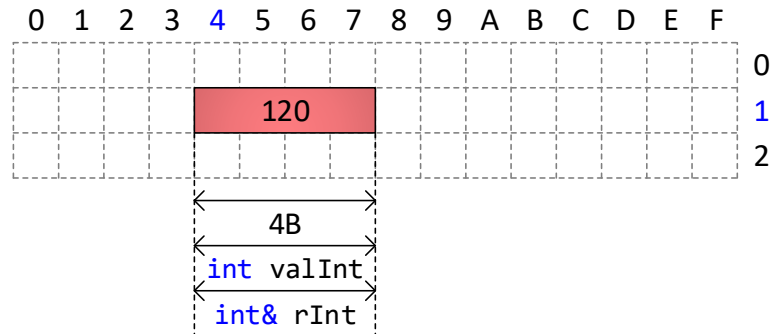


Odkaz – možná realizácia pomocou ukazovateľov na 32-bitovej architektúre

Odkaz

Práca s premennou prostredníctvom odkazu
v jazyku C++

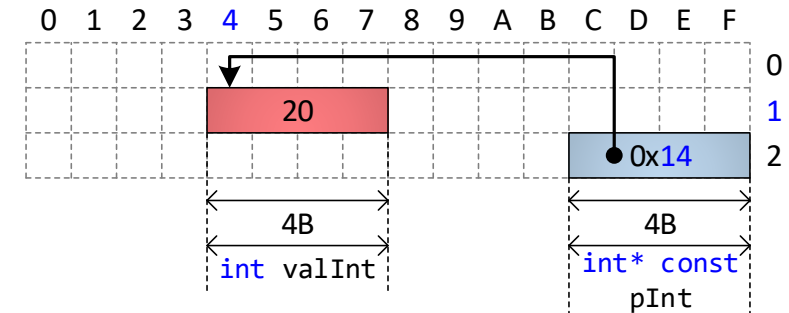
1. `int valInt = 20;`
2. `int& rInt = valInt;`
3. `rInt = rInt + 100;`



Odkaz realizovaný ukazovateľom

Možná transformácia kódu ilustrujúceho definovanie
odkazu a prácu s ním na premennú typu ukazovateľ
v jazyku C++

1. `int valInt = 20;`
2. `int* const pInt = &valInt;`
3. `*pInt = *pInt + 100;`



Odvedené typy

- Vlastné údajové typy, ktoré je možné z primitívnych typov vytvoriť dvomi spôsobmi:
 - **Agregácia** – zlúčením (agregovaním) viacerých existujúcich typov do jedného typu.
 - **Enumerácia** – vymenovaním (enumeráciou) všetkých možných hodnôt, ktoré môže nadobúdať.
- Agregácia môže byť:
 - **Homogénna** – pole
 - Principiálne slúži na uchovávanie údajov charakterizujúcich viaceré objekty toho istého typu.
 - **Heterogénna** – záznam
 - Principiálne slúži na uchovávanie údajov charakterizujúcich jeden objekt.

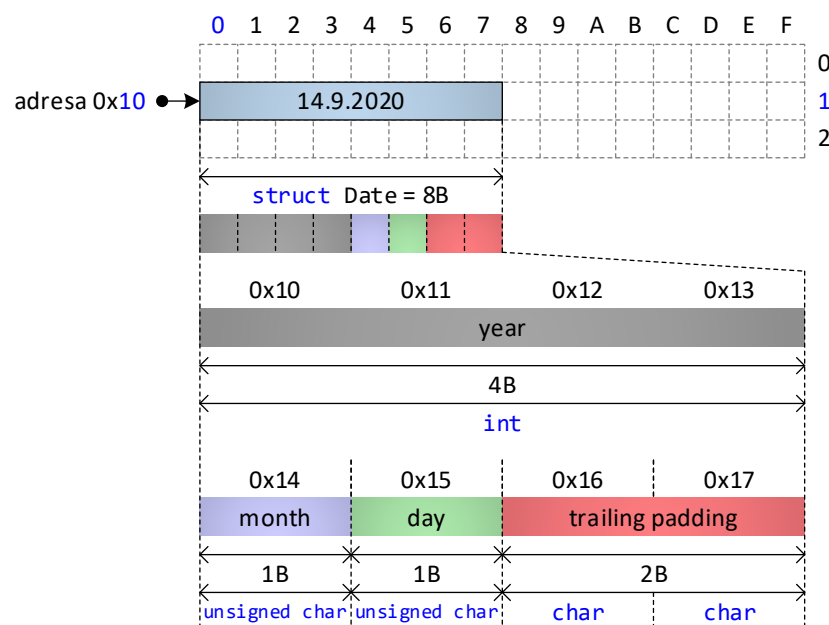
Záznam

- Uchováva údaje charakterizujúcich jeden objekt.
- V C++ `struct` a `class`.
- Pre efektívne spracovanie je potrebné **zarovnanie** (angl. „padding“) – prekladač doplní prázdne byty tak, aby údaje začínali na adrese zhodnej s násobkom šírky **slova** (angl. „word“) procesora :
 - Slovo je základné množstvo údajov, s ktorým dokáže procesor súčasne pracovať v jednom časovom okamihu a je definované šírkou v bitoch.
 - Primárne ovplyvnené hardvérovou architektúrou.
 - V prípade 64-bitových procesorov má hodnotu 64.
- Prekladač môže **doplniť** bajty za štruktúru – (angl. „trailing padding“), aby bolo zabezpečené uloženie ďalších údajov na „správnej“ adrese.
- Údaje v zázname definujte od pamäťovo najväčších typov po pamäťovo najmenšie typy.

Zarovnanie

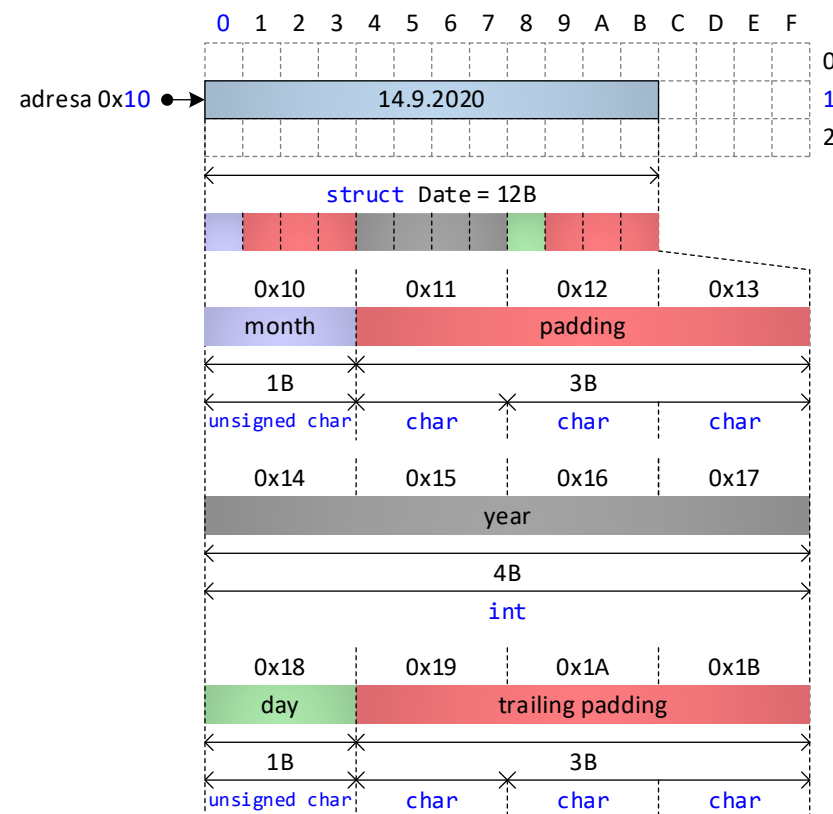
Definícia štruktúry dátum v jazyku C++ s menšími pamäťovými nárokmi po zohľadnení zarovnaní údajových typov

```
1. struct Date {
2.     int year;
3.     unsigned char month;
4.     unsigned char day;
5. };
```



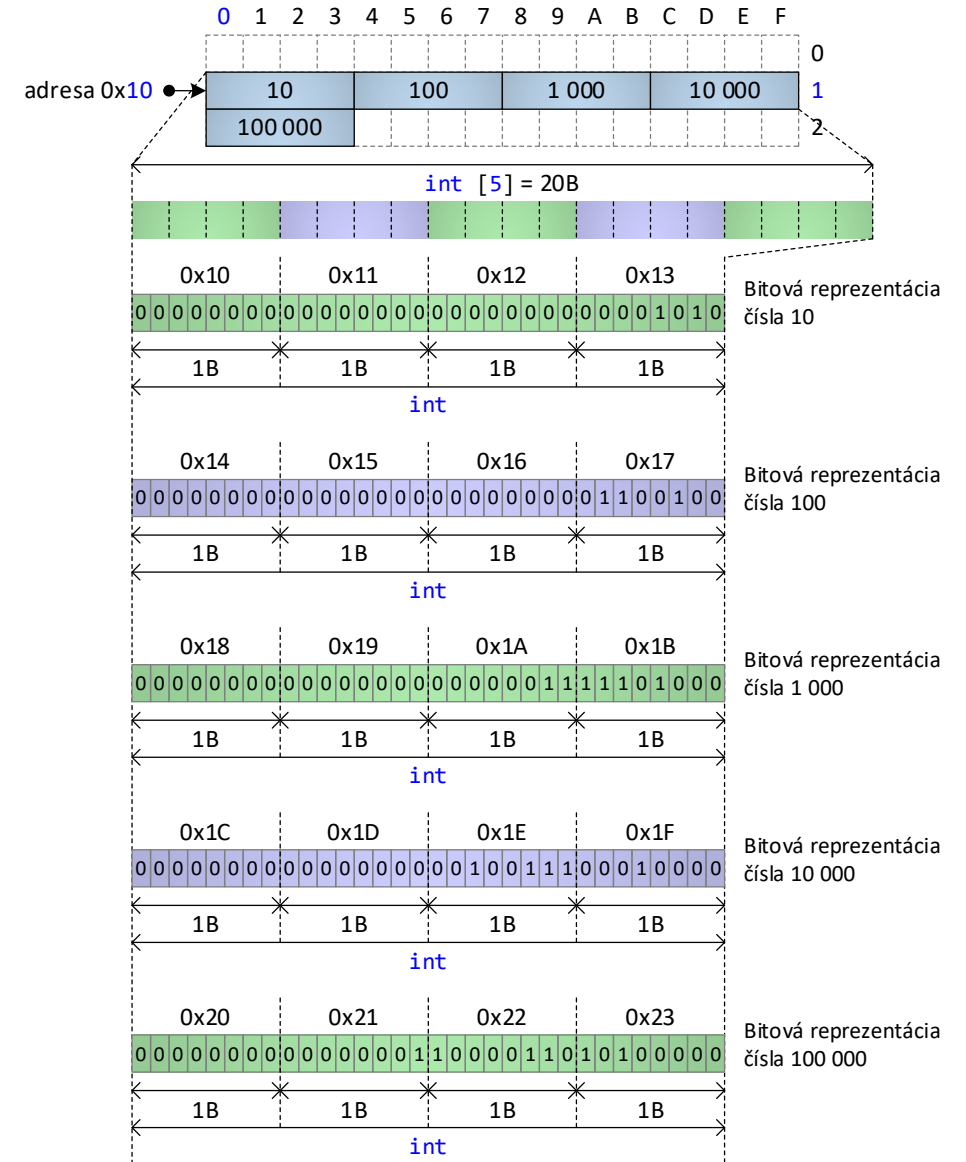
Definícia štruktúry dátum v jazyku C++ s „menšími pamäťovými nárokmi“ po nevhodnom preusporiadaní údajových typov

```
1. struct Date {
2.     unsigned char month;
3.     int year;
4.     unsigned char day;
5. };
```



Pole

- Uchováva údaje charakterizujúce viaceré objekty toho istého typu.
- V C++ sa definuje uvedením `[]` za identifikátor premennej, ktorej typ určuje, čo sa v poli nachádza (napr `int cisla[5]`).
- Z nízkoúrovňového pohľadu je to súvislý blok operačnej pamäte rozdelený na menšie rovnako veľké bloky.
- Pozor na pole záznamov a potenciálne plytvanie pamäte kvôli zarovnaniu!
- Ak sú prvky poľa ďalšie polia, je možné vytvárať dvoj- a viacrozmerné polia.
- Pole je jedna zo základných údajových štruktúr – o tom neskôr.



Pole 5 čísel v architektúre big-endian

Pole znakov – reťazec

Ret'azec

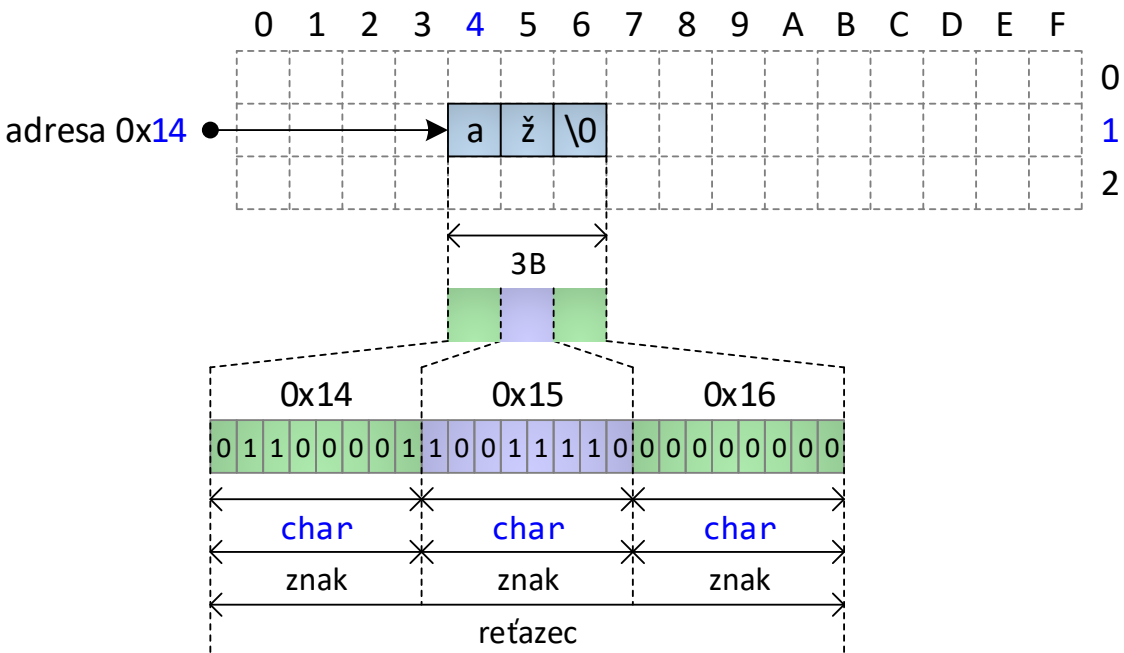
- Špecifický typ poľa, ktorého položkami sú znaky (údajový typ `char`).
- Posledná položka poľa je znak s kódom 0 („NULL character“).
- Endianita neovplyvňuje pamäťovú reprezentáciu položiek.
- Ak sú znaky reťazca kódované **v štandarde ASCII alebo jeho rozšíreniach**, v ktorých je veľkosť 1 znaku zhodná s veľkosťou 1 bajtu, tak potom platí, že **dĺžka reťazca** (počet znakov reťazca) sa **zhoduje s počtom položiek typu `char`**, ktoré predchádzajú v poli reprezentujúcom reťazec prvému výskytu nulového znaku, a **každá položka v poli reprezentuje práve jeden znak**.
- Ak sú znaky reťazca kódované **v štandarde Unicode**, v ktorom môže byť 1 znak reprezentovaný 1 až 4 bajtmi, tak **dĺžka reťazca** reprezentovaného polom položiek typu `char` môže byť **menšia, ako je počet položiek poľa** predchádzajúcich prvému výskytu nulového znaku.

„Široký reťazec“

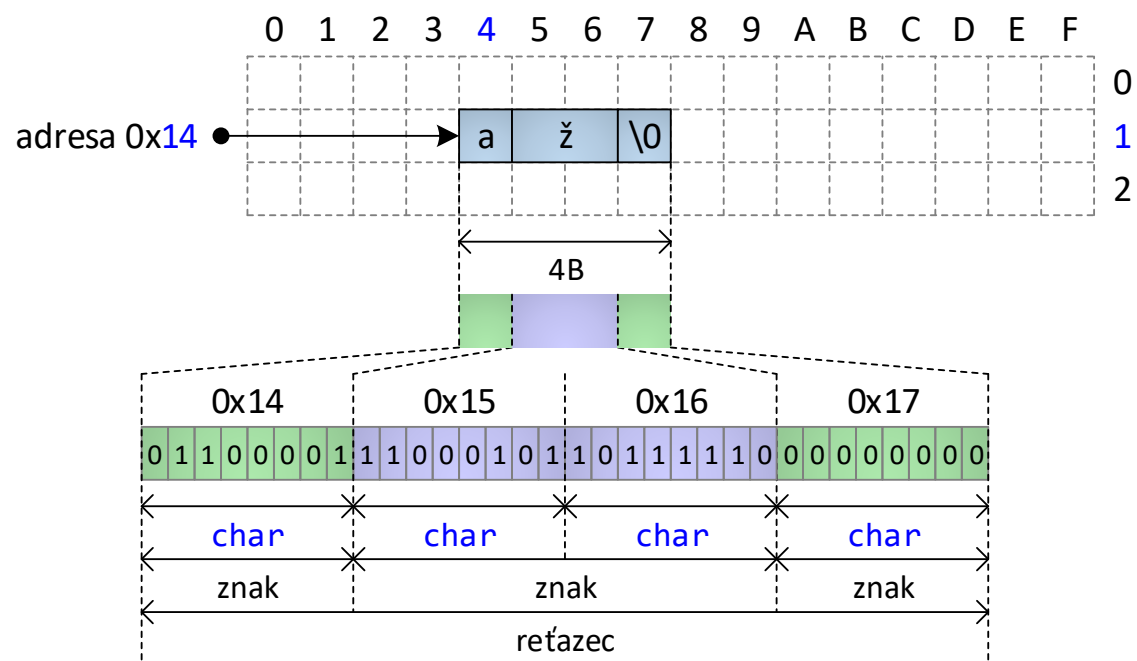
- Špecifický typ poľa, ktorého položkami sú „široké“ znaky (údajový typ `wchar_t`).
- Prvkami poľa sú viacbajtové položky, pričom v závislosti od kompilátora môže mať údajový typ `wchar_t` veľkosť 2 alebo 4 bajty.
- **Endianita ovplyvňuje pamäťovú reprezentáciu položiek.**
- Posledná položka poľa je znak s kódom 0 („NULL character“).
- Visual C++ kompilátor pre Windows definuje `wchar_t` ako 2-bajtový typ a na kódovanie znakov používa kódovanie UTF-16, v ktorom môžu mať kódy znakov dĺžku 2 alebo 4 bajty. **1 položka poľa tak predstavuje celý kód alebo polovicu kódu 1 znaku.**
- Kompilátor g++ pre Linux definuje `wchar_t` ako 4-bajtový údajový typ a na kódovanie znakov používa UTF-32. V tomto prípade platí, že **1 položka poľa položiek typu `wchar_t` vždy reprezentuje celý znak** a počet položiek predchádzajúcich prvému výskytu nulového znaku sa zhoduje s dĺžkou reťazca uloženého v tomto poli.

Údajový typ reťazec – ukážka pamäťovej reprezentácie

Rozšírenie ASCII



UTF-8



Vymenovaný typ

- Vzniká vymenovaním všetkých možných hodnôt, ktoré môže nadobúdať.
- V C++ enum.
- Interne implementovaný ako **celočíselný údajový typ** – jednotlivé hodnoty vymenovaného typu zodpovedajú konkrétnym hodnotám celočíselného údajového typu použitého na implementáciu vymenovaného typu.
- Použije sa taký celočíselný typ, ktorý dokáže pokryť všetky hodnoty vymenovaného typu.
- C++ umožňuje presne definovať celočíselný typ použitý na implementáciu vymenovaného typu (nepovinné).
- Java – vníma enum ako **extenziu** všetkých inštancií triedy, ktorá vystupuje ako vymenovaný typ.

Definícia vymenovaného typu v jazyku C++ s explicitným uvedením celočíselného typu použitého pre internú reprezentáciu vymenovaného typu a jeho hodnôt

```
1. enum DayOfWeek : short {  
2.     Monday,  
3.     Tuesday,  
4.     Wednesday,  
5.     Thursday,  
6.     Friday,  
7.     Saturday,  
8.     Sunday  
9. };
```

Zhrnutie prednášky

- Predstavenie predmetu
- Zložitosť algoritmov a údajových štruktúr
 - Časová a pamäťová zložitosť
 - Asymptotické odhady
 - Triedy zložitosť algoritmov
- Kódovanie údajov

Plán na cvičenie

- Opakovanie C++.
- Príprava univerzálneho testeru údajových štruktúr.

Ďakujem za pozornosť

Doplňujúce informácie?
Vaše otázky?

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu 5UI124 Algoritmy a údajové štruktúry 1 na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline.
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené.

Ing. Michal Varga, PhD.
Katedra informatiky
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
Michal.Varga@fri.uniza.sk

Algoritmy a údajové štruktúry 1

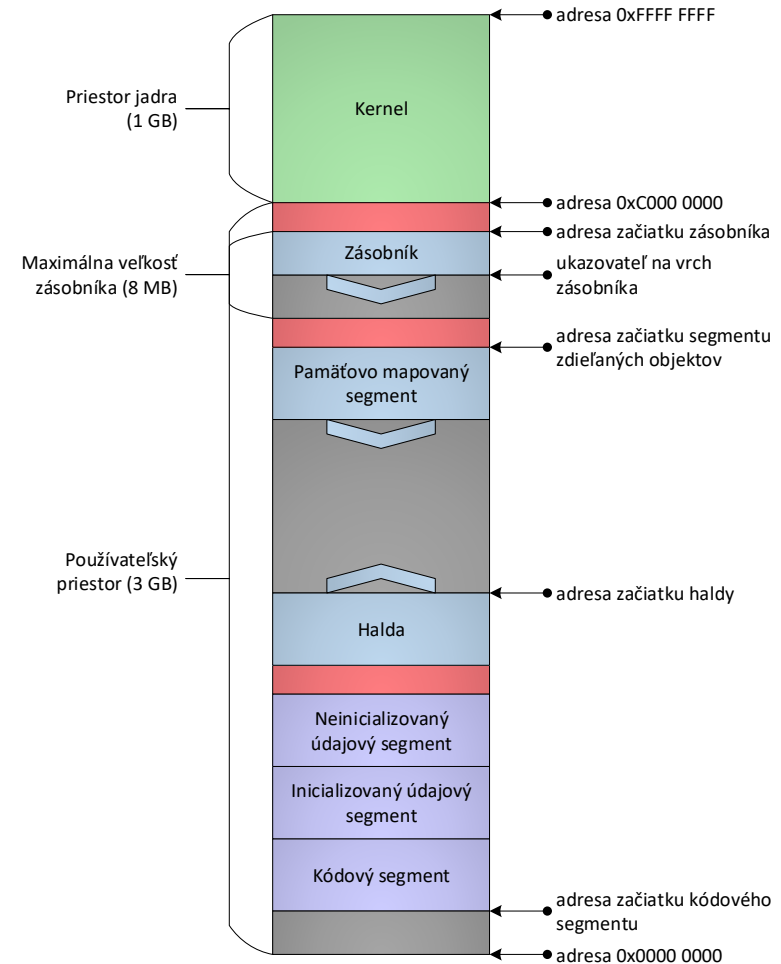
Prednáška č. 2

- Správa pamäte na úrovni operačného systému
- Správcovia pamäte



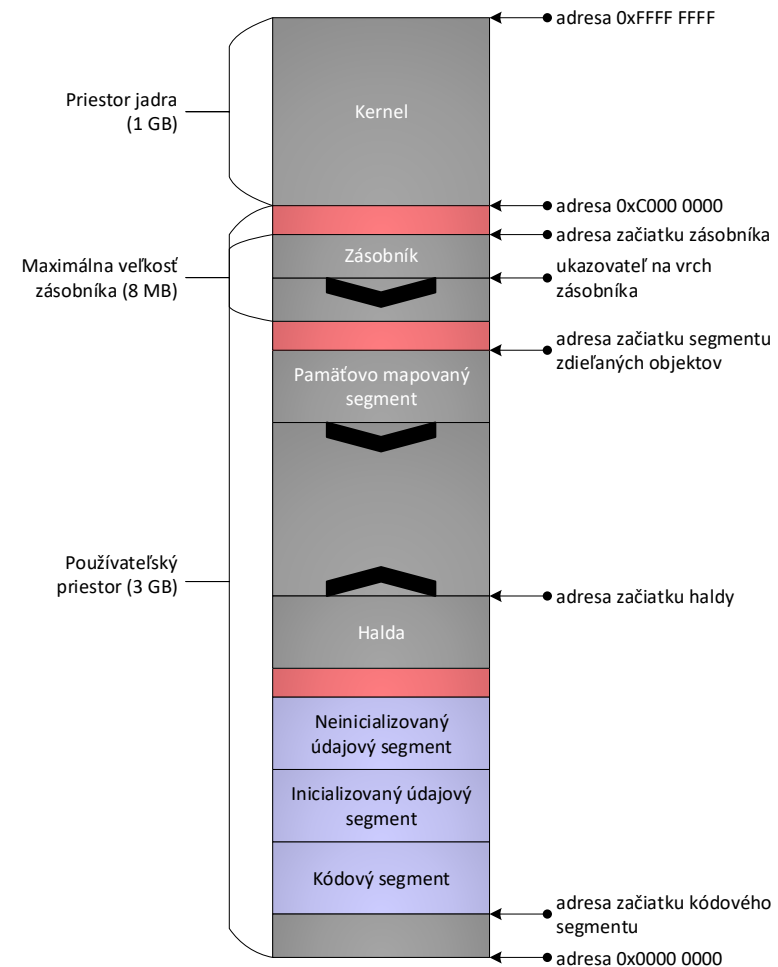
Správa pamäte na úrovni operačného systému

- **Program** je pasívna entita, postupnosť inštrukcií, ktorá hovorí, ako sa majú dané údaje spracovať.
- Aby mohol byť vykonaný, musí byť zavedený do pamäte – vzniká entita s názvom **proces**.
- Proces potrebuje od systému **pamäť**, ktorá sa delí do **segmentov**, podľa toho, čo je v nej uchovávané.
- Vpravo príklad 32-bitového OS Linux.



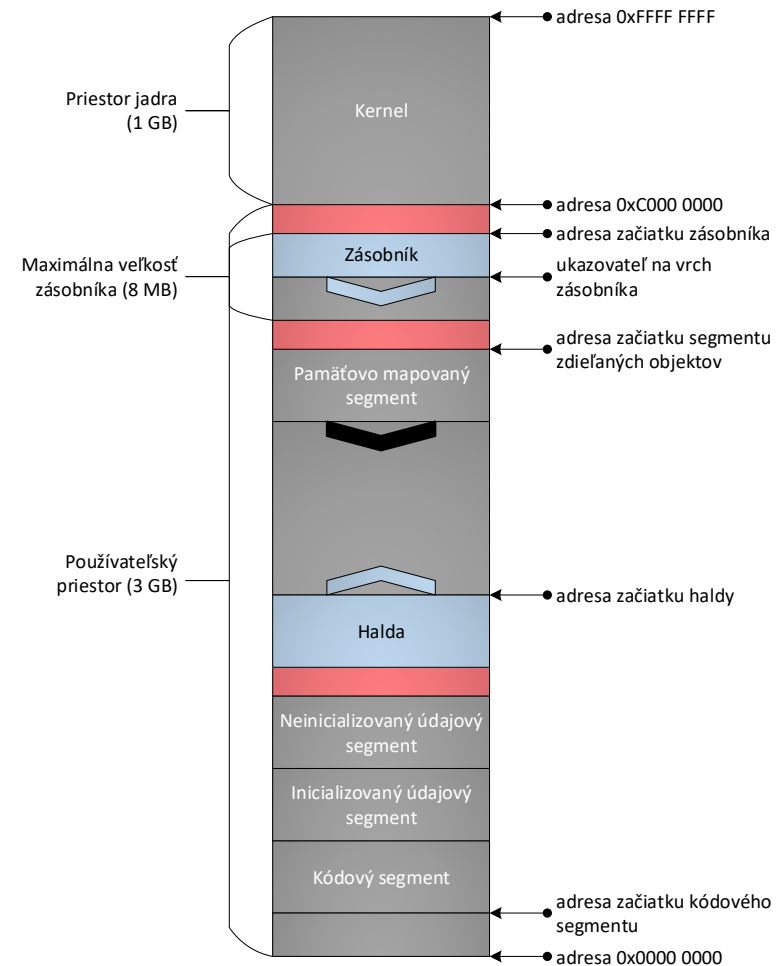
Segmenty s nemennou veľkosťou počas behu programu

- **Kódový segment** obsahuje postupnosť inštrukcií, ktoré má proces vykonávať = samotný program.
- **Inicializovaný údajový segment** obsahuje v kóde explicitne inicializované:
 - **globálne premenné:**
 - procedurálne programovanie: definované mimo funkcií a procedúr
 - objektové programovanie: definované mimo metód a tried
 - **statické premenné:**
 - definované v rámci funkcie (alebo procedúry/metódy) alebo triedy a ich aktuálny obsah sa zachováva medzi viacerými volaniami tej istej funkcie, alebo je zdieľaný medzi všetkými inštanciami danej triedy
- **Neinicializovaný údajový segment** (bss = block started by symbol) obsahuje v kóde explicitne neinicializované globálne a statické premenné.



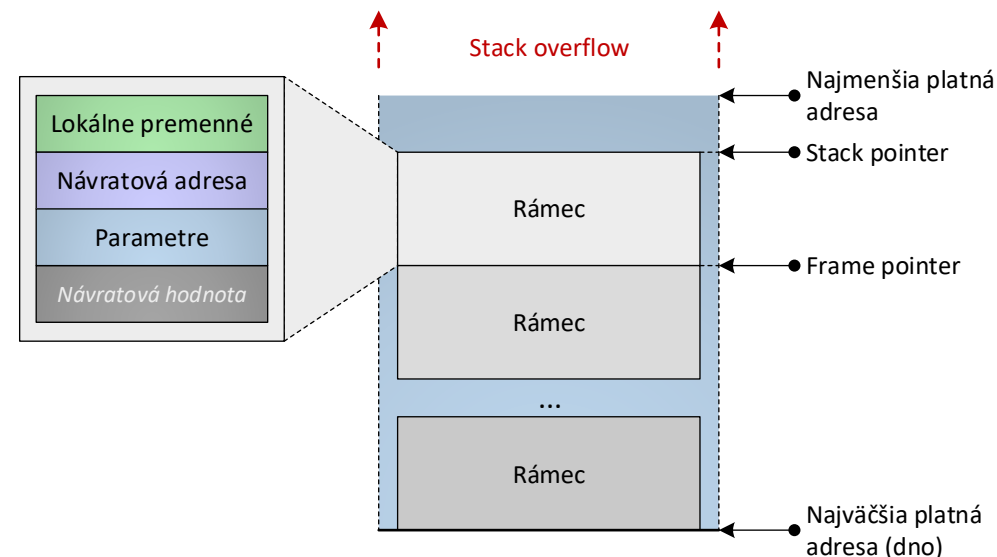
Segmenty s dynamickou veľkosťou počas behu programu

- Programy potrebujú nie len globálne a statické premenné, ale aj:
 - **lokálne premenné:**
 - definované v rámci funkcií.
 - ich obsah sa nezachováva medzi viacerými volaniami tej istej funkcie.
 - **parametre a návratové hodnoty z funkcií:**
 - pamäť pridelená od okamžiku zavolania funkcie do okamžiku jej opustenia.
 - **dynamicky alokované premenné:**
 - vznikajú na požiadanie.
 - zanikajú buď explicitne na požiadanie, alebo implicitne prostredníctvom mechanizmu sledujúceho nutnosť ďalšej existencie dynamicky alokovanej premennej
- **Zásobník** obsahuje premenné, ktorých životný cyklus je riadený životným cyklom funkcie (lokálne premenné, parametre a návratové hodnoty).
- **Halda** obsahuje dynamicky alokované premenné, ktorých životný cyklus je riadený explicitnými volaniami.



Zásobník – Call stack

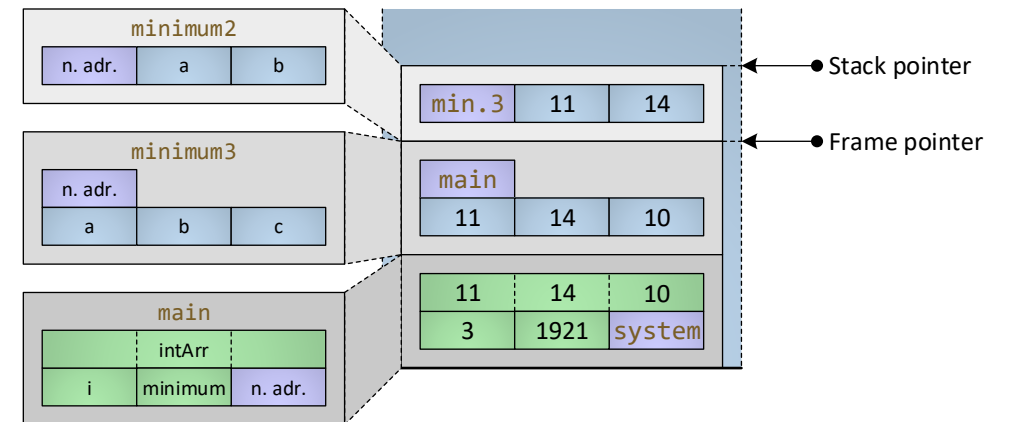
- Pomocou zásobníka si funkcie vymieňajú údaje (argumenty a návratové hodnoty).
- **Stack pointer** = vrchol zásobníka – tu aktuálne zásobník končí.
- Zásobník sa delí na **rámce** (1 rámec = 1 funkčné volanie). Rámec (angl. „frame“) obsahuje pamäť pre:
 - lokálne premenné definované vo funkcii,
 - parametre funkcie,
 - návratovú adresu (adresa inštrukcie, ktorá nasleduje po ukončení funkcie),
 - návratovú hodnotu (ak sa vracia pomocou zásobníka).
- **Frame pointer** = adresa začiatku rámca aktuálne vykonávanej funkcie.
- Pamäť pre rámec je automaticky uvoľnená po skončení funkcie.
- Ak dôjde k vyčerpaniu pamäte vyhradenej pre zásobník, tak dochádza k chybe označovanej ako **pretečenie zásobníka** (angl. „stack overflow“).



Zásobník – Call stack

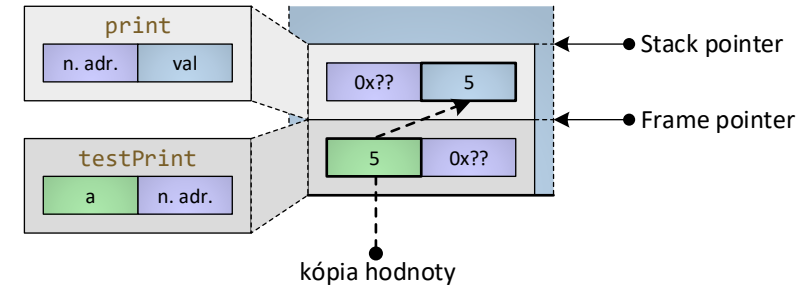
Ukážka jednoduchého programu v jazyku C++

```
1.  int minimum2(int a, int b) {  
2.    int min = a < b ? a : b;  
3.    return min;  
4.  }  
5.  
6.  int minimum3(int a, int b, int c) {  
7.    return minimum2(minimum2(a, b), c);  
8.  }  
9.  
10. int main() {  
11.   int intArr[3] = { 0 };  
12.   std::cout << "Zadajte 3 celé čísla" << std::endl;  
13.   for (int i = 0; i < 3; i++) {  
14.     std::cin >> intArr[i];  
15.   }  
16.   int minimum = minimum3(intArr[0], intArr[1], intArr[2]);  
17.   std::cout << "Minimum je " << minimum << std::endl;  
18.   return 0;  
19. }
```



Zásobník – Odovzdávanie parametrov hodnotou

- **Hodnota** parametra je **skopírovaná** na stack.
- Argument môže byť čokoľvek (literál, premenná, trieda..).
- Nikdy sa nezmení hodnota pôvodného argumentu.
- Pozor na odovzdávanie veľkých typov (triedy, polia) hodnotou!
- *pass by value*

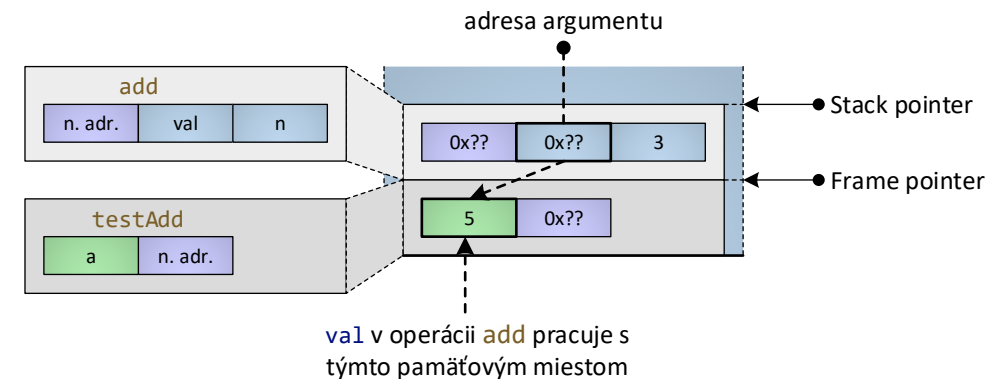


Ukážka časti programu v jazyku C++ využívajúceho odovzdávanie parametrov hodnotou

```
1. void print(int val) {  
2.     std::cout << val << std::endl;  
3. }  
4.  
5. void testPrint() {  
6.     int a = 5;  
7.     print(a);  
8.     print(5);  
9.     print(a + 5);  
10. }
```


Zásobník – Odovzdávanie parametrov odkazom

- Na stack je **skopírovaná adresa** parametra (rovnako ako v predchádzajúcom prípade).
- Pre prístup k hodnote však **nie je** parameter **potrebné dereferencovať!**
- Akákoľvek zmena parametra spôsobí zmenu argumentu (je to to isté pamäťové miesto - pozor na nechcené zmeny!).
- Nemusí sa kopírovať potenciálne veľká štruktúra – rýchle.
- Modifikáciám sa dá zabrániť cez `const`.
- *pass by reference*



Ukážka časti programu v jazyku C++ využívajúceho odovzdávanie parametrov odkazom

```
1. void add(int& val, int n) {  
2.     val += n;  
3. }  
4.  
5. void testAdd() {  
6.     int a = 5;  
7.     add(a, 3);  
8.     print(a); //8  
9. }
```

Zásobník – Návratová hodnota funkcie

- Funguje rovnako ako odovzdávanie parametrov, len je otočený tok dát.
- Môže byť odovzdaná hodnotou, odkazom aj adresou.
- Pozor na platnosť lokálnych premenných! **Nikdy nevracajte odkazom ani adresou premenné, ktoré sú lokálne** – pamäť po skončení funkcie nie je platná.
- Návratový typ adresou alebo odkazom používajte hlavne pre dynamicky alokovanú pamäť.

Halda

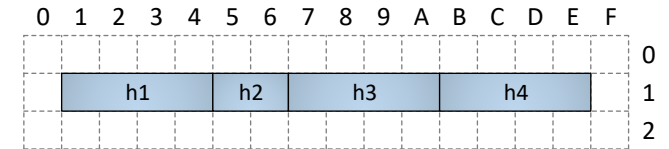
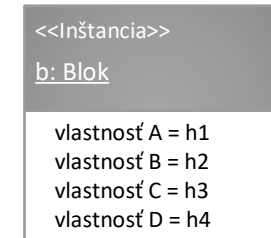
- Kopa („heap“) pamäte – doslova – jeden z najväčších segmentov pamäte.
- Obsahuje dynamicky alokované premenné, ktoré sú dostupné medzi funkčnými volaniami.
- Životný cyklus premenných riadi programátor:
 - **vznik**: v C++ `malloc`, `calloc`, `realloc`, `new` (pre objektové typy volá aj konštruktor) – výsledkom je ukazovateľ do haldy, kde objekt vznikol.
 - **zánik**: v C++ `free`, `delete` (pre objektové typy volá aj deštruktor) – ako parameter požadujú ukazovateľ na objekt, ktorý zaniká.
- Ukazovateľ, ktorý obsahuje adresu bloku pamäte vráteného operačnému systému, sa označuje ako tzv. **visiaci ukazovateľ** (angl. „dangling pointer“).
- Pre korektnosť a prípadné vyhnutie sa chybám počas behu programu by mal byť visiaci ukazovateľ vždy nastavený na hodnotu NULL alebo `nullptr`.

Halda

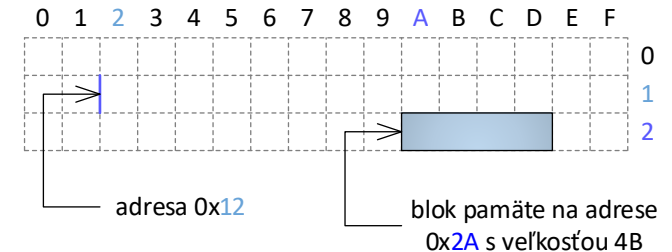
- Aby sa dalo prístupit' na pamäť v halde, musí existovať niekde v zásobníku príslušná premenná, pomocou ktorej sa dá dohľadať!
- Ak nejakým spôsobom prideme o adresu alokovaného bloku pamäte (napr. prepísaním adresy v referenčnej premennej pôvodne uchováajúcej predmetný alokovaný blok), už sa k nemu nikdy nedokážeme dostať, a teda ho nedokážeme ani uvoľniť. Vznikne **únik pamäte** („memory leak“).
- Ak by sa nám takýto problém v programe vyskytoval viackrát, mohlo by to viesť k postupnému vyčerpaniu voľnej pamäte nachádzajúcej sa v halde a následnému pádu bežiaceho programu.
- Problémom spojených s únikom pamäte sa dá predchádzať:
 - automatická správa pamäte (garbage collector).
 - inteligentný ukazovateľ (smart pointer).

Správca pamäte

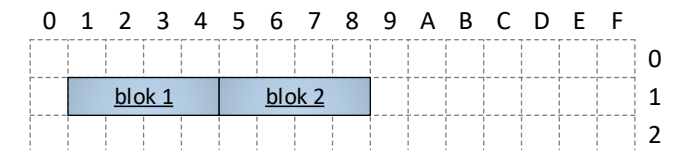
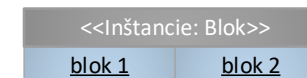
- Správca pamäte (SP) môže spravovať operačnú alebo externú pamäť.
- Za sebou idúce bajty v pamäti tvoria **kompaktnú pamäť** – z pohľadu SP nie je podstatné, čo v nej je zakódované.
- Kompaktnú pamäť môžeme pomyselne deliť na rovnako veľké **bloky pamäte**, v ktorých sú zakódované údaje s nejakým významom (a).
- Informáciu o polohe bloku pamäte v pamäti (adresa) ukladáme do referenčnej premennej (b).
- Spravované bloky pamäte sa môžu nachádzať v **súvislej** pamäti alebo v **nesúvislej** pamäti – tvoria adresný priestor (c).



a)



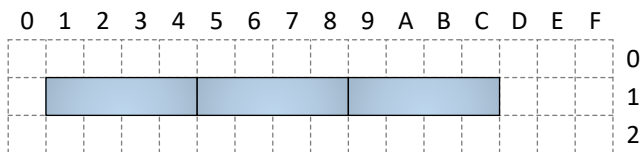
b)



c)

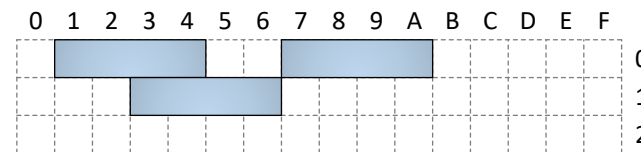
Adresný priestor

Lineárny



- Alokované bloky tvoria súvislú oblasť pamäte.
- Začína na **bázovej adrese**.
- **Referencia** môže byť číslo vyjadrujúce vzdialenosť bloku pamäte od bázovej adresy.
- Referencia je relatívna voči bázovej adrese.
- Efektívne pridelenie, kopírovanie a rušenie pamäte.
- ÚŠ je možné efektívne implementovať v operačnej a externej pamäti.

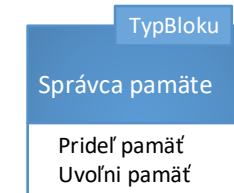
Nelineárny



- Alokované bloky netvoria súvislú oblasť pamäte.
- **Referencia** je adresa, ktorá je absolútna v celom adresnom priestore.

Správca pamäte

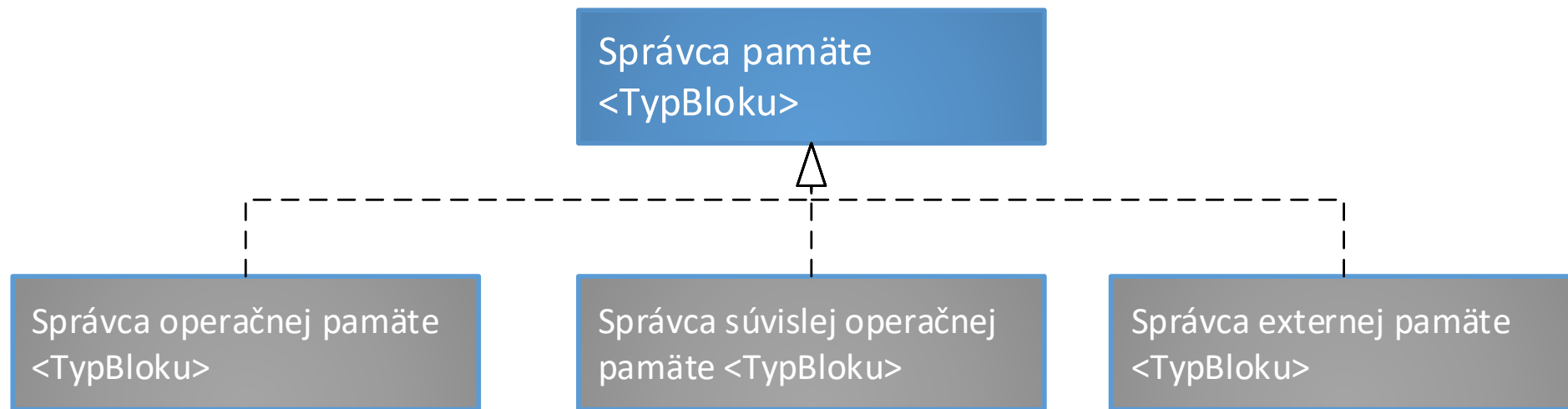
- Zodpovedný za pridelenie a vrátenie pamäte.
- Udržiava záznamy o polohe a veľkosti voľných segmentov pamäte.



Operácie poskytované správcom pamäte			
Názov operácie	Parametre	Návratová hodnota	Význam
Prideľ pamäť	int	↑TypBlok	Vyhradí pamäť pre bloky pamäte (ich počet je odovzdaný ako parameter) a vráti referenciu na prvý blok.
Uvoľni pamäť	↑TypBlok	-	Uvoľní bloky pamäte umiestnené v pamäti na odovzdanej adrese.

Príklady správcov pamäte

- **Správca operačnej pamäte:** operačná pamäť je alokovaná pomocou zvolenej stratégie (first fit, best fit, worst fit), uvoľniť je možné akýkoľvek blok, môže dochádzať k fragmentácii.
- **Správca súvislej operačnej pamäte:** operačná pamäť je postupne alokovaná, uvoľniť je možné naposledy alokovaný blok, nedochádza k fragmentácii.
- **Správca súvislej externej pamäte:** externá pamäť je postupne alokovaná, uvoľniť je možné naposledy alokovaný blok, nedochádza k fragmentácii.



Pridelenie pamäte

- SP pridelí pamäť a vráti jej počiatočnú fyzickú adresu (adresu prvého bajtu prvého bloku pamäte).
- Alokované bloky pamäte sa nachádzajú za sebou a tvoria kompaktnú pamäť.
- Na nájdenie vhodného voľného segmentu pamäte, kde alokuje kompaktnú pamäť využíva rôzne techniky:
 - **prvý vhodný** – správca pamäte nájde prvý vhodný segment, ktorý je dostatočne veľký pre alokáciu blokov pamäte. Ak po alokácii blokov pamäte ostane v segmente dostatok voľného miesta, je segment rozdelený a jeho voľnú časť bude možné použiť na ďalšiu alokáciu bloku pamäte. V opačnom prípade bude ostávajúca časť segmentu pamäte nevyužitá. S narastajúcim množstvom objektov, ktoré spôsobia nevyužívanie časti segmentov pamäte, stúpajú nároky na pamäť.
 - **najlepší vhodný** – správca pamäte nájde segment pamäte, ktorý sa svojou veľkosťou najviac približuje k požadovanej veľkosti alokovaných blokov pamäte. V tomto prípade je nevyužitá časť segmentu pamäte zväčša veľmi malá (alebo žiadna) a delenie segmentu, ako v prípade stratégie prvý vhodný, nie je efektívne.
 - **najhorší vhodný** – správca pamäte udržiava záznamy o voľných segmentoch pamäte zoradené od najväčších po najmenšie. Alokácia nového vždy prebieha do najväčšieho bloku pamäte, ktorý nie je potrebné špeciálne vyhľadávať. Alokácia pamäte je veľmi rýchla, avšak dochádza k jej fragmentácii a problémom popísaných pri stratégii prvý vhodný.

Inicializácia pamäte – neobjektové typy

- Uvažujme pamäť (a).
- Po pridelení môže byť nová pamäť neinicializovaná (b) alebo inicializovaná (c) na východiskové hodnoty.
- V prípade inicializácie je alokácia pomalšia, avšak pamäť môže byť hneď použitá na čítanie.

a)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
	41	135	14	247	66	63	28	254	179	176	75	211	83	183	191	254	0
	174	237	129	7	239	99	12	251	35	111	208	160	1	158	793	78	1
	103	212	43	140	14	151	110	166	255	216	135	24	40	98	0	4	2

b)

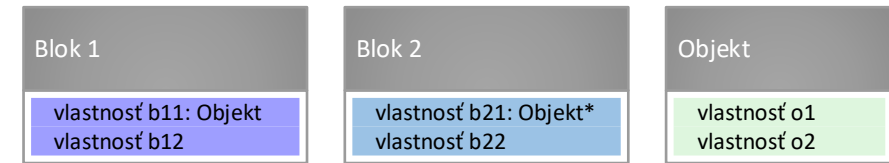
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
	41	135	14	247	66	63	28	254	179	176	75	211	83	183	191	254	0
	174	237	129	7	239	99	12	251	35	111	208	160	1	158	793	78	1
	103	212	43	140	14	151	110	166	255	216	135	24	40	98	0	4	2

c)

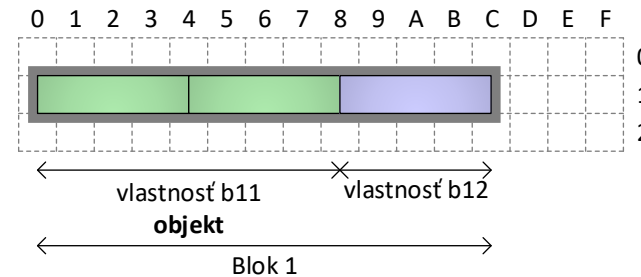
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
	41	135	14	247	66	63	28	254	179	176	75	211	83	183	191	254	0
	0	0	0	0	0	0	0	0	0	0	0	0	1	158	793	78	1
	103	212	43	140	14	151	110	166	255	216	135	24	40	98	0	4	2

Inicializácia pamäte – objektové typy

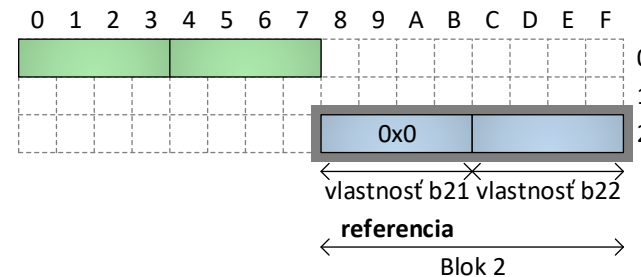
- Ak sú bloky pamäte **objektami**, **musí** byť zabezpečené volanie **konštruktorov**!
- Bloky pamäte môžu obsahovať objekt alebo referenciu na objekt (a) – rozdielny prístup k inicializácii.
 - Objekt**: konštruktor musí byť volaný nad pamäťovým miestom v rámci bloku pamäte. (b)
 - Objekt***: referencia môže, ale nemusí byť špeciálne inicializovaná (buď vznikne objekt, ktorého adresu uchová alebo v nej ostane NULL). (c)



a)



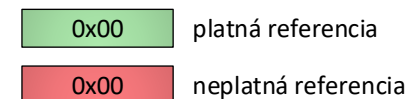
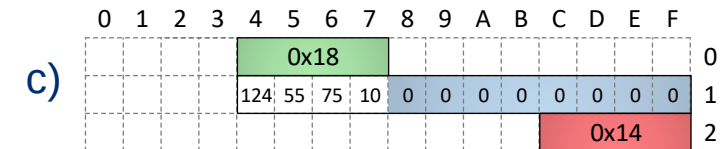
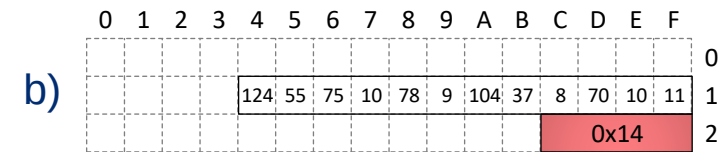
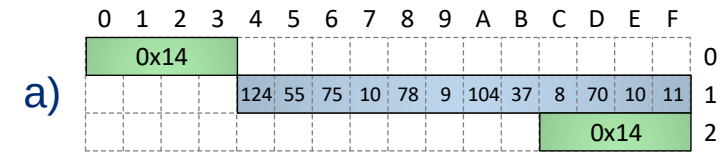
b)



c)

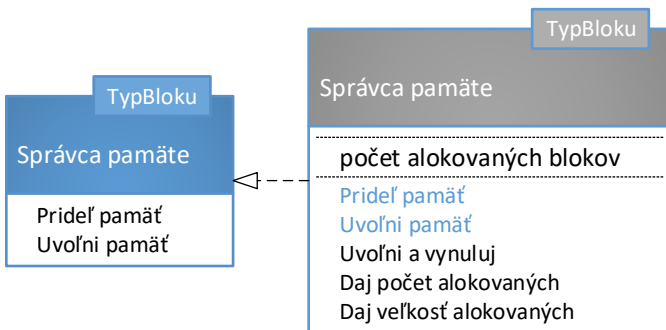
Uvoľnenie pamäte

- Koniec alokovanej pamäte si pamätá správca.
- Pozor na objektové typy – je potrebné volať **deštruktory** (inak by mohlo dôjsť k úniku pamäte).
- Pozor, ak na pamäte existuje viac referenčných premenných (v príklade na adresách 0x00 a 0x2C)! (a)
 - Ak sa uvoľní pamäť cez 0x00, tak 0x2C ukazuje na neplatnú pamäť (aj keď sa javí, že je platná). (b)
 - Ak sa prideli nová pamäť (od 0x18, uložená v 0x04), tak prístup cez 0x2C povedie k chybe! (c)



Správca pamäte

- Na pohodlnú prácu je možné definovať ďalšie operácie správcu pamäte.
- Univerzálny **správca** (nekompaktnej) operačnej **pamäte** môže byť navrhnutý nasledovne:



Názov operácie	Parametre	Návratová hodnota	Význam
Uvoľni a vynuluj	↑↑TypBoku	-	Uvoľní bloky pamäte umiestnené v pamäti na odovzdanej adrese a odovzdaný parameter nastaví na NULL (všimnite si odovzdanie referencie na referenciu).
Daj počet alokovaných	-	int	Vráti počet alokovaných blokov pamäte.
Daj veľkosť alokovaných	-	int	Vráti veľkosť alokovanej pamäte.

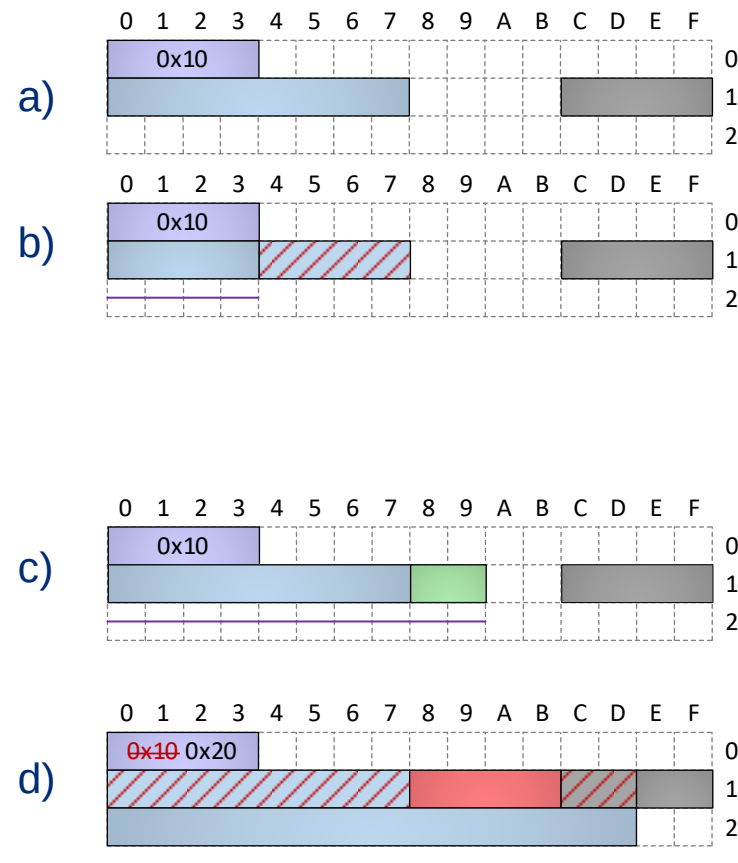
Operácie s kompaktnou pamäťou

- S kompaktnou pamäťou je možné efektívne vykonávať operácie z tabuľky vpravo.
- S ich využitím je možné navrhnuť **správca kompaktnej pamäte**
- Neskôr umožní efektívne implementovať údajové štruktúry, ak platí:
 - ukladané údaje sú homogénne,
 - nedochádza k častým modifikáciám štruktúry.

Názov operácie	Parametre	Návratová hodnota	Význam
Veľkosť pamäte	Referencia	int	Vráti veľkosť (počet bajtov) pamäte odovzdanej ako parameter.
	Údajový typ		
Zmeň veľkosť pamäte	Referencia int	Referencia	Zmení (zväčší alebo zmenší) veľkosť odovzdanej pamäte na novú hodnotu.
Skopíruj pamäť	Referencia Referencia int	-	Prekopíruje počet bajtov pamäte z jedného miesta (odovzdanom ako referencia) na iné. Nekontroluje prekrytie pamäte.
Premiestni pamäť	Referencia Referencia int	-	Prekopíruje počet bajtov pamäte z jedného miesta (odovzdanom ako referencia) na iné. Kontroluje prekrytie pamäte.
Porovnaj	Referencia Referencia int	Boolean	Vráti príznak, či je požadovaný počet bajtov od počiatočných referencií, odovzdaných ako parameter, rovnaký.
Prepíš	Referencia char int	-	Požadovaný počet krát skopíruje hodnotu jedného bajtu na od danej referencie.
Vymeň	Referencia Referencia int	-	Vymení obsah dvoch pamäťových miest.
	Údajový typ Údajový typ		

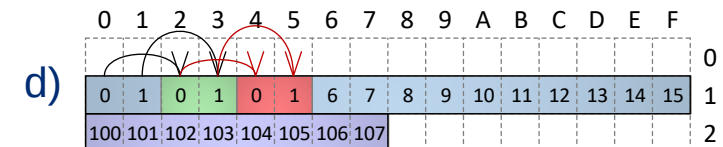
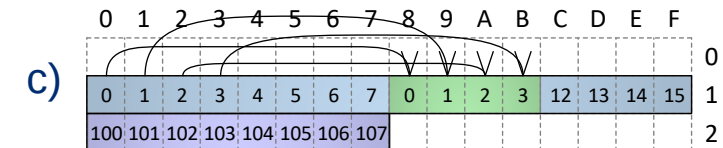
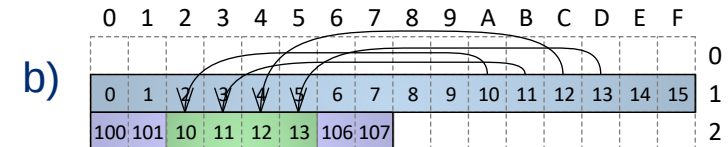
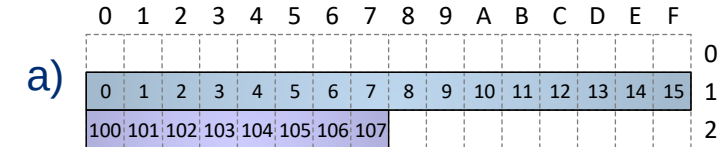
Operácia zmen veľkosť pamäte

- Operácia zmení veľkosť **pamäte**, ktorej **adresa** a **nová veľkosť** sú odovzdané ako parametre. Môžu nastať nasledujúce situácie
 - Ak je **nová veľkosť** menšia ako pôvodná veľkosť alokovanej pamäte, potom sú nadbytočné bajty uvoľnené (a→b).
 - Ak je **nová veľkosť** väčšia, ako pôvodná veľkosť alokovanej pamäte, tak je snaha prideliť dodatočnú pamäť bezprostredne za pôvodnou pamäťou, aby vznikla kompaktná pamäť.
 - Ak to je možné (**pridané bloky** nezasiahnu do iného alokovaného bloku), pamäť sa jednoducho vyhradí a pridá k existujúcej (a→c).
 - Ak to nie je možné (**pridané bloky** zasiahnu do iného alokovaného bloku), je potrebné alokovať **novú pamäť** s požadovanou veľkosťou, prekopírovať pôvodné údaje a následne pôvodnú pamäť uvoľniť (a→d).
- Keďže neviete predpokladať, ktorá situácia nastane, **vždy spracujte návratovú hodnotu!** (d)



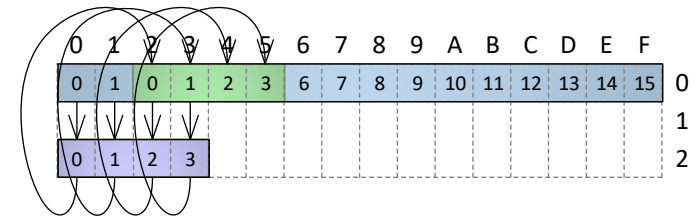
Operácia skopíruj pamäť

- Kvôli efektívnosti nevyužíva odkladaciu pamäť, nekontroluje prekrytie zdrojovej a cieľovej pamäte ani ich platnosť.
- Ak sa zdrojová a cieľová oblasť v pamäti prelína, môžu sa niektoré údaje stratiť.
- Uvažujme pamäť (a). Nasledujúce časti zobrazujú výsledok po volaní funkcie s parametrami:
 - b) (0x1A, 0x22, 4)
 - c) (0x10, 0x18, 4)
 - d) (0x10, 0x12, 4)



Operácia **premiestni** pamäť

- Obdobná, ako **skopíruj** Pamäť, ale využíva buffer.
- Ak sa zdrojová a cieľová oblasť v pamäti prelína, údaje sa nestratia.
- Na obrázku je zobrazené volanie tejto funkcie v problematickom prípade (d) z predchádzajúcej snímky.



Správca kompaktnej pamäte – výhody

1. **Nedochádza k fragmentácii pamäte**, je teda možné efektívne vytvárať jej kópie resp. priradovať pamäť.
2. S využitím operácií s kompaktnou pamäťou **je možné efektívne implementovať aj ďalšie operácie, ktoré budú pre údajové štruktúry typické** (napr. **prirad'**, **porovnaj**).
 - Budú rýchle, pretože je možné na pamäť nazerať nízkoúrovňovo ako na pole bajtov.
3. **Bloky pamäte je možné organizovať v externej pamäti, ktorá musí byť kompaktná.**
 - Ak je potrebné ukladať údaje v externej pamäti, potom údajová štruktúra spolupracuje so správcom externej kompaktnej pamäte.

Správca kompaktnej pamäte – obmedzenia

1. Alokácia blokov pamäte prebieha efektívne iba na konci kompaktnej pamäte.

- Pri alokácii pamäte bude potrebné zabezpečiť dostatok voľného miesta za pridelenou pamäťou (napr. s pomocou operácie **zmeňVeľkosťPamäte**).
- Pre alokáciu bloku pamäte v „strede“ kompaktnej pamäte, je potrebné najskôr posunúť už alokované bloky pamäte tak, aby vzniklo miesto v kompaktnej pamäti pre práve alokovaný blok.
- Takéto posunutie blokov pamäte má za následok zneplatnenie referencií na presunuté platné bloky pamäte a potrebu následnej opravy referencií.

2. Mazanie je možné efektívne vykonávať iba na konci kompaktnej pamäte.

- Pre mazanie blokov pamäte zo „stredy“ kompaktnej pamäte je potrebné nasledujúce bloky posunúť (aby nevznikla diera), čo má za následok zneplatnenie referencií na takto presunuté platné bloky pamäte a potrebu následnej opravy referencií, rovnako ako pri alokovaní bloku pamäte.

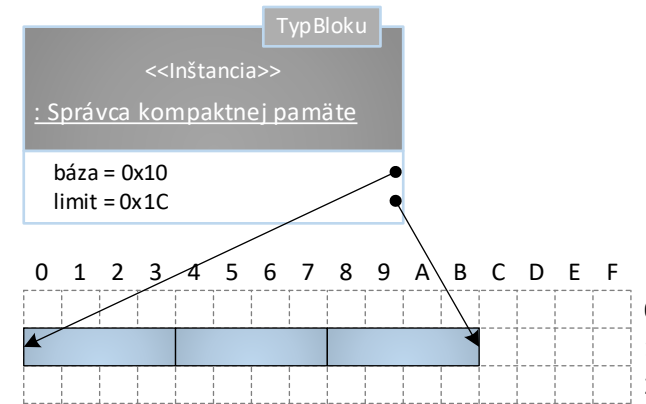
Oprava referencií po modifikácii v strede kompaktnnej pamäte

- Ak **údajová štruktúra neobsahuje referencie na bloky pamäte** a posunutím blokov pamäte teda nezneplatní referencie, **môže** správca kompaktnnej pamäte **povoliť alokovanie resp. odstránenie bloku pamäte zo stredu** a nasledujúce prvky posunúť operáciou **premiestniPamäť bez ďalších následkov**.
- Ak bude údajová štruktúra uchovávať referencie na bloky pamäte, tak pri mazaní typicky najskôr mazaný blok vymení s posledným blokom pamäte v kompaktnnej pamäti, kde ho následne nebude problém dealokovať. Keďže údajová štruktúra pozná ňou spravované bloky pamäte, dokáže po prípadnej výmene aktualizovať referencie na výmenou dotknuté bloky pamäte. Ak je potrebné blok pamäte alokovať, potom musí takto opraviť referencie na všetky bloky od práve alokovaného bloku až po posledný blok v kompaktnnej pamäti.
- Ak nie je možné vymeniť resp. inak posunúť bloky (napr. poloha bloku pamäte určuje jeho vzťahy s inými blokmi – viac sa budeme tejto téme venovať pri popise implicitných APS) a údajová štruktúra obsahuje referencie na bloky pamäte, potom nebude možné blok pamäte zo stredu kompaktnnej pamäte odstrániť a to, že je neplatný, bude potrebné poznačiť v správcovi pamäte. Alokácia bloku pamäte v strede kompaktnnej pamäte bude možná iba na pamäťovom mieste, ktoré je označené ako neplatné (práve nepoužívané).

Správca kompaktnej pamäte – návrh

- Musí si pamätať platnú pamäť (vytýčená *bázovou* a *limitnou* adresou).
- **Referencia** je číslo vyjadrujúce vzdialenosť bloku pamäte od *bázovej* adresy.
- Ďalšie operácie:

Názov operácie	Parametre	Návratová hodnota	Význam
Vypočítaj adresu	<code>int</code>	<code>↑TypBloku</code>	Vypočíta adresu bloku pamäte s daným poradím.
Vypočítaj poradie	<code>↑TypBloku</code>	<code>int</code>	Vypočíta poradie bloku pamäte v kompaktnej pamäti
Daj blok pamäte	<code>int</code>	<code>↑TypBloku</code>	Vráti blok pamäte s daným poradím
Vymeň	<code>int</code> <code>int</code>	-	Navzájom vymení bloky pamäte s daným poradím



Vypočítaj adresu

$$A(i) = A_{\text{báza}} + (i * d)$$

$$A_{\text{báza}} = \text{bázová adresa (báza)}$$

$$d = \text{veľkosť jedného bloku pamäte } (|TypBloku|)$$

Vypočítaj poradie

$$I(\text{blok}) = \frac{(A_{\text{blok}} - A_{\text{báza}})}{d}$$

$$A_{\text{blok}} = \text{adresa bloku pamäte}$$

Správca kompaktnej pamäte – pseudokódy základných operácií

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.vypočítajAdresu($\uparrow$ TypBloku):
adresa

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.vypočítajAdresu(  
    dáta:  $\uparrow$ TypBloku  
): adresa {  
2.   definuj premennú p:  $\uparrow$ TypBloku  $\leftarrow$  báza  
3.   Pokiaľ (p  $\neq$  koniec  $\wedge$  p  $\neq$  dajAdresu(dáta)) opakuj {  
4.     p++  
5.   }  
6.   Keď platí (p = koniec)  
       tak vráť NULL  
       inak vráť p  
7. }
```

Definícia operácie

SprávcaKompaktnejPamäte<TypBloku>.vypočítajPoradie($\uparrow$ TypBloku): int

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.vypočítajPoradie(  
    dáta:  $\uparrow$ TypBloku  
): int {  
2.   Keď platí (dajAdresu(dáta)  $\leq$  koniec  $\wedge$   
               dajAdresu(dáta)  $\geq$  báza)  
       tak vráť dajAdresu(dáta) - báza  
       inak vráť -1  
3. }
```

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.dajBlokPamäte(int):
 $\uparrow$ TypBloku

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.dajBlokPamäte(  
    index: int  
):  $\uparrow$ TypBloku {  
2.   Vráť (báza + index) $\downarrow$   
3. }
```

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.vymeň(int, int)

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.vymeň(  
    index1: int,  
    index2: int  
) {  
2.   vymeň(dajBlokPamäte(index1), dajBlokPamäte(index2))  
3. }
```


Správca kompaktnej pamäte – prirad' a porovnaj – pseudokódy

Definícia operácie

SprávcaKompaktnejPamäte<TypBloku>.prirad'($\uparrow$ SprávcaKompaktnejPamäte<TypBloku>): $\uparrow$ SprávcaKompaktnejPamäte<TypBloku>

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.prirad'(
    iný:  $\uparrow$ SprávcaKompaktnejPamäte<TypBloku>
):  $\uparrow$ SprávcaKompaktnejPamäte<TypBloku> {
2.   Ak (self  $\neq$  dajAdresu(iný)) potom {
3.     uvoľniPamäť(báza)
4.     početAlokovanýchBlokov  $\leftarrow$  iný $\rightarrow$ početAlokovanýchBlokov
5.     definuj premennú novýPrvý: adresa  $\leftarrow$ 
        zmenVeľkosťPamäte(báza, iný $\rightarrow$ dajVeľkosťAlokovaných())
6.     Ak (novýPrvý = NULL) potom {
7.       CHYBA
8.     }
9.     báza  $\leftarrow$  novýPrvý
10.    koniec  $\leftarrow$  báza + početAlokovanýchBlokov
11.    limit  $\leftarrow$  báza + (iný $\rightarrow$ limit - iný $\rightarrow$ báza)
12.    skopírujPamäť(báza, iný $\rightarrow$ báza, iný $\rightarrow$ dajVeľkosťAlokovaných())
13.  }
14.  Vráť self $\downarrow$ 
15. }
```

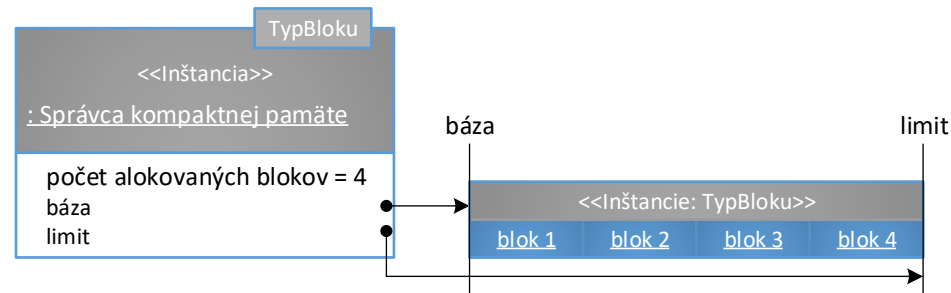
Definícia operácie *SprávcaKompaktnejPamäte<TypBloku>.porovnaj*
($\uparrow$ SprávcaKompaktnejPamäte<TypBloku>): bool

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.porovnaj(
    iný:  $\uparrow$ SprávcaKompaktnejPamäte<TypBloku>
): bool {
2.   Vráť (self = dajAdresu(iný))  $\vee$ 
        (početAlokovanýchBlokov = iný $\rightarrow$ početAlokovanýchBlokov  $\wedge$ 
        porovnajPamäť(báza,
            iný $\rightarrow$ báza,
            početAlokovanýchBlokov * |TypBloku|) = 0)
3. }
```

Správca kompaktnej pamäte – vznik

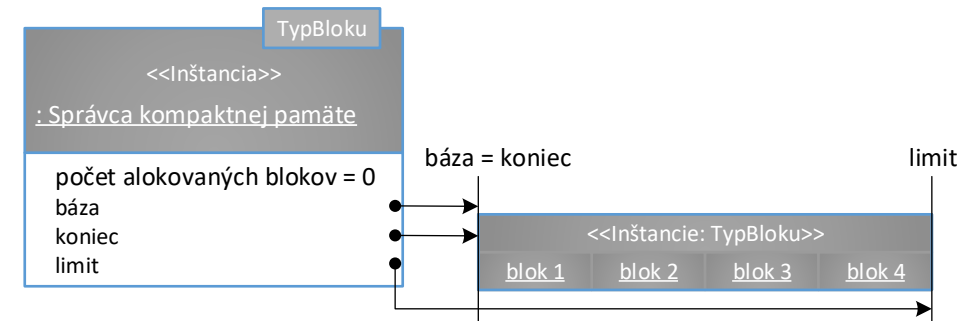
Statický

- Alokujú presne toľko pamäte, koľko bude počas životného cyklu štruktúry potrebné.
- Všetky bloky pamäte sú dostupné od začiatku.
- Nemožné alokovať/dealokovať bloky.



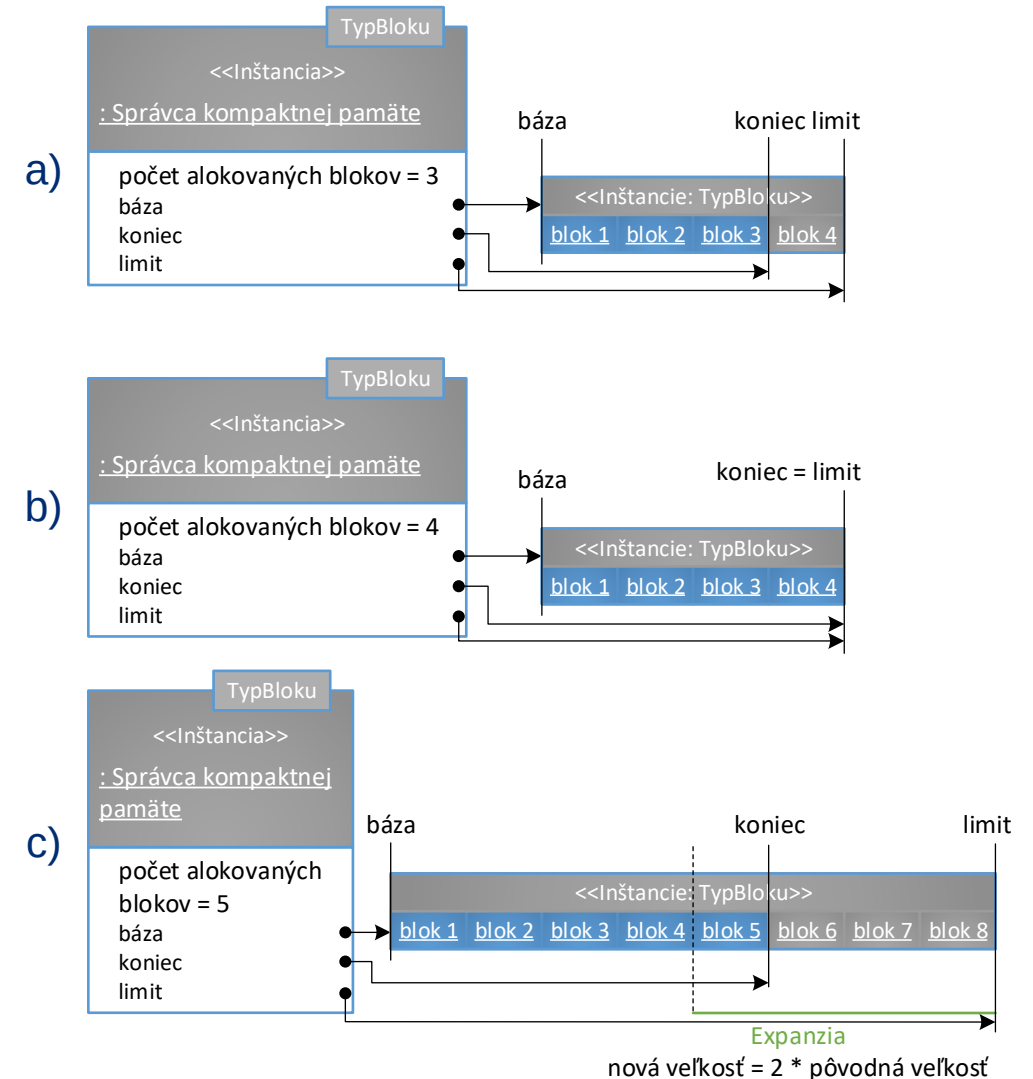
Dynamický (ďalej uvažujeme tohto SP)

- Pri vzniku typicky alokuje viac pamäte, ako je potrebné a uchováva **koniec** platnej pamäte – zvýšená pamäťová náročnosť.
- Bloky pamäte sú dostupné až po volaní operácie **pridel pamäť**.
- Alokujú bloky pamäte na požiadanie.



Správca kompaktnej pamäte – alokovanie blokov

- Alokácia za *koniec* (a).
- Ak pre ďalší blok pamäte nie je dostatok miesta (b), je potrebné využiť operáciu s kompaktnou pamäťou **zmeň veľkosť pamäte** a vyhradiť väčšiu kapacitu s novou veľkosťou a blok alokovať potom (c) – zapuzdrená v operácii **zmeň veľkosť**.
- **Expanzná stratégia:**
 - **konštantná**
 $\text{nová veľkosť} \leftarrow \text{pôvodná veľkosť} + 10$
 - **lineárna**
 $\text{nová veľkosť} \leftarrow \text{pôvodná veľkosť} * 2$
 - **geometrická**
 $\text{nová veľkosť} \leftarrow \text{pôvodná veľkosť}^2$
- Alokácia je možná aj do stredu – blok sa alokuje na konci a následne sa bloky od vkladaneho miesta posunú – veľmi časovo aj pamäťovo náročná operácia!



Správca kompaktnej pamäte – alokovanie blokov – pseudokódy

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.prideIPamät(): ↑TypBloku

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.prideIPamät(  
    ): ↑TypBloku {  
2.   Vráť prideIPamätNa(koniec - báza)  
3. }
```

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.prideIPamätNa(int):
↑TypBloku

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.prideIPamätNa(  
    index: int  
    ): ↑TypBloku {  
2.   Ak (koniec = limit) potom {  
3.     zmeňVeľkosť(2 * početAlokovanýchBlokov)  
4.   }  
5.   Ak (koniec - báza > index) potom {  
6.     presuňPamät(báza + index + 1,  
        báza + index,  
        ((koniec - báza) - index) * |TypBloku|)  
7.   }  
8.   početAlokovanýchBlokov++  
9.   koniec++  
10.  Vráť vytvor TypBloku() na (báza + index)  
11. }
```

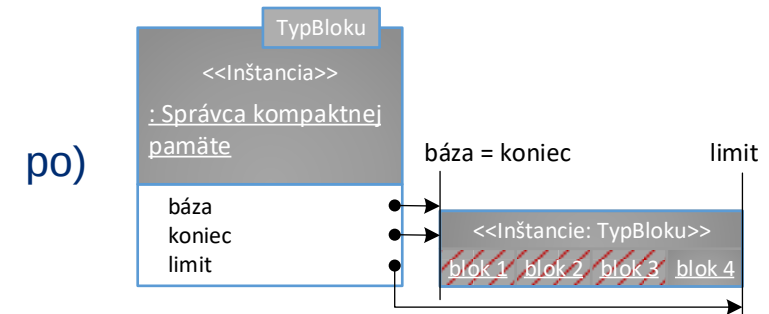
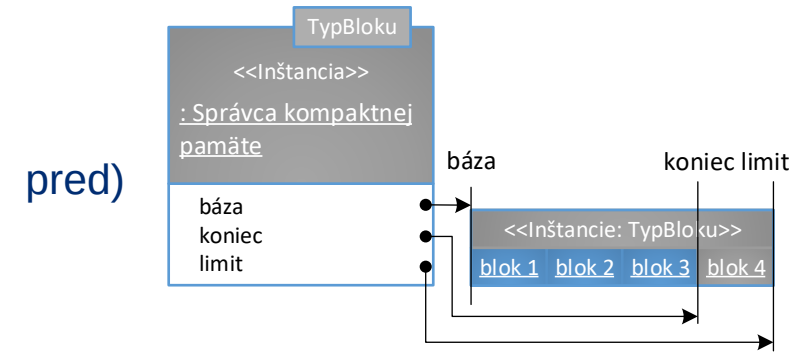
Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.zmeňVeľkosť(int)

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.zmeňVeľkosť(  
    nováVeľkosť: int  
    ) {  
2.   definuj premennú novýPrvý: adresa ←  
        zmeňVeľkosťPamäte(báza, nováVeľkosť * |TypBloku|)  
3.   Ak (novýPrvý = NULL) potom {  
4.     CHYBA  
5.   }  
6.   báza ← novýPrvý  
7.   koniec ← báza + početAlokovanýchBlokov  
8.   limit ← báza + nováVeľkosť  
9. }
```



Správca kompaktnej pamäte – mazanie blokov

- Operáciu je možné využiť v deštruktore.
- Pamäť sa automaticky nezmenšuje (expanzia je drahá, nechceme ju prípadne robiť opäť).
 - je možné pridať operáciu (napr. **zmenši pamäť**), ktorá nepotrebnú pamäť vráti.
 - explicitne volaná v správnej situácii programátorom.
 - nezabudnite zabezpečiť minimálnu kapacitu, aby po zmenšení pamäte fungovala expanzná stratégia.
- Pozor na životné cykly objektových typov v blokoch pamäte!
- Mazanie je možné aj zo stredu – bloky od konca sa posunú o 1 až po vkladané miesto – veľmi časovo aj pamäťovo náročná operácia!



Správca kompaktnej pamäte – mazanie blokov – pseudokódy

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.uvoľniPamäť()

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.uvoľniPamäť() {  
2.   uvoľniPamäť(koniec - 1)  
3. }
```

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.uvoľniPamäťNa(int)

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.uvoľniPamäťNa(  
   index: int  
) {  
2.   presuňPamäť(báza + index,  
                 báza + index + 1,  
                 ((koniec - báza) - index - 1) * |TypBloku|)  
3.   uvoľniPamäť()  
4. }
```

Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.uvoľniPamäť(↑TypBloku)

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.uvoľniPamäť(  
   ukazovateľ: ↑TypBloku  
) {  
2.   Ak (ukazovateľ < báza v ukazovateľ > koniec) potom {  
3.     CHYBA  
4.   }  
5.   definuj premennú p: ↑TypBloku ← ukazovateľ  
6.   Pokiaľ (p ≠ koniec) opakuj {  
7.     zruš p  
8.     p++  
9.   }  
10.  koniec ← ukazovateľ  
11.  početAlokovanýchBlokov ← koniec - báza  
12. }
```

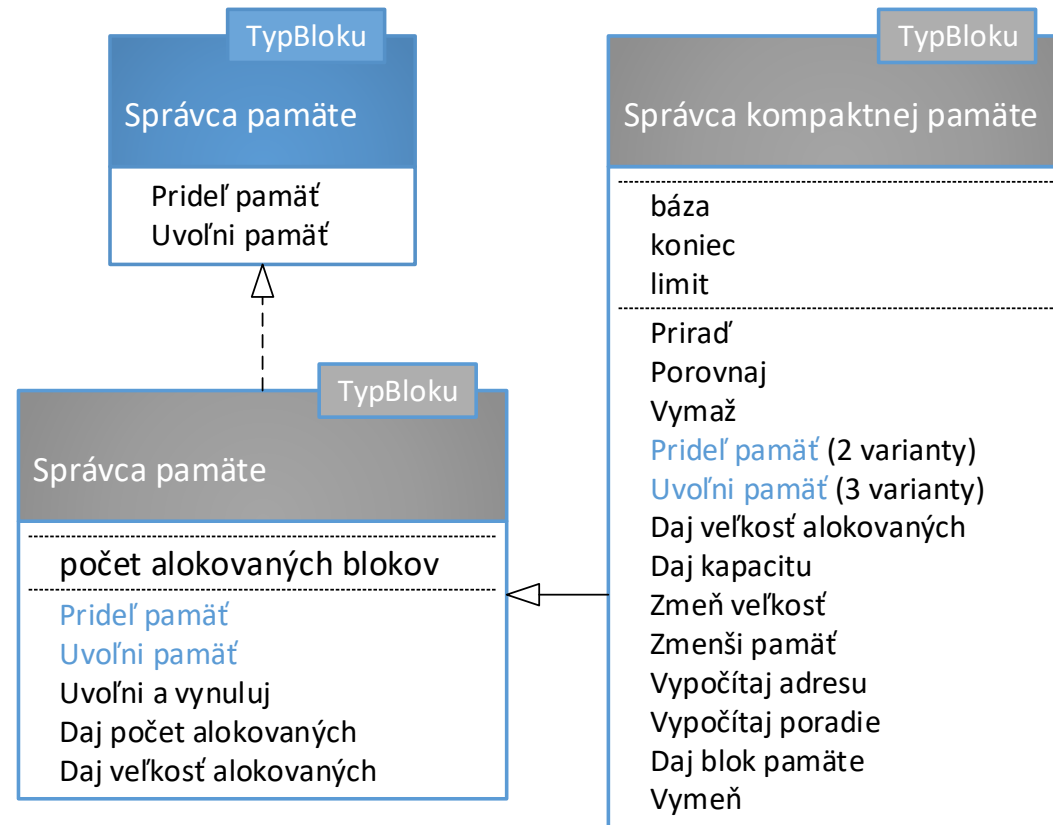
Definícia operácie *SprávcaKompaktnejPamäte*<TypBloku>.zmenšiPamäť()

```
1. operácia SprávcaKompaktnejPamäte<TypBloku>.zmenšiPamäť() {  
2.   definuj premennú nováVeľkosť: int ← koniec - báza  
3.   Ak (nováVeľkosť < VEĽKOSŤ_INIT) potom {  
4.     nováVeľkosť ← VEĽKOSŤ_INIT  
5.   }  
6.   zmenšVeľkosť(nováVeľkosť)  
7. }
```

Definícia deštruktora *SprávcaKompaktnejPamäte*<TypBloku>()

```
1. deštruktör SprávcaKompaktnejPamäte<TypBloku>() {  
2.   uvoľniPamäť(báza)  
3.   vráťPamäť(báza)  
4.   báza ← NULL  
5.   koniec ← NULL  
6.   limit ← NULL  
7.   finalizuj predka SprávcaPamäte<TypBloku>  
8. }
```

Diagram správcov pamäte



Zhrnutie prednášky

- Správa pamäte na úrovni operačného systému
 - Kódový segment
 - (Ne)Inicializovaný údajový segment
 - Zásobník
 - Halda
- Správcovia pamäte
 - Správca pamäte
 - Operácie s kompaktnou pamäťou
 - Správca kompaktnej pamäte

Plán na cvičenie

- Správca pamäte.
- Aritmetika ukazovateľov.
 - Rozdiel medzi ukazovateľom a odkazom.
- Správca kompaktnej pamäte.
 - Riadenie životného cyklu objektov.

Ďakujem za pozornosť

Doplňujúce informácie?
Vaše otázky?

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu 5UI124 Algoritmy a údajové štruktúry 1 na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline.
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené.

Ing. Michal Varga, PhD.
Katedra informatiky
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
Michal.Varga@fri.uniza.sk

Algoritmy a údajové štruktúry 1

Prednáška č. 3

- Abstraktné pamäťové typy
 - Prehliadky APT
 - Iterátory APT



Dôležité otázky z 1. prednášky

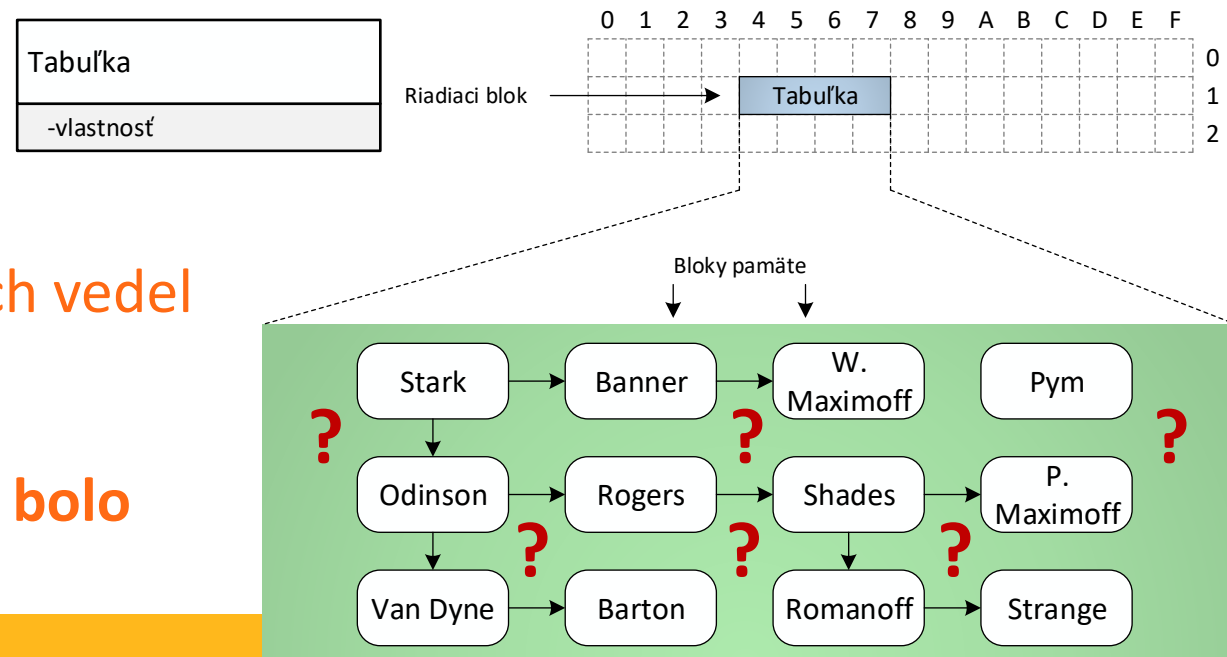
Ako mám implementovať tabuľku osôb, v ktorej ich budem môcť efektívne vyhľadávať?

Tabuľka osôb				
ID	Titul	Meno	Stredné meno	Priezvisko
1		Tony	Edward	Stark
2		Thor		Odinson
3		Janet		Van Dyne
4	Dr.	Henry	Jonathan	Pym
5	Dr.	Robert	Bruce	Banner
6		Steven	Grant	Rogers
7		Clint	Francis	Barton
8		Pietro		Maximoff
9		Wanda		Maximoff
10		Victor		Shade
11		Natasha	Alianova	Romanoff
12	Dr.	Stephen		Strange

Ako implementovať údajovú štruktúru, ktorá umožní efektívne spracovávať údaje v danej situácii?

Aké sú vzťahy medzi údajmi v štruktúre?

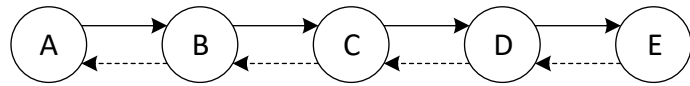
- Údajová štruktúra pozostáva z 2 typov **blokov**:
 - **Riadiaci blok** – objekt, ktorý reprezentuje samotnú údajovú štruktúru, v ktorom sú uložené jej vlastnosti (spravidla 1, typicky objekt).
 - **Blok pamäte**: ukladá údaje spravované údajovou štruktúrou (spravidla mnoho, môže to byť objekt, záznam alebo štandardný typ).



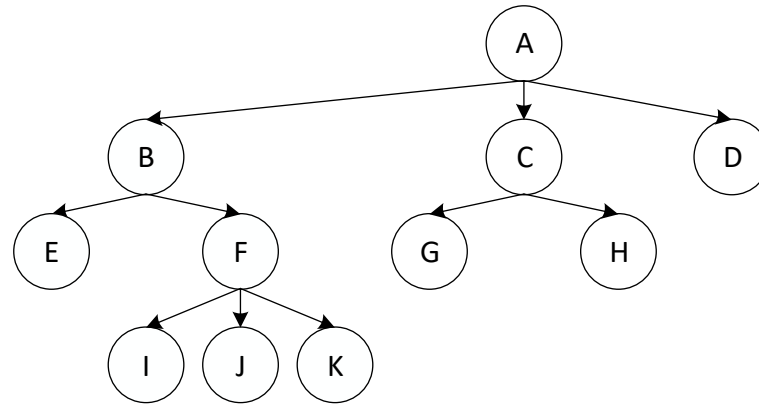
Ako mám organizovať osoby, aby som ich vedel efektívne vyhľadať v tabuľke?

Ako organizovať bloky pamäte, aby ich bolo možné efektívne spracovať?

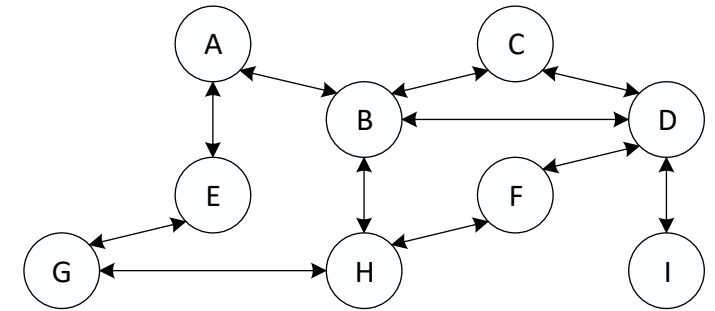
Logické usporiadanie údajových štruktúr



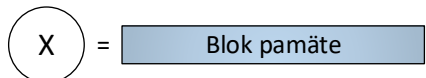
Sekvencia



Hierarchia



Sieť



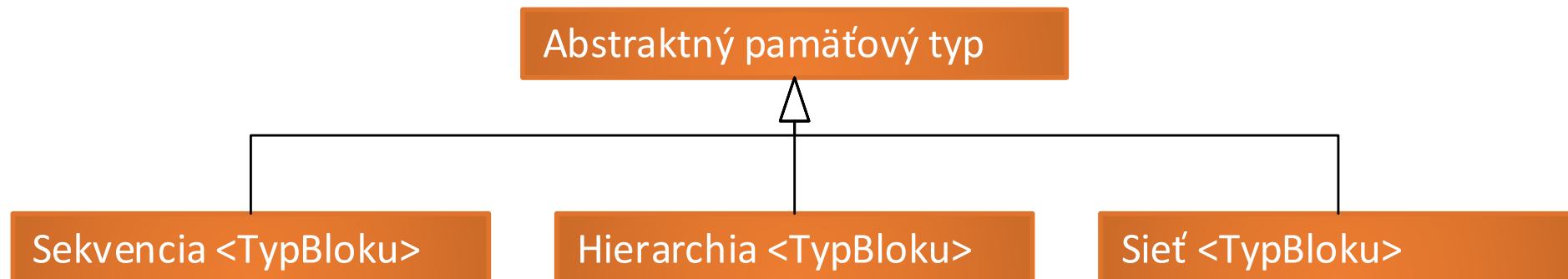
→ = Vzťah medzi blokmi pamäte

Abstraktný pamäťový typ

- Na všetky údajové štruktúry, ktoré **organizujú bloky pamäte rovnakým spôsobom** (napr. Zoznam, Sekvenčný prioritný front, Sekvenčná tabuľka), sa môžeme pozerat' jednotne – definujeme **abstraktný pamäťový typ** (APT).
- APT určuje
 - ako údajová štruktúra **organizuje bloky pamäte v pamäti**.
 - operácie typické pre dané pamäťové usporiadanie.
 - požiadavky na **bloky pamäte**.
- Implementačne:
 - APT je typicky **rozhranie**.
 - Bloky pamäte je možné špecifikovať s využitím generík, šablón, konceptov..
 - Ten istý APT je možné realizovať viacerými spôsobmi s odlišnou organizáciou blokov pamäte, čím je možné vybrať najvhodnejšiu realizáciu pre konkrétnu situáciu.

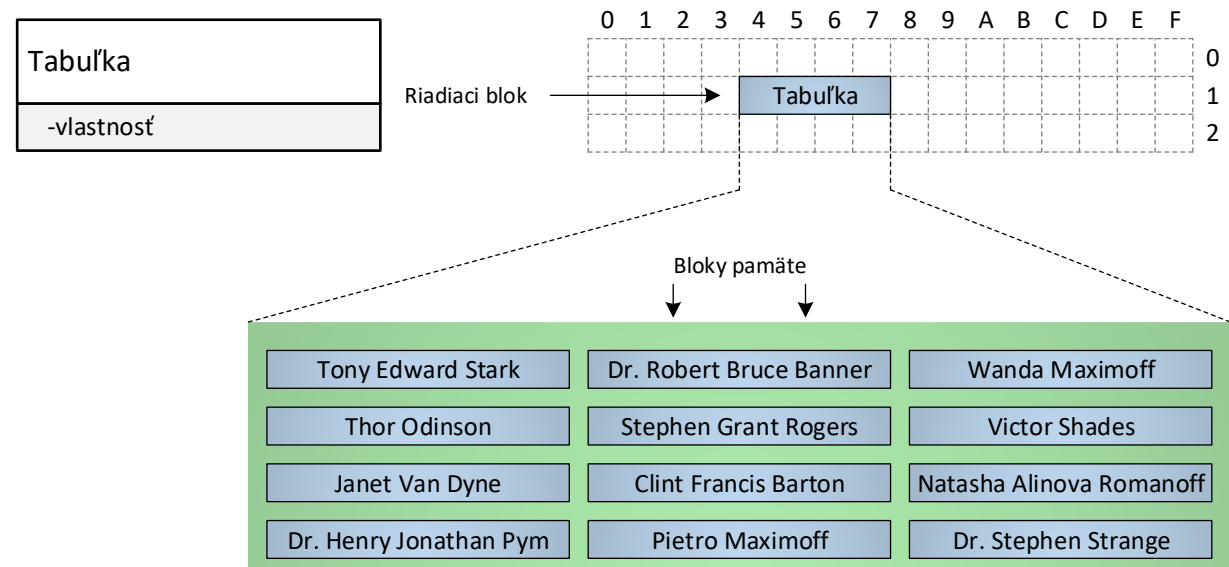
Abstraktný pamäťový typ

- **Sekvencia:** blok pamäte má najviac jedného predchodcu a najviac jedného nasledovníka – vzťah **1:1**.
- **Hierarchia:** blok pamäte má najviac jedného predchodcu (budeme ho nazývať otec) a N nasledovníkov (budeme ich nazývať synovia) – vzťah **1:N**.
- **Sieť:** blok pamäte eviduje vzťahy k maximálne n-1 iným prvkom – potenciálne môže mať teda každý blok pamäte v sieti vzťahy so všetkými ostatnými blokmi pamäte v sieti – vzťah **N:N**.



Ako sú fyzicky uložené údaje?

- Bloky pamäte sú uložené v pamäti:
 - Podľa umiestnenia:
 - Operačná
 - Externá
 - Podľa súvislosti
 - Súvislá
 - Nesúvislá



Ako a kde mám alokovať bloky pamäte s osobami aby tvorili sekvenciu?

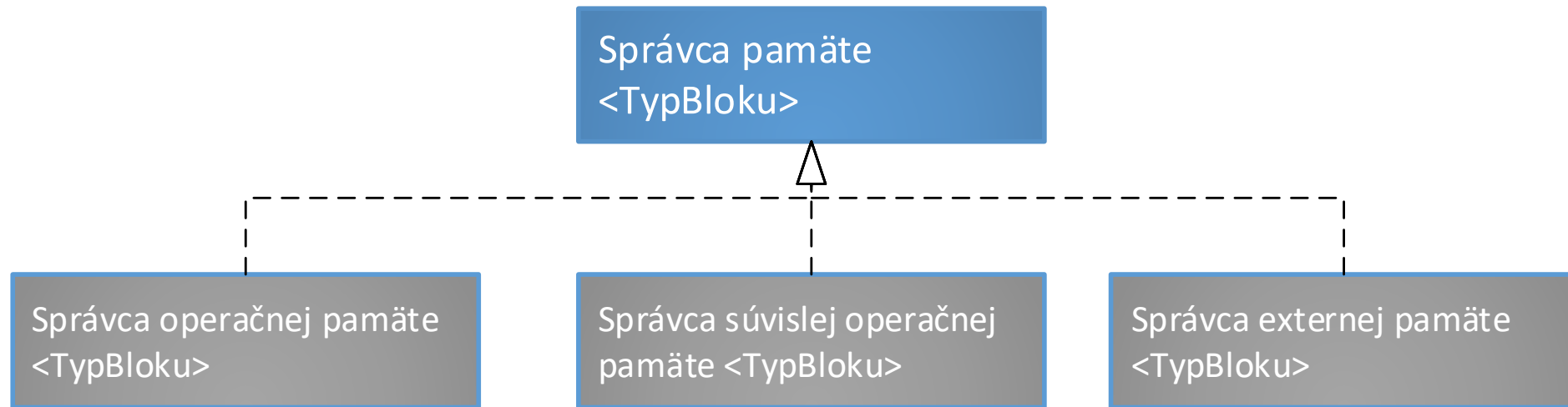
V akej pamäti efektívne alokovať bloky pamäte s údajmi aby tvorili logické usporiadanie?

Správca pamäte

- Všetky pamäťové štruktúry, ktoré **ukladajú bloky pamäte do rovnakého typu pamäte** (napr. Sekvencia v súvislej pamäti, Hierarchia v súvislej pamäti), budú požadovať pri pridelovaní pamäte špecifické správanie (v našom príklade súvislosť pamäte) – definujeme **správca pamäte** (SP).
- SP určuje:
 - Spôsob **pridelovania a uvoľňovania pamäte**.
- Implementačne:
 - SP je typicky **rozhranie**.
 - Nerozlišuje bloky pamäte.
 - Rovnakého správcu pamäte (napr. správcu súvislej operačnej pamäte) je teda možné využívať pre realizáciu rôznych APT.

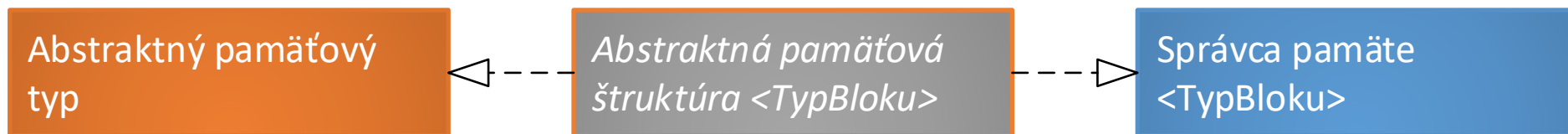
Príklady správcov pamäte (opakovanie)

- **Správca operačnej pamäte:** operačná pamäť je alokovaná pomocou zvolenej stratégie (first fit, best fit, worst fit), uvoľniť je možné akýkoľvek blok, môže dochádzať k fragmentácii.
- **Správca súvislej operačnej pamäte:** operačná pamäť je postupne alokovaná, uvoľniť je možné naposledy alokovaný blok, nedochádza k fragmentácii.
- **Správca súvislej externej pamäte:** externá pamäť je postupne alokovaná, uvoľniť je možné naposledy alokovaný blok, nedochádza k fragmentácii.

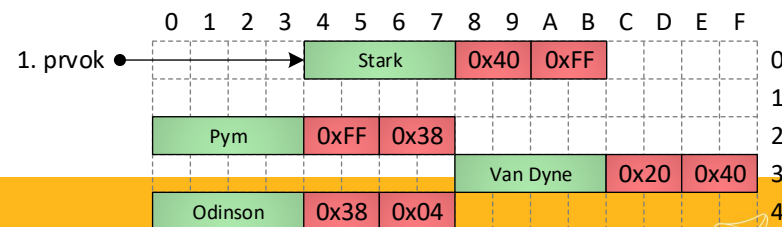
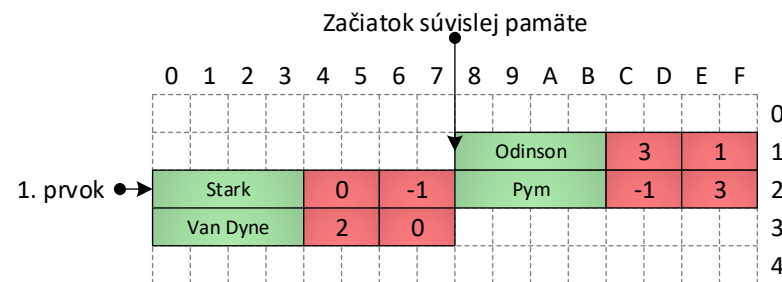
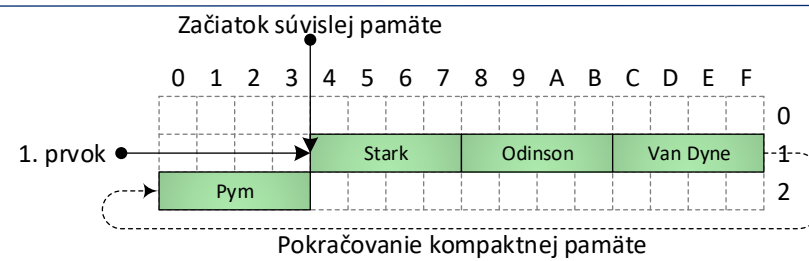
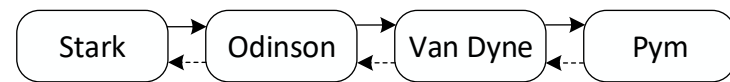
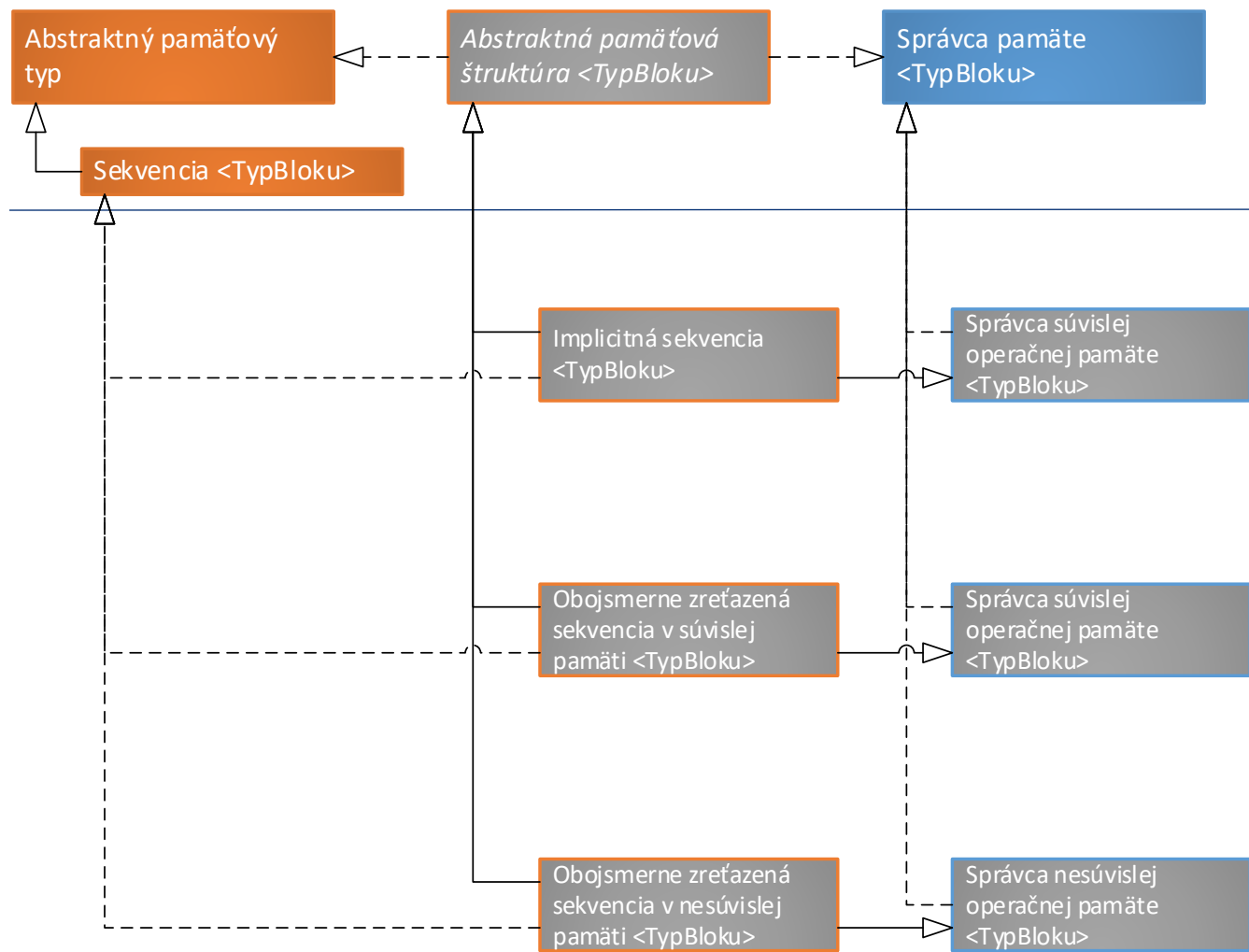


Abstraktná pamäťová štruktúra

- Abstraktná pamäťová štruktúra (APS) je realizácia APT aj SP:
 - APT: aké sú vzťahy medzi údajmi?
 - SP: v akej pamäti sú údaje uložené?
- Ani APT ani SP nešpecifikujú ani neobmedzujú bloky pamäte - **znalosť o konkrétnych blokoch pamäte, ktoré spracúva, má APS.**



Abstraktný pamäťový typ a štruktúra

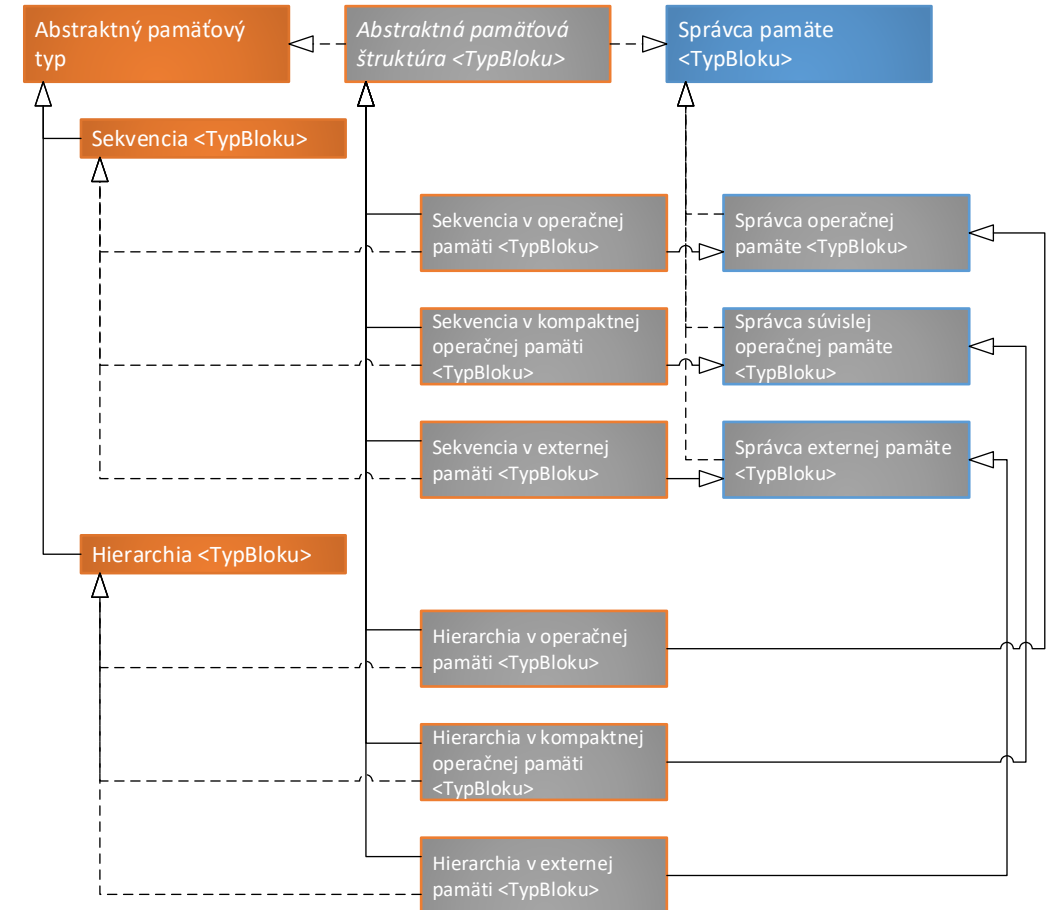


Logická úroveň

Fyzická úroveň

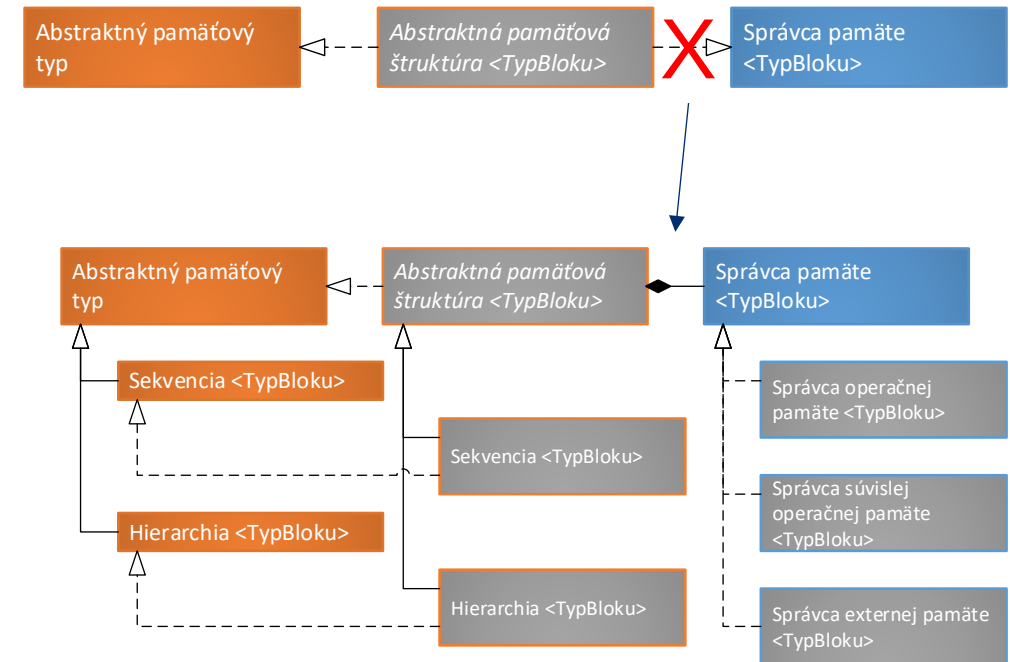
Problém návrhu – kombinatorická explózia tried

- Využitie dedičnosti (realizácie) limituje znovupoužiteľnosť tried.
- Príklad na obrázku vpravo: ak chceme pokryť všetky kombinácie APT (2 – sekvencia a hierarchia) a SP (3 – správca operačnej, súvislej operačnej a externej pamäte) musíme vytvoriť $3 \times 2 = 6$ tried APS.
- Pridanie ďalšieho APT spôsobí nutnosť definovania toľkých ďalších APS, koľko je SP (a opačne – pridanie SP spôsobí nutnosť definovania toľkých ďalších APS, koľko je APT) = **kombinatorická explózia!**



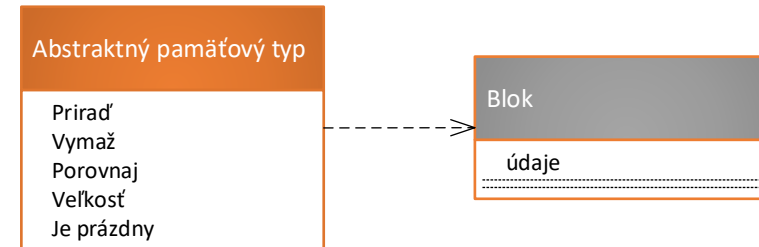
Riešenie problému - kompozícia

- Realizáciu SP v APS môžeme zameniť za **kompozíciu** – APS nebude rozhranie realizovať priamo, ale bude sa **skladať** z objektu, ktorý ho realizuje.
- Realizácie jednotlivých APT budú existovať iba raz, jednotlivé realizácie SP budú existovať tiež iba raz – spoja sa pri vzniku APS.

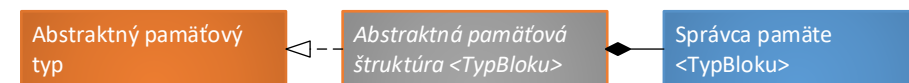


Abstraktný pamäťový typ

- Spravuje bloky pamäte (BP), každý BP má:
 - Údajovú časť
 - Vzťahovú časť
- Je realizovaný **Abstraktnými pamäťovými štruktúrami (APS)**.
 - Implicitné (vzťahová časť = 0B)
 - Explicitné (vzťahová časť > 0B)
- Je potrebné zabezpečiť, aby údajová časť blokov pamäte mohla ukladať akékoľvek údaje. Je možné využiť 2 prístupy:
 - Dedičnosť
 - Kompozícia

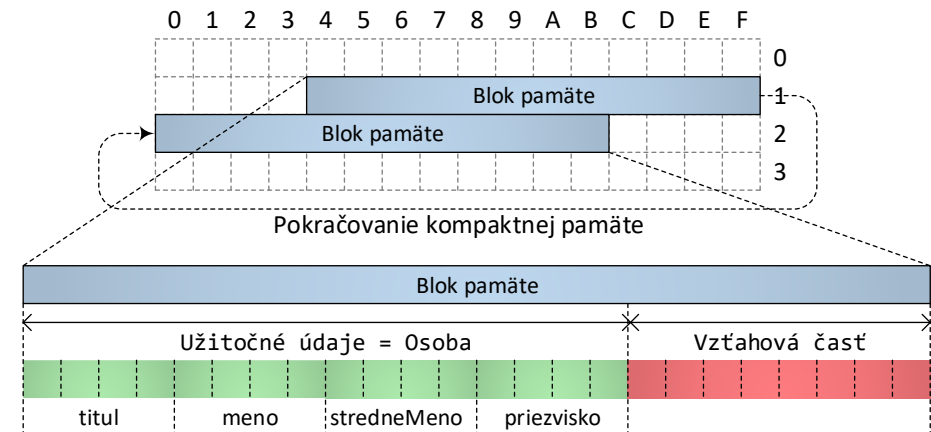
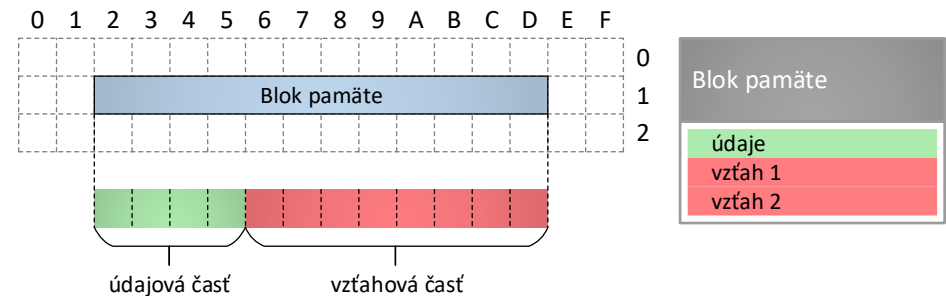


Základné operácie abstraktných pamäťových typov			
Názov operácie	Parametre	Návratová hodnota	Význam
Priradiť	↑Abstraktný PamäťovýTyp	↑Abstraktný PamäťovýTyp	Priradí prvky z iného APT.
Vymazať	-	-	Vymaže všetky prvky z APT.
Dotazy			
Názov dotazu	Parametre	Návratová hodnota	Dotaz vráti
Porovnať	↑Abstraktný PamäťovýTyp	bool	Príznak, či majú APT rovnaký obsah.
Veľkosť	-	int	Aktuálny počet prvkov v APT.
Je prázdny	-	bool	Príznak, či sa v APT nenachádzajú žiadne prvky.



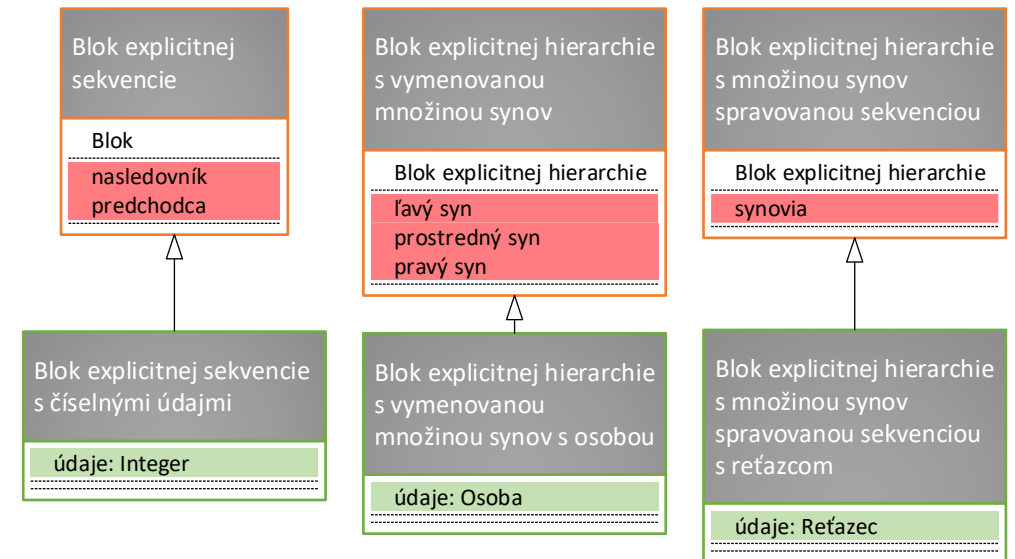
Bloky pamäte na fyzickej úrovni

- V bloku pamäte môžeme rozlišovať 2 časti:
 - **Údajová** (informačná) časť:
 - (užitočné) údaje, ktoré chce používateľ ukladať v údajovej štruktúre.
 - z pohľadu ÚŠ nedôležité.
 - **Vzťahová** časť:
 - údaje pre zabezpečenie prístupu k ďalším blokom pamäte.
 - túto časť je potrebné minimalizovať.



Definovanie údajovej časti blokov pamäte pomocou dedičnosti

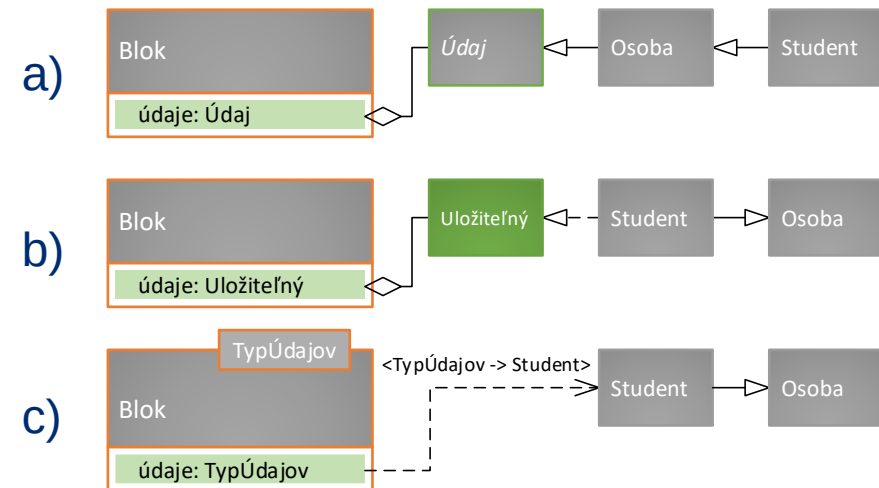
- Každý prvok ÚŠ bude potomkom bloku pamäte, v ktorom je definovaná iba údajová časť.
- Problém, ak jazyk nepodporuje viacnásobnú dedičnosť (nemôže byť zdedený z iného predka, ako z bloku pamäte)!
- Jeden prvok iba v jednej štruktúre (lebo má iba jednu vzťahovú časť).
- Veľká výhoda je prístup k iným blokom pamäte, ktoré sú vo vzťahu s aktuálnymi údajmi (lebo sú blokom) – pozor na zapúzdrenie.



Definovanie údajovej časti blokov pamäte pomocou kompozície

- Blok pamäte bude mať atribút (kompozícia), ktorý bude uchovávať údaje – **ako definovať typ atribútu?**

- a) **Predok údajov** – atribút bude typu **Údaj**.
- Všetky údaje budú potomkom napr. triedy **Údaj**.
 - Problémy s dedičnosťou.
 - Pre primitívne typy treba **obaľovacie triedy** (jednotlivé triedy majú jediný atribút obaľovaného primitívneho typu).
 - Potrebné pretypovanie pri používaní!
- b) **Rozhranie** – atribút bude typu **Uložiteľný**.
- Všetky údaje budú realizovať toto rozhranie.
 - Stačí, aby bolo **značkovacie** (bez metód).
 - Potrebné pretypovanie pri používaní!
- c) **Šablóna** (generikum) – atribút bude typu **T** (šablónového parametra).
- Nekladie žiadne obmedzenia.
 - Pohodlná práca (nie je potrebné pretypovanie).
 - Flexibilita v závislosti od programovacieho jazyka.



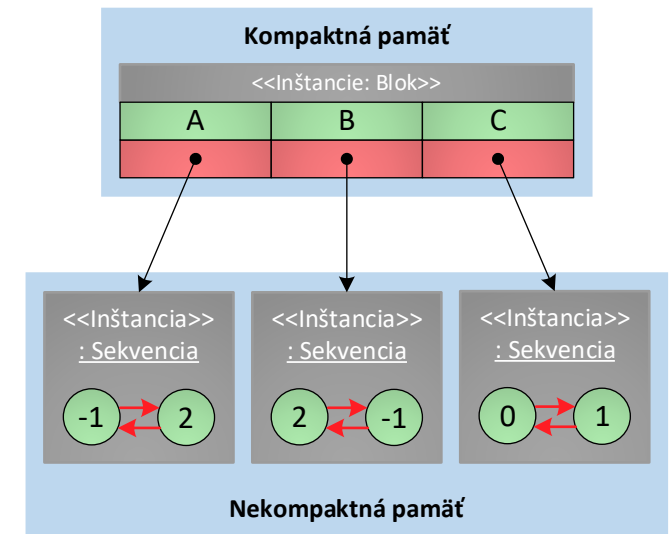
Definovanie **vzt'ahovej časti** blokov pamäte explicitných APS vymenovaním v bloku pamäte

- Počet vzt'ahov musí byť vopred známy.
- Po jednom alebo pomocou poľa (na mieste).
- Vzt'ahy sú uložené v pamäti „za sebou“.
- Vhodné pre operačnú aj externú pamäť.

Kompaktná pamäť		
<<Inštancie: Blok>>		
A	B	C
-1	2	0
2	-1	1

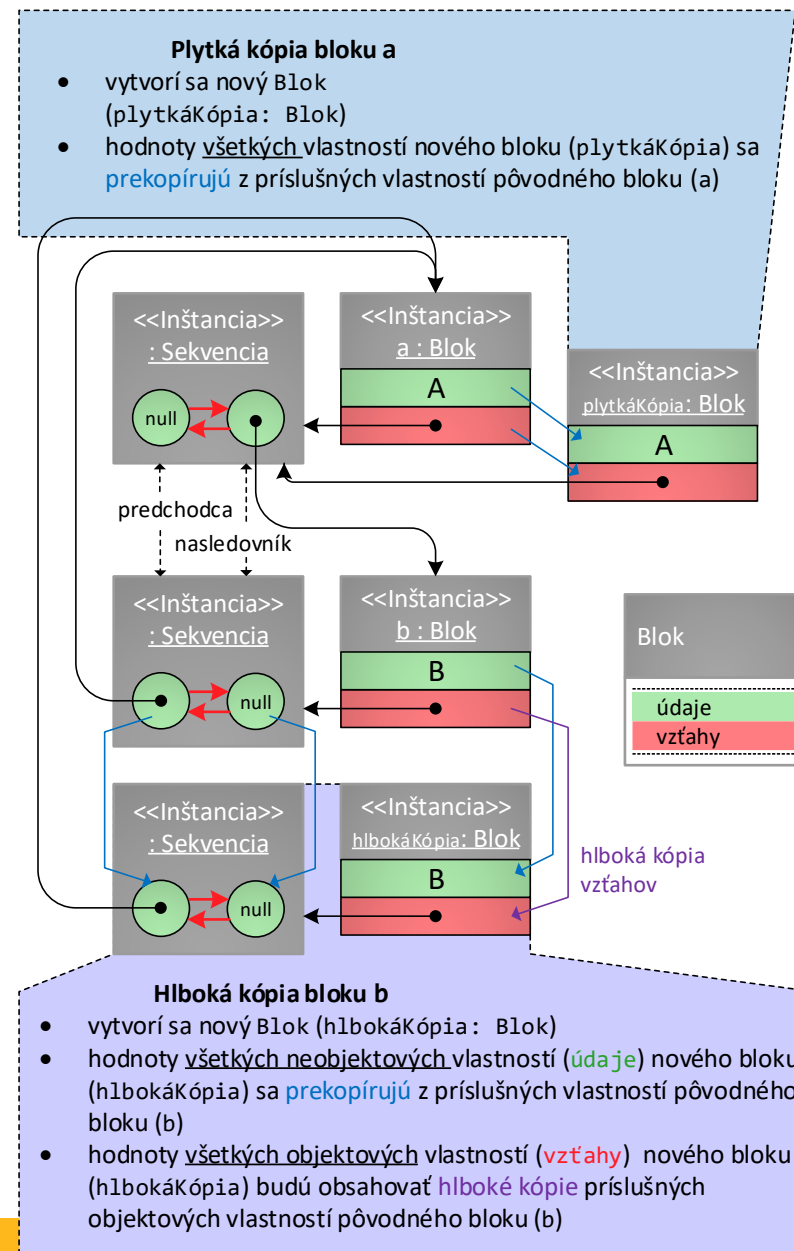
Definovanie **vzt'ahovej časti** blokov pamäte explicitných APS **uložením v inej** (typicky sekvenčnej) **APS**

- Počet vzťahov nie je vopred známy alebo sa môže dynamicky meniť.
- V bloku pamäte je uchovaná referencia na APS – je potrebné riadiť životný cyklus tejto APS.
- Vhodné iba pre operačnú pamäť.
- Univerzálnejší blok pamäte!
- **Ak sú vzťahy v blokoch pamäte uložené v inej APS a je využitý správca kompaktnej pamäte, tak nie je garantované, že pamäť celej štruktúry bude súvislá!**



Kopírovanie blokov pamäte

- Veľmi užitočná operácia, ktorú je potrebné správne implementovať.
- Jednoduché, ak sú bloky bez údajovej časti alebo ak je vzťahová časť vymenovaná.
- Ak je (správne) využitý **správca kompaktnej pamäte**, môže byť celá funkčnosť ponechaná na neho.
- Ak sú vzťahy v blokoch pamäte uchovávané v ďalšej APS, musí sa pri kopírovaní vytvoriť tiež kópia tejto APS, inak by dva bloky pamäte (originál a kópia) zdieľali tú istú množinu vzťahov.
- Rozlišujeme 2 kópie:
 - **Plytká** (shallow) **kópia**: objekt nevytvorí kópie objektov, z ktorých sa skladá (blok pamäte kópiu APS)
 - **Hlboká** (deep) **kópia**: objekt vytvorí kópie objektov, z ktorých sa skladá (a ak sa tie skladajú z ďalších objektov, tak sa princíp aplikuje opäť).



Implicitné APS

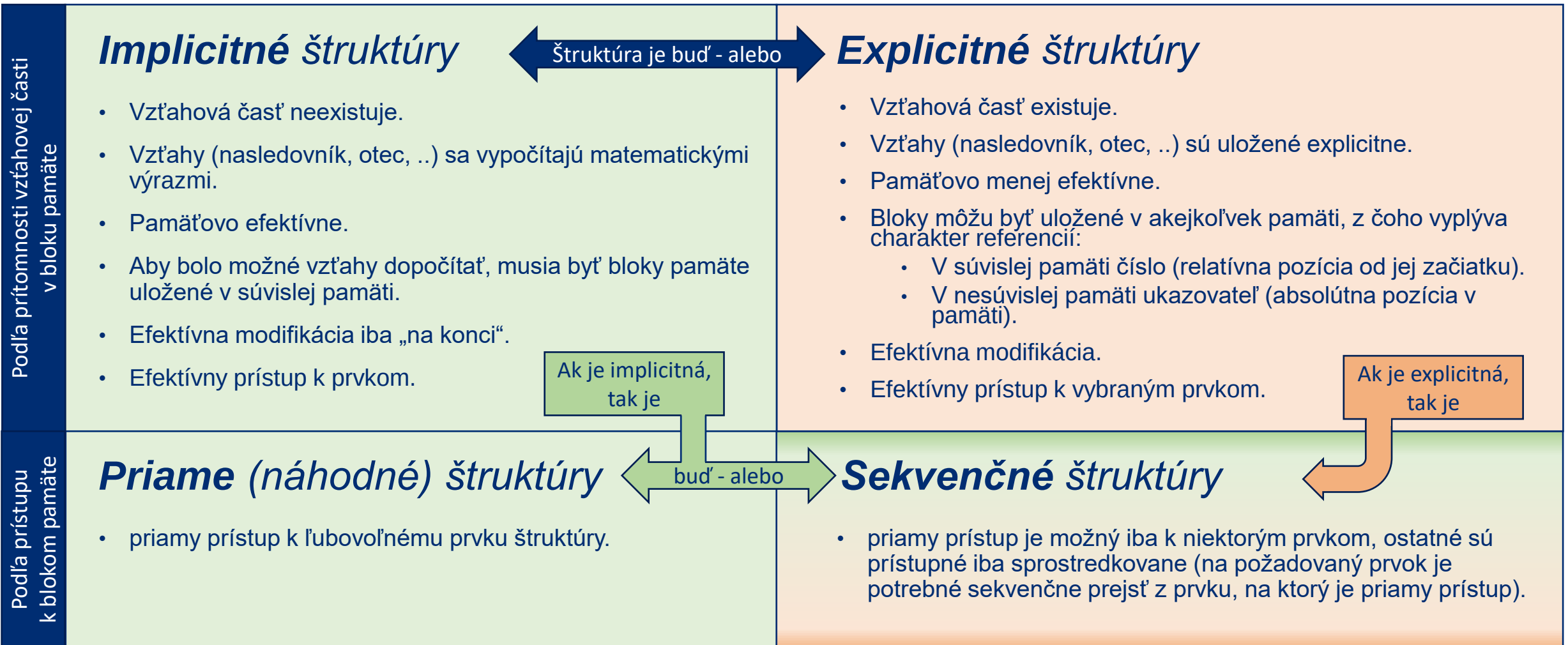
- Bloky pamäte nemajú vzťahovú časť!
- Na vyjadrenie vzťahov medzi blokmi pamäte definujú implicitné APS matematické výrazy.

Aby bolo možné definovať operácie vracajúce konkrétnu adresu bloku pamäte podľa požadovaného vzťahu, musia implicitné APS ukladať spravované bloky pamäte výhradne do kompaktnej pamäte.

Explicitné APS

- Bloky pamäte majú vzťahovú časť s explicitne vyjadrenými vzťahmi.
- Explicitná APS môže ukladať bloky pamäte do pamäte, ktorá môže byť:
 - **Súvislá**
 - referencia môže byť číslo, prázdna referencia je typicky -1.
 - Správca kompaktnej pamäte.
 - **Nesúvislá**
 - referencia musí byť adresa, prázdna referencia je NULL.
 - Správca pamäte.
- Rozdiel medzi explicitnými APS v súvislej a nesúvislej pamäti sa v operáciách prejavuje iba na úrovni modifikátorov (selektory a prehliadky sú v explicitných APS toho istého druhu implementované rovnako, líšia sa iba vo forme referencie, a teda aj prázdnej referencie).

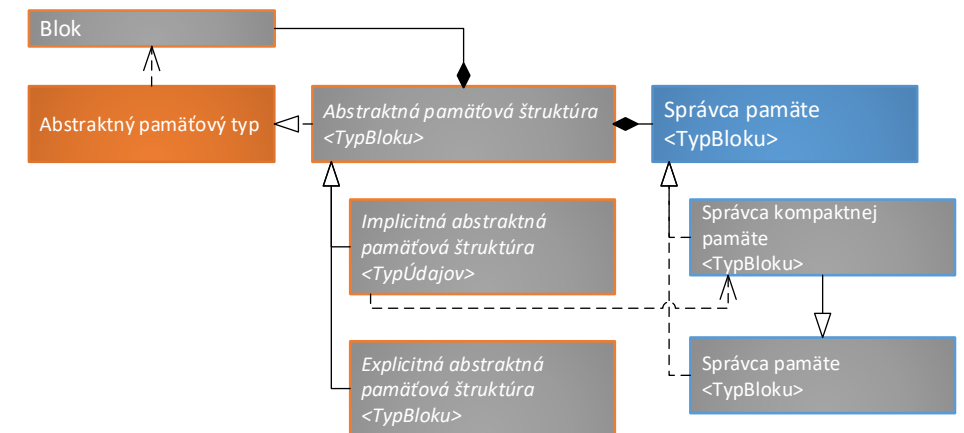
Organizácia blokov pamäte v pamäti



Implementácia univerzálnych APS

- Je možné definovať univerzálnych predkov akýchkoľvek APS.
- Operácie **prirad'**, **vymaž'**, **porovnaj**, **kapacita implicitných APS** je možné vykonať správcom kompaktnej pamäte.
- Ďalej budeme za explicitné APS považovať štruktúry v nekompaktnej pamäti (pokiaľ nebude priamo uvedené inak).

Charakteristika APS		Bloky pamäte sú uložené v	
		Súvislej pamäti	Nesúvislej pamäti
Vzťahová časť bloku pamäte	Neexistuje	Implicitná APS	Nie je možné
	Existuje	Explicitná APS v kompaktnej pamäti	Explicitná APS v nekompaktnej pamäti



Iterátory APS

Iterátor je objekt, ktorý zapúzdruje prechod údajovou štruktúrou a zároveň efektívne sprístupňuje jej prvky bez odhalenia jej implementačných detailov

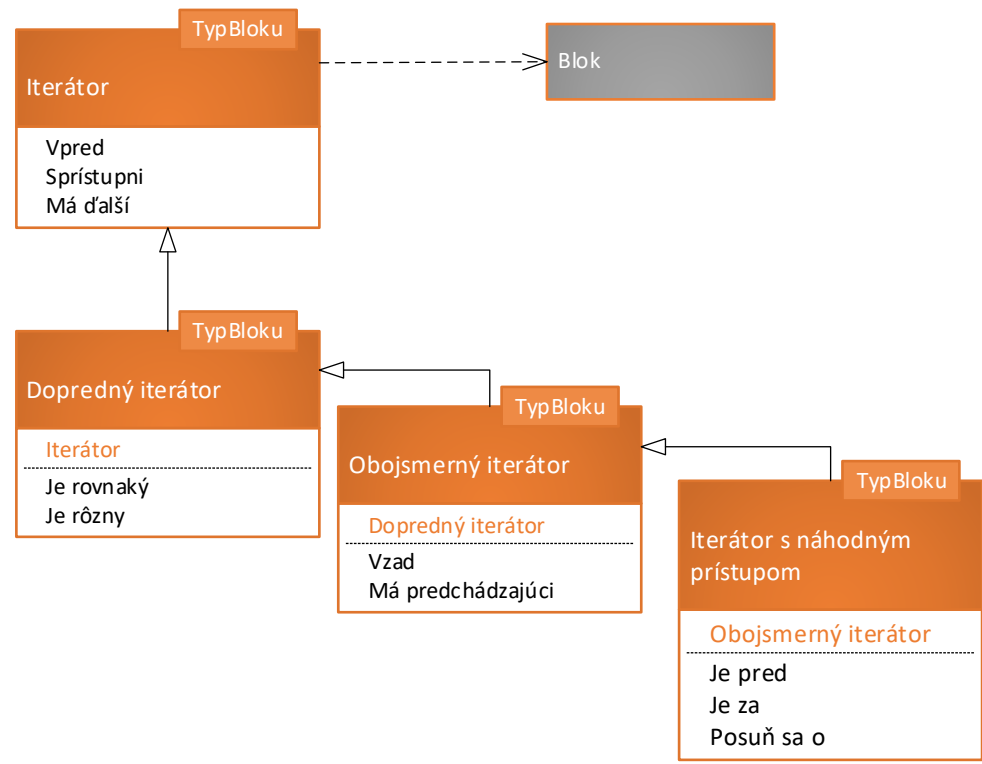
Iterátory APS – výhody

- Iterátor umožňuje sprístupňovať prvky bez toho, aby volajúci získal prístup k implementačným detailom štruktúry (teda, aká je organizácia pamäte, aké dodatočné informácie ukladá údajová štruktúra v blokoch pamäte, atď.). Tým sa stáva prechod komplikovanými štruktúrami pre volajúceho transparentný.
- Cez tú istú štruktúru je možné vykonávať viac prehliadok naraz (bez toho, aby jedna nutne skončila), keďže je iterátor objekt a uchováva si vlastný vnútorný stav.
- Je možné navrhnuť univerzálne algoritmy nezávislé na pamäťovej organizácii štruktúry tak, že budú tieto algoritmy založené na iterátoroch. Je teda možné využívať ten istý algoritmus bez zmeny jeho kódu v rôznych štruktúrach.
- Zavedením iterátorov do štruktúr sa získa jednotné rozhranie na prechod týchto štruktúr.
- zložitosti všetkých operácii sú väčša $O(1)$.

Iterátory APS – obmedzenia

- Štruktúra typicky nesmie počas prehliadky iterátormi podliehať zmenám (teda nesmú sa volať modifikátory štruktúry, pokiaľ existujú aktívne iterátory nad danou štruktúrou).
- Tým, že iterátor je objekt, zvýšia sa pamäťové (o veľkosť iterátora) a výpočtové (o dobu potrebnú na vytvorenie resp. zrušenie iterátora) zložitosti.

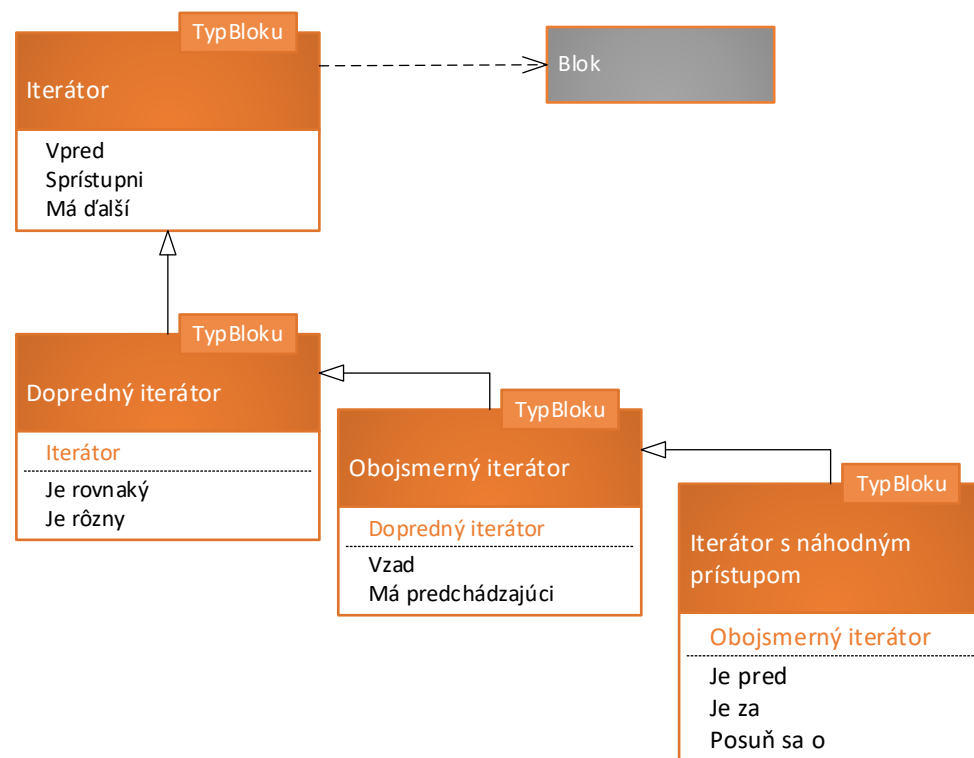
Iterátory APS



Všetky iterátory			
Názov operácie	Parametre	Návratová hodnota	Význam
Vpred	-	-	Efektívne posunie iterátor na nasledujúci prvok v štruktúre
Sprístupni	-	↑TypBloku	Sprístupní aktuálny prvok v štruktúre
Má ďalší	-	bool	Vráti príznak, či existuje nasledujúci prvok v štruktúre

Dopredné iterátory (navyš k operáciám všetkých iterátorov)			
Názov operácie	Parametre	Návratová hodnota	Význam
Je rovnaký	↑DoprednýIterátor <TypBloku>	bool	Vráti príznak, či ukazuje na rovnaký prvok štruktúry, ako iterátor z parametra
Je rôzny			Vráti príznak, či ukazuje na rozdielny prvok štruktúry, ako iterátor z parametra

Iterátory APS



Obojsmerné iterátory (navyš k operáciám dopredných iterátorov)			
Názov operácie	Parametre	Návratová hodnota	Význam
Vzad	-	-	Efektívne posunie iterátor na predchádzajúci prvok v štruktúre
Má predchádzajúci	-	bool	Vráti príznak, či existuje predchádzajúci prvok v štruktúre

Iterátory s náhodným prístupom (navyš k operáciám spätných iterátorov)			
Názov operácie	Parametre	Návratová hodnota	Význam
Je pred	↑IterátorS NáhodnýmPrístupom <TypBloku>	bool	Vráti príznak, či ukazuje na prvok štruktúry pred iterátorom odovzdaným ako parameter
Je za			Vráti príznak, či ukazuje na prvok štruktúry za iterátorom odovzdaným ako parameter
Posuň sa o	int	-	Efektívne posunie iterátor na prvok v štruktúre vzdialený o parameter od aktuálneho prvku

For-each v jazyku C++

Metódy, ktoré musí implementovať štruktúra			
Názov operácie	Parametre	Návratová hodnota	Význam
<code>begin</code>	-	„iterátor“	Vráti „iterátor“ (čokoľvek, čo má požadované operácie) na začiatok sekvencie (na prvý platný prvok).
<code>end</code>	-	„iterátor“	Vráti „iterátor“ (čokoľvek, čo má požadované operácie) na koniec sekvencie (na prvý neplatný prvok).
Operátory, ktoré musí implementovať „iterátor“			
Názov operácie	Parametre	Návratová hodnota	Význam a prislúchajúca operácia zo snímok vyššie
<code>operator!=</code>	„iterátor“	<code>bool</code>	Vráti true, ak sa iterátor nerovná s iterátorom odovzdaným ako parameter. Operácia <i>jeRôzny</i> .
<code>operator*</code>	-	<code>T</code>	Sprístupní aktuálny prvok v iterácii. Operácia <i>sprístupni</i> .
<code>operator++</code>	-	„iterátor“&	Posunie sa ďalej. Operácia <i>vpred</i> .

Cyklus for-each v jazyku C++

```
1. auto it = struktura.begin();
2. auto end = struktura.end();
3. while (it != end) {
4.     T prvok = *it;
5.     ++it;
6.     // spracovanie prvku
7. }
```

Príklad využitia cyklu for-each v jazyku C++

```
1. std::vector<int> cisla {10, 20, 30};
2. for (auto cislo : cisla) {
3.     std::cout << cislo << std::endl;
4. }
```

For-each v jazyku Java

Iterable<T>			
Názov operácie	Parametre	Návratová hodnota	Význam
iterator	-	Iterator<T>	Vráti iterátor na prvky typu T.
Iterator<E>			
Názov operácie	Parametre	Návratová hodnota	Význam
hasNext	-	boolean	Vráti true , ak sa v iterácii ešte nachádzajú prvky. Reflektuje operáciu máĎalší .
next	-	E	Sprístupní aktuálny prvok v iterácii a posunie sa ďalej. Reflektuje operácie sprístupni a vpred .

Cyklus for-each v jazyku Java

```
1. for (Iterator<T> i = struktura.iterator(); i.hasNext(); ) {  
2.     T prvok = i.next();  
3.     // spracovanie prvku  
4. }
```

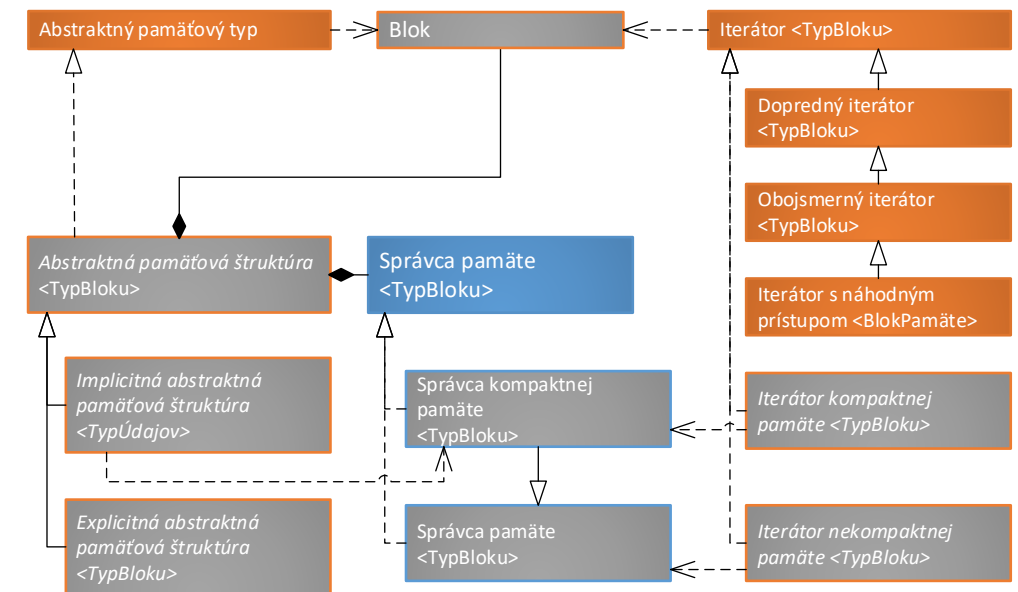
Príklad využitia cyklu for-each v jazyku Java

```
1. List<String> retazce = List.of("foo", "bar", "baz");  
2. for (String retazec : retazce) {  
3.     System.out.println(retazec);  
4. }
```

Implementácia univerzálnych APS s iterátormi

- Podľa uloženia APS môžeme vytvoriť všeobecné iterátory:
 - kompaktnej pamäte - musia uchovávať 2 vlastnosti: *referenciu na APS* a *index* bloku pamäte.
 - nekompaktnej pamäte – musia uchovávať *referenciu na blok pamäte*.
- Všeobecné iterátory dokážu implementovať operáciu *sprístupni*.
- Ostatné operácie implementuje konkrétna APS.
- APT rozšírime o operácie *prvý iterátor* a *posledný iterátor*.

Názov operácie	Parametre	Návratová hodnota	Význam
Prvý iterátor	-	Iterator <TypBloků>	Vráti iterátor na prvý platný prvok.
Posledný iterátor	-	Iterator <TypBloků>	Vráti iterátor na prvý neplatný prvok.



Sekvenčné prehliadky APS s využitím iterátorov

- Typickými operáciami realizovanými s APS sú ich **prehliadky** – teda algoritmy, ktoré musia efektívne spracovať všetky jej bloky pamäte. Vo všeobecnosti môžeme definovať algoritmy pre:
 - sekvenčné spracovanie všetkých blokov pamäte,
 - získanie prvého bloku pamäte s danou vlastnosťou,
 - získanie poradia spracovania bloku pamäte s danou vlastnosťou (ako špeciálny prípad získania bloku pamäte s danou vlastnosťou).
- **S využitím iterátorov je možné (nie len) takéto algoritmy zapísať všeobecne, bez ohľadu na to, aké majú bloky pamäte vzájomné vzťahy alebo ako sú interne uchovávané.**

Prehliadky podmnožiny APS s využitím iterátorov – pseudokódy

Definícia operácie *spracujVšetkyBloky*(DoprednýIterátor<TypBloku>, DoprednýIterátor<TypBloku>, $\lambda(\uparrow \text{TypBloku})$)

```
1. operácia spracujVšetkyBloky(  
    začiatok: DoprednýIterátor<TypBloku>,  
    koniec: DoprednýIterátor<TypBloku>,  
    operácia:  $\lambda(\uparrow \text{TypBloku})$   
    ) {  
2.   Pokiaľ (začiatok.jeRôzny(koniec)) opakuj {  
3.     operácia(začiatok.sprístupni())  
4.     začiatok.vpred()  
5.   }  
6. }
```

Definícia operácie *nájdíBlokSVlastnosťou*(DoprednýIterátor<TypBloku>, DoprednýIterátor<TypBloku>, $\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$): $\uparrow \text{TypBloku}$

```
1. operácia nájdíBlokSVlastnosťou(  
    začiatok: DoprednýIterátor<TypBloku>,  
    koniec: DoprednýIterátor<TypBloku>,  
    skontroluj:  $\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$   
    ) :  $\uparrow \text{TypBloku}$  {  
2.   Pokiaľ (začiatok.jeRôzny(koniec)) opakuj {  
3.     definuj premennú blok:  $\uparrow \text{TypBloku} \leftarrow$  začiatok.sprístupni()  
4.     Ak (skontroluj(blok)) potom {  
5.       Vráť blok  
6.     }  
7.     začiatok.vpred()  
8.   }  
9.   Vráť NULL  
10. }
```

Definícia operácie *poradieBlokSVlastnosťou*(DoprednýIterátor <TypBloku>, DoprednýIterátor<TypBloku>, $\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$): int

```
1. operácia poradieBlokSVlastnosťou(  
    začiatok: DoprednýIterátor<TypBloku>,  
    koniec: DoprednýIterátor<TypBloku>,  
    skontroluj:  $\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$   
    ) : int {  
2.   definuj premennú poradie: int  $\leftarrow$  0  
3.   Pokiaľ (začiatok.jeRôzny(koniec)) opakuj {  
4.     Ak (skontroluj(začiatok.sprístupni())) potom {  
5.       Vráť poradie  
6.     }  
7.     inak {  
8.       poradie++  
9.       začiatok.vpred()  
10.    }  
11.  }  
12.  Vráť -1  
13. }
```

Prehliadky **celej** APS s využitím iterátorov – pseudokódy

Definícia operácie *APS.spracujVšetkyBloky*($\lambda(\uparrow \text{TypBloku})$)

1. **operácia** *APS.spracujVšetkyBloky*(
 operácia: $\lambda(\uparrow \text{TypBloku})$
) {
2. *spracujVšetkyBloky*(
 prvýIterátor(), *poslednýIterátor*(), *operácia*)
3. }

Definícia operácie

APS.nájdiBlokSVlastnosťou($\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$): $\uparrow \text{TypBloku}$

1. **operácia** *APS.nájdiBlokSVlastnosťou* (
 skontroluj: $\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$
) : $\uparrow \text{TypBloku}$ {
2. **Vráť** *nájdiBlokSVlastnosťou*(
 prvýIterátor(), *poslednýIterátor*(), *skontroluj*)
3. }

Definícia operácie

APS.poradieBlokuSVlastnosťou($\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$): *int*

1. **operácia** *APS.poradieBlokuSVlastnosťou*(
 skontroluj: $\lambda(\uparrow \text{TypBloku}) \rightarrow \text{bool}$
) : *int* {
2. **Vráť** *poradieBlokuSVlastnosťou*(
 prvýIterátor(), *poslednýIterátor*(), *skontroluj*)
3. }

Sekvenčné prehliadky APS s využitím iterátorov – zložitosti

Prehliadky		
Prehliadka	Zložitosť	Parametre
Spracuj všetky bloky	$O(n) * O(a)$	n – počet blokov pamäte spracovaných prehliadkov $O(a)$ – zložitosť operácie spracovania bloku pamäte
Nájdi blok s vlastnosťou	$O(n) * O(k)$	n – počet blokov pamäte spracovaných prehliadkov $O(k)$ – zložitosť kontroly bloku pamäte
Poriadie bloku s vlastnosťou		

Zhrnutie prednášky

- Abstraktné pamäťové typy
 - Prehliadky APT
 - Iterátory APT

Plán na cvičenie

- Univerzálny návrh APT
 - Viacnásobná dedičnosť v jazyku C++.
 - Návrh spoločných predkov pre implicitnú a explicitnú APS.
 - Implementácia univerzálnych metód v jednotlivých predkoch APS.

Ďakujem za pozornosť

Doplňujúce informácie?
Vaše otázky?

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu 6UI0004 Algoritmy a údajové štruktúry 1 na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline.
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené.

Ing. Michal Varga, PhD.
Katedra informatiky
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
Michal.Varga@fri.uniza.sk

Algoritmy a údajové štruktúry 1

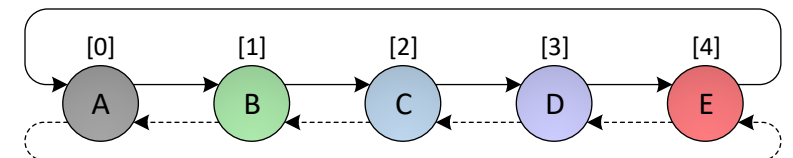
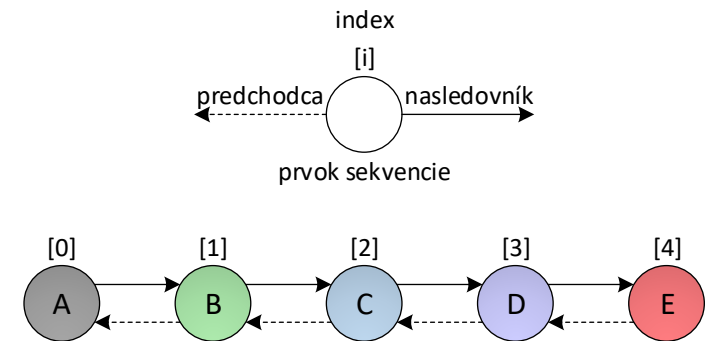
Prednáška č. 4

- Sekvencie a ich iterátory
 - Implicitná sekvencia
 - Explicitná sekvencia

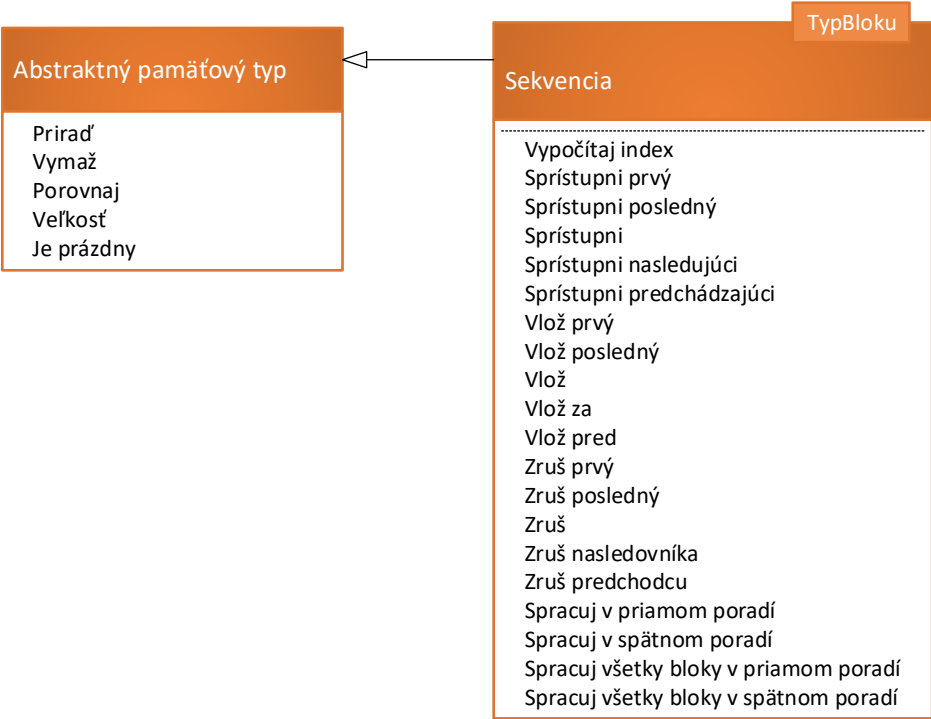


Lineárne abstraktné pamäťové typy (IAPT) – **sekvencie**

- Pre ľubovoľné dva bloky pamäte v sekvencii je možné jednoznačne definovať ich vzťah **nasledovník** – **predchodca**.
 - Nasledovník ((D)) = (E)
 - Predchodca ((D)) = (C)
 - Index ((A)) = 0
- Každý blok pamäte má určené svoje poradie – **index**.
- Bloku pamäte spolu s prideleným indexom budeme hovoriť **prvok sekvencie**.
- Ak sú navzájom previazané prvý a posledný prvok, hovoríme o **cyklickej** sekvencii.



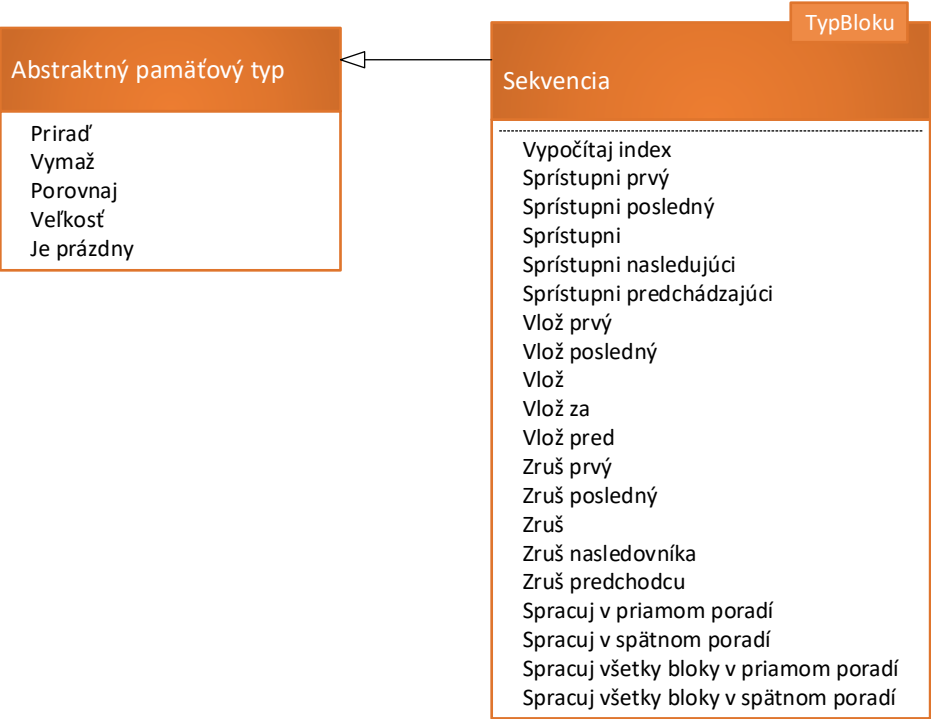
Sekvence



Základné operácie so sekvenciou			
Názov operácie	Parametre	Návratová hodnota	Význam
Vypočítaj index	↑TypBloku	int	Vráti index bloku pamäte odovzdaného ako parameter.

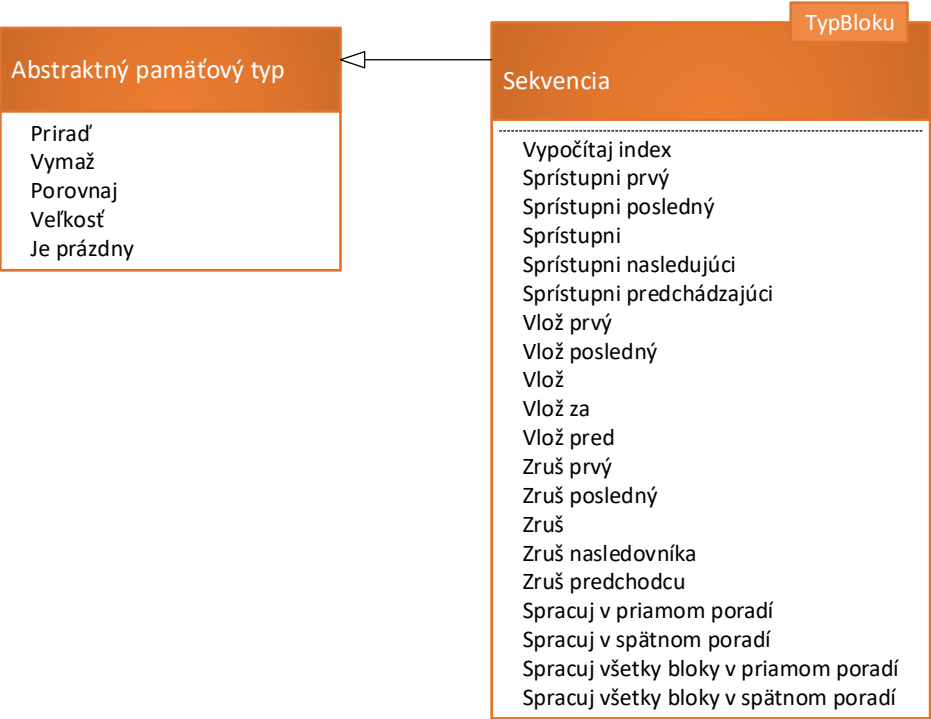
Selektory			
Názov selektora	Parametre	Návratová hodnota	Selektor sprístupní
Sprístupni prvý	-	↑TypBloku	Prvý blok pamäte sekvencie.
Sprístupni posledný	-		Posledný blok pamäte sekvencie.
Sprístupni	int		Blok pamäte s daným indexom.
Sprístupni nasledujúci	↑TypBloku		Nasledujúci blok pamäte od bloku odovzdaného ako parameter.
Sprístupni predchádzajúci	↑TypBloku		Predchádzajúci blok pamäte od bloku odovzdaného ako parameter.

Sekvence



Modifikátory			
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor vloží nový blok pamäte
Vlož prvý	-	↑TypBloku	Na začiatok sekvencie.
Vlož posledný	-		Na koniec sekvencie.
Vlož	int		Na miesto v sekvencii určené parametrom index.
Vlož za	↑TypBloku		Za blok pamäte odovzdaný ako parameter.
Vlož pred	↑TypBloku		Pred blok pamäte odovzdaný ako parameter.
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor odstráni zo sekvencie blok pamäte
Zruš prvý	-	-	Prvý.
Zruš posledný	-		Posledný.
Zruš	int		Určený parametrom index.
Zruš nasledovníka	↑TypBloku		Za blokom pamäte určeným parametrom.
Zruš predchodcu	↑TypBloku		Pred blokom pamäte určeným parametrom.

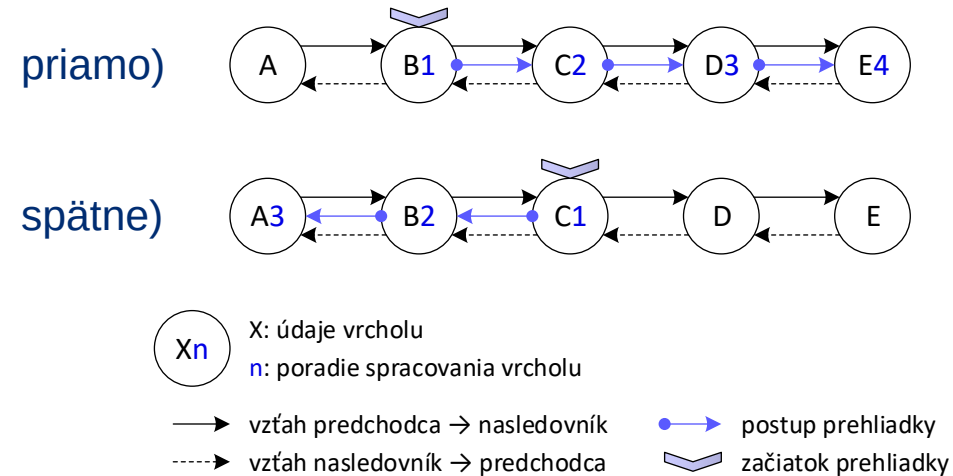
Sekvence



Prehliadky			
Názov prehliadky	Parametre	Návratová hodnota	Princíp prehliadky
Spracuj v priamom poradí	↑TypBlok λ(↑TypBlok)	-	Pomocou operácie z parametra spracuje v priamom poradí sekvenciu, ktorej začiatok je odovzdaný ako parameter.
Spracuj v spätnom poradí			Pomocou operácie z parametra spracuje v spätnom poradí sekvenciu, ktorej začiatok je odovzdaný ako parameter.
Spracuj všetky bloky v priamom poradí	↑TypBlok		Pomocou operácie z parametra spracuje v priamom poradí všetky bloky pamäte sekvencie.
Spracuj všetky bloky v spätnom poradí			Pomocou operácie z parametra spracuje v spätnom poradí všetky bloky pamäte sekvencie.

Prehliadky sekvencií

- Možné v **priamom** aj **spätnom** poradí.
- Implementačne sa nelíši prehliadka implicitnej a explicitnej sekvencie.



Definícia operácie *Sekvencia*<TypBlok>.spracujVPriamomPoradí(↑TypBlok, λ(↑TypBlok))

```

1. operácia Sekvencia<TypBlok>.spracujVPriamomPoradí(
    blok: ↑TypBlok,
    operácia: λ(↑TypBlok)
) {
2.   Pokiaľ (blok ≠ NULL) opakuje {
3.     operácia(blok)
4.     blok ← sprístupniNasledujúci(blok↓)
5.   }
6. }
```

Definícia operácie *Sekvencia*<TypBlok>. spracujVšetkyBlokyVPriamomPoradí (λ(↑TypBlok))

```

1. operácia Sekvencia<TypBlok>.spracujVšetkyBlokyVPriamomPoradí(
    operácia: λ(↑TypBlok)
) {
2.   spracujVPriamomPoradí(sprístupniPrvý(), operácia)
3. }
```

Definícia operácie *Sekvencia*<TypBlok>.spracujVSpätnomPoradí(↑TypBlok, λ(↑TypBlok))

```

1. operácia Sekvencia<TypBlok>.spracujVSpätnomPoradí(
    blok: ↑TypBlok,
    operácia: λ(↑TypBlok)
) {
2.   Pokiaľ (blok ≠ NULL) opakuje {
3.     operácia(blok)
4.     blok ← sprístupniPredchádzajúci(blok↓)
5.   }
6. }
```

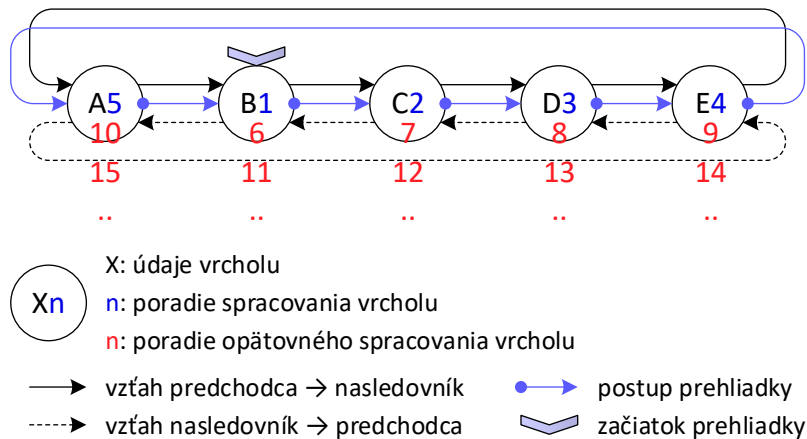
Definícia operácie *Sekvencia*<TypBlok>.spracujVšetkyBlokyVSpätnomPoradí (λ(↑TypBlok))

```

1. operácia Sekvencia<TypBlok>.spracujVšetkyBlokyVSpätnomPoradí(
    operácia: λ(↑TypBlok)
) {
2.   spracujVSpätnomPoradí(sprístupniPosledný(), operácia)
3. }
```

Prehliadka cyklickej sekvencie

- Pozor na nekonečný cyklus!
- Stačí opraviť operáciu spracuj bloky v priamom poradí (analogicky operáciu v spätnom poradí).



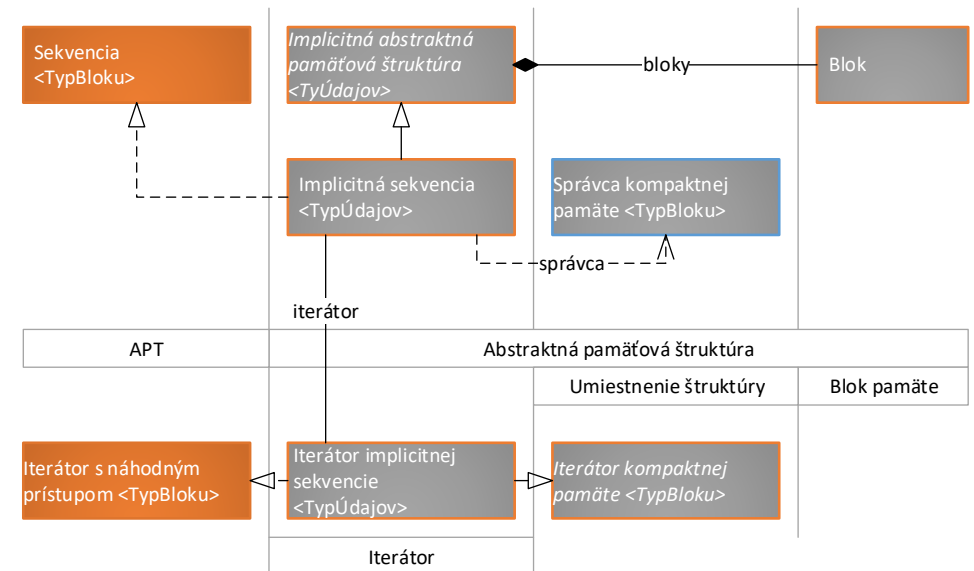
Definícia operácie *CyklickáSekvencia*<TypBlok>.spracujVPriamomPoradí($\uparrow$ TypBlok, $\lambda(\uparrow$ TypBlok))

```
1. operácia CyklickáSekvencia<TypBlok>.spracujVPriamomPoradí(  
    blok:  $\uparrow$ TypBlok,  
    operácia:  $\lambda(\uparrow$ TypBlok)  
    ) {  
2.   definuj premennú prvýBlokSpracovaný: bool  $\leftarrow$  nepravda  
3.   definuj premennú prvýBlok:  $\uparrow$ TypBlok  $\leftarrow$  blok  
4.   Pokiaľ ((blok  $\neq$  NULL)  $\wedge$   
        ( $\neg$ prvýBlokSpracovaný  $\vee$  blok  $\neq$  prvýBlok)) opakuj {  
5.     operácia(blok)  
6.     blok  $\leftarrow$  sprístupniNasledujúci(blok $\downarrow$ )  
7.     prvýBlokSpracovaný  $\leftarrow$  pravda  
8.   }  
9. }
```

Implicitná sekvencia

- Vzťahy pre získanie nasledovníka / predchodcu sú jednoduché:
 - $Nasledovník(i) = i + 1$
 - $Predchodca(i) = i - 1$
- Je vhodné zapúzdriť ich pre cyklickú sekvenciu, kde je potrebné vyriešiť „zacyklenie“ z prvého na posledný prvok a opačne.

Názov operácie	Parametre	Návratová hodnota	Význam
Index nasledovníka	<code>int</code>	<code>int</code>	Vráti index nasledovníka.
Index predchodcu	<code>int</code>	<code>int</code>	Vráti index predchodcu.



Implicitná sekvencia – selektory

- Selektory je možné implementovať pomocou univerzálneho selektora `sprístupni(int): ↑ImplicitnáAPS.TypBloku`.
- Ten využije operáciu `správcu kompaktnnej pamäte` `daj blok pamäte`, ktorá vráti blok pamäte predstavujúci návratovú hodnotou z toho selektora.
- Ostatné selektory využijú vhodné metódy `sekvencie` resp. `správcu kompaktnnej pamäte` na získanie správneho indexu (príklad. `sprístupni nasledujúci`).

Selektory		
Operácia	Zložitosť	Parametre
<code>Sprístupni prvý</code>	O(1)	-
<code>Sprístupni posledný</code>		
<code>Sprístupni</code>		
<code>Sprístupni nasledujúci</code>		
<code>Sprístupni predchádzajúci</code>		

Implicitná sekvencia – selektory – pseudokódy

Definícia operácie *IS.sprístupni(int)*: $\uparrow$ ImplicitnáAPS.TypBlok

```
1. operácia IS.sprístupni(index: int):  $\uparrow$ ImplicitnáAPS.TypBlok {  
2.   Keď platí (index  $\geq 0 \wedge$  index < veľkosť())  
   tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(index))  
   inak vráť NULL  
3. }
```

Definícia operácie *IS.sprístupniPrvý()*: $\uparrow$ ImplicitnáAPS.TypBlok

```
1. operácia IS.sprístupniPrvý():  $\uparrow$ ImplicitnáAPS.TypBlok {  
2.   Keď platí (veľkosť() > 0)  
   tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(0))  
   inak vráť NULL  
3. }
```

Definícia operácie *IS.sprístupniPosledný()*: $\uparrow$ ImplicitnáAPS.TypBlok

```
1. operácia IS.sprístupniPosledný():  $\uparrow$ ImplicitnáAPS.TypBlok {  
2.   definuj premennú veľkosť: int  $\leftarrow$  veľkosť()  
3.   Keď platí (veľkosť > 0)  
   tak vráť dajAdresu(  
       správcaPamäte→dajBlokPamäte(veľkosť - 1))  
   inak vráť NULL  
4. }
```

Definícia operácie

IS.sprístupniNasledujúci($\uparrow$ ImplicitnáAPS.TypBlok): $\uparrow$ ImplicitnáAPS.TypBlok

```
1. operácia IS.sprístupniNasledujúci(  
   blok:  $\uparrow$ ImplicitnáAPS.TypBlok  
) :  $\uparrow$ ImplicitnáAPS.TypBlok {  
2.   definuj premennú index: int  $\leftarrow$   
       indexNasledovníka(správcaPamäte→vypočítajPoradie(blok))  
3.   Keď platí (index < veľkosť())  
   tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(index))  
   inak vráť NULL  
4. }
```

Definícia operácie

IS.sprístupniPredchádzajúci($\uparrow$ ImplicitnáAPS.TypBlok): $\uparrow$ ImplicitnáAPS.TypBlok

```
1. operácia IS.sprístupniPredchádzajúci(  
   blok:  $\uparrow$ ImplicitnáAPS.TypBlok  
) :  $\uparrow$ ImplicitnáAPS.TypBlok {  
2.   definuj premennú index: int  $\leftarrow$   
       indexPredchodcu(správcaPamäte→vypočítajPoradie(blok))  
3.   Keď platí (index  $\geq 0$ )  
   tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(index))  
   inak vráť NULL  
4. }
```

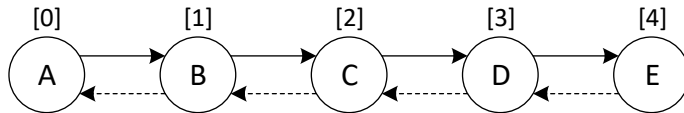
Implicitná sekvencia – modifikátory – vkladanie

- Alokovanie blokov pamäte ma za úlohu správca kompaktnej pamäte – stačí zavolať príslušnú operáciu.
- Vkladanie na koniec je jednoduché (ak nedôjde k expanzii).
- Vkladanie do stredu spôsobí potenciálne expanziu a pomalé kopírovanie pamäte!

Modifikátory		
Operácia	Zložitosť	Parametre
Vlož prvý	$2 \cdot O(b)$	b: $\lceil n/B \rceil$ B: veľkosť buffra (koľko prvkov sa do neho zmestí) n: počet prvkov
Vlož posledný	$O(b)$	
Vlož	$2 \cdot O(b)$	
Vlož za	$2 \cdot O(b)$	
Vlož pred	$2 \cdot O(b)$	

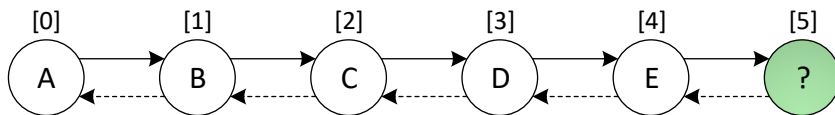
Implicitná sekvencia – modifikátory – vkladanie – príklad

pred vkladáním



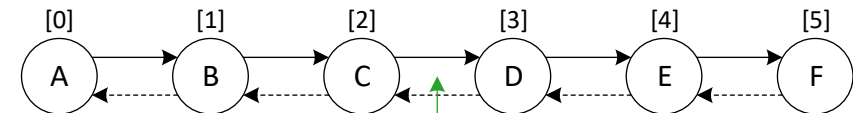
<<Inštancie: Blok>>							
A	B	C	D	E	?	?	?

po volaní operácie vlož(5)



<<Inštancie: Blok>>							
A	B	C	D	E	?	?	?

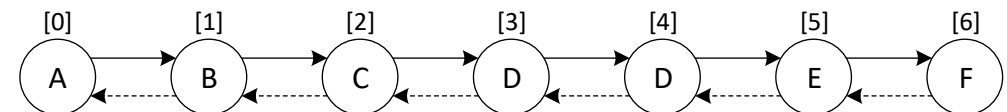
po volaní operácie vlož(3)



<<Inštancie: Blok>>							
A	B	C	D	E	F	?	?

<<Inštancie: Blok>>							
A	B	C	D	D	E	F	?

<<Inštancie: Blok>>							
A	B	C	D	D	E	F	?



Implicitná sekvencia – modifikátory – vkladanie – pseudokódy

Definícia operácie IS.vlož(int): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IS.vlož(index: int):  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Vráť správcaPamäte $\rightarrow$ prideIPamätNa(index) $\downarrow$   
3. }
```

Definícia operácie IS.vložPosledný(): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IS.vložPosledný():  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Vráť správcaPamäte $\rightarrow$ prideIPamät() $\downarrow$   
3. }
```

Definícia operácie IS.vložZa($\uparrow$ ImplicitnáAPS.TypBloku): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IS.vložZa(  
   blok:  $\uparrow$ ImplicitnáAPS.TypBloku  
):  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Vráť správcaPamäte $\rightarrow$ prideIPamätNa(  
       správcaPamäte $\rightarrow$ vypočítajPoradie(blok) + 1) $\downarrow$   
3. }
```

Definícia operácie IS.vložPrvý(): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IS.vložPrvý():  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Vráť správcaPamäte $\rightarrow$ prideIPamätNa(0) $\downarrow$   
3. }
```

Definícia operácie IS.vložPred($\uparrow$ ImplicitnáAPS.TypBloku): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IS.vložPred(  
   blok:  $\uparrow$ ImplicitnáAPS.TypBloku  
):  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Vráť správcaPamäte $\rightarrow$ prideIPamätNa(  
       správcaPamäte $\rightarrow$ vypočítajPoradie(blok)) $\downarrow$   
3. }
```

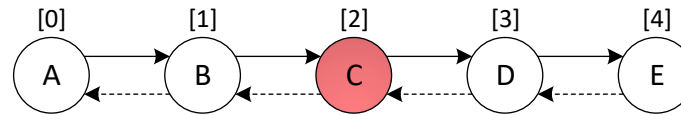

Implicitná sekvencia – modifikátory – mazanie

- Analogicky ku vkladaniu – rušenie blokov pamäte ma za úlohu správca kompaktnéj pamäte – stačí zavolať príslušnú operáciu.
- Mazanie z konca je jednoduché.
- Mazanie zo stredu spôsobí pomalé kopírovanie pamäte!
- Alokované, ale nepoužívané, miesto sa automaticky nevráti.

Modifikátory		
Operácia	Zložitosť	Parametre
Zruš prvý	$O(b)$	b: $\lfloor n/B \rfloor$ B: veľkosť buffra (koľko prvkov sa do neho zmestí) n: počet prvkov
Zruš posledný	$O(1)$	
Zruš	$O(b)$	
Zruš nasledovníka	$O(b)$	
Zruš predchodcu	$O(b)$	

Implicitná sekvencia – modifikátory – mazanie – príklad

volenie operácie zruš(2)



<<Inštancie: Blok>>							
A	B	C	D	E	?	?	?

<<Inštancie: Blok>>							
A	B	C	D	E	?	?	?



<<Inštancie: Blok>>							
A	B	D	E	E	?	?	?

<<Inštancie: Blok>>							
A	B	D	E	E	?	?	?

Implicitná sekvencia – modifikátory – mazanie – pseudokódy

Definícia operácie IS.zruš(int)

```
1. operácia IS.zruš(index: int) {  
2.   správcaPamäte→uvoľniPamäťNa(index)  
3. }
```

Definícia operácie IS.zrušPrvý()

```
1. operácia IS.zrušPrvý() {  
2.   správcaPamäte→uvoľniPamäťNa(0)  
3. }
```

Definícia operácie IS.zrušNasledovníka(↑ImplicitnáAPS.TypBloku)

```
1. operácia IS.zrušNasledovníka(blok: ↑ImplicitnáAPS.TypBloku) {  
2.   správcaPamäte→uvoľniPamäťNa(  
       indexNasledovníka(správcaPamäte→vypočítajPoradie(blok)))  
3. }
```

Definícia operácie IS.zrušPosledný()

```
1. operácia IS.zrušPosledný() {  
2.   správcaPamäte→uvoľniPamäťNa()  
3. }
```

Definícia operácie IS.zrušPredchodcu(↑ImplicitnáAPS.TypBloku)

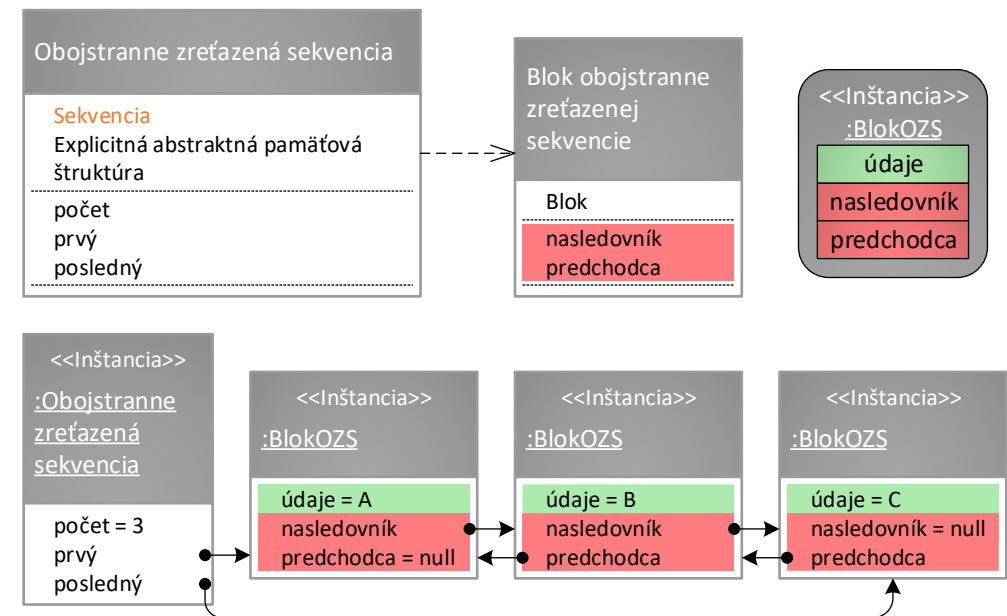
```
1. operácia IS.zrušPredchodcu(blok: ↑ImplicitnáAPS.TypBloku) {  
2.   správcaPamäte→uvoľniPamäťNa(  
       indexPredchodcu(správcaPamäte→vypočítajPoradie(blok)))  
3. }
```

Implicitná sekvencia – iterátory

- Iterátory implicitných sekvencií je možné implementovať ako iterátory s náhodným prístupom.
- Iterátor je založený na všeobecnom iterátore kompaktnej pamäte, ktorý obsahuje dva atribúty:
 - referenciu na štruktúru (implicitnú sekvenciu, cez ktorú iteruje) a
 - referenciu na aktuálny prvok (vo forme indexu).
- Posun iterátora implicitnej sekvencie je realizovaný jednoducho prostredníctvom prepočtu hodnoty vlastnosti aktuálny, čím je tento iterátor veľmi efektívny – zložitosti všetkých operácií sú $O(1)$.
- Implicitná sekvencia vytvára iterátory tak, že inicializuje atribút aktuálny na index správneho prvku:
 - Prvý iterátor: index prvého platného prvku, čo je 0.
 - Posledný iterátor: index prvého neplatného prvku, čo je počet.

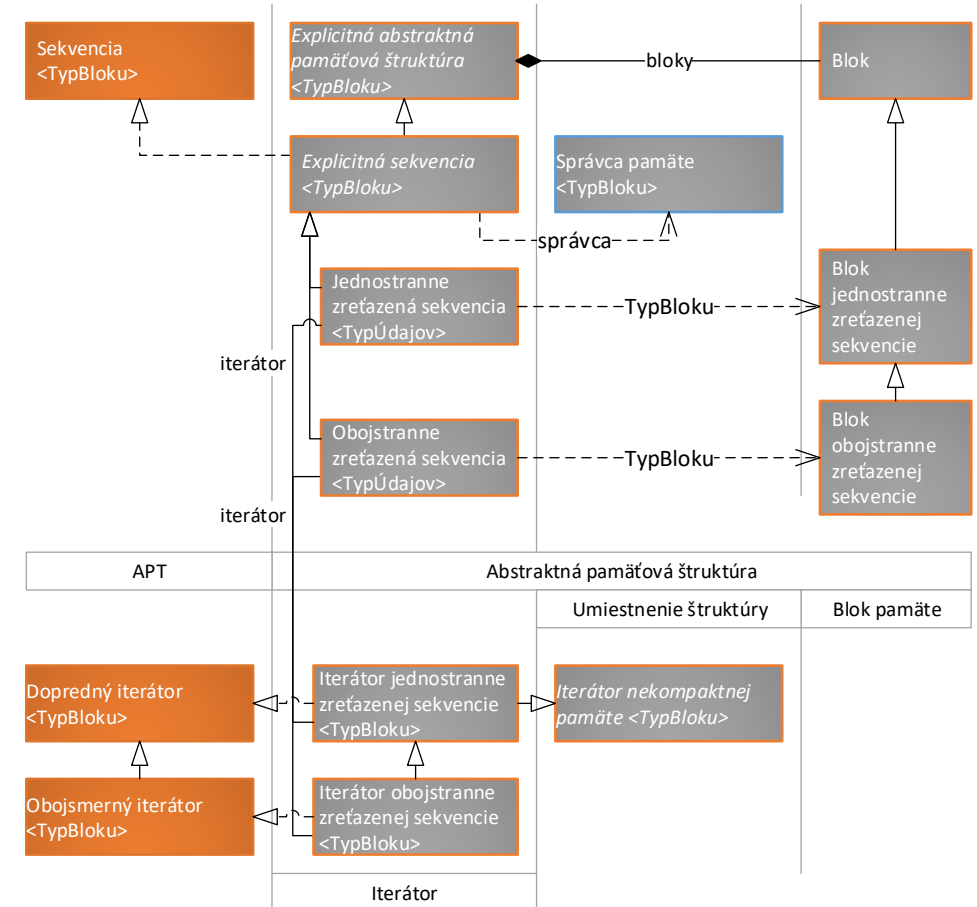
Explicitná sekvencia

- Riadiaci blok **explicitnej** sekvencie typicky ukladá referencie na *prvý* a *posledný* blok pamäte a kvôli efektívite aj ich *počet*.
- Ak blok pamäte obsahuje iba referenciu na nasledovníka tak hovoríme o **jednostranne zreťazenej (explicitnej) sekvencii**.
- Ak obsahuje aj referenciu na predchodcu, hovoríme o **obojustranne zreťazenej (explicitnej) sekvencii**.
- Z pohľadu APS je nepodstatné, čo bude v údajovej časti uložené.



Explicitná sekvencia

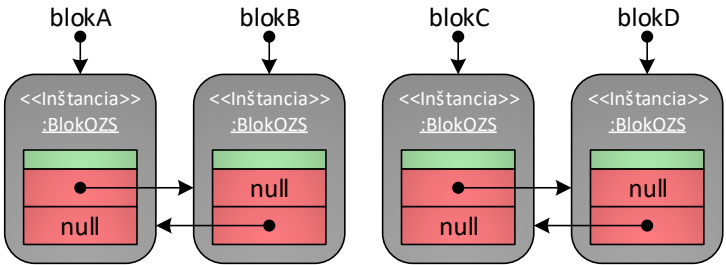
- **Explicitné sekvencie** je možné do veľkej miery navrhnuť jednotne.
- Obojstranne zreťazené sekvencie umožňujú efektívnejšiu implementáciu niektorých modifikátorov, avšak za cenu vyšších pamäťových nárokov.
- Ak bude možné v **obojsmerne zreťazenej sekvencii** zefektívniť algoritmus niektorej operácie vďaka vlastnosti *predchodca* v blokoch pamäte, potom táto štruktúra môže príslušnú operáciu prepísať.
- **Jednostranne aj obojsmerne zreťazená sekvencia** špecifikuje typ bloku pamäte explicitnej sekvencie.



Explicitná sekvencia – pomocné operácie

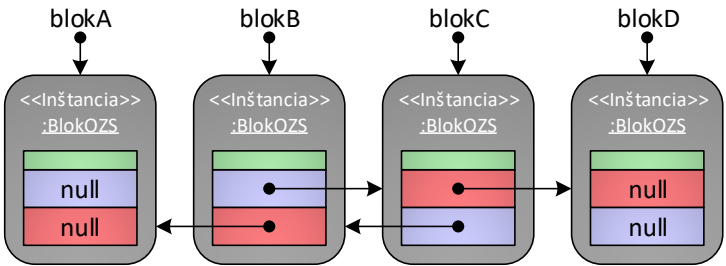
Názov operácie	Parametre	Návratová hodnota	Význam
spojBloky	↑TypBloku ↑TypBloku	-	Navzájom prepojí bloky pamäte v poradí podľa parametrov.
odpojBlok	↑TypBloku	-	Vypojí zo zreťazenia blokov pamäte blok odovzdaný ako parameter.

pred)

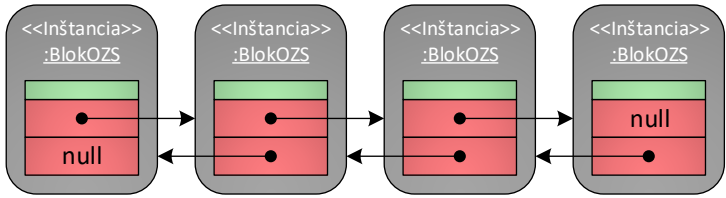


spojBloky(null, blokA)
spojBloky(blokB, blokC)
spojBloky(blokD, null)

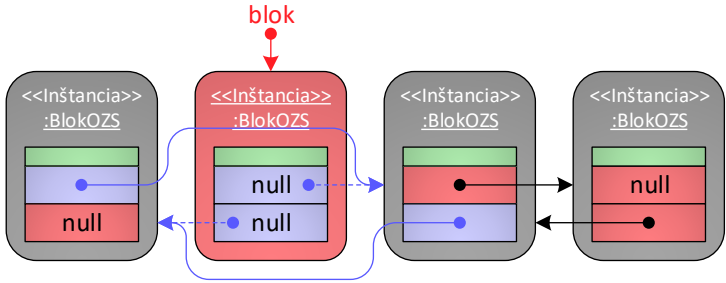
po)



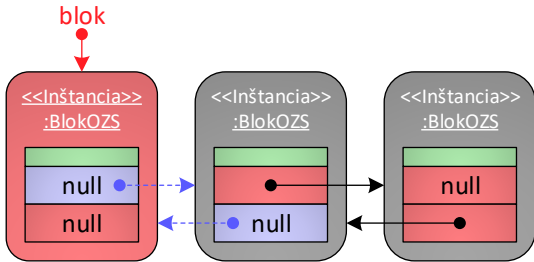
pred)



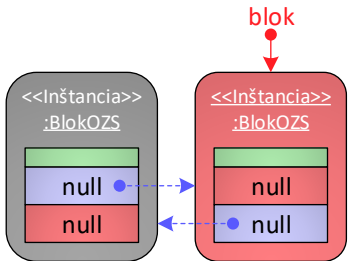
volanie 1)



volanie 2)



volanie 3)



Postupné volanie operácie odpoj blok

Explicitná sekvencia – pomocné operácie - pseudokódy

Definícia operácie *ES.spojBloky*(↑TypBloku, ↑TypBloku)

```
1. operácia ES.spojBloky(  
    predchádzajúci: ↑TypBloku,  
    nasledujúci: ↑TypBloku  
    ) {  
2.   Ak (predchádzajúci ≠ NULL) potom {  
3.     predchádzajúci→nasledovník ← nasledujúci  
4.   }  
5. }
```

Definícia operácie *ES.odpojBlok*(↑TypBloku)

```
1. operácia ES.odpojBlok(blok: ↑TypBloku) {  
2.   spojBloky(sprístupniPredchádzajúci(blok↓),  
              sprístupniNasledujúci(blok↓))  
3.   blok→nasledovník ← NULL  
4. }
```

Definícia operácie *ObojstranneZS.spojBloky*(↑TypBloku, ↑TypBloku)

```
1. operácia ObojstranneZS.spojBloky(  
    predchádzajúci: ↑TypBloku,  
    nasledujúci: ↑TypBloku  
    ) {  
2.   ES.spojBloky(predchádzajúci, nasledujúci)  
3.   Ak (nasledujúci ≠ NULL) potom {  
4.     nasledujúci→predchodca ← predchádzajúci  
5.   }  
6. }
```

Definícia operácie *ObojstranneZS.odpojBlok*(↑TypBloku)

```
1. operácia ObojstranneZS.odpojBlok(blok: ↑TypBloku) {  
2.   ES.odpojBlok(blok)  
3.   blok→predchodca ← NULL  
4. }
```


Explicitná sekvencia – kostry typických algoritmov

- Posun z bloku na blok je v ES typicky realizovaný volaním
 $\text{aktuálny} \leftarrow \text{aktuálny} \rightarrow \text{nasledovník}$
- Pomocou tohto volania je možné definovať kostry 3 typických algoritmov (vpravo).
- Tieto slúžia na implementáciu ostatných operácií (napr. **prirad'**, **vymaž**, **porovnaj**).

Kostra 1

Kostra algoritmu pre získanie bloku pamäte s daným poradím v explicitnej sekvencii

```
1. definuj premennú blok:  $\uparrow \text{BlokPamäteExplicitnejSekvencie} \leftarrow \text{prvý}$ 
2. Opakuj pre premennú i: int od 0 do k - 1 {
3.   blok  $\leftarrow$  blok $\rightarrow$ nasledovník
4. }
5. // v premennej blok je blok pamäte s poradím „k“
6. // k musí byť menšie ako počet a väčšie alebo rovné ako 0
```

Kostra 2

Kostra algoritmu pre získanie bloku pamäte s danou vlastnosťou v explicitnej sekvencii

```
1. definuj premennú blok:  $\uparrow \text{BlokPamäteExplicitnejSekvencie} \leftarrow \text{prvý}$ 
2. Pokiaľ (blok  $\neq$  NULL  $\wedge$  „blok nespĺňa vlastnosť“) opakuj {
3.   blok  $\leftarrow$  blok $\rightarrow$ nasledovník
4. }
5. // v premennej blok je buď
6. // NULL, ak žiadny blok nespĺňa vlastnosť alebo
7. // referencia na prvý blok s danou vlastnosťou
```

Kostra 3

Kostra algoritmu pre spracovanie všetkých blokov pamäte v explicitnej sekvencii

```
1. definuj premennú blok:  $\uparrow \text{BlokPamäteExplicitnejSekvencie} \leftarrow \text{prvý}$ 
2. Pokiaľ (blok  $\neq$  NULL) opakuj {
3.   // spracuj blok
4.   blok  $\leftarrow$  blok $\rightarrow$ nasledovník
5. }
```

Explicitná sekvencia – základné operácie

- **Priradenie:** zahodia sa všetky údaje (operácia **vymaž**) a následne sa pridajú všetky prvky s parametrom odovzdanej **explicitnej sekvencie** (na to je možné využiť 3. kostru).
- **Vymazanie:** Využije sa 3. kostra, spracovaním bloku pamäte je samotné vymazanie. Pre posun je potrebné použiť ďalšiu pomocnú premennú, do ktorej si je možné nasledovníka bloku pamäte poznačiť pred jeho vymazaním (je možné využiť vlastnosti *prvý* a *posledný* a po mazaní ich nastaviť na NULL).
- **Porovnanie:** simultánna prehliadku dvoch sekvencií. Dve sekvencie nemôžu byť rovnaké, ak neobsahujú rovnaký počet prvkov. Tento test je jednoduchý a jeho zaradenie do algoritmu v mnohých prípadoch ušetrí pomalé vzájomné porovnávanie príslušných blokov pamäte v sekvenciách.
- **Výpočet indexu:** je možné využiť prehliadku pre nájdenie bloku s vlastnosťou (samého seba) a počas prehliadky napočítavať index.

Explicitná sekvencia – základné operácie – pseudokódy

Definícia operácie ES.prirad'(↑APT)

```
1. operácia ES.prirad'(iný: ↑APT): ↑APT {
2.   Ak (¬(iný je typu ES)) potom {
3.     CHYBA
4.   }
5.   Ak (self ≠ dajAdresu(iný)) potom {
6.     vymaž()
7.     iný → spracujVšetkyBlokyVPriamomPoradí(
8.       λ(b: ↑TypBloku) { vložPosledný() → dáta ← b → dáta })
9.   }
10.  Vráť self↓
```

Definícia operácie ES.vypočítajIndex(↑TypBloku): int

```
1. operácia ES.vypočítajIndex(dáta: ↑TypBloku): int {
2.   definuj premennú výsledok: int ← 0
3.   definuj premennú blok: ↑TypBloku ←
4.     najdiBlokSVlastnosťou(λ(b: ↑TypBloku) {
5.       výsledok++
6.       Vráť dajAdresu(dáta) = b
7.     })
8.   Keď platí (blok ≠ NULL)
9.     tak vráť výsledok
10.  inak vráť -1
```

Definícia operácie ES.porovnaj(↑APT): bool

```
1. operácia ES.porovnaj(iný: ↑APT): bool {
2.   Ak (¬(iný je typu ES)) potom {
3.     Vráť nepravda
4.   }
5.   Ak (self = dajAdresu(iný)) potom {
6.     Vráť pravda
7.   }
8.   Ak (veľkosť() ≠ iný → veľkosť()) potom {
9.     Vráť nepravda
10.  }
11.  definuj premennú môjAktuálny: ↑TypBloku ← prvý
12.  definuj premennú inýAktuálny: ↑TypBloku ← iný → prvý
13.  Pokiaľ (môjAktuálny ≠ NULL) opakuj {
14.    Ak (¬(môjAktuálny → dáta = inýAktuálny → dáta)) potom {
15.      Vráť nepravda
16.    }
17.    inak {
18.      môjAktuálny ← sprístupniNasledujúci(môjAktuálny↓)
19.      inýAktuálny ← iný → sprístupniNasledujúci(inýAktuálny↓)
20.    }
21.  }
22.  Vráť pravda
23. }
```

Definícia operácie ES.vymaž()

```
1. operácia ES.vymaž() {
2.   posledný ← prvý
3.   Pokiaľ (prvý ≠ NULL) opakuj {
4.     prvý ← sprístupniNasledujúci(prvý↓)
5.     správcaPamäte → uvoľniPamäť(posledný)
6.     posledný ← prvý
7.   }
8. }
```

Explicitná sekvencia – základné operácie – zložitosti

Základné operácie so sekvenciou			
Operácia	Zložitosť		Parametre
Spoj bloky	O(1)		-
Odpoj blok	Obojstranne zreťazená sekvencia?		n: počet prvkov
	áno	nie	
	O(1)	O(n)	
Priradiť	O(n + m)		n: pôvodný počet prvkov m: nový počet prvkov
Vymaž	O(n)		n: počet prvkov
Porovnaj	O(n)		
Vypočítaj index	O(n)		

Explicitná sekvencia – selektory

- Efektívnosť selektorov je podmienená (ne)prítomnosťou referencie na *predchodcu* v blokoch pamäte a referencie na *posledný* prvok v riadiacom bloku štruktúry.
- Selektory v *explicitných sekvenciách* nie je možné efektívne napísať univerzálne s odvolaním sa na jediný univerzálny selektor, ako v *implicitných sekvenciách*, ale je potrebné rozlíšiť jednotlivé prípady.
- Podľa jednotlivých prípadov je možné využiť kostry algoritmov na prechod *explicitnou sekvenciou*, ako boli uvedené vyššie.
- Sprístupnenie *prvého* prvku je vždy jednoduché, nakoľko referencia naň je vždy uložená v riadiacom bloku štruktúry.
- Rovnako je jednoduché sprístupnenie *nasledovníka* prvku, nakoľko referencia naň je vždy uložená v samotnom prvku.

Explicitná sekvencia – selektory – pseudokódy

Definícia operácie ES.sprístupni(int): $\uparrow$ TypBloku

```
1. operácia ES.sprístupni(index: int):  $\uparrow$ TypBloku {
2.   definuj premennú výsledok:  $\uparrow$ TypBloku  $\leftarrow$  NULL
3.   Ak (index  $\geq$  0  $\wedge$  index < veľkosť()) potom {
4.     výsledok  $\leftarrow$  prvý
5.     Opakuj pre premennú i: int od 0 do index - 1 {
6.       výsledok  $\leftarrow$  výsledok $\rightarrow$ nasledovník
7.     }
8.   }
9.   Vráť výsledok
10. }
```

Definícia operácie ES.sprístupniPrvý(): $\uparrow$ TypBloku

```
1. operácia ES.sprístupniPrvý():  $\uparrow$ TypBloku {
2.   Vráť prvý
3. }
```

Definícia operácie ES.sprístupniPosledný(): $\uparrow$ TypBloku
(referencia na posledný prvok je uchovávaná)

```
1. // referencia na posledný prvok je uchovávaná
2. operácia ES.sprístupniPosledný():  $\uparrow$ TypBloku {
3.   Vráť posledný
4. }
```

Definícia operácie ES.sprístupniNasledujúci($\uparrow$ TypBloku): $\uparrow$ TypBloku

```
1. operácia ES.sprístupniNasledujúci(
   blok:  $\uparrow$ TypBloku
):  $\uparrow$ TypBloku {
2.   Vráť blok $\rightarrow$ nasledovník
3. }
```

Definícia operácie

ES.sprístupniPredchádzajúci($\uparrow$ TypBloku): $\uparrow$ TypBloku

```
1. operácia ES.sprístupniPredchádzajúci(
   blok:  $\uparrow$ TypBloku
):  $\uparrow$ TypBloku {
2.   Vráť nájdíBlokSVlastnosťou(
        $\lambda$ (b:  $\uparrow$ TypBloku) { Vráť b $\rightarrow$ nasledovník = dajAdresu(blok) })
3. }
```

Definícia operácie ES.sprístupniPosledný(): $\uparrow$ TypBloku
(referencia na posledný prvok nie je uchovávaná)

```
1. // referencia na posledný prvok nie je uchovávaná
2. operácia ES.sprístupniPosledný():  $\uparrow$ TypBloku {
3.   definuj premennú blok: TypBloku  $\leftarrow$  prvý
4.   Opakuj pre premennú i: int od 0 do veľkosť() - 1 {
5.     blok  $\leftarrow$  blok $\rightarrow$ nasledovník
6.   }
7.   Vráť blok
8. }
```

Obojstranne zreťazená ES – selektory – pseudokódy

Definícia operácie *ObojstranneZS.sprístupni(int): ↑TypBloku*

```
1. operácia ObojstranneZS.sprístupni(index: int): ↑TypBloku {  
2.   definuj premennú výsledok: ↑TypBloku ← NULL  
3.   Ak (index ≥ 0 ∧ index < veľkosť()) potom {  
4.     Ak (index < veľkosť() / 2) potom {  
5.       výsledok ← prvý  
6.       Opakuj pre premennú i: int od 1 do index {  
7.         výsledok ← výsledok→nasledovník  
8.       }  
9.     }  
10.  inak {  
11.    výsledok ← posledný  
12.    Opakuj pre premennú i: int od veľkosť() - index do 2 {  
13.      výsledok ← výsledok→predchodca  
14.    }  
15.  }  
16. }  
17. Vráť výsledok  
18. }
```

Definícia operácie

ObojstranneZS.sprístupniPredchádzajúci(↑TypBloku): ↑TypBloku

```
1. operácia ObojstranneZS.sprístupniPredchádzajúci(  
   blok: ↑TypBloku  
): ↑TypBloku {  
2.   Vráť blok→predchodca  
3. }
```

Explicitná sekvencia – selektory – zložitosti

Selektory		
Operácia	Zložitosť	
Sprístupni prvý	O(1)	
Sprístupni posledný	Obsahuje v riadiacom bloku referenciu na posledný?	
	áno	nie
	O(1)	O(n)
Sprístupni	Obsahuje v riadiacom bloku referenciu na posledný a v bloku pamäte referenciu na predchádzajúci?	
	áno	nie
	$\frac{1}{2} * O(n)$	O(n)
Sprístupni nasledujúci	O(1)	
Sprístupni predchádzajúci	Obojstranne zreťazená sekvencia?	
	áno	nie
	O(1)	O(n)

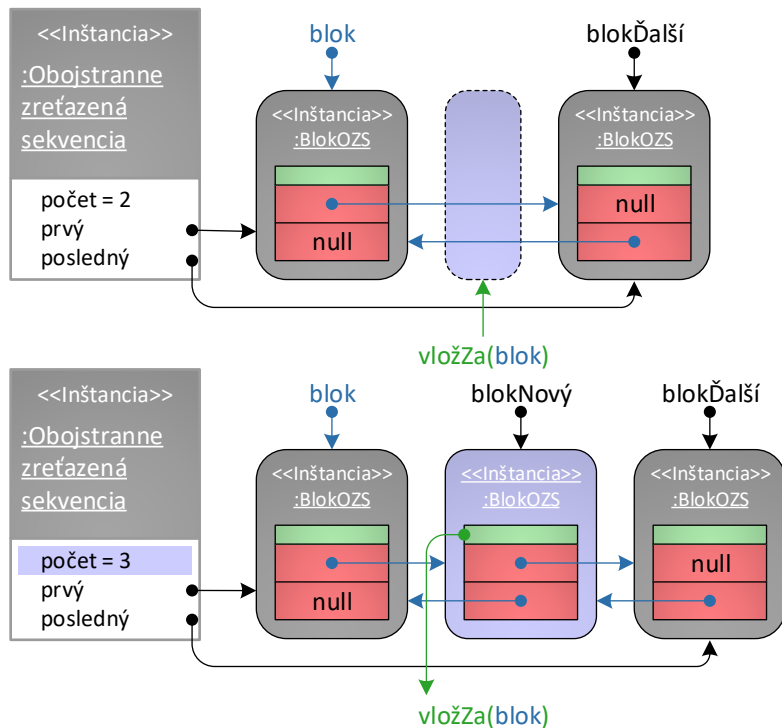
n: počet prvkov v sekvencii

Explicitná sekvencia – modifikátory – vkladanie

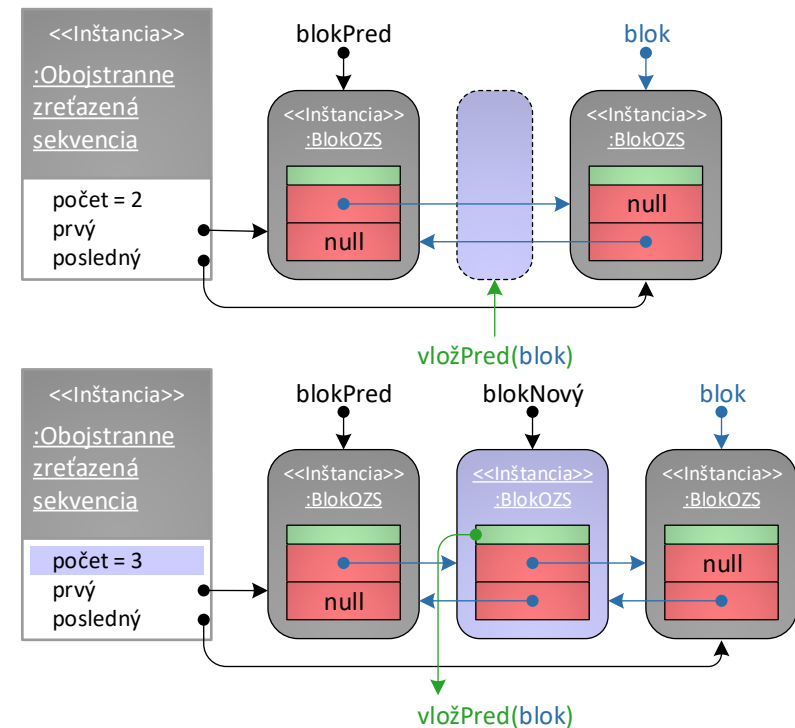
- Modifikátory explicitných sekvencií typicky nespôsobia posun prvkov v pamäti (pozor v na správcu kompaktnéj pamäte v prípade implementácie explicitných sekvencií v kompaktnéj pamäti, ktorý môže potrebovať zväčšiť pamäť, keď mu dôjde kapacita).
- Pre nové prvky stačí vytvoriť blok pamäte a zabezpečiť správnosť vzťahov v dotknutých blokoch pamäte.
- Efektívnosť modifikátorov je podmienená (ne)prítomnosťou referencie na *predchodcu* v blokoch pamäte a referencie na *posledný* prvok v riadiacom bloku štruktúry - nie je ich možné efektívne napísať univerzálne s odvolaním sa na jediný univerzálny modifikátor.
- Pre vytvorenie cyklickej explicitnej sekvencie stačí upraviť všetky modifikátory, ktoré upravujú hodnotu vlastností *prvý* a *posledný*, o volanie operácie *spoj* s parametrami *prvý* a *posledný*.

Explicitná sekvencia – modifikátory – vkladanie – príklad

vložZa



vložPred



Explicitná sekvencia – modifikátory – vkladanie – pseudokódy

Definícia operácie ES.vlož(int): $\uparrow$ TypBlok

```
1. operácia ES.vlož(index: int):  $\uparrow$ TypBlok {
2.   Keď platí (index = 0)
      tak vráť vložPrvý()
      inak vráť Keď platí (index = veľkosť())
      tak vráť vložPosledný()
      inak vráť vložZa(sprístupni(index - 1)↓)
3. }
```

Definícia operácie ES.vložPosledný(): $\uparrow$ TypBlok

```
1. operácia ES.vložPosledný():  $\uparrow$ TypBlok {
2.   Ak (veľkosť() = 0) potom {
3.     prvý  $\leftarrow$  posledný  $\leftarrow$  správcaPamäte→prideIPamät()
4.     Vráť posledný↓
5.   }
6.   inak {
7.     Vráť vložZa(posledný↓)
8.   }
9. }
```

Definícia operácie ES.vložZa($\uparrow$ TypBlok): $\uparrow$ TypBlok

```
1. operácia ES.vložZa(blok:  $\uparrow$ TypBlok):  $\uparrow$ TypBlok {
2.   definuj premennú nasledujúciBlok:  $\uparrow$ TypBlok  $\leftarrow$ 
      sprístupniNasledujúci(blok)
3.   definuj premennú novýBlok:  $\uparrow$ TypBlok  $\leftarrow$ 
      správcaPamäte→prideIPamät()
4.   spojBloky(blok, novýBlok)
5.   spojBloky(novýBlok, nasledujúciBlok)
6.   Ak (posledný = dajAdresu(blok)) potom {
7.     posledný  $\leftarrow$  novýBlok
8.   }
9.   Vráť novýBlok↓
10. }
```

Definícia operácie ES.vložPrvý(): $\uparrow$ TypBlok

```
1. operácia ES.vložPrvý():  $\uparrow$ TypBlok {
2.   Ak (veľkosť() = 0) potom {
3.     prvý  $\leftarrow$  posledný  $\leftarrow$  správcaPamäte→prideIPamät()
4.     Vráť prvý↓
5.   }
6.   inak {
7.     Vráť vložPred(prvý↓)
8.   }
9. }
```

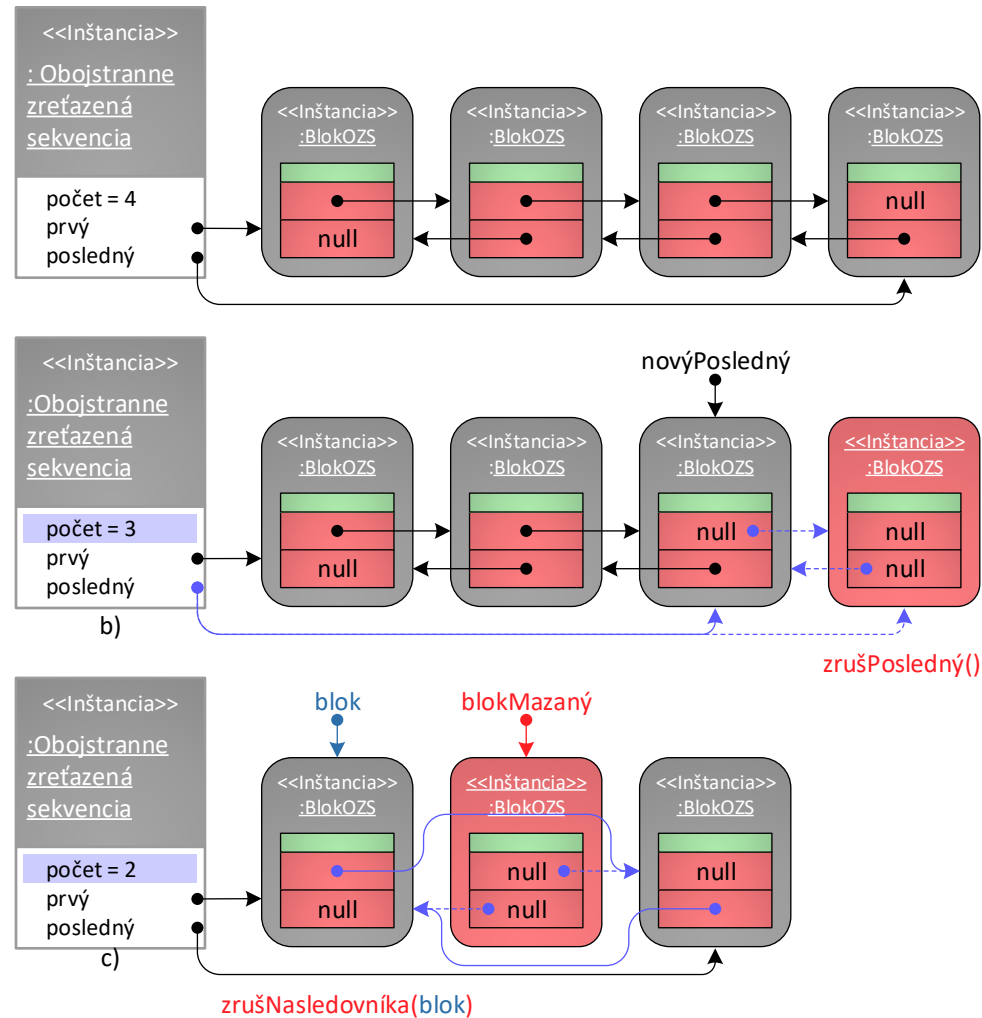
Definícia operácie ES.vložPred($\uparrow$ TypBlok): $\uparrow$ TypBlok

```
1. operácia ES.vložPred(blok:  $\uparrow$ TypBlok):  $\uparrow$ TypBlok {
2.   definuj premennú predchádzajúciBlok:  $\uparrow$ TypBlok  $\leftarrow$ 
      sprístupniPredchádzajúci(blok)
3.   definuj premennú novýBlok:  $\uparrow$ TypBlok  $\leftarrow$ 
      správcaPamäte→prideIPamät()
4.   spojBloky(predchádzajúciBlok, novýBlok)
5.   spojBloky(novýBlok, blok)
6.   Ak (prvý = dajAdresu(blok)) potom {
7.     prvý  $\leftarrow$  novýBlok
8.   }
9.   Vráť novýBlok↓
10. }
```

Explicitná sekvencia – modifikátory - mazanie

- Mazanie je možné realizovať inverzne k vkladaniu prvkov, vo všeobecnosti pozostávať z volania operácií **odpojBlok** (pre odstránenie prvku zo zreťazenia) a **uvolniPamäť**.
- Efektívnosť modifikátorov je opäť podmienená (ne)prítomnosťou referencie na *predchodcu* v blokoch pamäte a referencie na *posledný* prvok v riadiacom bloku štruktúry - nie je ich možné efektívne napísať univerzálne s odvolaním sa na jediný univerzálny modifikátor.

ES v nekompaktnej pamäti – modifikátory – mazanie – príklad



Explicitná sekvencia v kompaktnej pamäti – modifikátory - mazanie

- Ak sa jedná o explicitnú sekvenciu v kompaktnej pamäti, potom na rozdiel od implicitných sekvencií mazanie prvkov z konkrétneho indexu nemusí spôsobiť posun všetkých prvkov v kompaktnej pamäti:
 - **Neplatí**, že pozícia prvku v kompaktnej pamäti priamo vyjadruje index.
 - Keďže správca kompaktnej pamäte dokáže efektívne odstrániť iba prvky z konca spravovanej kompaktnej pamäte, je pred mazaním potrebné vymeniť mazaný prvok s posledným prvkom v kompaktnej pamäti (to nemusí byť nutne posledný prvok sekvencie).
 - Explicitná sekvencia v kompaktnej pamäti musí zabezpečiť správnosť vzťahov – je možné definovať operáciu opravSpojenie, ktorá opraví referencie v dotknutých blokoch pamäte.
 - Pre mazanie sú postupne volané operácie odpoj, vymeň, opravSpojenie a vráťPamäť.

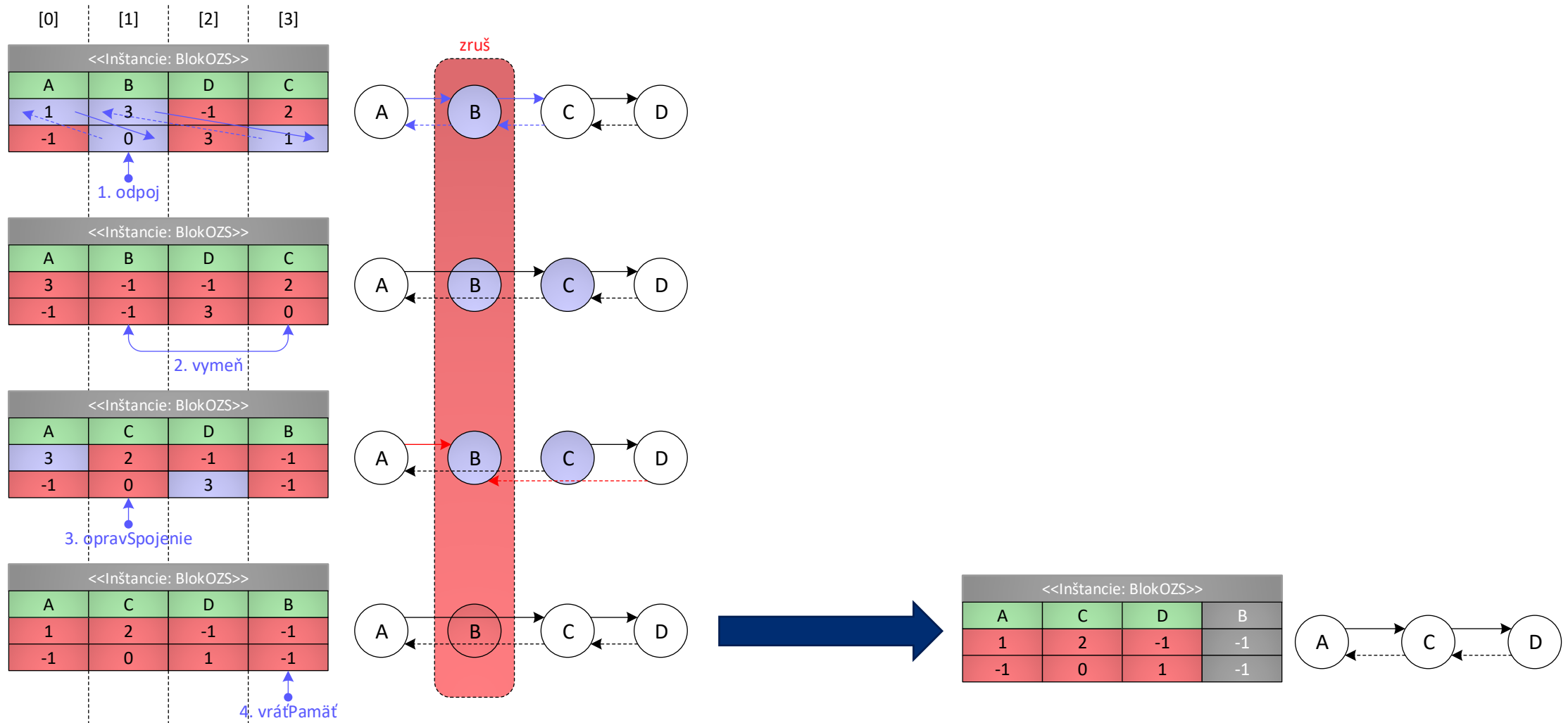
ES v kompaktnej pamäti – modifikátory – oprav spojenie

Názov operácie	Parametre	Návratová hodnota	Význam
opravSpojenie	int	-	Opravi vzťahy svojho nasledovníka a predchodcu na zmenenú adresu podľa parametra

Definícia operácie ES.opravSpopjenie(int)

```
1. operácia ES.opravSpopjenie(index: int) {
2.   definuj premennú blok: ↑TypBloku ←
      správcaPamäte→dajBlokPamäte(index)↓
3.   spoj(správcaPamäte→vypočítajPoradie(
      sprístupniPredchádzajúci(blok)↓), index)
4.   spoj(index, správcaPamäte→vypočítajPoradie(
      sprístupniNasledujúci(blok)↓))
5. }
```

ES v kompaktnej pamäti – modifikátory – mazanie – príklad



Explicitná sekvencia – modifikátory – mazanie – pseudokódy

Definícia operácie ES.zruš(int)

```
1. operácia ES.zruš(index: int) {
2.   Ak (index = 0) potom {
3.     zrušPrvý()
4.   }
5.   inak {
6.     zrušNasledovníka(sprístupni(index - 1)↓)
7.   }
8. }
```

Definícia operácie ES.zrušPrvý()

```
1. operácia ES.zrušPrvý() {
2.   Ak (prvý = posledný) potom {
3.     správcaPamäte→uvoľniPamäť(prvý)
4.     prvý ← posledný ← NULL
5.   }
6.   inak {
7.     definuj premennú novýPrvý: ↑TypBloku ←
       sprístupniNasledujúci(prvý↓)
8.     správcaPamäte→uvoľniPamäť(prvý)
9.     prvý ← novýPrvý
10.  }
11. }
```

Definícia operácie ES.zrušNasledovníka(↑TypBloku)

```
1. operácia ES.zrušNasledovníka(blok: ↑TypBloku) {
2.   definuj premennú mazanýBlok: ↑TypBloku ←
       sprístupniNasledujúci(blok)
3.   Ak (mazanýBlok = posledný) potom {
4.     zrušPosledný()
5.   }
6.   inak {
7.     odpojBlok(mazanýBlok)
8.     správcaPamäte→uvoľniPamäť(mazanýBlok)
9.   }
10. }
```

Definícia operácie ObojstranneZS.zrušPrvý()

```
1. operácia ObojstranneZS.zrušPrvý() {
2.   ES.zrušPrvý()
3.   Ak (prvý ≠ NULL) potom {
4.     prvý→predchodca ← NULL
5.   }
6. }
```

Definícia operácie ES.zrušPosledný()

```
1. operácia ES.zrušPosledný() {
2.   Ak (prvý = posledný) potom {
3.     správcaPamäte→uvoľniPamäť(posledný)
4.     prvý ← posledný ← NULL
5.   }
6.   inak {
7.     definuj premennú novýPosledný: ↑TypBloku ←
       sprístupniPredchádzajúci(posledný↓)
8.     správcaPamäte→uvoľniPamäť(posledný)
9.     posledný ← novýPosledný
10.    posledný→nasledovník ← NULL
11.  }
12. }
```

Definícia operácie ES.zrušPredchodcu(↑TypBloku)

```
1. operácia ES.zrušPredchodcu(blok: ↑TypBloku) {
2.   definuj premennú mazanýBlok: ↑TypBloku ←
       sprístupniPredchádzajúci(blok)
3.   Ak (mazanýBlok = prvý) potom {
4.     zrušPrvý()
5.   }
6.   inak {
7.     odpojBlok(mazanýBlok)
8.     správcaPamäte→uvoľniPamäť(mazanýBlok)
9.   }
10. }
```

Explicitná sekvencia – modifikátory – zložitosti

Modifikátory			n: počet prvkov
Operácia	Zložitosť		
Vlož prvý	O(1)		
Vlož posledný	Obsahuje v riadiacom bloku referenciu na posledný?		
	áno	nie	
	O(1)	O(n)	
Vlož	Obsahuje v riadiacom bloku referenciu na posledný a v bloku pamäte referenciu na predchádzajúci?		
	áno	nie	
	$\frac{1}{2} * O(n)$	O(n)	
Vlož za	O(1)		
Vlož pred	Obsahuje v bloku pamäte referenciu na predchádzajúci?		
	áno	nie	
	O(1)	O(n)	

Modifikátory			
Operácia	Zložitosť		Parametre
Zruš prvý	O(1)		n: počet prvkov
Zruš posledný	Obsahuje v riadiacom bloku referenciu na posledný a v bloku pamäte referenciu na predchádzajúci??		
	áno	nie	
	O(1)	O(n)	
Zruš	Obsahuje v riadiacom bloku referenciu na posledný a v bloku pamäte referenciu na predchádzajúci?		
	áno	nie	
	$\frac{1}{2} * O(n)$	O(n)	
Zruš nasledovníka	O(1)		
Zruš predchodcu	Obsahuje v bloku pamäte referenciu na predchádzajúci?		
	áno	nie	
	O(1)	O(n)	

Explicitná sekvencia – iterátory

- Iterátor explicitnej sekvencie je založený na všeobecnom iterátore nekompaktnej pamäte, ktorý obsahuje jeden atribút:
 - referenciu na aktuálny prvok.
- Explicitné sekvencie nedokážu efektívne implementovať iterátory s náhodným prístupom, pretože sa bloky pamäte nenachádzajú v kompaktnej pamäti za sebou, a teda neplatí, že poloha bloku pamäte od nejakej bázevej adresy vyjadruje jeho index.
- Efektívnosť implementácie obojsmerných iterátorov je podmienená existenciou referencie na *predchodcu* v blokoch pamäte.
- Zložitosť všetkých operácií sú $O(1)$.
- Explicitná sekvencia vytvára iterátory tak, že inicializuje atribút *pozícia* ako referenciu na správny prvok:
 - Prvý iterátor: referencia na prvý platný prvok je uložená v atribúte *prvý*.
 - Posledný iterátor: referencia na prvý neplatný prvok je NULL.

Zhrnutie prednášky

- Sekvencie a ich iterátory
 - Implicitná sekvencia
 - Explicitná sekvencia

Plán na cvičenie

- Implicitná sekvencia.

Ďakujem za pozornosť

Doplňujúce informácie?
Vaše otázky?

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu 6UI0004 Algoritmy a údajové štruktúry 1 na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline.
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené.

Ing. Michal Varga, PhD.
Katedra informatiky
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
Michal.Varga@fri.uniza.sk

Algoritmy a údajové štruktúry 1

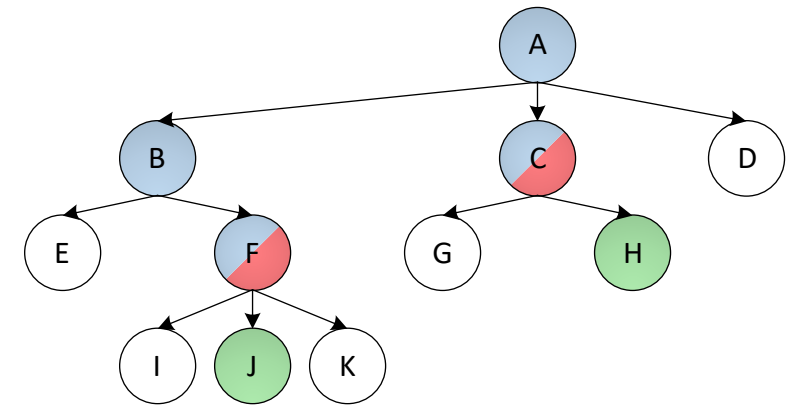
Prednáška č. 5

- Hierarchie a ich iterátory
 - Implicitná hierarchia
 - Explicitná hierarchia



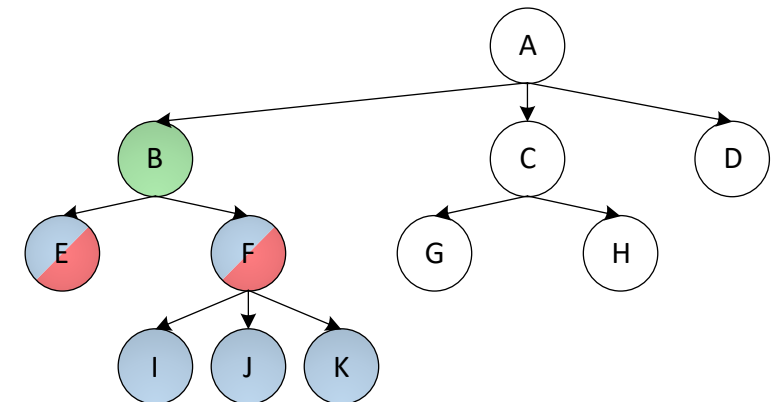
Hierarchické abstraktné pamäťové typy (hAPT) – hierarchie

- Bloky pamäte sa označujú **vrcholy**.
- Pre ľubovoľné dva vrcholy je možné jednoznačne definovať ich vzťah **predok** – **potomok**.
 - Priamy predok vrcholu sa nazýva **otec**,
 - priamy potomok vrcholu sa nazýva **syn**.



vizualizácia vzťahov otec a predok v hierarchii

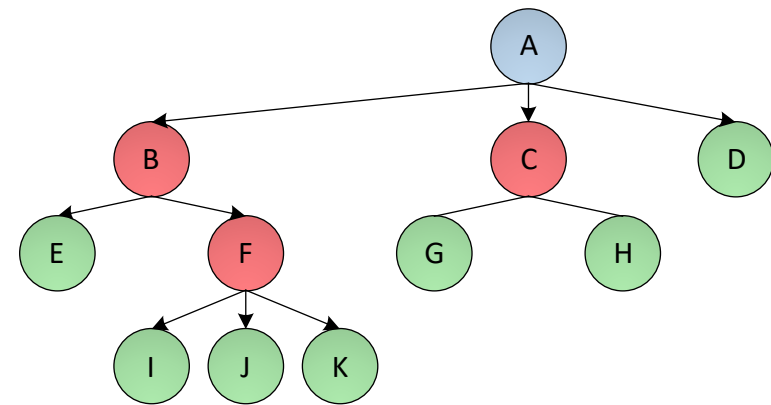
- Otec (H) = C Predkovia (H) = {C, A}
- Otec (J) = F Predkovia (J) = {F, B, A}
- Synovia (B) = {E, F}
- Potomkovia (B) = {E, F, I, J, K}



vizualizácia vzťahov syn a potomok v hierarchii

Klasifikácia vrcholov v hierarchiách

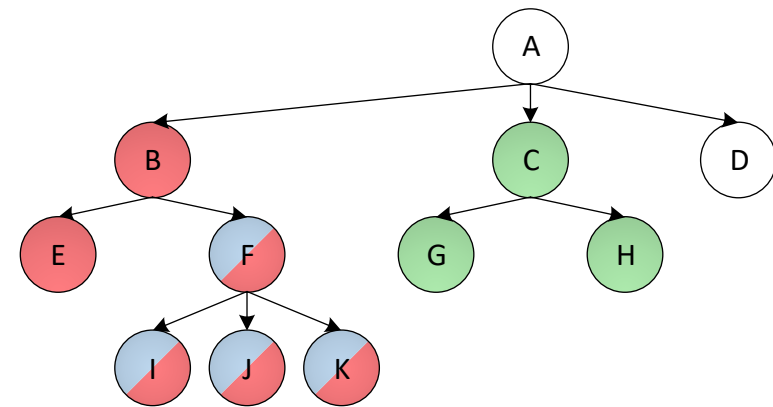
- Na základe vzťahov medzi vrcholmi rozlišujeme tri druhy vrcholov:
 - **Koreň** – nemá otca.
 - **List** – nemá synov.
 - **Vnútorňý vrchol** – má synov a otca.
- Koreň = 
- Listy = {, , , , , , 
- Vnútorňé vrcholy = {, , 



Podhierarchie

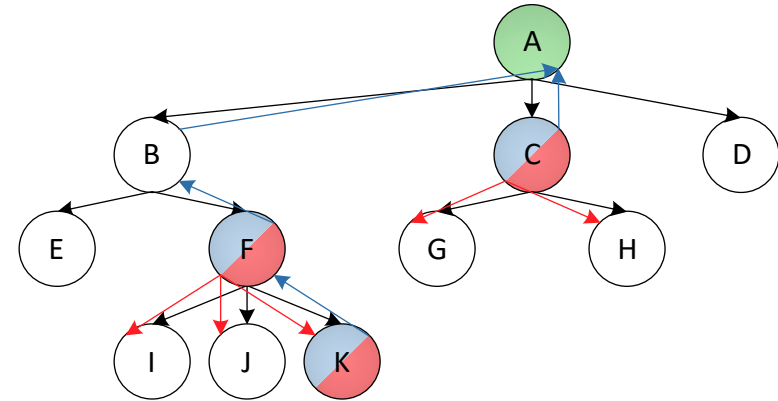
- Každý vrchol môžeme považovať za koreň vlastnej hierarchie.
- Každá hierarchia môže mať najviac jeden koreň.
- Celkový počet listov a vnútorných vrcholov nie je obmedzený.

- Hierarchia (C) = {C, G, H}
- Hierarchia (B) = {B, E, F, I, J, K}
- Hierarchia (F) = {F, I, J, K}



Stupeň a úroveň vrcholu, hĺbka a mohutnosť hierarchie

- **Stupeň** vrcholu vyjadruje počet synov.
- **Úroveň** vrcholu predstavuje jeho najkratšiu vzdialenosť od koreňa. Koreň je v úrovni 0.
- **Hĺbka** hierarchie (charakterizovanej koreňom) je maximálna úroveň vrcholu.
- **Mohutnosť** hierarchie (charakterizovanej koreňom) vyjadruje počet vrcholov v hierarchii.

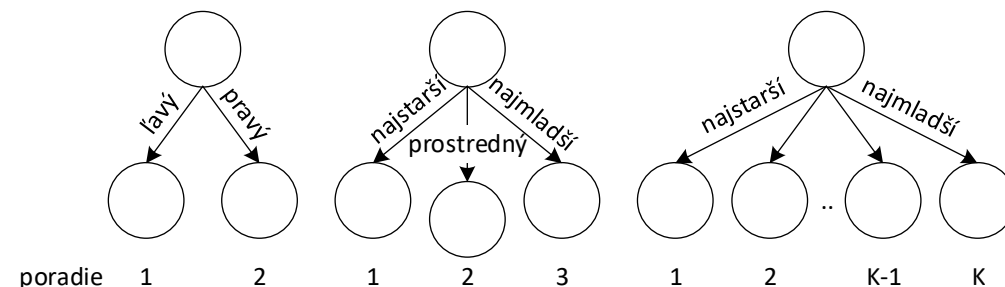


- Stupeň (C) = 2 Úroveň (C) = 1
- Stupeň (F) = 3 Úroveň (F) = 2
- Stupeň (K) = 0 Úroveň (K) = 3
- Hĺbka (A) = 3
- Mohutnosť (A) = 11

Hierarchie podľa obmedzenia stupňa vrcholu

- **K-cestné hierarchie** – každý vrchol má najviac K synov. K -cestné hierarchie môžeme ďalej špecifikovať a hovoriť o
 - binárnych hierarchiách ($K = 2$),
 - trojcestných hierarchiách ($K = 3$),
 - štvorcestných hierarchiách ($K = 4$), atď.
- **Viaccestné hierarchie** – počet synov vrcholu nie je obmedzený. Môžeme povedať, že sa jedná o špeciálny prípad k -cestnej hierarchie, kde $K = \infty$.

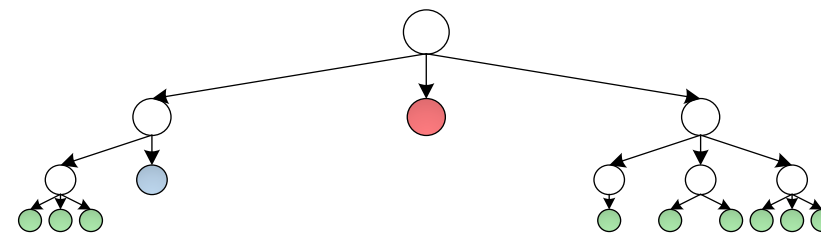
- K -cestné hierarchie je možné ďalej klasifikovať na:
 - **Usporiadané K -cestné hierarchie** – synovia tvoria lineárne usporiadanú množinu. Synov je potom možné pomenovať, napr. najstarší syn, jeho mladší brat, najmladší syn; ľavý syn a pravý syn; prvý, druhý, tretí..



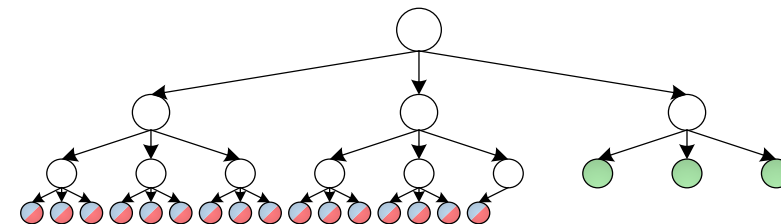
- **Neusporiadané K -cestné hierarchie** – synovia netvoria lineárne usporiadanú množinu.

Hierarchie podľa topológie

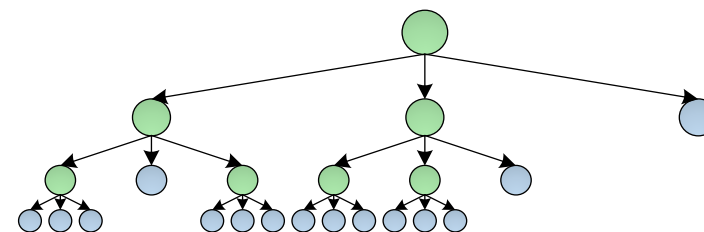
- Hierarchia je **vyvážená**, keď každý list nie je od koreňa vzdialený „o mnoho viac“ (faktor vyváženosti je potrebné definovať – napr. o 2) ako ľubovoľný iný list.
- Pre K-cestné hierarchie môžeme určiť ďalšie vlastnosti:
 - Kompletnosť**: všetky listy majú hĺbku n alebo $n-1$ a všetky listy s hĺbkou n ležia **vľavo**.
 - Plnosť**: každý vrchol má buď práve n synov alebo 0 synov (teda je list).
 - Perfektnosť**: Všetky listy sú v rovnakej hĺbke a všetky vnútorné vrcholy majú stupeň n .



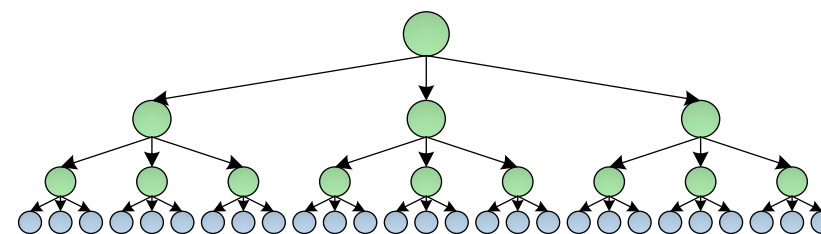
Vyvážená hierarchia



Kompletná trojcestná hierarchia

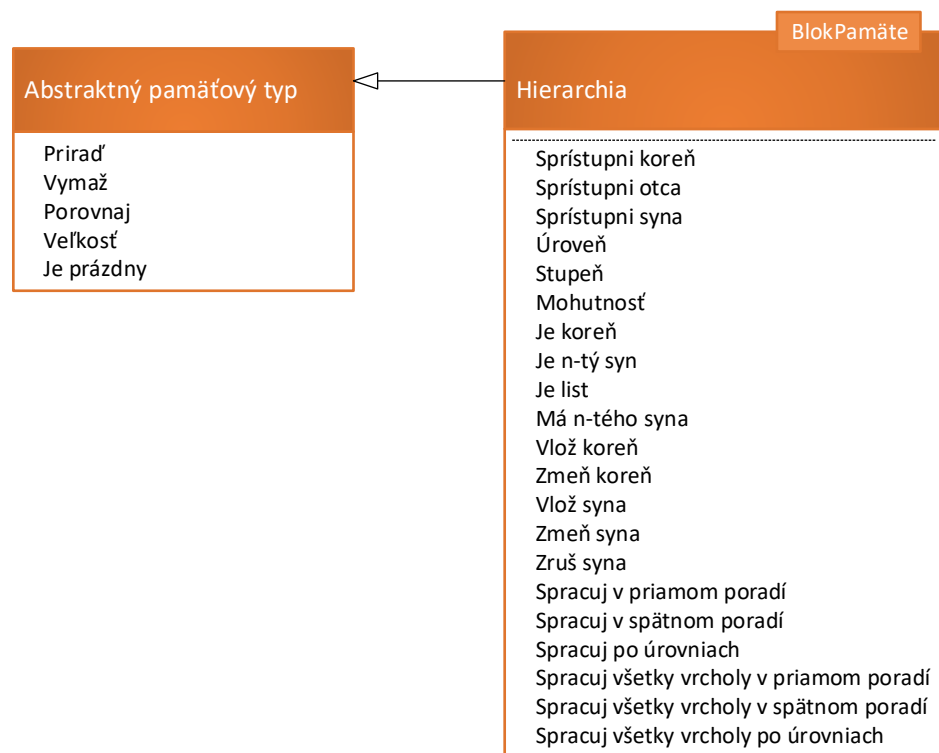


Plná trojcestná hierarchia



Perfektná trojcestná hierarchia

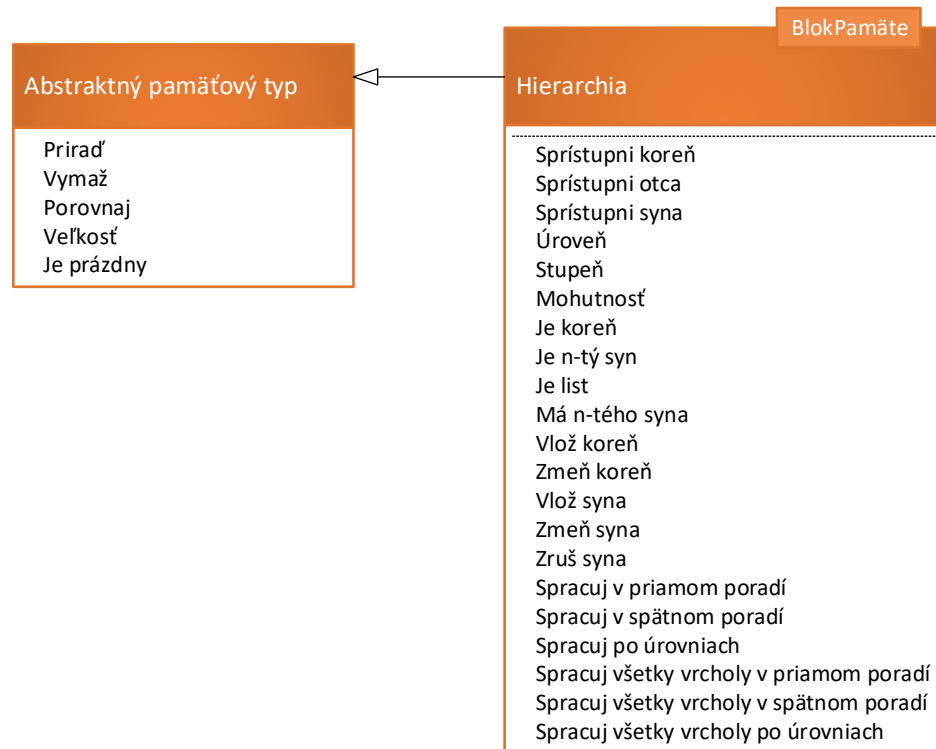
Hierarchie



Selektory			
Názov selektora	Parametre	Návratová hodnota	Selektor sprístupní
Sprístupni koreň	-	↑TypBloku	Koreň hierarchie
Sprístupni otca	↑TypBloku		Otca bloku odovzdaného ako parameter.
Sprístupni syna	↑TypBloku int		Syna bloku odovzdaného ako parameter s daným poradím.

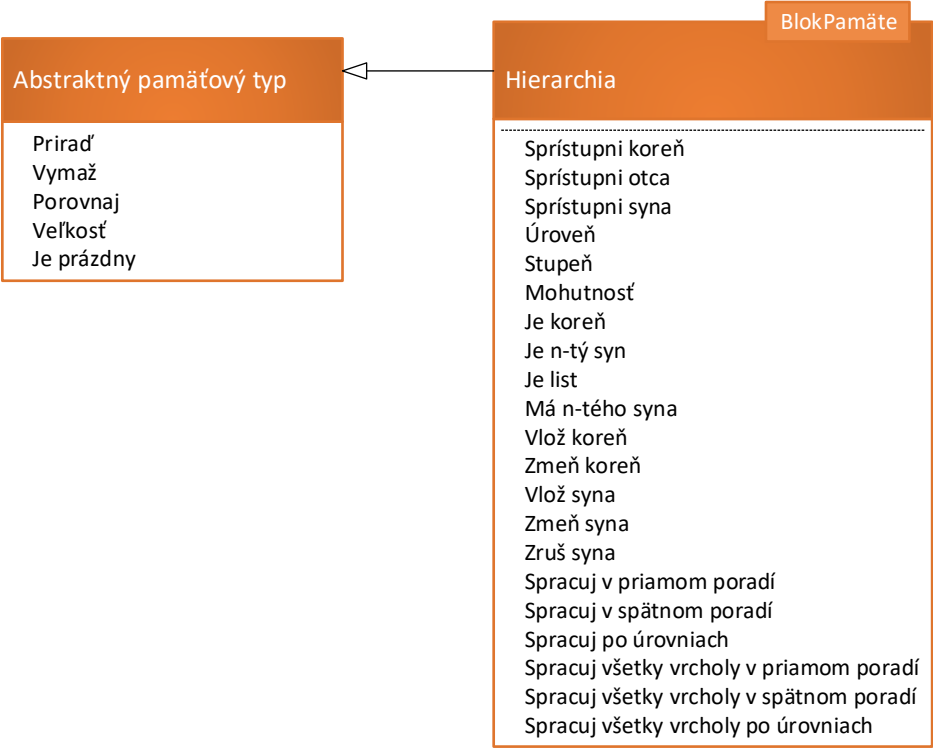
Dotazy			
Názov dotazu	Parametre	Návratová hodnota	Dotaz vráti
Úroveň	↑TypBloku	int	Úroveň vrcholu odovzdaného ako parameter.
Stupeň	↑TypBloku		Stupeň vrcholu odovzdaného ako parameter.
Mohutnosť	↑TypBloku		Počet vrcholov v hierarchii, ktorej koreň je odovzdaný ako parameter.
Je koreň	↑TypBloku	bool	Príznak, či je vrchol koreňom hierarchie.
Je N-tý syn	↑TypBloku int		Príznak, či je vrchol n-tým synom svojho otca.
Je list	↑TypBloku		Príznak, či je vrchol listom.
Má N-tého syna	↑TypBloku int		Príznak, či má vrchol n-tého syna.

Hierarchie



Modifikátory			
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor vykoná
Vlož koreň	-	↑TypBloku	Vloží nový koreň do hierarchie. Pôvodný koreň nedealokuje.
Zmeň koreň	↑TypBloku	-	Ustanoví nový koreň hierarchie. Pôvodný koreň nedealokuje.
Vlož syna	↑TypBloku int	↑TypBloku	Vloží nového syna na miesto s daným poradím (syn bude mať u otca požadované poradie)
Zmeň syna	↑TypBloku int ↑TypBloku	-	Zmení syna s daným poradím. Pôvodného syna nedealokuje.
Zruš syna	↑TypBloku int	-	Zruší syna s daným poradím.

Hierarchie



Prehliadky			
Názov prehliadky	Parametre	Návratová hodnota	Princíp prehliadky
Spracuj v priamom poradí	$\uparrow \text{TypBloku}$ $\lambda(\uparrow \text{TypBloku})$	-	Pomocou operácie z parametra spracuje v priamom poradí vrcholy hierarchie, ktorej koreň je odovzdaný ako parameter.
Spracuj v spätnom poradí			Pomocou operácie z parametra spracuje v spätnom poradí vrcholy hierarchie, ktorej koreň je odovzdaný ako parameter.
Spracuj po úrovniach			Pomocou operácie z parametra spracuje v poradí po úrovniach vrcholy hierarchie, ktorej koreň je odovzdaný ako parameter.
Spracuj všetky vrcholy v priamom poradí	$\lambda(\uparrow \text{TypBloku})$	-	Pomocou operácie z parametra spracuje v priamom poradí všetky vrcholy hierarchie.
Spracuj všetky vrcholy v spätnom poradí			Pomocou operácie z parametra spracuje v spätnom poradí všetky vrcholy hierarchie.
Spracuj všetky vrcholy po úrovniach			Pomocou operácie z parametra spracuje v poradí po úrovniach všetky vrcholy hierarchie.

K-cestné hierarchie

- Pre K-cestné hierarchie môžeme zaviesť samostatné rozhranie, ktoré špecifikuje K.

BlokPamäte, K

<<Rozhranie>>

K-cestná hierarchia

Hierarchia
- Usporiadané K-cestné hierarchie typicky implementujú operácie pracujúce so synmi vrcholov tak, aby presne pomenovali syna, ktorého sa operácia týka.
- Potomok K-cestnej hierarchie operácie implementuje jednoducho volaním operácie predka so správnym parametrom (vyjadrujúcim poradie syna).
- Príklad pre binárne hierarchie vpravo.

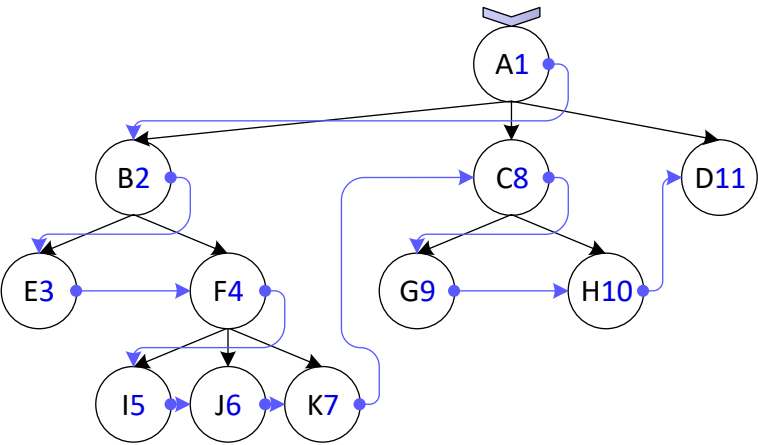
Selektory			
Názov selektora	Parametre	Návratová hodnota	Selektor sprístupní
Sprístupni ľavého syna	↑TypBloku	↑TypBloku	Ľavého syna bloku odovzdaného ako parameter.
Sprístupni pravého syna			Pravého syna bloku odovzdaného ako parameter.
Dotazy			
Názov dotazu	Parametre	Návratová hodnota	Dotaz vráti
Je ľavý syn	↑TypBloku	bool	Príznak, či je vrchol ľavým synom svojho otca.
Je pravý syn			Príznak, či je vrchol pravým synom svojho otca.
Má ľavého syna			Príznak, či má vrchol ľavého syna.
Má pravého syna			Príznak, či má vrchol pravého syna.
Modifikátory			
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor vykoná
Vlož ľavého syna	↑TypBloku	↑TypBloku	Vloží otcovi ľavého syna.
Vlož pravého syna			Vloží otcovi pravého syna.
Zmeň ľavého syna	↑TypBloku ↑TypBloku	-	Zmení ľavého syna. Pôvodného syna nedealokuje.
Zmeň pravého syna			Zmení pravého syna. Pôvodného syna nedealokuje.
Zruš ľavého syna	↑TypBloku	-	Zruší ľavého syna vrcholu.
Zruš pravého syna			Zruší pravého syna vrcholu.

Prehliadky hierarchií

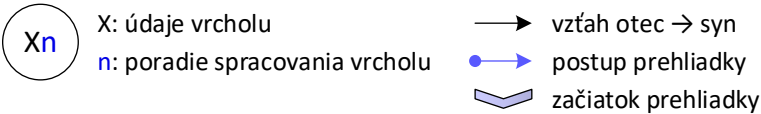
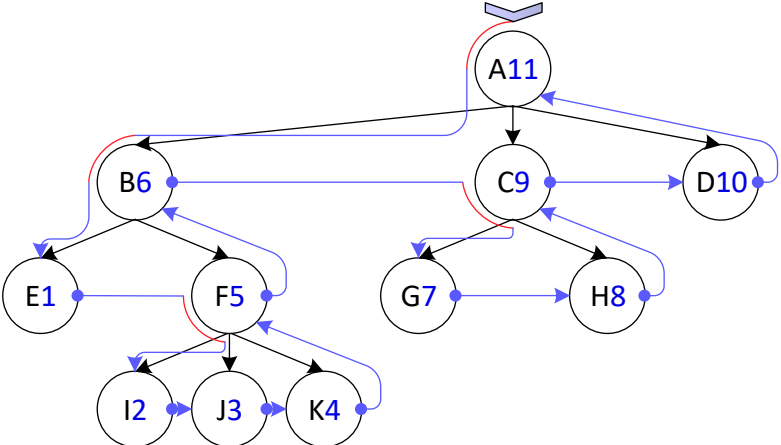
- Možné do **hĺbky** a do **šírky**.
- Prehliadky do **hĺbky** sú založené na rekurzívnom prehliadaní podhierarchií. Prehliadka do hĺbky sa skladá z dvoch častí:
 1. spracovanie vrcholu (vykonanie operácie nad daným vrcholom) a
 2. následné spustenie prehliadky na synoch vrcholu.
- Vo všeobecnosti existujú dve prehliadky do hĺbky, ktoré sa líšia poradím krokov:
 - **Prehliadka v priamom poradí** (preorder) – vykoná kroky v poradí 1 a 2.
 - **Prehliadka v spätnom poradí** (postorder) – vykoná kroky v poradí 2 a 1.
- Implementačne sa nelíši prehliadka **implicitnej** a **explicitnej** hierarchie.

Prehliadky hierarchií do hĺbky

prehliadka v priamom poradí



prehliadka v spätnom poradí



Prehliadky hierarchií do hĺbky

prehliadka v priamom poradí

Definícia operácie `Hierarchia<TypBloku>.spracujVPriamomPoradí(↑TypBloku, λ(↑TypBloku))`

```
1. operácia Hierarchia<TypBloku>.spracujVPriamomPoradí(  
    vrchol: ↑TypBloku,  
    operácia: λ(↑TypBloku)  
  ) {  
2.   Ak (vrchol ≠ NULL) potom {  
3.     operácia(vrchol)  
4.     definuj premennú stupeňVrcholu: int ← stupeň(vrchol↓)  
5.     definuj premennú poradieSyna: int ← 0  
6.     definuj premennú početSpracovanýchSynov: int ← 0  
7.     Pokiaľ (početSpracovanýchSynov < stupeňVrcholu) opakuje {  
8.       definuj premennú syn: ↑TypBloku ←  
9.         sprístupniSyna(vrchol↓, poradieSyna)  
10.      Ak (syn ≠ NULL) potom {  
11.        spracujVPriamomPoradí(syn, operácia)  
12.        početSpracovanýchSynov++  
13.      }  
14.      poradieSyna++  
15.    }  
16.  }  
17. }
```

Definícia operácie `Hierarchia<TypBloku>.spracujVšetkyVrcholyVPriamomPoradí (λ(↑TypBloku))`

```
1. operácia Hierarchia<TypBloku>.  
    spracujVšetkyVrcholyVPriamomPoradí(  
    operácia: λ(↑TypBloku)  
  ) {  
2.   spracujVPriamomPoradí(sprístupniKoreň(), operácia)  
3. }
```

prehliadka v spätnom poradí

Definícia operácie `Hierarchia<TypBloku>.spracujVSpätnomPoradí(↑TypBloku, λ(↑TypBloku))`

```
1. operácia Hierarchia<TypBloku>.spracujVSpätnomPoradí(  
    vrchol: ↑TypBloku,  
    operácia: λ(↑TypBloku)  
  ) {  
2.   Ak (vrchol ≠ NULL) potom {  
3.     definuj premennú stupeňVrcholu: int ← stupeň(vrchol↓)  
4.     definuj premennú poradieSyna: int ← 0  
5.     definuj premennú početSpracovanýchSynov: int ← 0  
6.     Pokiaľ (početSpracovanýchSynov < stupeňVrcholu) opakuje {  
7.       definuj premennú syn: ↑TypBloku ←  
8.         sprístupniSyna(vrchol↓, poradieSyna)  
9.       Ak (syn ≠ NULL) potom {  
10.        spracujVSpätnomPoradí(syn, operácia)  
11.        početSpracovanýchSynov++  
12.      }  
13.      poradieSyna++  
14.    }  
15.    operácia(vrchol)  
16.  }  
17. }
```

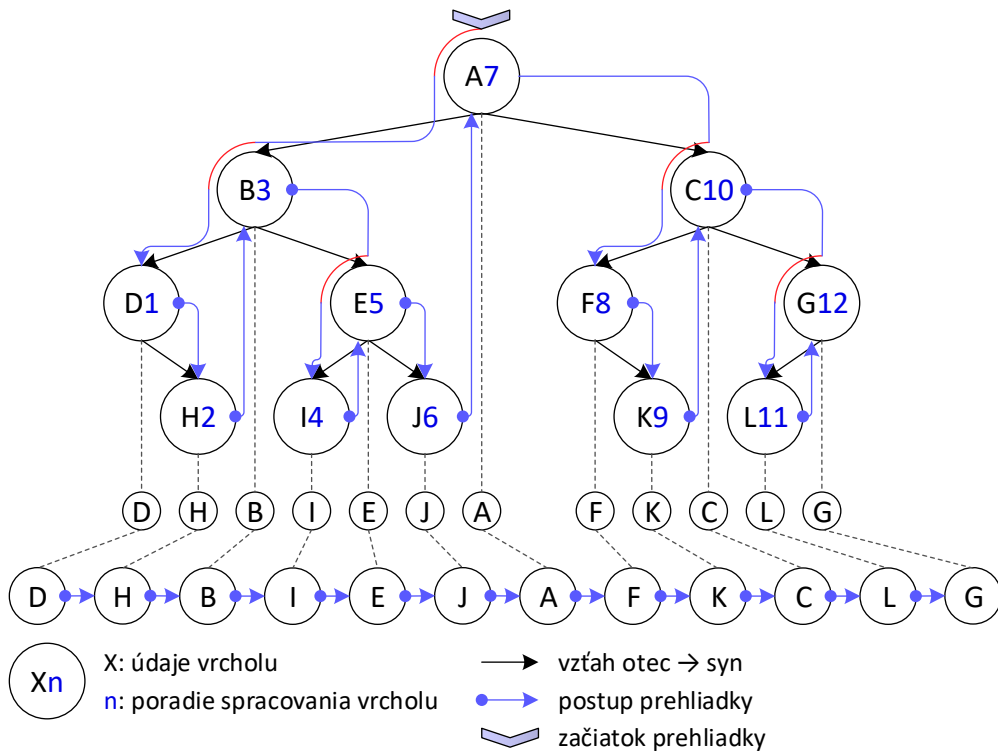
Definícia operácie `Hierarchia <TypBloku>.spracujVšetkyVrcholyVSpätnomPoradí (λ(↑TypBloku))`

```
1. operácia Hierarchia<TypBloku>.  
    spracujVšetkyVrcholyVSpätnomPoradí (  
    operácia: λ(↑TypBloku)  
  ) {  
2.   spracujVSpätnomPoradí(sprístupniKoreň(), operácia)  
3. }
```

Prehliadky hierarchií do hĺbky – binárna hierarchia

- Binárna hierarchia umožňuje ďalší druh prehliadky do hĺbky – **prehliadku vo vnútornom poradí** (inorder)
- Vykonanie operácie nad vrcholom je vsunuté medzi spracovanie ľavej a pravej podhierarchie vrcholu.
- Takáto prehliadka spracuje vrcholy v poradí „zľava doprava“.

Prehliadky hierarchií do hĺbky – binárna hierarchia



Definícia operácie

BinárnaHierarchia<TypBlok>.spracujVoVnútoranomPoradí(↑TypBlok, λ(↑TypBlok))

```

1. operácia BinárnaHierarchia<TypBlok>.spracujVoVnútoranomPoradí(
   vrchol: ↑TypBlok,
   operácia: λ(↑TypBlok)
) {
2.   Ak (vrchol ≠ NULL) potom {
3.     spracujVoVnútoranomPoradí(sprístupniĽavéhoSyna(vrchol↓),
                                   operácia)
4.     operácia(vrchol)
5.     spracujVoVnútoranomPoradí(sprístupniPravéhoSyna(vrchol↓),
                                   operácia)
6.   }
7. }
    
```

Definícia operácie

BinárnaHierarchia<TypBlok>.spracujVšetkyVrcholyVoVnútoranomPoradí(λ(↑TypBlok))

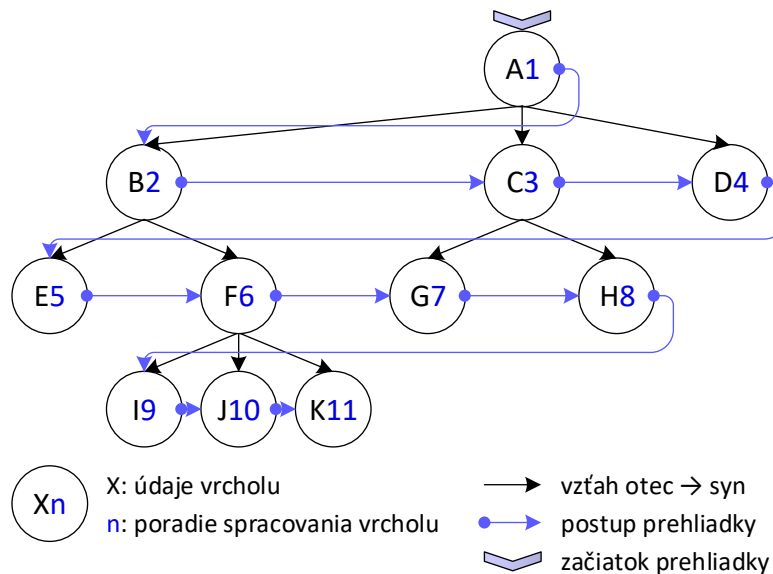
```

1. operácia BinárnaHierarchia<TypBlok>.
   spracujVšetkyVrcholyVoVnútoranomPoradí(
   operácia: λ(↑TypBlok)
) {
2.   spracujVoVnútoranomPoradí(sprístupniKoreň(), operácia)
3. }
    
```

Prehliadky hierarchií do šírky

- Prehliadka do šírky spracováva vrcholy postupne po jednotlivých úrovniach „zľava doprava“.
- Často býva označovaná ako **prehliadka po úrovniach** (levelorder).

Prehliadky hierarchií do šírky



Definícia operácie *Hierarchia<TypBloku>.spracujPoÚrovniach*(↑TypBloku, λ(↑TypBloku))

```

1. operácia Hierarchia<TypBloku>.spracujPoÚrovniach(
    vrchol: ↑TypBloku,
    operácia: λ(↑TypBloku)
) {
2.   Ak (vrchol ≠ NULL) potom {
3.     definuj premennú sekvencia: JednostranneZS<↑TypBloku>
4.     sekvencia→vložPrvý()→dáta ← vrchol
5.     Pokiaľ (¬sekvencia→jePrázdny()) opakuje {
6.       definuj premennú aktuálny: ↑TypBloku ←
           sekvencia→sprístupniPrvý()→dáta
7.       sekvencia→zrušPrvý()
8.       Ak (aktuálny ≠ NULL) potom {
9.         operácia(aktuálny)
10.        definuj premennú stupeňVrcholu: int ←
            stupeň(aktuálny↓)
11.        Opakuje pre premennú n: int od 0 do stupeňVrcholu - 1 {
12.          sekvencia→vložPosledný()→dáta ←
            sprístupniSyna(aktuálny↓, n)
13.        }
14.      }
15.    }
16.  }
17. }
    
```

Definícia operácie *Hierarchia<TypBloku>.spracujVšetkyVrcholyPoÚrovniach*(λ(↑TypBloku))

```

1. operácia Hierarchia<TypBloku>.spracujVšetkyVrcholyPoÚrovniach(
    operácia: λ(↑TypBloku)
) {
2.   spracujPoÚrovniach(sprístupniKoreň(), operácia)
3. }
    
```

Hierarchie – dotazy (vrcholy)

- Dotazy **jeKoreň**, **jeNTýSyn** a **máNTéhoSyna** je možné implementovať univerzálne pre všetky **hierarchie**, ktoré ukladajú množinu synov v **blokoch** pamäte do **sekvencie**, s využitím volaní iných selektorov alebo dotazov.
- Efektívnu implementáciu selektorov potom špecifikuje konkrétna **hierarchia**.
- Zložitosť operácie **jeList** závisí od operácie **stupeň** – nie je ju možné univerzálne implementovať (vrcholy môžu ukladať vzťahy rôzne), preto sa zložitosť operácie **jeList** bude zhodovať s neskôr uvedenou operáciou **stupeň**.
- **úroveň** je možné chápať výpočet indexu vrcholu v pomyselnej **explicitnej** **sekvencii** (zreťazenie v smere syn → otec).

Dotazy		
Operácia	Zložitosť	Parametre
Je koreň	O(1)	-
Je N-tý syn		
Má N-tého syna		
Úroveň	O(h)	h: hĺbka hierarchie

Hierarchie – dotazy (vrcholy) – pseudokódy

Definícia operácie *Hierarchia<TypBloku>.jeKoreň(↑TypBloku): bool*

```
1. operácia Hierarchia<TypBloku>.jeKoreň(  
    vrchol: ↑TypBloku  
): bool {  
2.   Vráť sprístupniOtca(vrchol) = NULL  
3. }
```

Definícia operácie *Hierarchia<TypBloku>.jeList(↑TypBloku): bool*

```
1. operácia Hierarchia<TypBloku>.jeList(  
    vrchol: ↑TypBloku  
): bool {  
2.   Vráť stupeň(vrchol) = 0  
3. }
```

Definícia operácie *Hierarchia<TypBloku>.úroveň(vrchol: ↑TypBloku): int*

```
1. operácia Hierarchia<TypBloku>.úroveň(vrchol: ↑TypBloku): int {  
2.   definuj premennú úroveň: int ← 0  
3.   definuj premennú otec: ↑TypBloku ← sprístupniOtca(vrchol)  
4.   Pokiaľ (otec ≠ NULL) opakuje {  
5.     úroveň++  
6.     otec ← sprístupniOtca(otec↓)  
7.   }  
8.   Vráť úroveň  
9. }
```

Definícia operácie *Hierarchia<TypBloku>.jeNTýSyn(↑TypBloku, int): bool*

```
1. operácia Hierarchia<TypBloku>.jeNTýSyn(  
    vrchol: ↑TypBloku,  
    poradieSyna: int  
): bool {  
2.   definuj premennú otec: ↑TypBloku ← sprístupniOtca(vrchol)  
3.   Vráť otec ≠ NULL ∧  
        sprístupniSyna(otec↓, poradieSyna) = dajAdresu(vrchol)  
4. }
```

Definícia operácie *Hierarchia<TypBloku>.máNTéhoSyna(↑TypBloku, int): bool*

```
1. operácia Hierarchia<TypBloku>.máNTéhoSyna(  
    vrchol: ↑TypBloku,  
    poradieSyna: int  
): bool {  
2.   Vráť sprístupniSyna(vrchol, poradieSyna) ≠ NULL  
3. }
```

Hierarchie – dotazy (hierarchia)

- Na **mohutnosť** je možné využiť ľubovoľnú prehliadku – veľmi drahá operácia (aj všetky operácie, ktoré ju použijú – napr. **veľkosť**)!!!
- Ak sa dá, vyvarujte sa jej (pozri implementáciu dotazu **jePrázdny**).

Základné operácie s hierarchiou		
Operácia	Zložitosť	Parametre
Mohutnosť	$O(n)$	n : počet prvkov v hierarchii
Veľkosť		
Je prázdny	$O(1)$	-

Hierarchie – dotazy (hierarchia) – pseudokódy

Definícia operácie *Hierarchia*<TypBlok>.veľkosť(): int

```
1. operácia Hierarchia<TypBlok>.veľkosť(): int {  
2.   Vrátť mohutnosť(sprístupniKoreň())  
3. }
```

Definícia operácie *Hierarchia*<TypBlok>.jePrázdny(): bool

```
1. operácia Hierarchia<TypBlok>.jePrázdny(): bool {  
2.   Vrátť sprístupniKoreň() = NULL  
3. }
```

Definícia operácie *Hierarchia*.mohutnosť(↑BlokPamäte): int

```
1. operácia Hierarchia<TypBlok>.mohutnosť(  
2.   vrchol: ↑BlokPamäte  
3. ): int {  
4.   definuj λ spočítaj(vrchol: ↑TypBlok) → int {  
5.     Ak (vrchol ≠ NULL) tak {  
6.       definuj premennú počet: int ← 1  
7.       Opakuj pre premennú i: int od 0 do stupeň(vrchol↓) - 1 {  
8.         počet ← počet + spočítaj(sprístupniSyna(vrchol↓, i))  
9.       }  
10.      Vrátť počet  
11.    }  
12.    inak {  
13.      Vrátť 0  
14.    }  
15.  }  
16.  Vrátť spočítaj(sprístupniKoreň())  
17. }
```

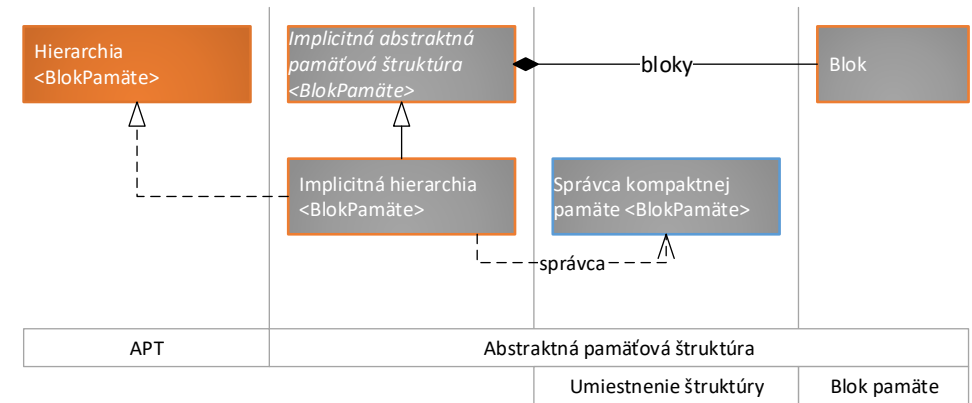
Alternatíva operácie *mohutnosť* s využitím prehliadky:

Zjednodušená definícia operácie *Hierarchia*<TypBlok>.mohutnosť(↑TypBlok): int

```
1. operácia Hierarchia<TypBlok>.mohutnosť(  
2.   vrchol: ↑TypBlok  
3. ): int {  
4.   definuj premennú počet: int ← 0  
5.   spracujVPriamomPoradí(vrchol, λ(b: ↑TypBlok) { počet++ })  
6.   Vrátť počet  
7. }
```

Implicitná hierarchia

- Musí byť v kompaktnej pamäti, aby bolo možné vypočítať vzťahy otec $\leftrightarrow$ syn.
- **Možné len usporiadané K-cestné hierarchie!** Toto obmedzenie zaručí, že bude možné získať presné poradie každého vrcholu v súvislej pamäti a tým bude možné určiť vzťahy pre získanie otca resp. synov vrcholu.

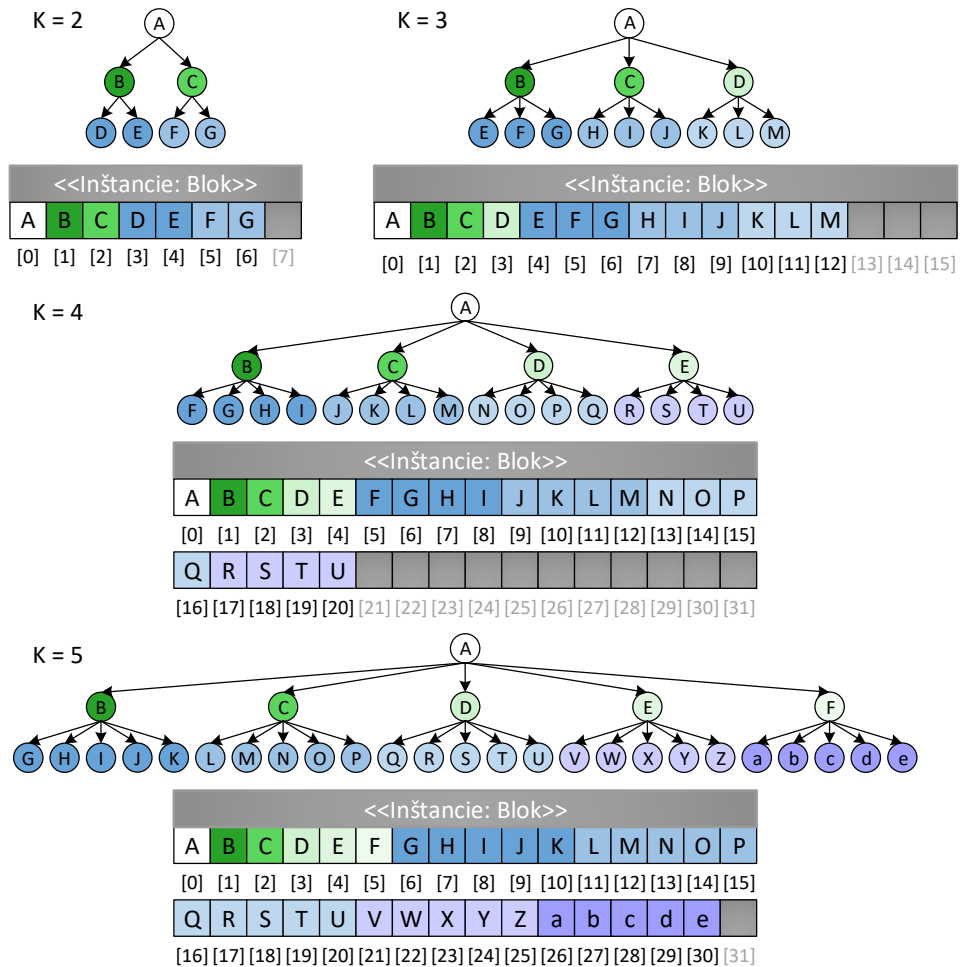


Implicitná hierarchia – výpočet vzťahov

K = 2		K = 3		K = 4		K = 5		
Index (poradie v súvislej pamäti)								
otca	synov	otca	synov	otca	synov	otca	synov	
0	1	0	1	0	1	0	1	
	2		2		2			
1	3		3		3		3	
	4	1	4		4		4	
2	5		1	5	5			
	6			6	6			
		3		10	1	7	1	7
				11		8		8
			12	9		9		
				4	17	5		10
					18		26	
					19		27	
				20	28			
							29	
							30	

Index synov S_1 až S_K vrcholu K -cestnej s indexom i je možné v súvislej pamäti vypočítať podľa nasledujúcich vzťahov:

K = 2	K = 3	K = 4	K = 5
$S_1(i) = 2 * i + 1$ $S_2(i) = 2 * i + 2$	$S_1(i) = 3 * i + 1$ $S_2(i) = 3 * i + 2$ $S_3(i) = 3 * i + 3$	$S_1(i) = 4 * i + 1$ $S_2(i) = 4 * i + 2$ $S_3(i) = 4 * i + 3$ $S_4(i) = 4 * i + 4$	$S_1(i) = 5 * i + 1$ $S_2(i) = 5 * i + 2$ $S_3(i) = 5 * i + 3$ $S_4(i) = 5 * i + 4$ $S_5(i) = 5 * i + 5$



Implicitná hierarchia – výpočet vzťahov

- Pre vypočítanie indexu syna v súvislej pamäti pre akúkoľvek K -cestnú hierarchiu je potrebná informácia o **indexe** otca (i) a o **poradí** (nie indexe) syna u svojho otca (p).
- Úpravou vzťahu pre výpočet indexu syna je možné vyjadriť vzťah pre sprístupnenie indexu otca v súvislej pamäti, ak je známy index syna (i) v súvislej pamäti. Informácia o poradí syna u otca (p) nie je potrebná, nakoľko ju dokáže zastúpiť operácia celočíselné delenie.

Názov operácie	Parametre	Návratová hodnota	Význam
Index otca	int	int	Vráti index otca vrcholu v súvislej pamäti.
	↑ImplicitnáAPS.TypBloku		
Index syna	int	int	Vráti index syna v súvislej pamäti.
	↑ImplicitnáAPS.TypBloku int		

$$\text{Syn}(i, p) = K * i + p$$

K = druh hierarchie

i = index otca v súvislej pamäti

p = poradie syna u svojho otca; $p \in \langle 1; K \rangle$

$$\text{Otec}(i) = (i - 1) \text{ div } K$$

K = druh hierarchie

i = index syna v súvislej pamäti

$$\text{Úroveň}(i) = \lfloor \log_K((K - 1) * (i + 1)) \rfloor$$

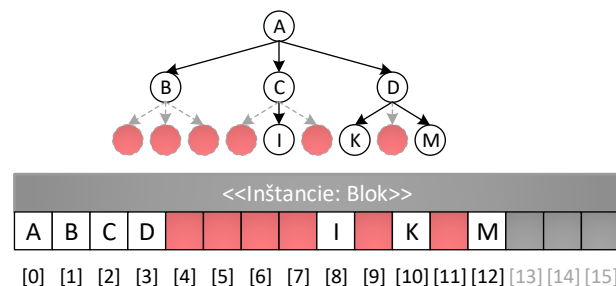
K = druh hierarchie

i = index vrcholu v súvislej pamäti

Základné operácie s implicitnou hierarchiou		
Operácia	Zložitosť	Parametre
Index otca (obe verzie)	O(1)	-
Index syna (obe verzie)		

Implicitná hierarchia – výpočet vzťahov

- Implicitne je možné implementovať iba usporiadané K-cestné hierarchie, keďže je vzťah pre získanie indexov vrcholov pevne daný a opiera sa o dané K.
- Ak vrchol nemá niektorého syna, bude pamäťové miesto vyhradené, hoci nebude používané (tu je rozdiel od implicitných sekvencií, kde sa nepoužívané pamäťové miesta prepisovali posunutím nasledujúcich blokov pamäte).
- Aby nedochádzalo k zbytočnému plytvaniu pamäťovým miestom, musí byť hierarchia naviac **kompletná**.



Príklad pamäťovo neefektívnej ternárnej hierarchie.

Implicitná hierarchia – selektory

- Selektory je možné implementovať pomocou vzťahov pre sprístupnenie otca resp. synov.
- Ako implicitnú je možné implementovať iba kompletnú hierarchiu - efektívne pracuje iba s posledným listom (najpravejší list poslednej úrovne).
- Je vhodné pridať operáciu **sprístupniPoslednýList**:

Názov selektora	Parametre	Návratová hodnota	Selektor sprístupní
Sprístupni posledný list	-	↑ImplicitnáAPS.TypBloku	List na poslednej úrovni najviac vpravo.

Selektory		
Operácia	Zložitosť	Parametre
Sprístupni koreň	O(1)	-
Sprístupni otca		
Sprístupni syna		
Sprístupni posledný list		

Implicitná hierarchia – selektory – pseudokódy

Definícia operácie *IH.sprístupniKoreň()*: $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IH.sprístupniKoreň():  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Keď platí (veľkosť() > 0)  
      tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(0))  
      inak vráť NULL  
3. }
```

Definícia operácie *IH.sprístupniPoslednýList()*: $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IH.sprístupniPoslednýList(  
  ):  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   definuj premennú veľkosť: int  $\leftarrow$  veľkosť()  
3.   Keď platí (veľkosť  $\neq$  0)  
      tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(  
        veľkosť - 1))  
      inak vráť NULL  
4. }
```

Definícia operácie

IH.sprístupniOtca($\uparrow$ ImplicitnáAPS.TypBloku): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IH.sprístupniOtca(  
  vrchol:  $\uparrow$ ImplicitnáAPS.TypBloku  
):  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   definuj premennú index: int  $\leftarrow$  indexOtca(vrchol)  
3.   Keď platí (index  $\neq$  NEPLATNÝ_INDEX)  
      tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(index))  
      inak vráť NULL  
4. }
```

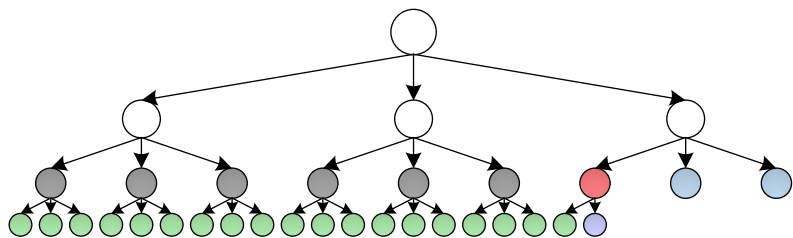
Definícia operácie

IH.sprístupniSyna($\uparrow$ ImplicitnáAPS.TypBloku, int): $\uparrow$ ImplicitnáAPS.TypBloku

```
1. operácia IH.sprístupniSyna(  
  vrchol:  $\uparrow$ ImplicitnáAPS.TypBloku,  
  poradieSyna: int  
):  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   definuj premennú index: int  $\leftarrow$   
      indexSyna(vrchol, poradieSyna)  
3.   Keď platí (index < veľkosť())  
      tak vráť dajAdresu(správcaPamäte→dajBlokPamäte(index))  
      inak vráť NULL  
4. }
```

Implicitná hierarchia – dotazy

- Dotazy je možné jednoducho implementovať s využitím predtým definovaných vzťahov a pomocou vlastností **správca** **kompaktnej** **pamäte**.
- Pre **stupeň** je možné oprieť sa o kompletnosť **implicitnej** **hierarchie**:
 - **zelené** vrcholy - vrchol sa nachádza v poslednej úrovni, musí byť list a má stupeň 0.
 - **biele** vrcholy - vrchol sa nachádza pred predposlednou úrovňou, potom musí mať K synov.
 - Špeciálne ošetriť preto stačí predposlednú úroveň a situáciu, kedy by bol vrchol otcom (**červený** vrchol) „posledného“ listu (**fialový** vrchol) v **hierarchii**:
 - **šedé** vrcholy (od červeného naľavo) musia mať stupeň K.
 - **modré** vrcholy (of červeného napravo) musia mať stupeň 0.
 - **červený** vrchol - využijeme operáciu modulo.



Dotazy		
Operácia	Zložitosť	Parametre
Úroveň (obe verzie)	O(1)	-
Stupeň (obe verzie)		
Mohutnosť	O(1)	vrchol je koreň
	O(n)	n – počet vrcholov v hierarchii vrcholu, vrchol nie je koreň

Implicitná hierarchia – dotazy – pseudokódy

Definícia operácie $IH.stupeň(int): int$

```
1. operácia  $IH.stupeň(index: int): int$  {
2.   definuj premennú aktuálnaÚroveň:  $int \leftarrow úroveň(index)$ 
3.   definuj premennú indexPosledného:  $int \leftarrow veľkosť() - 1$ 
4.   definuj premennú hĺbka:  $int \leftarrow úroveň(indexPosledného)$ 
5.   Ak (aktuálnaÚroveň = hĺbka) potom {
6.     Vráť 0
7.   }
8.   inak {
9.     Ak (aktuálnaÚroveň = hĺbka - 1) potom {
10.      definuj premennú indexOtcaPosledného:  $int \leftarrow$ 
        indexOtca(indexPosledného)
11.      Ak (index < indexOtcaPosledného) potom {
12.        Vráť K
13.      }
14.      inak {
15.        Ak (index > indexOtcaPosledného) potom {
16.          Vráť 0
17.        }
18.        inak {
19.          definuj premennú mod:  $int \leftarrow (veľkosť() - 1) \bmod K$ 
20.          Keď platí (mod = 0)
            tak vráť K
            inak vráť mod
21.        }
22.      }
23.    }
24.    inak {
25.      Vráť K
26.    }
27.  }
28. }
```

Definícia operácie $IH.stupeň(\uparrow ImplicitnáAPS.TypBloku): int$

```
1. operácia  $IH.stupeň(vrchol: \uparrow ImplicitnáAPS.TypBloku): int$  {
2.   Vráť  $stupeň(správcaPamäte \rightarrow vypočítajPoradie(vrchol))$ 
3. }
```

Definícia operácie $IH.úroveň(int): int$

```
1. operácia  $IH.úroveň(index: int): int$  {
2.   Vráť  $\text{floor}(\log((K - 1) * (index + 1)) / \log(K))$ 
3. }
```

Definícia operácie $IH.úroveň(\uparrow ImplicitnáAPS.TypBloku): int$

```
1. operácia  $IH.úroveň(vrchol: \uparrow ImplicitnáAPS.TypBloku): int$  {
2.   Vráť  $úroveň(správcaPamäte \rightarrow vypočítajPoradie(vrchol))$ 
3. }
```

Zefektívnenie operácie mohutnosť:

Definícia operácie $IH.mohutnosť(\uparrow ImplicitnáAPS.TypBloku): int$

```
1. operácia  $IH.mohutnosť(vrchol: \uparrow ImplicitnáAPS.TypBloku): int$  {
2.   Keď platí ( $správcaPamäte \rightarrow vypočítajPoradie(vrchol) = 0$ )
        tak vráť veľkosť()
        inak vráť  $Hierarchia<ImplicitnáAPS.TypBloku>.mohutnosť(vrchol)$ 
3. }
```

Implicitná hierarchia – modifikátory

- Výrazne limitovaná množina - musíme zabezpečiť, že hierarchia bude kompletná.
- Namiesto univerzálnych modifikátorov iba 2, ktoré pracujú na konci kompaktnej pamäte
- Pripomienka: správca kompaktnej pamäte automaticky nevracia nepoužívané miesto.

Modifikátory		
Operácia	Zložitosť	Parametre
Vlož posledný list	$O(b)$	b: $\lceil n/B \rceil$ B: veľkosť buffra (koľko prvkov sa do neho zmestí) n: počet prvkov
Zruš posledný list	$O(1)$	

Implicitná hierarchia – modifikátory – pseudokódy

Definícia operácie *IH.vložPoslednýList()*: $\uparrow$ ImplicitnáAPS.TypBloku

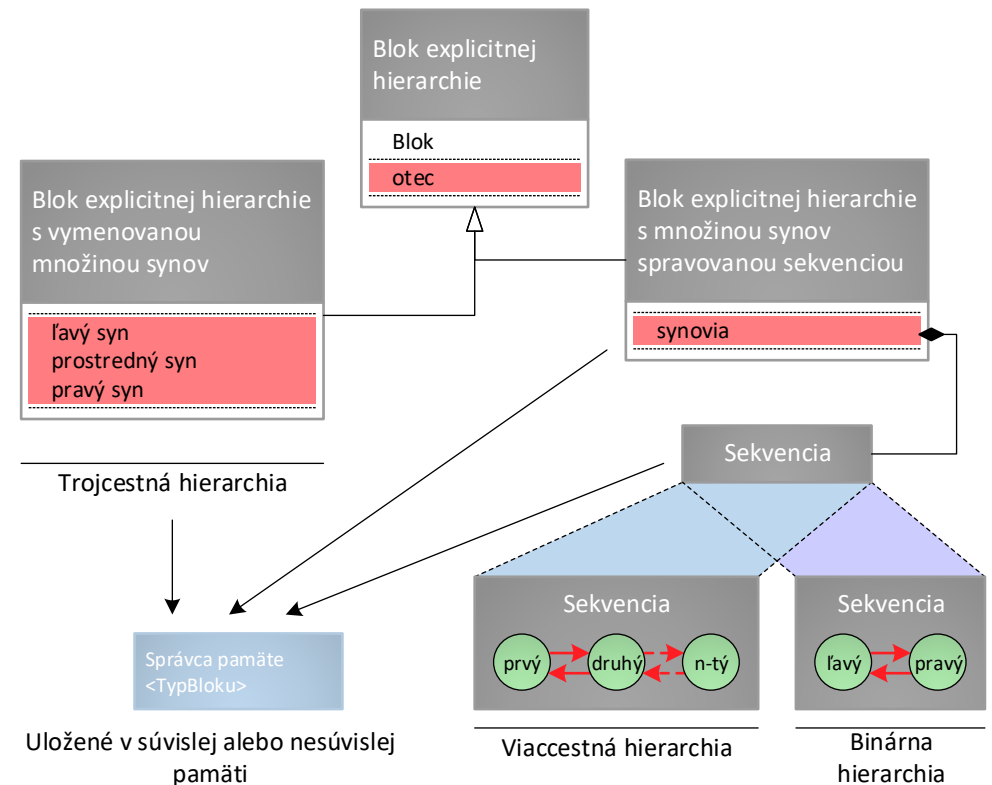
```
1. operácia IH.vložPoslednýList():  $\uparrow$ ImplicitnáAPS.TypBloku {  
2.   Vráť správcaPamäte  $\rightarrow$  pridajPamäť()  $\downarrow$   
3. }
```

Definícia operácie *IH.odstráňPoslednýList()*

```
1. operácia IH.odstráňPoslednýList() {  
2.   správcaPamäte  $\rightarrow$  uvoľniPamäť()  
3. }
```

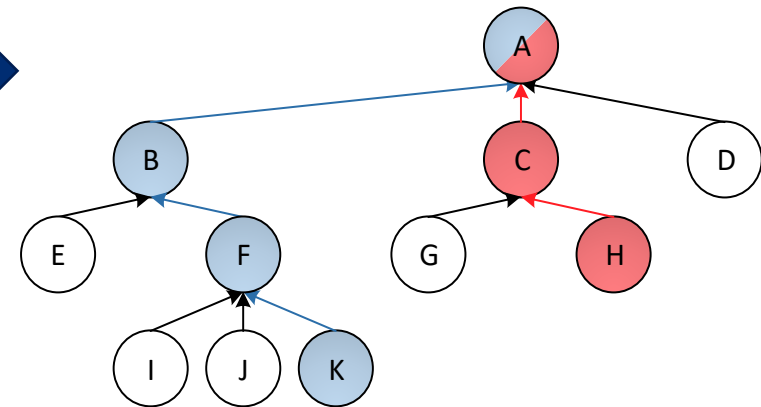
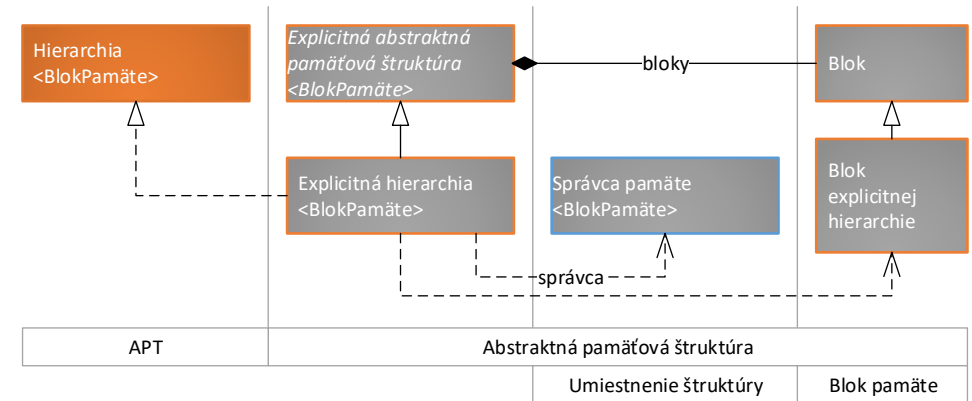
Explicitná hierarchia

- Riadiaci blok **explicitnej hierarchie** typicky ukladá referencie na *koreň*.
- Bloky pamäte typicky uchovávajú referencie na *otca* a na *synov*.
- Spôsob organizácie množiny synov určuje, či je možné hierarchie uložiť v externej pamäti:
 - Viaccestné hierarchie ($K = \infty$) – množina synov je neobmedzená – nie je to možné.
 - K – cestné hierarchie:
 - Ak je použitá iná APS – nie je to možné.
 - Ak sú vymenované vo vrchole – je to možné.
- Pozor na pamäťovú efektívnosť!



Explicitná hierarchia

- Explicitné hierarchie je možné do veľkej miery navrhnuť jednotne.
- Líšia sa iba v reprezentácii množiny synov vo vrcholoch – úloha potomkov.
- Vrcholy spolu s explicitne vyjadrenými vzťahmi so svojimi otcami tvoria explicitnú sekvenciu – preto bude možné operácie explicitných hierarchií, ktoré sú založené na práci s otcom vrcholu, založiť na podobných algoritmoch, ako operácie explicitných sekvencií.



Explicitná hierarchia – základné operácie

- **Priradenie:** zahodia sa všetky údaje (operácia **vymaž**) a následne sa pridajú všetky prvky z parametrom odovzdanej **explicitnej hierarchie** - na to je možné využiť prehliadku v priamom poradí (zabezpečí, že pri pridávaní vrcholu bude určite existovať jeho otec).
- **Vymazanie:** Využije sa prehliadka v spätnom poradí, spracovaním vrcholu je samotné vymazanie (ak by sa vrchol najskôr vymazal, synovia by už neboli dostupní).

Základné operácie s hierarchiou		
Operácia	Zložitosť	Parametre
Prirad'	$O(n + m)$	n – pôvodný počet prvkov m – nový počet prvkov
Vymaž	$O(n)$	n – počet prvkov

Explicitná hierarchia – základné operácie – pseudokódy

Definícia operácie EH.prirad'(↑APT): ↑APT

```
1. operácia EH.prirad'(iný: ↑APT): ↑APT {
2.   Ak (¬(iný je typu EH)) potom {
3.     CHYBA
4.   }
5.   Ak (self = dajAdresu(iný)) potom {
6.     Vráť self↓
7.   }
8.   definuj λ kopíruj(môjBlok: ↑TypBlok, inýBlok: ↑TypBlok) {
9.     môjBlok→dáta ← inýBlok→dáta
10.    definuj premennú početSynov: int ← iný→stupeň(inýBlok↓)
11.    definuj premennú početSpracovanýchSynov: int ← 0
12.    definuj premennú indexSyna: int ← 0
13.    Pokiaľ (početSpracovanýchSynov < početSynov) opakuj {
14.      definuj premennú inýSyn: ↑TypBlok ←
15.        iný→sprístupniSyna(inýBlok↓, indexSyna)
16.      Ak (inýSyn ≠ NULL) potom {
17.        kopíruj(dajAdresu(vložSyna(môjBlok↓, indexSyna)),
18.          inýSyn)
19.      }
20.    }
21.    vymaž()
22.    Ak (iný→koreň ≠ NULL) potom {
23.      vložKoreň()
24.      kopíruj(koreň, iný→koreň)
25.    }
26.    Vráť self↓
27. }
```

Definícia operácie EH.vymaž()

```
1. operácia EH.vymaž() {
2.   definuj λ vymažVrchol(vrchol: ↑TypBlok) {
3.     správcaPamäte→uvoľniPamäť(vrchol)
4.   }
5.   spracujVSpätnomPoradí(koreň, vymažVrchol)
6.   koreň ← NULL
7. }
```

Explicitná hierarchia – selektory

- Veľmi jednoduché, vzťahy sú priamo vyjadrené.
 - a) Ak sú uložené v **sekvencii** s (ne)obmedzenou kapacitou, stačí získať údaje z príslušnej **sekvencie** – zložitosť sa odvíja od použitej implementácie.
 - b) Ak sú vzťahy vymenované, je v univerzálnom selektore potrebné identifikovať správneho syna.
 - c) Vymenovaní synovia umožňujú vytvoriť špecializované, veľmi efektívne selektory.

Selektory		
Operácia	Zložitosť	Parametre
Sprístupni otca Sprístupni koreň	O(1)	-
Sprístupni syna	O(1)	synovia sú uložení v implicitnej sekvencii alebo sú vymenovaní v bloku pamäte
	O(s)	s – stupeň vrcholu, synovia sú uložení v explicitnej sekvencii

Explicitná hierarchia – selektory – pseudokódy

a)

Definícia operácie	
ViaccestnáEH.sprístupniSyna($\uparrow$ TypBlok, int): $\uparrow$ TypBlok	
1.	operácia ViaccestnáEH.sprístupniSyna(vrchol: $\uparrow$ TypBlok, poradieSyna: int): $\uparrow$ TypBlok {
2.	definuj premennú blokSynov: $\uparrow$ BlokPamäte< $\uparrow$ TypBlok> $\leftarrow$ vrchol $\rightarrow$ synovia $\rightarrow$ sprístupni(poradieSyna)
3.	Ked' platí (blokSynov $\neq$ NULL) tak vráť blokSynov $\rightarrow$ dáta inak vráť NULL
4.	}

b)

Definícia operácie	
BinárnaEH.sprístupniSyna($\uparrow$ TypBlok, int): $\uparrow$ TypBlok	
1.	operácia BinárnaEH.sprístupniSyna(vrchol: $\uparrow$ TypBlok, poradieSyna: int): $\uparrow$ TypBlok {
2.	Ked' (poradieSyna) nadobúda {
3.	hodnotu INDEX_LAVÉHO_SYNA tak {
4.	Vráť vrchol $\rightarrow$ ľavý
5.	}
6.	hodnotu INDEX_PRAVÉHO_SYNA tak {
7.	Vráť vrchol $\rightarrow$ pravý
8.	}
9.	žiadnu z uvedených hodnôt {
10.	Vráť NULL
11.	}
12.	}
13.	}

c)

Definícia operácie	
BinárnaEH.sprístupniĽavéhoSyna($\uparrow$ TypBlok): $\uparrow$ TypBlok	
1.	operácia BinárnaEH.sprístupniĽavéhoSyna(vrchol: $\uparrow$ TypBlok): $\uparrow$ TypBlok {
2.	Vráť vrchol $\rightarrow$ ľavý
3.	}

Explicitná hierarchia – dotazy – porovnanie

- Simultánna prehliadka dvoch hierarchií.
- Prehliadka musí byť v rovnakom poradí, avšak nezabezpečí rovnakú topológiu!
- Rovnako ako v prípade sekvencií, ani dve hierarchie nemôžu byť rovnaké, ak neobsahujú rovnaký počet prvkov. Na rozdiel od explicitných sekvencií však explicitné hierarchie túto hodnotu neevidujú a jej získanie by bolo rovnako časovo zložité, ako porovnanie všetkých prvkov.

Dotazy		
Operácia	Zložitosť	Parametre
Porovnaj	$O(n)$	n: počet prvkov

Explicitná hierarchia – dotazy – porovnanie – pseudokódy

Definícia operácie $EH.porovnaj(\uparrow APT): bool$

```
1. operácia EH.porovnaj(iný:  $\uparrow APT$ ): bool {
2.   Ak ( $\neg$ (iný je typu EH)) potom {
3.     Vráť nepravda
4.   }
5.   Ak (self = dajAdresu(iný)) potom {
6.     Vráť pravda
7.   }
8.   definuj  $\lambda$  porovnanie(
9.     môjBlok:  $\uparrow TypBlok$ ,
10.    inýBlok:  $\uparrow TypBlok$ 
11.  )  $\rightarrow bool$  {
12.    Ak (môjBlok = NULL  $\wedge$  inýBlok = NULL) potom {
13.      Vráť pravda
14.    }
15.    inak Ak (môjBlok = NULL  $\vee$  inýBlok = NULL) potom {
16.      Vráť nepravda
17.    }
18.    inak Ak (stupeň(môjBlok $\downarrow$ )  $\neq$ 
19.      ináHierarchia $\rightarrow$ stupeň(inýBlok $\downarrow$ )) potom {
20.      Vráť nepravda
21.    }
22.    inak Ak (môjBlok $\rightarrow$ dáta  $\neq$  inýBlok $\rightarrow$ dáta) potom {
23.      Vráť nepravda
24.    }
25.    inak {
26.      definuj premennú maxStupeň: int  $\Leftarrow$  stupeň(môjBlok $\downarrow$ )
27.      Opakuj pre premennú i: int od 0 do maxStupeň - 1 {
28.        Ak ( $\neg$ porovnanie(sprístupniSyňa(môjBlok $\downarrow$ , i),
29.          iný $\rightarrow$ sprístupniSyňa(inýBlok $\downarrow$ , i))) potom {
30.          Vráť nepravda
31.        }
32.      }
33.    }
34.    Vráť pravda
35.  }
36. }
37. Vráť porovnanie(koreň, iný $\rightarrow$ koreň)
```

Explicitná hierarchia – dotazy – stupeň

- Je potrebné rozlíšiť, či sa jedná o usporiadané alebo neusporiadané hierarchie.
 - a) Synovia vrcholov v **neusporiadaných** hierarchiách sú typicky ukladaní v **sekvencií**, čím stačí požiadať **sekvenciu** o aktuálny počet synov v nej.
- Ak sa jedná o **usporiadanú** hierarchiu, potom:
 - b) ak sú synovia ukladaní v **sekvencií** s danou kapacitou (alebo sú vymenovaní v bloku pamäte), a potom nemusí byť niektorá z referencií nastavená (má hodnotu NULL). V takomto prípade je potrebné počet platných referencií zrátať.
 - c) ak sú priamo vymenovaní, treba ich postupne otestovať a zrátať.

Dotazy		
Operácia	Zložitosť	Parametre
Stupeň	O(1)	v neusporiadaných hierarchiách
	O(K)	v usporiadaných K-cestných hierarchiách

Explicitná hierarchia – dotazy – stupeň – pseudokódy

a)

Definícia operácie *ViaccestnáEH.stupeň*(↑TypBloku): int

```
1. operácia ViaccestnáEH.stupeň(vrchol: ↑TypBloku): int {  
2.   Vráť vrchol→synovia→veľkosť()  
3. }
```

b)

Definícia operácie *KCestnáEH.stupeň*(↑TypBloku): int

```
1. operácia KCestnáEH.stupeň(vrchol: ↑TypBloku): int {  
2.   definuj premennú výsledek: int ← 0  
3.   Pre každý syn v vrchol→synovia opakuj {  
4.     Ak (syn ≠ NULL) potom {  
5.       výsledek++  
6.     }  
7.   }  
8.   Vráť výsledek  
9. }
```

c)

Definícia operácie *BinárnaEH.stupeň*(↑TypBloku): int

```
1. operácia BinárnaEH.stupeň(vrchol: ↑TypBloku): int {  
2.   definuj premennú výsledek: int ← 0  
3.   Ak (vrchol→ľavý ≠ NULL) potom {  
4.     ++výsledek  
5.   }  
6.   Ak (vrchol→pravý ≠ NULL) potom {  
7.     ++výsledek  
8.   }  
9.   Vráť výsledek  
10. }
```

Explicitná hierarchia – modifikátory

- Modifikátory pracujúce s koreňom hierarchie sú principiálne rovnaké, ako modifikátory pracujúce so všeobecným vrcholom hierarchie. Rozdiel spočíva iba v potrebe aktualizácie referencie na koreň v riadiacom bloku štruktúry.
- Analógia:
 - $\text{vložSyna} \leftrightarrow \text{vložKoreň}$;
 - $\text{zmeňSyna} \leftrightarrow \text{zmeňKoreň}$;
 - $\text{zrušSyna} \leftrightarrow \text{vymaž}$.
- Modifikátory nekontrolujú, či pridaním koreňa resp. syna vrcholu hierarchie, nedôjde k poškodeniu hierarchie (napr. vznikom cyklu) – je to z dôvodu ich efektivity.
- Modifikátory zmeňKoreň a zmeňSyna nerušia bloky pamäte (ani prípadne celé podhierarchie), pretože sa predpokladá ich budúce ustanovenie za syna iného vrcholu.

Explicitná hierarchia – modifikátory – koreň – pseudokódy

Pseudokód operácie EH.vložKoreň(): ↑TypBloku

```
1. operácia EH.vložKoreň(): ↑TypBloku {  
2.   koreň ← správcaPamäte→prideIPamäť()  
3.   Vráť koreň↓  
4. }
```

Pseudokód operácie EH.zmeňKoreň(↑TypBloku)

```
1. operácia EH.zmeňKoreň(novýKoreň: ↑TypBloku) {  
2.   Ak (novýKoreň ≠ NULL) potom {  
3.     novýKoreň→otec ← NULL  
4.   }  
5.   koreň ← novýKoreň  
6. }
```

Explicitná hierarchia – modifikátory – vloženie syna

- a) Viaccestné explicitné hierarchie pridajú syna jednoducho využitím vhodnej operácie sekvencie, ktorá ukladá synov.
- K-cestné explicitné hierarchie s obmedzeným K neumožňujú pridať ďalší blok pamäte do množiny synov, pretože množina synov v blokoch pamäte je buď realizovaná implicitnou sekvenciou s nemennou veľkosťou alebo je vymenovaná. Vloženie syna v takýchto hierarchiách nepridá nový blok pamäte do množiny synov, ale zapíše nového syna na príslušné pamäťové miesto.
 - b) Ak sú synovia uložení v sekvencii, využije sa vhodná operácia na sprístupnenie bloku pamäte, ktorého údajová časť sa prepíše.
 - c) Hierarchie s vymenovanou množinou synov v blokoch pamäte môžu operáciu napísať pre konkrétne prípady.

Explicitná hierarchia – modifikátory – vloženie syna – pseudokódy

a)

Pseudokód operácie
ViaccestnáEH.vložSyna(↑TypBloku, int): ↑TypBloku

```
1. operácia ViaccestnáEH.vložSyna(  
    otec: ↑TypBloku,  
    poradieSyna: int  
): ↑TypBloku {  
2.   definuj premennú novýSyn: ↑TypBloku ←  
      správcaPamäte→prideIPamät()  
3.   otec→synovia→vlož(poradieSyna)→dáta ← novýSyn  
4.   novýSyn→otec ← dajAdresu(otec)  
5.   Vráť novýSyn↓  
6. }
```

b)

Pseudokód operácie *KCestnáEH.vložSyna*(↑TypBloku, int): ↑TypBloku

```
1. operácia KCestnáEH.vložSyna(  
    otec: ↑TypBloku,  
    poradieSyna: int  
): ↑TypBloku {  
2.   definuj premennú novýSyn: ↑TypBloku ←  
      správcaPamäte→prideIPamät()  
3.   otec→synovia→sprístupni(poradieSyna)→dáta ← novýSyn  
4.   novýSyn→otec ← dajAdresu(otec)  
5.   Vráť novýSyn↓  
6. }
```

c)

Definícia operácie *BinárnaEH.vložĽavéhoSyna*(↑TypBloku): ↑TypBloku

```
1. operácia BinárnaEH.vložĽavéhoSyna(  
    otec: ↑TypBloku  
): ↑TypBloku {  
2.   definuj premennú novýSyn: ↑TypBloku ←  
      správcaPamäte→prideIPamät()  
3.   otec→Ľavý ← novýSyn  
4.   novýSyn→otec ← dajAdresu(otec)  
5.   Vráť novýSyn↓  
6. }
```

Explicitná hierarchia – modifikátory – zmena syna

- Je potrebné aktualizovať vzťahy všetkých dotknutých vrcholov:
 - Pôvodný syn už nemá otca, je potrebné odstrániť referenciu na neho.
 - Novému synovi je nutné aktualizovať referenciu na otca a zaradiť ho do množiny synov nového otca.
- a) modifikátor je možné implementovať rovnako pre viaccestné aj pre k-cestné explicitné hierarchie.
- b) Hierarchie s vymenovanou množinou synov v blokoch pamäte môžu operáciu napísať pre konkrétne prípady.

Explicitná hierarchia – modifikátory – zmena syna – pseudokódy

a)

Pseudokód operácie <i>ViaccestnáEH.zmeňSyna(↑TypBloku, int, ↑TypBloku)</i>	
1.	operácia <i>ViaccestnáEH.zmeňSyna</i> (otec: ↑TypBloku, poradieSyna: int, novýSyn: ↑TypBloku) {
2.	definuj premennú blokSynov: ↑BlokPamäte<↑TypBloku> ← otec→synovia→sprístupni(poradieSyna)
3.	definuj premennú pôvodnýSyn: ↑TypBloku ← blokSynov→dáta
4.	blokSynov→dáta ← novýSyn
5.	Ak (pôvodnýSyn ≠ NULL) potom {
6.	pôvodnýSyn→otec ← NULL
7.	}
8.	Ak (novýSyn ≠ NULL) potom {
9.	novýSyn→otec ← dajAdresu(otec)
10.	}
11.	}

b)

Pseudokód operácie <i>BinárnaEH.zmeňĽavéhoSyna(↑TypBloku, ↑TypBloku)</i>	
1.	operácia <i>BinárnaEH.zmeňĽavéhoSyna</i> (otec: ↑TypBloku, novýSyn: ↑TypBloku) {
2.	definuj premennú pôvodnýSyn: ↑TypBloku ← otec→Ľavý
3.	otec→Ľavý ← novýSyn
4.	Ak (pôvodnýSyn ≠ NULL) potom {
5.	pôvodnýSyn→otec ← NULL
6.	}
7.	Ak (novýSyn ≠ NULL) potom {
8.	novýSyn→otec ← dajAdresu(otec)
9.	}
10.	}

Explicitná hierarchia – modifikátory – zrušenie syna

- Pre zrušenie syna je možné využiť rovnaký princíp ako pri operácií **zruš** – zrušiť vrchol aj s celou jeho podhierarchiou pomocou prehliadky v spätnom poradí.
 - a) V neusporiadaných **explicitných hierarchiách** je možné využiť operáciu **zruš** zo **sekvencie** (sekvencia môže meniť svoju veľkosť).
 - b) V usporiadaných K-cestných **explicitných hierarchiách** vznikne na mieste odstráneného syna „diera“ (na miesto zrušeného syna sa zapíše NULL, **sekvencia** nemôže meniť svoju veľkosť),
 - c) **Hierarchie** s vymenovanou množinou synov v blokoch pamäte môžu operáciu napísať pre konkrétne prípady.

Explicitná hierarchia – modifikátory – zrušenie syna – pseudokódy

a)

Definícia operácie ViaccestnáEH.zrušSyna($\uparrow$ TypBloku, int)

```
1. operácia ViaccestnáEH.zrušSyna(  
    otec:  $\uparrow$ TypBloku,  
    poradieSyna: int  
    ) {  
2.   definuj premennú blokSynov:  $\uparrow$ BlokPamäte< $\uparrow$ TypBloku>  $\Leftarrow$   
    otec $\rightarrow$ synovia $\rightarrow$ sprístupni(poradieSyna)  
3.   definuj premennú mazanýSyn:  $\uparrow$ TypBloku  $\Leftarrow$  blokSynov $\rightarrow$ dáta  
4.   spracujVSpätnomPoradí(mazanýSyn,  $\lambda$ (b:  $\uparrow$ TypBloku) {  
    správcaPamäte $\rightarrow$ uvoľniPamäť(b)  
    })  
5.   otec $\rightarrow$ synovia $\rightarrow$ zruš(poradieSyna)  
6. }
```

b)

Definícia operácie KCestnáEH.zrušSyna($\uparrow$ TypBloku, int)

```
1. operácia KCestnáEH.zrušSyna(  
    otec:  $\uparrow$ TypBloku,  
    poradieSyna: int  
    ) {  
2.   definuj premennú blokSynov:  $\uparrow$ BlokPamäte< $\uparrow$ TypBloku>  $\Leftarrow$   
    otec $\rightarrow$ synovia $\rightarrow$ sprístupni(poradieSyna)  
3.   definuj premennú mazanýSyn:  $\uparrow$ TypBloku  $\Leftarrow$  blokSynov $\rightarrow$ dáta  
4.   spracujVSpätnomPoradí(mazanýSyn,  $\lambda$ (b:  $\uparrow$ TypBloku) {  
    správcaPamäte $\rightarrow$ uvoľniPamäť(b)  
    })  
5.   blokSynov $\rightarrow$ dáta  $\Leftarrow$  NULL  
6. }
```



c)

Pseudokód operácie BinárnaEH.zrušĽavéhoSyna($\uparrow$ TypBloku)

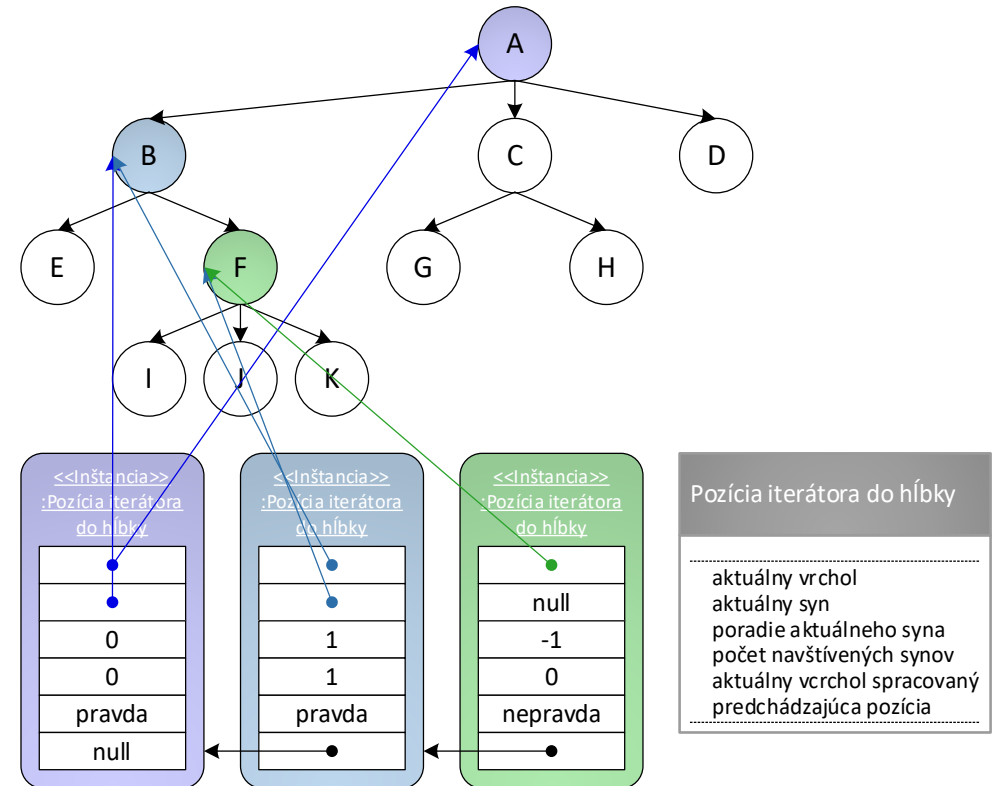
```
1. operácia BinárnaEH.zrušĽavéhoSyna(otec:  $\uparrow$ TypBloku) {  
2.   definuj premennú mazanýSyn:  $\uparrow$ TypBloku  $\Leftarrow$  otec $\rightarrow$ Ľavý  
3.   spracujVSpätnomPoradí(mazanýSyn,  $\lambda$ (b:  $\uparrow$ TypBloku) {  
    správcaPamäte $\rightarrow$ uvoľniPamäť(b)  
    })  
4.   otec $\rightarrow$ Ľavý  $\Leftarrow$  NULL  
5. }
```

Explicitná hierarchia – modifikátory – zložitosti

Modifikátory		
Operácia	Zložitosť	Parametre
Vlož koreň	O(1)	-
Zmeň koreň		
Vlož syna	O(s)	s: stupeň vrcholu, synovia sú uložení v sekvencií, prípustné iba pre viaccestné hierarchie
Zmeň syna	O(1)	synovia sú uložení v implicitnej sekvencií
	O(s)	s: stupeň vrcholu, synovia sú uložení v explicitnej sekvencií
Zruš syna	O(m)	m: mohutnosť podhierarchie mazaného syna, synovia sú uložení v implicitnej sekvencií
	O(s) + O(m)	s: stupeň vrcholu, m: mohutnosť podhierarchie mazaného syna, synovia sú uložení v explicitnej sekvencií

Hierarchia – univerzálne iterátory do hĺbky

- Založené na rekurzívnych prehliadkach – treba vytvoriť sekvenčnú verziu týchto algoritmov:
 - **rekurzia** využíva *zásobník volaní* a *rámce*.
 - **iterátory** využijú *sekvenciu pozícií*.
- Hierarchia vytvára iterátory rovnakých typov tak, že inicializuje referenciu na správny prvok:
 - **Prvý iterátor**: referencia na prvý platný prvok je uložená vo vlastnosti *koreň*.
 - **Posledný iterátor**: referencia na prvý neplatný prvok je NULL.



Hierarchia – univerzálne iterátory do hĺbky

- Univerzálne iterátory sa musia vysporiadať aj s usporiadanými K-cestnými hierarchiami, - niektorý z K synov nemusí byť nastavený, a teda navýšenie spomenutej premennej o jedna by spôsobilo znefunkčnenie iterátora (jeho pozícia by bola na hodnote vrcholu NULL).
- Komplikovaná je operácia **vpred**.
- Operácia **skús ísť na ďalšieho syna** túto funkčnosť realizuje jednotne pre akékoľvek hierarchie.
- Iterátory je vhodné rozšíriť o operácie pracujúce so sekvenciou pozícií.

Operácie pre prácu s postupnosťou pozícií iterátorov hierarchií do hĺbky			
Názov operácie	Parametre	Návratová hodnota	Význam
Ulož pozíciu	↑TypBloku	-	Vytvorí a uchová objekt PozíciaIterátoraDoHĺbky zodpovedajúci spracovaniu vrcholu odovzdaného ako parameter (vytvorenie rámca funkčného volania).
Odstráň pozíciu	-	-	Zruší prvý objekt PozíciaIterátoraDoHĺbky v postupnosti (odstráni rámec funkčného volania)
Skús ísť na ďalšieho syna	-	bool	Posunie index aktuálne spracovávaného syna vrcholu v objekte PozíciaIterátoraDoHĺbky na ďalšieho platného syna. Vrátí príznak podľa toho, či bolo možné prejsť na ďalšieho syna.

Hierarchia – univerzálne iterátory do hĺbky – pseudokódy

Definícia konštruktora IterátorHierarchieDoHĺbky ($\uparrow$ IterátorHierarchieDoHĺbky)

```
1. konštruktor IterátorHierarchieDoHĺbky(  
    iný:  $\uparrow$ IterátorHierarchieDoHĺbky  
  ) {  
2.   inicializuj IterátorHierarchieDoHĺbky ()  
3.   definuj premennú mojaPozícia:  
       $\uparrow$ PozíciaIterátoraDoHĺbky  $\leftarrow$  NULL  
4.   Opakuj pre premennú ináPozícia:  $\uparrow$ PozíciaIterátoraDoHĺbky  
      od iný $\rightarrow$ aktuálnaPozícia do NULL - 1 {  
5.     Ak (aktuálnaPozícia = NULL) potom {  
6.       aktuálnaPozícia  $\leftarrow$   
          vytvor PozíciaIterátoraDoHĺbky(ináPozícia $\downarrow$ )  
7.       mojaPozícia  $\leftarrow$  aktuálnaPozícia  
8.     }  
9.     inak {  
10.      mojaPozícia $\rightarrow$ predchádzajúcaPozícia  $\leftarrow$   
          vytvor PozíciaIterátoraDoHĺbky(ináPozícia $\downarrow$ )  
11.      mojaPozícia  $\leftarrow$  mojaPozícia $\rightarrow$ predchádzajúcaPozícia  
12.    }  
13.  }  
14. }
```

Definícia operácie IterátorHierarchieDoHĺbky.jeRovnaký($\uparrow$ IterátorHierarchieDoHĺbky)

```
1. operácia IterátorHierarchieDoHĺbky.jeRovnaký(  
    iný:  $\uparrow$ IterátorHierarchieDoHĺbky  
  ): bool {  
2.   Ak (hierarchia  $\neq$  iný $\rightarrow$ hierarchia) potom {  
3.     Vráť nepravda  
4.   }  
5.   definuj premennú mojaPozícia:  $\uparrow$ PozíciaIterátoraDoHĺbky  $\leftarrow$   
      aktuálnaPozícia  
6.   definuj premennú ináPozícia:  $\uparrow$ PozíciaIterátoraDoHĺbky  $\leftarrow$   
      iný $\rightarrow$ aktuálnaPozícia  
7.   Ak (mojaPozícia  $\neq$  NULL  $\wedge$  ináPozícia  $\neq$  NULL) potom {  
8.     Ak (mojaPozícia $\rightarrow$ aktuálnyVrchol  $\neq$   
          ináPozícia $\rightarrow$ aktuálnyVrchol  $\vee$   
          mojaPozícia $\rightarrow$ poradieAktuálnehoSyna  $\neq$   
          ináPozícia $\rightarrow$ poradieAktuálnehoSyna) potom {  
9.       Vráť nepravda  
10.    }  
11.  }  
12.  Vráť mojaPozícia = NULL  $\wedge$  ináPozícia = NULL  
13. }
```

Definícia operácie IterátorHierarchieDoHĺbky.sprístupni(): $\uparrow$ TypBloku

```
1. operácia IterátorHierarchieDoHĺbky.sprístupni():  $\uparrow$ TypBloku {  
2.   aktuálnaPozícia $\rightarrow$ aktuálnyVrcholSpracovaný  $\leftarrow$  pravda  
3.   Vráť aktuálnaPozícia $\rightarrow$ aktuálnyVrchol $\downarrow$   
4. }
```

Hierarchia – univerzálne iterátory do hĺbky – pseudokódy

Definícia operácie

IterátorHierarchieDoHĺbky.skúsÍstNaĎalšiehoSyna(): bool

```
1. operácia IterátorHierarchieDoHĺbky.skúsÍstNaĎalšiehoSyna(  
): bool {  
2.   aktuálnaPozícia → početNavštívenýchSynov++  
3.   definuj premennú aktualnyStupen: int ←  
       hierarchia → stupeň(aktuálnaPozícia → aktuálnyVrchol↓)  
4.   Ak (aktuálnaPozícia → početNavštívenýchSynov ≤  
       aktualnyStupen) potom {  
5.     Opakuj {  
6.       aktuálnaPozícia → poradieAktuálnehoSyna++  
7.       aktuálnaPozícia → aktuálnySyn ←  
           hierarchia → sprístupniSyna(  
               aktuálnaPozícia → aktuálnyVrchol↓,  
               aktuálnaPozícia → poradieAktuálnehoSyna)  
8.     } pokiaľ (aktuálnaPozícia → aktuálnySyn = NULL)  
9.     Vráť pravda  
10.  }  
11.  inak {  
12.    aktuálnaPozícia → poradieAktuálnehoSyna ← -1  
13.    aktuálnaPozícia → aktuálnySyn ← NULL  
14.    Vráť nepravda  
15.  }  
16. }
```

Definícia operácie IterátorHierarchieDoHĺbky.uložPozíciu(↑TypBloku)

```
1. operácia IterátorHierarchieDoHĺbky.uložPozíciu(  
   aktuálnyVrchol: ↑TypBloku  
) {  
2.   aktuálnaPozícia ←  
       vytvor PozíciaIterátoraDoHĺbky(aktuálnyVrchol,  
       aktuálnaPozícia)  
3. }
```

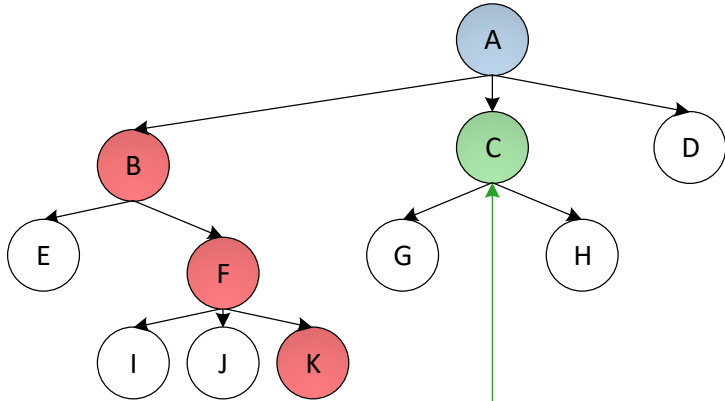
Definícia operácie IterátorHierarchieDoHĺbky.odstráňPozíciu()

```
1. operácia IterátorHierarchieDoHĺbky.odstráňPozíciu() {  
2.   definuj premennú pozíciaNaOdstránenie:  
       ↑PozíciaIterátoraDoHĺbky ← aktuálnaPozícia  
3.   aktuálnaPozícia ← aktuálnaPozícia → predchádzajúcaPozícia  
4.   zruš pozíciaNaOdstránenie  
5. }
```

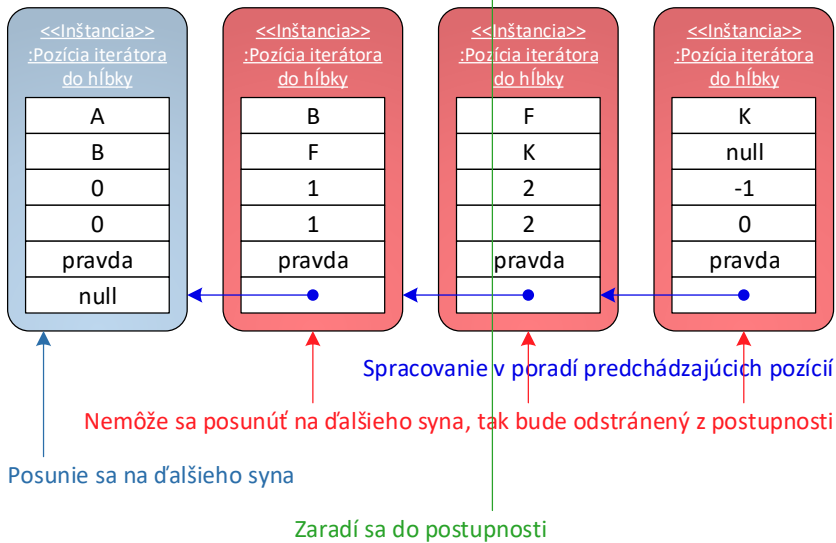
Hierarchia – iterátor v priamom poradí

- Pri vytvorení iterátora sa do sekvencie pozícií teda vloží objekt `PozíciaIterátoraDoHĺbky` reprezentujúci *koreň* hierarchie, čím sa tento spracuje ako prvý.
- Operácia `vpred` je potom principiálne jednoduchá.
 - Do postupnosti pozícií sa vloží objekt `PozíciaIterátoraDoHĺbky` pre prvého syna, čím sa bude tento spracovávať v ďalšom volaní operácie `sprístupni` (aktuálny objekt `PozíciaIterátoraDoHĺbky` bude druhý v postupnosti pozícií).
 - Po spracovaní syna sa vloží objekt `PozíciaIterátoraDoHĺbky` pre ďalšieho existujúceho syna (čím sa v ďalšom volaní operácie `sprístupni` spracuje tento), atď.
- Takúto funkčnosť operácie `vpred` vykonávajú aj synovia, čím sa pôvodný objekt `PozíciaIterátoraDoHĺbky` pre *koreň* hierarchie bude posúvať v postupnosti pozícií ďalej od jej začiatku (vlastnosti *aktuálnaPozícia* v iterátore).
- Po spracovaní všetkých synov je potrebné objekt `PozíciaIterátoraDoHĺbky` z postupnosti pozícií odstrániť a pokračovať spracovaním ďalšieho syna rodiča – teda brata aktuálne spracovaného vrcholu. To vieme zabezpečiť opätovným volaním operácie `vpred`

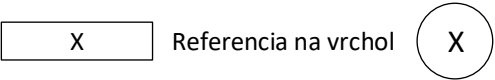
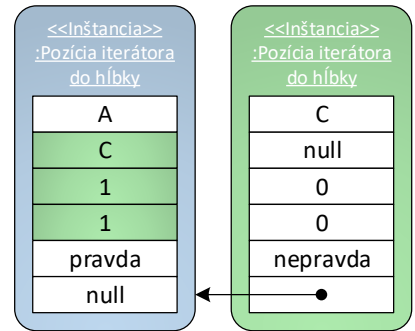
Hierarchia – iterátor v priamom poradí – príklad



pred volaním **vpred**:



po volaní **vpred**:



Hierarchia – iterátor v priamom poradí – pseudokódy

Definícia konštruktora `IterátorHierarchieVPriamomPoradí(↑Hierarchia<TypBloku>, ↑TypBloku)`

```
1. konštruktor IterátorHierarchieVPriamomPoradí(  
    hierarchia: ↑Hierarchia<TypBloku>,  
    vrchol: ↑TypBloku  
    ) {  
2.   inicializuj predka IterátorHierarchieDoHíbký(hierarchia)  
3.   Ak (vrchol ≠ NULL) potom {  
4.     uložPozíciu(vrchol)  
5.   }  
6. }
```

Definícia operácie `IterátorHierarchieVPriamomPoradí.vpred()`

```
1. operácia IterátorHierarchieVPriamomPoradí.vpred() {  
2.   Ak (skúsÍstNadálšiehoSyna()) potom {  
3.     uložPozíciu(aktuálnaPozícia→aktuálnySyn)  
4.   }  
5.   inak {  
6.     odstráňPozíciu()  
7.     Ak (aktuálnaPozícia ≠ NULL) potom {  
8.       vpred()  
9.     }  
10.  }  
11. }
```

Hierarchia – iterátor v spätnom poradí

- Pri zaradení ďalšieho vrcholu do postupnosti pozícií musí iterátor „klesnúť“ (vytvoriť objekty `PozíciaIterátoraDoHĺbky` v postupnosti pozícií) na posledného syna najviac vľavo.
- To je potrebné, vykonať aj ihneď pri vzniku, kedy sa do postupnosti pozícií zaradí objekt `PozíciaIterátoraDoHĺbky` reprezentujúci *koreň* hierarchie.
- Po spracovaní všetkých synov nebude už žiadny objekt `PozíciaIterátoraDoHĺbky` zaradený do postupnosti pozícií, čím sa spracuje vrchol samotný (bude pri volaní operácie `sprístupni` na prvom mieste v postupnosti pozícií).
- Následne sa môže vykonať odstránenie tejto pozície zo sekvencie a pokračovať v spracovávaní rodičovského vrcholu.
- To vieme zabezpečiť opätovným volaním operácie `vpred`.

Hierarchia – iterátor v spätnom poradí – pseudokódy

Definícia konštruktora IterátorHierarchieVSpätnomPoradí($\uparrow$ Hierarchia<TypBloku>, $\uparrow$ TypBloku)

```
1. konštruktor IterátorHierarchieVSpätnomPoradí(  
    hierarchia:  $\uparrow$ Hierarchia<TypBloku>,  
    vrchol:  $\uparrow$ TypBloku  
    ) {  
2.   inicializuj predka IterátorHierarchieDoHĺbky(hierarchia)  
3.   Ak (vrchol  $\neq$  NULL) potom {  
4.     uložPozíciu(vrchol)  
5.     vpred()  
6.   }  
7. }
```

Definícia operácie IterátorHierarchieVSpätnomPoradí.vpred()

```
1. operácia IterátorHierarchieVSpätnomPoradí.vpred() {  
2.   Ak ( $\neg$ aktuálnaPozícia $\rightarrow$ aktuálnyVrcholSpracovaný  $\wedge$   
        skúsÍstNadálšiehoSyna()) potom {  
3.     uložPozíciu(aktuálnaPozícia $\rightarrow$ aktuálnySyn)  
4.     vpred()  
5.   }  
6.   inak {  
7.     Ak (aktuálnaPozícia $\rightarrow$ aktuálnyVrcholSpracovaný) potom {  
8.       odstráňPozíciu()  
9.       Ak (aktuálnaPozícia  $\neq$  NULL) potom {  
10.        vpred()  
11.      }  
12.    }  
13.  }  
14. }
```

Hierarchia – iterátor vo vnútornom poradí

- Principiálne sleduje prehliadku v spätnom poradí. Iterátor musí pri zaradení ďalšieho vrcholu do postupnosti pozícií „klesnúť“ na posledného syna najviac vľavo a rovnako je tento krok potrebné vykonať ihneď pri vzniku.
- Spracovanie vrcholu prebieha po návrate spracovania ľavej podhierarchie (ak existovala)!
- Je potrebné rozlíšiť, či už bol alebo nebol spracovaný aktuálny vrchol a až potom pokračovať spracovaním pravého vrcholu (využitie vlastnosti *aktuálnyVrcholSpracovaný* v objekte *PozíciaIterátoraDoHĺbky*).
- Po spracovaní pravého syna nasleduje odstránenie objektu *PozíciaIterátoraDoHĺbky* z postupnosti pozícií a pokračuje sa v spracovávaní rodičovského vrcholu.
- To je principiálne rovnaké, ako v prípade prehliadky v priamom resp. spätnom poradí.

Hierarchia – iterátor vo vnútornom poradí – pseudokódy

Definícia konštruktora IterátorHierarchieVoVnútornomPoradí($\uparrow$ Hierarchia<TypBloku>,
 $\uparrow$ TypBloku)

```
1. konštruktor IterátorHierarchieVoVnútornomPoradí(  
    hierarchia:  $\uparrow$ BinárnaHierarchia<TypBloku>,  
    vrchol:  $\uparrow$ TypBloku  
    ) {  
2.   inicializuj predka IterátorHierarchieDoHíbký(hierarchia)  
3.   Ak (vrchol  $\neq$  NULL) potom {  
4.     uložPozíciu(vrchol)  
5.     vpred()  
6.   }  
7. }
```

Definícia operácie IterátorHierarchieVoVnútornomPoradí.vpred()

```
1. operácia IterátorHierarchieVoVnútornomPoradí.vpred() {  
2.   Ak ( $\neg$ aktuálnaPozícia $\rightarrow$ aktuálnyVrcholSpracovaný) potom {  
3.     Ak (aktuálnaPozícia $\rightarrow$ poradieAktuálnehoSyna  $\neq$   
        INDEX_LAVÉHO_SYNA  $\wedge$   
        skúsÍstNaĽavéhoSyna()) potom {  
4.       uložPozíciu(aktuálnaPozícia $\rightarrow$ aktuálnySyn)  
5.       vpred()  
6.     }  
7.   }  
8.   inak {  
9.     Ak (aktuálnaPozícia $\rightarrow$ poradieAktuálnehoSyna  $\neq$   
        INDEX_PRAVÉHO_SYNA  $\wedge$   
        skúsÍstNaPravéhoSyna()) potom {  
10.      uložPozíciu(aktuálnaPozícia $\rightarrow$ aktuálnySyn)  
11.      vpred()  
12.    }  
13.    inak {  
14.      odstráňPozíciu()  
15.      Ak (aktuálnaPozícia  $\neq$  NULL) potom {  
16.        vpred()  
17.      }  
18.    }  
19.  }  
20. }
```

Hierarchia – iterátor do šírky

- Veľmi jednoduchý, algoritmus nie je rekurzívny, jednotlivé časti preto stačí rozdeliť medzi príslušné metódy iterátora.
- Vďaka uloženiu v pamäti je iterátor do šírky pre `implicitné hierarchie` rovnako, ako pre `implicitné sekvencie`.

Hierarchia – iterátor do šírky

Definícia konštruktora `IterátorHierarchiePoÚrovniah(↑Hierarchia<↑TypBloku>, ↑TypBloku)`

```
1. konštruktor IterátorHierarchiePoÚrovniah(  
    hierarchia: ↑Hierarchia<↑TypBloku>,  
    aktuálnyVrchol: ↑TypBloku  
    ) {  
2.   hierarchia ← hierarchia  
3.   sekvencia ← vytvor JednostranneZS<↑TypBloku>()  
4.   Ak (aktuálnyVrchol ≠ NULL) potom {  
5.     sekvencia→vložPosledný()→dáta ← aktuálnyVrchol  
6.   }  
7. }
```

Definícia operácie

`IterátorHierarchiePoÚrovniah.jeRovnaký(↑IterátorHierarchiePoÚrovniah): bool`

```
1. operácia IterátorHierarchiePoÚrovniah.jeRovnaký(  
    iný: ↑IterátorHierarchiePoÚrovniah  
    ): bool {  
2.   Vráť sekvencia→porovnaj(iný→sekvencia)  
3. }
```

Definícia operácie `IterátorHierarchiePoÚrovniah.sprístupni(): ↑TypBloku`

```
1. operácia IterátorHierarchiePoÚrovniah.sprístupni(  
    ): ↑TypBloku {  
2.   Vráť sekvencia→sprístupniPrvý()→dáta  
3. }
```

Definícia operácie `IterátorHierarchiePoÚrovniah.máĎalší(): bool`

```
1. operácia IterátorHierarchiePoÚrovniah.máĎalší(): bool {  
2.   Vráť sekvencia→veľkosť() > 0  
3. }
```

Definícia operácie `IterátorHierarchiePoÚrovniah.vpred()`

```
1. operácia IterátorHierarchiePoÚrovniah.vpred() {  
2.   definuj premennú vrchol: ↑TypBloku ←  
       sekvencia→sprístupniPrvý()→dáta  
3.   sekvencia→zrušPrvý()  
4.   Opakuj pre premennú i: int  
       od 0 do vrchol→synovia→veľkosť() - 1 {  
5.     definuj premennú syn: ↑TypBloku ←  
       hierarchia→sprístupniSyna(vrchol↓, i)  
6.     Ak (syn ≠ NULL) potom {  
7.       sekvencia→vložPosledný()→dáta ← syn  
8.     }  
9.   }  
10. }
```

Zhrnutie prednášky

- Hierarchie a ich iterátory
 - Implicitná hierarchia
 - Explicitná hierarchia

Plán na cvičenie

- Explicitná sekvencia.
- Iterátory sekvencií.

Ďakujem za pozornosť

Doplňujúce informácie?
Vaše otázky?

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu 6UI0004 Algoritmy a údajové štruktúry 1 na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline.
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené.

Ing. Michal Varga, PhD.
Katedra informatiky
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
Michal.Varga@fri.uniza.sk

Algoritmy a údajové štruktúry 1

Prednáška č. 6

- Abstraktné údajové štruktúry a multištruktúry
- Zoznamy
 - Implicitné (cyklické)
 - Jednostranne a obojstranne zreťazené (cyklické)
- Stromy
 - Viaccestné
 - K-cestné implicitné a explicitné
 - Binárne implicitné a explicitné



Abstraktný údajový typ

- Abstraktný údajový typ (AUT) určuje
 - **doménu prvkov** (prípadne viac domén prvkov) – určuje požiadavky kladené na prvky (napr. prítomnosť kľúča, priority, nemožnosť duplicít atď.);
 - **operácie na prvkoch domény** (viacerých domén) – operácie sú nezávislé na konkrétnej implementácii AUT.
- Analógia k abstraktnému pamäťovému typu (APT):
 - AUT aj APT sú „čierne skrinky“ (implementačne sú rozhranie).
 - AUT je realizovateľný viacerými abstraktnými údajovými štruktúrami - AUS (APT je realizovateľný viacerými abstraktnými pamäťovými štruktúrami - APS).

Abstraktné údajové štruktúry

- AUS ukladajú prvky v pamäti pomocou vhodnej APS.
- Podľa použitej APS sa budú odvíjať typické vlastnosti AUS:

*Ak AUS využívajú **implicitnú** APS*

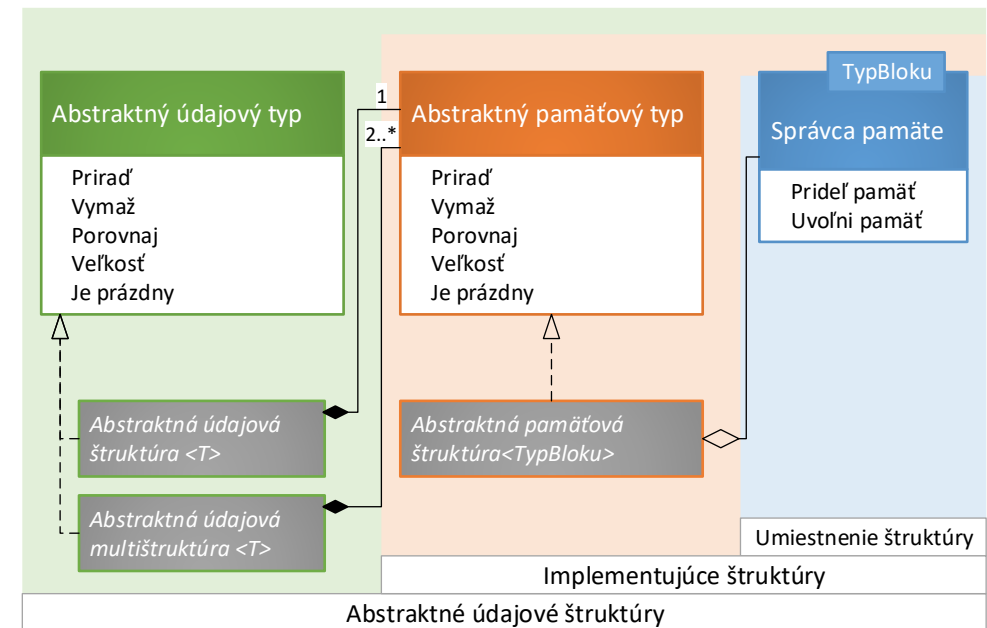
- **nižšie** pamäťové nároky na uloženie jedného prvku,
- **efektívne** operácie pre prístup k prvkom, avšak
- **pomalé** modifikácie štruktúry;

*Ak AUS využívajú **explicitnú** APS*

- **vyššie** pamäťové nároky, pretože je potrebné explicitne ukladať referencie na ostatné prvky štruktúry,
- **neefektívne** operácie pre prístup k prvkom, pre ktoré nie je uložená referencia priamo v riadiacom bloku štruktúry,
- **efektívne** modifikácie štruktúry.

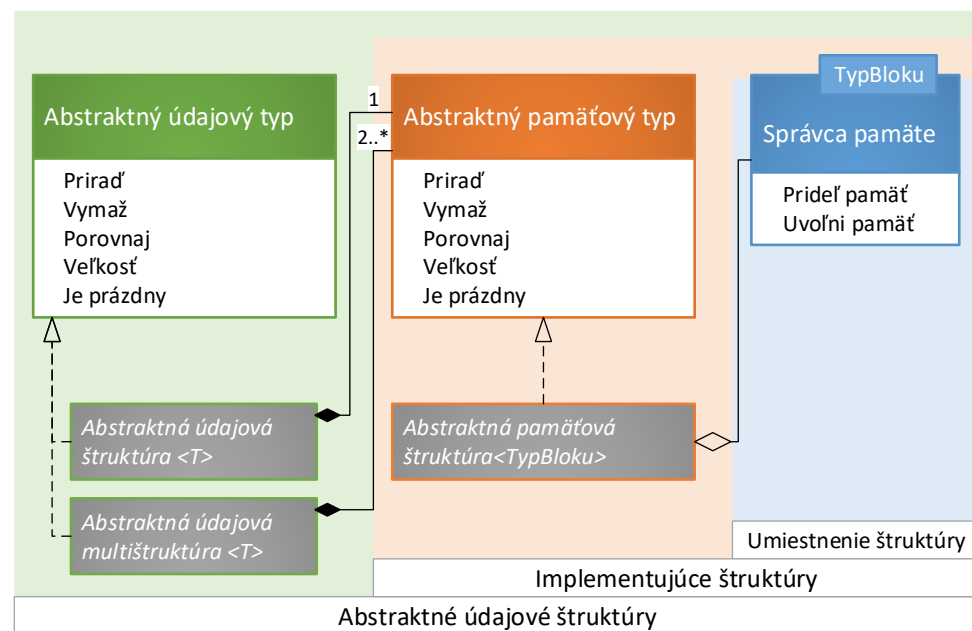
Abstraktné údajové štruktúry a multištruktúry

- Vzťah medzi AUS a APS je typicky 1:1.
- Pokročilé realizácie AUT využívajú kombináciu rôznych APS kvôli zefektívneniu niektorých operácií.
- Ak sa abstraktná údajová štruktúra skladá z viac ako jednej abstraktnej pamäťovej štruktúry, budeme o nej hovoriť ako o **multištruktúre** (AUMS).
- **APS** pracujú s **blokami pamäte**:
 - šablónový parameter **TypBloku**.
 - pre APS nie je podstatné, čo je v údajovej časti bloku pamäte.
 - **APT** – pre univerzálnosť nemá šablónový parameter.
- **AUS** pracujú s **údajmi**.
 - šablónový parameter **T** = typ údajov.
 - AUS definuje údajovú časť bloku pamäte.
 - **AUT** – pre univerzálnosť nemá šablónový parameter.



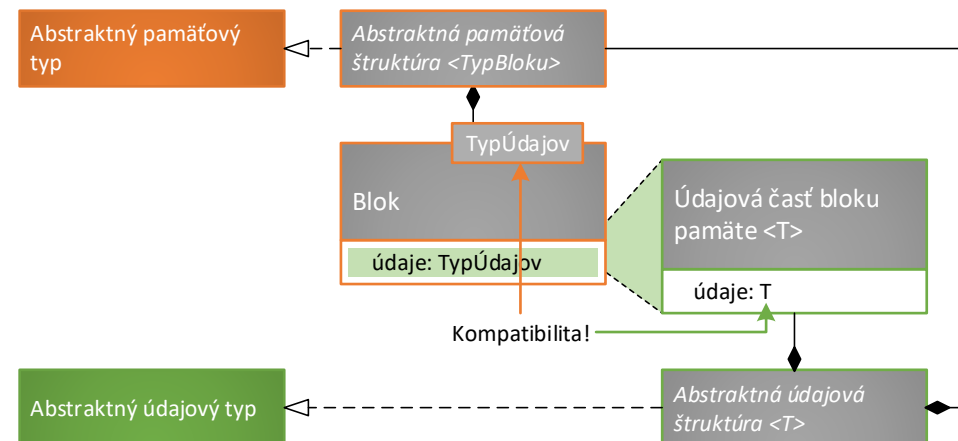
Abstraktné údajové štruktúry a multištruktúry

- Každá AUS:
 - musí kontrolovať vstupy (napr. platnosť indexu, unikátnosť prvku atď.).
 - môže zefektívniť operácie priradiť a porovnať (porovnaním referencie self s odovzdaným parametrom).
- Ak AUS nie je multištruktúra, potom:
 - musí riadiť koniec životného cyklu APS (ktorú typicky prijme v konštruktore).
 - sa nemusí zaoberať implementáciou základných operácií (ak ich podporuje), nakoľko všetky operácie sú univerzálne implementované v príslušnej APS.
- Ak AUS je multištruktúra, potom:
 - riadi celý životný cyklus APS, z ktorých sa skladá.
 - musí základné operácie korektne vyvolať na jednotlivých APS, z ktorých sa skladá.



Kompatibilita AUS a APS

- **AUS** musí byť kompatibilná s **APS** → bloky pamäte **APS** musia byť kompatibilné s typom údajov v **AUS**:
 - inak povedané šablónový parameter **AUS** musí byť typovo kompatibilný so šablónovým parametrom bloku pamäte.
 - Ak sa $AUS<T>$ skladá z $APS<TypBloku>$, potom šablónový parameter **TypBloku** musí byť typovo kompatibilný s triedou $BlokPamäte<T>$.
- Pri popise **AUT** a **AUS** budeme abstrahovať od **APT** a **APS** a organizácie ich blokov pamäte, budeme pracovať iba s údajovou časťou bloku pamäte, preto:
 - o údajovej časti bloku pamäte budeme pomyseľne uvažovať ako o predkovi údajov, ktoré spravujú **AUS**.
 - atribút *údaje* typu **T**, ktorý je definovaný v triede $BlokPamäte<T>$ pomyseľne stotožníme s predkom $ÚdajováČasťBlokuPamäte<T>$.



AUT, AUS a AUMS – definícia – pseudokódy

Definícia rozhrania AbstraktnýÚdajovýTyp

```
1. Rozhranie AbstraktnýÚdajovýTyp
2.   má skratku AUT {
3.     operácia prirad(iný: ↑AUT): ↑AUT
4.     operácia vymaž()
5.     operácia veľkosť(): int
6.     operácia jePrázdny(): bool
7.     operácia porovnaj(iný: ↑AUT): bool
8. }
```

Definícia triedy AbstraktnáÚdajováŠtruktúra<T>

```
1. Trieda AbstraktnáÚdajováŠtruktúra<T>
2.   realizuje AUT,
3.   má skratku AUS {
4.     konštruktor(pamäťováŠtruktúra: ↑APT)
5.     konštruktor(iná: ↑AUS)
6.     deštruktor()
7.     operácia prirad(iný: ↑AUT): ↑AUT
8.     operácia vymaž()
9.     operácia veľkosť(): int
10.    operácia jePrázdny(): bool
11.    operácia porovnaj(iný: ↑AUT): bool
12.    vlastnosť pamäťováŠtruktúra: ↑APT
13.    vlastnosť
14. }
```

Definícia triedy AbstraktnáÚdajováMultištruktúra<T>

```
1. Trieda AbstraktnáÚdajováMultištruktúra<T>
2.   realizuje AUT,
3.   má skratku AUMS {
4. }
```

AUS – základné operácie - pseudokódy

Definícia konštruktora AUS($\uparrow$ APT)

```
1. konštruktor AUS(pamäťováŠtruktúra:  $\uparrow$ APT) {  
2.   pamäťováŠtruktúra  $\leftarrow$  pamäťováŠtruktúra  
3. }
```

Definícia konštruktora AUS($\uparrow$ AUS)

```
1. konštruktor AUS(iná:  $\uparrow$ AUS) {  
2.   inicializuj AUS()  
3.   prirad(iná)  
4. }
```

Definícia deštruktora AUS

```
1. deštruktor AUS() {  
2.   zruš pamäťováŠtruktúra  
3.   pamäťováŠtruktúra  $\leftarrow$  NULL  
4. }
```

Definícia operácie AUS.veľkosť(): int

```
1. operácia AUS.veľkosť(): int {  
2.   Vráť pamäťováŠtruktúra $\rightarrow$ veľkosť()  
3. }
```

Definícia operácie AUS.jePrázdny(): bool

```
1. operácia AUS.jePrázdny(): bool {  
2.   Vráť pamäťováŠtruktúra $\rightarrow$ jePrázdny()  
3. }
```

Definícia operácie AUS.prirad($\uparrow$ AUT): $\uparrow$ AUT

```
1. operácia AUS.prirad(iný:  $\uparrow$ AUT):  $\uparrow$ AUT {  
2.   Ak ( $\neg$ (iný je typu AUS)) potom {  
3.     CHYBA  
4.   }  
5.   Ak (self  $\neq$  dajAdresu(iný)) potom {  
6.     pamäťováŠtruktúra $\rightarrow$ prirad(iný $\rightarrow$ pamäťováŠtruktúra $\downarrow$ )  
7.   }  
8.   Vráť self $\downarrow$   
9. }
```

Definícia operácie AUS.porovnaj($\uparrow$ AUT): bool

```
1. operácia AUS.porovnaj(iný:  $\uparrow$ AUT): bool {  
2.   Ak ( $\neg$ (iný je typu AUS)) potom {  
3.     Vráť nepravda  
4.   }  
5.   Ak (self  $\neq$  dajAdresu(iný)) potom {  
6.     Vráť pamäťováŠtruktúra $\rightarrow$ porovnaj(iný $\rightarrow$ pamäťováŠtruktúra $\downarrow$ )  
7.   }  
8.   inak {  
9.     Vráť pravda  
10.  }  
11. }
```

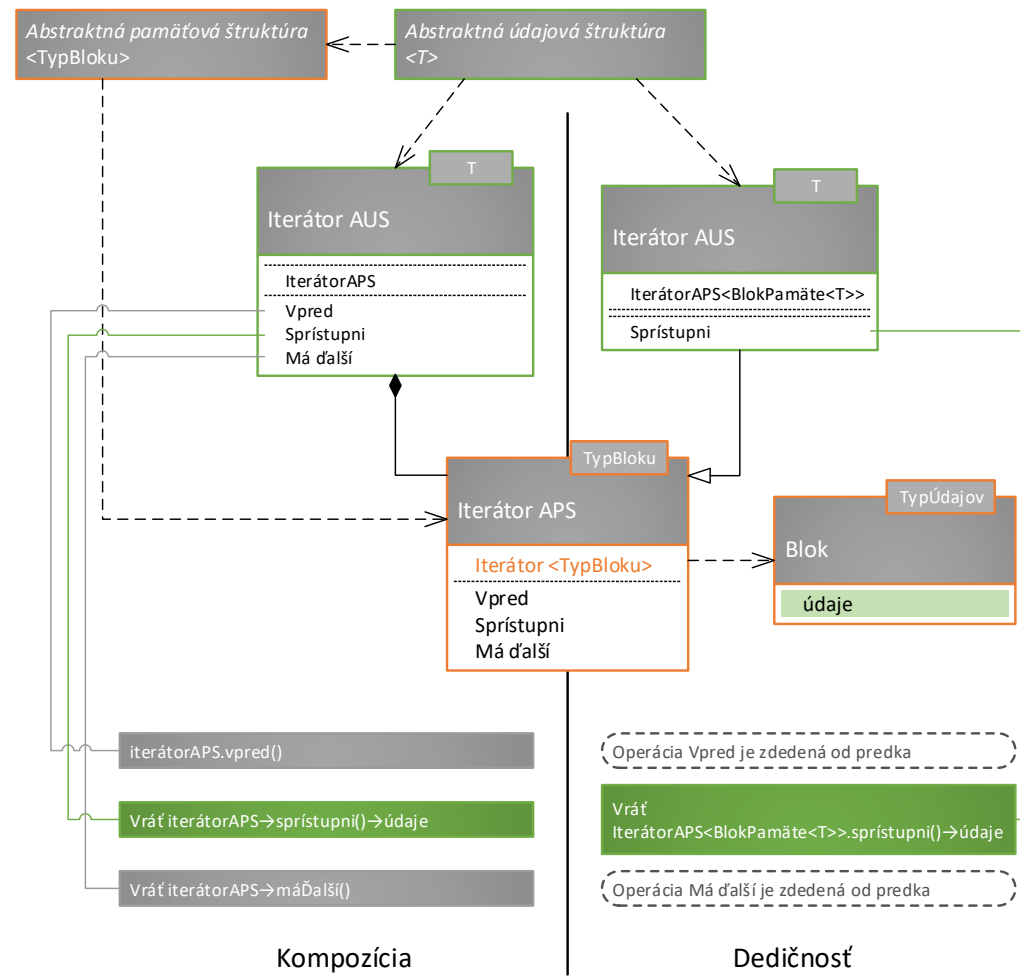
Definícia operácie AUS.vymaž()

```
1. operácia AUS.vymaž() {  
2.   pamäťováŠtruktúra $\rightarrow$ vymaž()  
3. }
```

Prehliadky AUS

- **Nie všetky AUT podporujú iterovanie (na rozdiel od všeobecnej APS).**
- Ak AUT bude špecifikovať iterátory potom stačí vrátiť príslušné iterátory z APS, z ktorej sa AUS skladá.
- Pozor na operáciu **sprístupni** v iterátoroch APS!
 - Sprístupní celý blok pamäte - v prípade explicitných APS to nemusí byť žiadúce. **Toto znamená aj sprístupnenie vzťahovej časti, čím by mohol volajúci tejto metódy poškodiť integritu APS.**
 - Riešenia:
 - **Dedičnosť:** Iterátor explicitných APS je možné zdediť a prekryť operáciu sprístupni tak, aby vrátila priamo údaje.
 - **Kompozícia:** Vytvoriť novú triedu IterátorAUS, ktorá obalí (prevezme ako atribút) pôvodný iterátor APS. Volania všetkých metód bude „posúvať“ iterátoru uloženom v atribúte a prípadne upraví návratové hodnoty.
- Iterátory AUMS je potrebné navrhnuť na mieru danej AUMS.

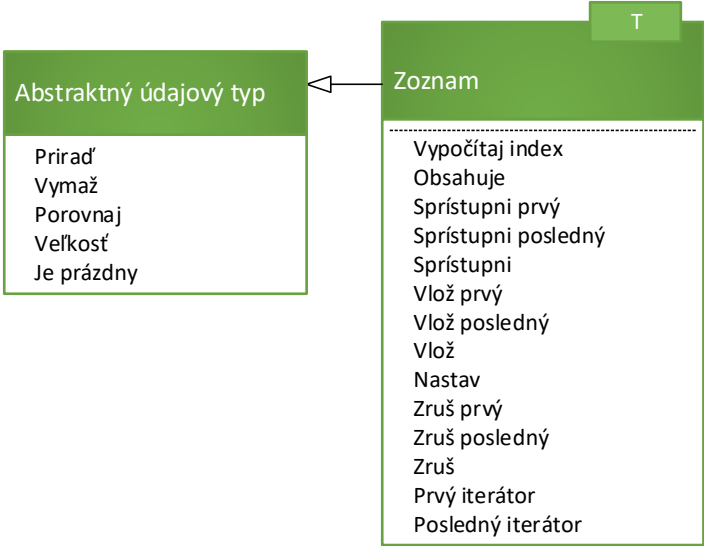
Univerzálne iterátory AUS



AUT Zoznam

- **AUT zoznam nekladie žiadne obmedzenia na doménu prvkov** (teda akékoľvek prvky môžu tvoriť zoznamy).
- Pre prvky v zozname je charakteristický **prístup na základe indexu** – ich poradia v zozname.
- Špeciálnym typom zoznamu je **cyklický zoznam** – nasledovník posledného prvku je prvý prvok a predchodca prvého prvku je posledný prvok.
- Zoznamy **je možné iterovať**.

AUT Zoznam

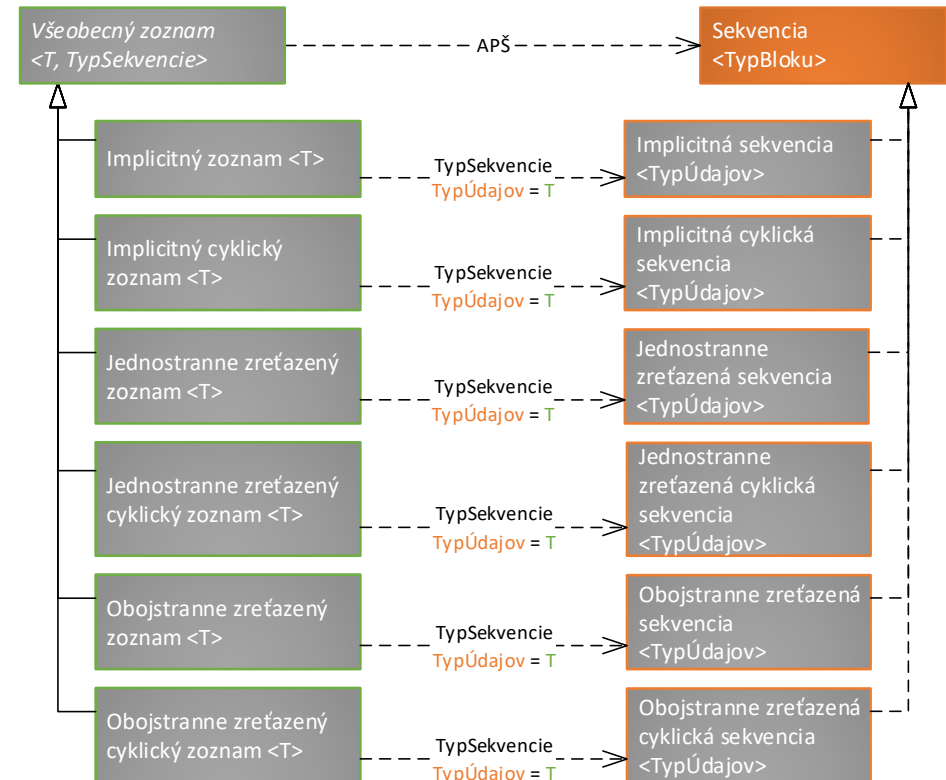
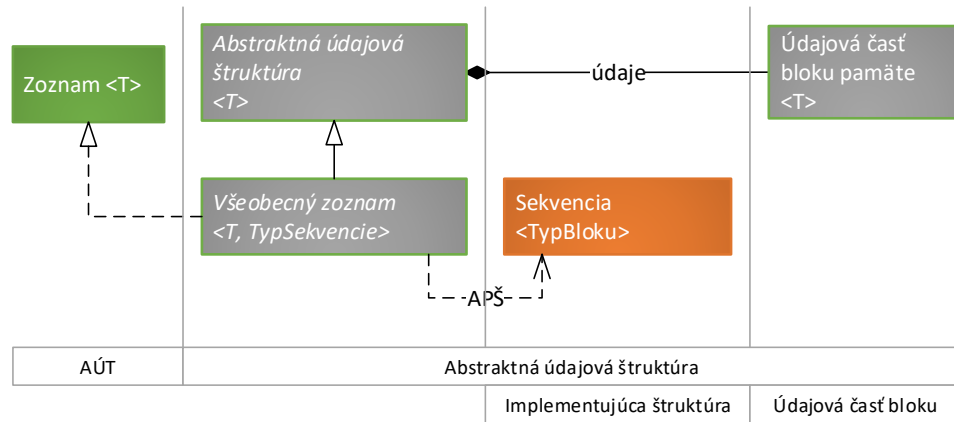


Základné operácie so zoznamom			
Názov operácie	Parametre	Návratová hodnota	Význam
Vypočítaj index	T	int	Vráti index prvého výskytu prvku odovzdaného ako parameter.
Selektory			
Názov selektora	Parametre	Návratová hodnota	Selektor sprístupní
Sprístupni prvý	-	T	Prvý prvok zoznamu.
Sprístupni posledný			Posledný prvok zoznamu.
Sprístupni	int		Prvok zoznamu s daným indexom.
Dotazy			
Názov dotazu	Parametre	Návratová hodnota	Dotaz vráti
Obsahuje	T	bool	Vráti príznak, či sa prvok vyskytuje v zozname.
Modifikátory			
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor vloží prvok na
Vlož prvý	T	-	Začiatok zoznamu.
Vlož posledný			Koniec zoznamu.
Vlož	int T		Miesto v zozname určené parametrom index
Názov modifikátora	Parametre	Návratová hodnota	Význam
Nastav	int T	-	Modifikátor nastaví údaje na danom indexe na novú hodnotu.
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor odstráni zo zoznamu prvok
Zruš prvý	-	-	Prvý.
Zruš posledný			Posledný.
Zruš	int		Určený parametrom index.

AUS všeobecný zoznam

- Podobnosť rozhrania `Zoznam<T>` s rozhraním `Sekvencia<BlokPamäte>`.
- **Zoznamy sú sekvencie.**
- Je možné vytvoriť všeobecný zoznam pracujúci so všeobecnou sekvenciou -
AUS `VšeobecnýZoznam<T, TypSekvencie>`
- V závislosti na voľbe sekvencie (implicitná/explicitná, acyklická/cyklická) získa zoznam dané vlastnosti a obmedzenia – úloha potomkov doplniť `TypSekvencie`.
- (takmer všetky) Operácie zoznamu je možné implementovať jednoducho zavolaním príslušnej operácie zo sekvencie.

AUS všeobecný zoznam



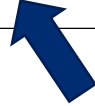
AUS všeobecný zoznam – základné operácie

- Pre vypočítanie indexu prvku nemôžeme využiť operáciu **vypočítajIndex** z rozhrania **Sekvencia<TypBloku>**:
 - V sekvencii je zaručená unikátnosť blokov pamäte, avšak unikátnosť údajov v zoznamoch zaručená nie je.
 - Je potrebné realizovať prehliadku sekvencie, ktorú zoznam spravuje.
 - Ak sa údaje v zozname nenachádzajú (teda v sekvencii neexistuje blok pamäte s údajovou časťou zhodnou s hľadanými údajmi), typická návratová hodnota je v takom prípade -1.

AUS všeobecný zoznam – základné operácie a dotazy – pseudokódy

Definícia konštruktora $VšeobecnýZoznam<T, TypSekvencie>()$

1. **konštruktor** $VšeobecnýZoznam<T, TypSekvencie>()$ {
2. **inicializuj predka** $AUS(vytvor TypSekvencie())$
3. }



Definícia konštruktora

$VšeobecnýZoznam<T, TypSekvencie>(\uparrow VšeobecnýZoznam<T, TypSekvencie>)$

1. **konštruktor** $VšeobecnýZoznam<T, TypSekvencie>(\text{iný: } \uparrow VšeobecnýZoznam<T, TypSekvencie>)$ {
2. **inicializuj predka** $AUS(vytvor TypSekvencie(), \text{iný})$
3. }

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.vypočitajIndex(T): int$

1. **operácia** $VšeobecnýZoznam<T, TypSekvencie>.vypočitajIndex(prvok: T): int$ {
2. **definuj premennú** $výsledok: int \leftarrow 0$
3. **definuj premennú** $blok: \uparrow TypSekvencie.TypBloku \leftarrow \text{pamäťováŠtruktúra} \rightarrow \text{nájdíBlokSVlastnosťou}(\lambda(blok: \uparrow TypSekvencie.TypBloku) \{$
4. **Ak** $(blok \rightarrow dáta = prvok)$ **potom** {
5. **Vráť** $pravda$
6. }
7. **inak** {
8. $výsledok++$
9. **Vráť** $nepravda$
10. }
11. }
12. }
13.)
14. **Keď platí** $(blok \neq NULL)$ **tak vráť** $výsledok$ **inak vráť** -1
15. }

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.obsahuje(T): bool$

1. **operácia** $VšeobecnýZoznam<T, TypSekvencie>.obsahuje(prvok: T): bool$ {
2. **Vráť** $vypočitajIndex(prvok) \neq -1$
3. }

AUS všeobecný zoznam – selektory – pseudokódy

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.sprístupniPrvý(): T$

```
1. operácia  $VšeobecnýZoznam<T, TypSekvencie>.$   
    $sprístupniPrvý(): T$  {  
2.   definuj premennú blok:  $\uparrow TypSekvencie.TypeBloku \Leftarrow$   
      $pamäťováŠtruktúra \rightarrow sprístupniPrvý()$   
3.   Ak (blok = NULL) potom {  
4.     chyba("Zoznam je prázdny!")  
5.   }  
6.   Vráť blok  $\rightarrow$  dáta  
7. }
```

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.sprístupniPosledný(): T$

```
1. operácia  $VšeobecnýZoznam<T, TypSekvencie>.$   
    $sprístupniPosledný(): T$  {  
2.   definuj premennú blok:  $\uparrow TypSekvencie.TypeBloku \Leftarrow$   
      $pamäťováŠtruktúra \rightarrow sprístupniPosledný()$   
3.   Ak (blok = NULL) potom {  
4.     chyba("Zoznam je prázdny!")  
5.   }  
6.   Vráť blok  $\rightarrow$  dáta  
7. }
```

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.sprístupni(int): T$

```
1. operácia  $VšeobecnýZoznam<T, TypSekvencie>.sprístupni($   
    $index: int$   
    $): T$  {  
2.   definuj premennú blok:  $\uparrow TypSekvencie.TypeBloku \Leftarrow$   
      $pamäťováŠtruktúra \rightarrow sprístupni(index)$   
3.   Ak (blok = NULL) potom {  
4.     chyba("Nesprávny index!")  
5.   }  
6.   Vráť blok  $\rightarrow$  dáta  
7. }
```

AUS všeobecný zoznam – modifikátory – vkladanie – pseudokódy

Definícia operácie *VšeobecnýZoznam*<T, *TypSekvencie*>.vložPrvý(T)

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.vložPrvý(  
    prvok: T  
    ) {  
2.   pamäťováŠtruktúra→vložPrvý()→dáta ← prvok  
3. }
```

Definícia operácie *VšeobecnýZoznam*<T, *TypSekvencie*>.vložPosledný(T)

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.vložPosledný(  
    prvok: T  
    ) {  
2.   pamäťováŠtruktúra→vložPosledný()→dáta ← prvok  
3. }
```

Definícia operácie *VšeobecnýZoznam*<T, *TypSekvencie*>.vlož(T, int)

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.vlož(  
    prvok: T,  
    index: int  
    ) {  
2.   Ak (index < 0 v index > veľkosť()) potom {  
3.     chyba("Nesprávny index!")  
4.   }  
5.   pamäťováŠtruktúra→vlož(index)→dáta ← prvok  
6. }
```



Definícia operácie *VšeobecnýZoznam*<T, *TypSekvencie*>.nastav(int, T)

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.nastav(  
    index: int,  
    prvok: T  
    ): T {  
2.   definuj premennú blok: ↑TypSekvencie.TypBloku ←  
     pamäťováŠtruktúra→sprístupni(index)  
3.   Ak (blok = NULL) potom {  
4.     chyba("Nesprávny index!")  
5.   }  
6.   blok→dáta ← prvok  
7. }
```



AUS všeobecný zoznam – modifikátory – mazanie – pseudokódy

- Je možné aby operácie **zruš** vracali vymazané údaje - **vždy to bude kópia údajov**, pretože miesto, kde boli údaje v bloku pamäte uložené, fyzicky zanikne spolu s **blokom pamäte**.
- Ak by bolo potrebné kópiu údajov z mazaného bloku pamäte vrátiť, potom je možné upraviť modifikátory tak, aby
 - najskôr uložili údaje do lokálnej premennej pomocou príslušného selektora **sekvencie**,
 - následne zavolali operáciu **zruš** a
 - nakoniec údaje vrátili.
- **Pozor na negatívny dopad volania selektora na zložitosť operácie **zruš**:**
 - V prípade **zrušPrvý** a **zrušPosledný** by nebol výrazný.
 - Pozor na rušenie na indexe v explicitných sekvenciách! Kvôli efektívnosti je potrebné sprístupniť predchádzajúci blok pamäte, aby bolo možné efektívne aj sprístupniť údaje nasledovníka pomocou selektora **sprístupniNasledujúci** a aj efektívne vymazať blok pamäte pomocou modifikátora **zrušNasledovníka**.

AUS všeobecný zoznam – modifikátory – mazanie – pseudokódy

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.zrušPrvý(): T$

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.zrušPrvý(  
  ) {  
2.   Ak (jePrázdny()) potom {  
3.     chyba("Zoznam je prázdny!")  
4.   }  
5.   pamäťováŠtruktúra→zrušPrvý()  
6. }
```

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.zrušPosledný(): T$

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.zrušPosledný(  
  ) {  
2.   Ak (jePrázdny()) potom {  
3.     chyba("Zoznam je prázdny!")  
4.   }  
5.   pamäťováŠtruktúra→zrušPosledný()  
6. }
```

Definícia operácie $VšeobecnýZoznam<T, TypSekvencie>.zruš(int): T$

```
1. operácia VšeobecnýZoznam<T, TypSekvencie>.zruš(  
  index: int  
  ) {  
2.   Ak (index < 0 v index ≥ veľkosť()) potom {  
3.     chyba("Nesprávny index!")  
4.   }  
5.   pamäťováŠtruktúra→zruš(index)  
6. }
```

AUS všeobecný zoznam – potomkovia

Definícia triedy ImplicitnýZoznam<T>

```
1. Trieda ImplicitnýZoznam<T>  
2.   rozširuje VšeobecnýZoznam<T, IS<T>> {  
3. }
```

Definícia triedy ImplicitnýCyklickýZoznam<T>

```
1. Trieda ImplicitnýCyklickýZoznam<T>  
2.   rozširuje VšeobecnýZoznam<T, CIS<T>> {  
3. }
```

Definícia triedy JednostranneZreťazenýZoznam<T>

```
1. Trieda JednostranneZreťazenýZoznam<T>  
2.   rozširuje VšeobecnýZoznam<T, JednostranneZS<T>> {  
3. }
```

Definícia triedy JednostranneZreťazenýCyklickýZoznam<T>

```
1. Trieda JednostranneZreťazenýCyklickýZoznam<T>  
2.   rozširuje VšeobecnýZoznam<T, JednostranneCZS<T>> {  
3. }
```

Definícia triedy ObojstranneZreťazenýZoznam<T>

```
1. Trieda ObojstranneZreťazenýZoznam<T>  
2.   rozširuje VšeobecnýZoznam<T, ObojstranneZS<T>> {  
3. }
```

Definícia triedy ObojstranneZreťazenýCyklickýZoznam<T>

```
1. Trieda ObojstranneZreťazenýCyklickýZoznam<T>  
2.   rozširuje VšeobecnýZoznam<T, ObojstranneCZS<T>> {  
3. }
```


AUS zoznam - zložitosti

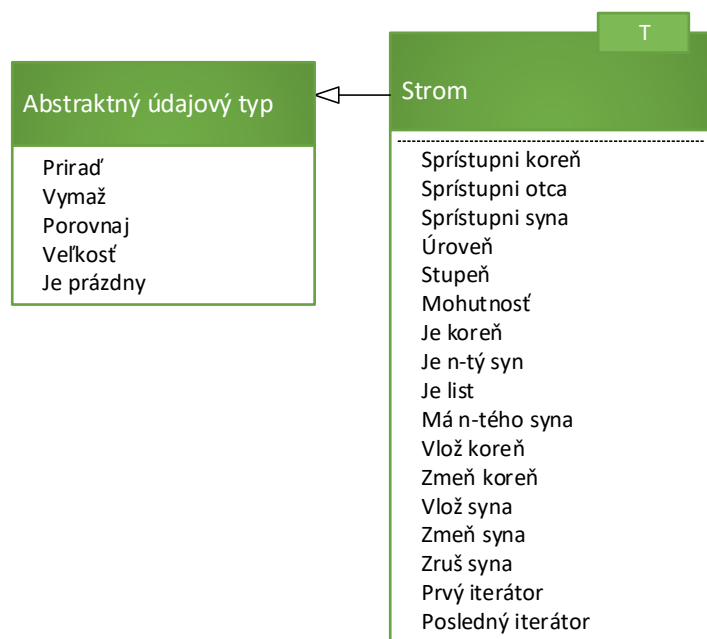
- Zložitosti operácií v zoznamoch vychádzajú zo zložitostí operácií použitej sekvencie.
- Cyklické sekvencie majú zložitosti operácií rovnaké ako acyklické, preto ich v tabuľke neuvádzame.
- Pri zreťazených sekvenciách predpokladáme existenciu referencie na posledný prvok (ak to tak nie je, tak zložitosti zodpovedajú zložitostiam v príslušných sekvenciách).

Operácie zoznamu				
Operácia	Zložitosť operácie podľa použitej sekvencie			Parametre
	Implicitná	Zreťazená		
		Jednostranne	Obojstranne	
Vypočítaj index	O(n)	O(n)	O(n)	n: počet prvkov v zozname b: $\lceil n/B \rceil$ B: veľkosť buffra v správcovi pamäte implicitnej sekvencie (koľko prvkov sa do neho zmestí)
Obsahuje	O(n)	O(n)	O(n)	
Sprístupni prvý	O(1)	O(1)	O(1)	
Sprístupni posledný	O(1)	O(1)	O(1)	
Sprístupni	O(1)	O(n)	O(n/2)	
Vlož prvý	2*O(b)	O(1)	O(1)	
Vlož posledný	O(b)	O(1)	O(1)	
Vlož	2*O(b)	O(n)	O(n/2)	
Nastav	O(1)	O(n)	O(n/2)	
Zruš prvý	O(b)	O(1)	O(1)	
Zruš posledný	O(1)	O(n)	O(1)	
Zruš	O(b)	O(n)	O(n/2)	

AUT Strom

- **AUT strom nekladie žiadne obmedzenia na doménu prvkov** (teda akékoľvek prvky môžu tvoriť stromy).
- Pre prvky v strome je typické, že je možné **priamo pristúpiť na prvky, s ktorými je aktuálny prvok vo vzťahu** (otec/syn).
- Najefektívnejší **prístup je priamo z vrcholu** hierarchie, preto budeme pri práci so stromami priamo pracovať s týmito objektami (priamy prístup k nasledovníkom resp. predchodcom je niekedy vhodné využiť aj pri zreťazených zoznamoch) – pozor na zapúzdrenie.
- Stromy **je možné iterovať**.
- Stromy tak, ako tu sú popísané, nie sú typicky zahrnuté do knižníc údajových štruktúr najmä z dôvodu implementačnej rozdielnosti vrcholov.
- Naš návrh však umožňuje navrhnuť jednotné rozhranie, pretože môžeme pracovať s univerzálnymi blokmi pamäte a všeobecnou hierarchiou.

AUT Strom

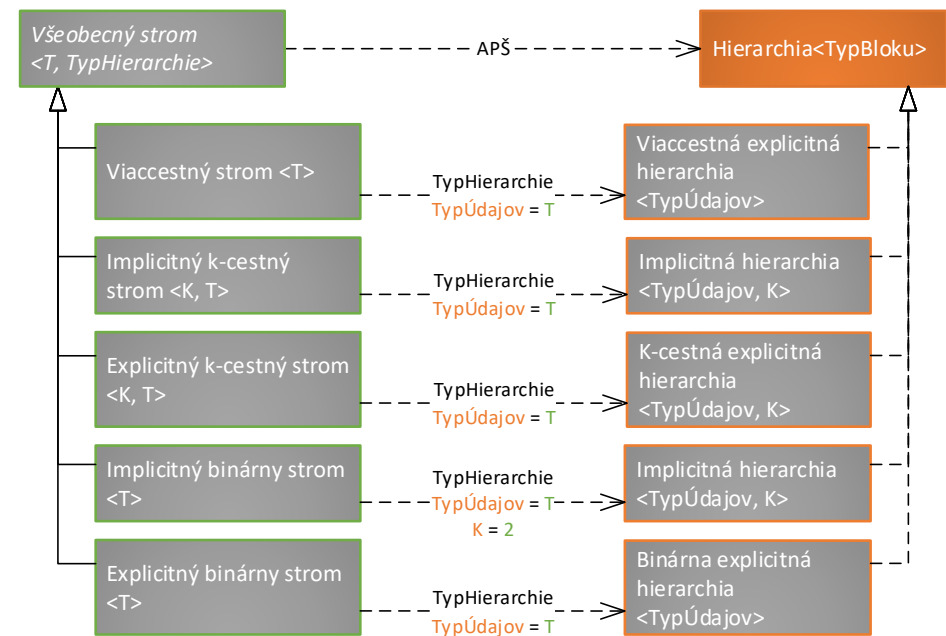
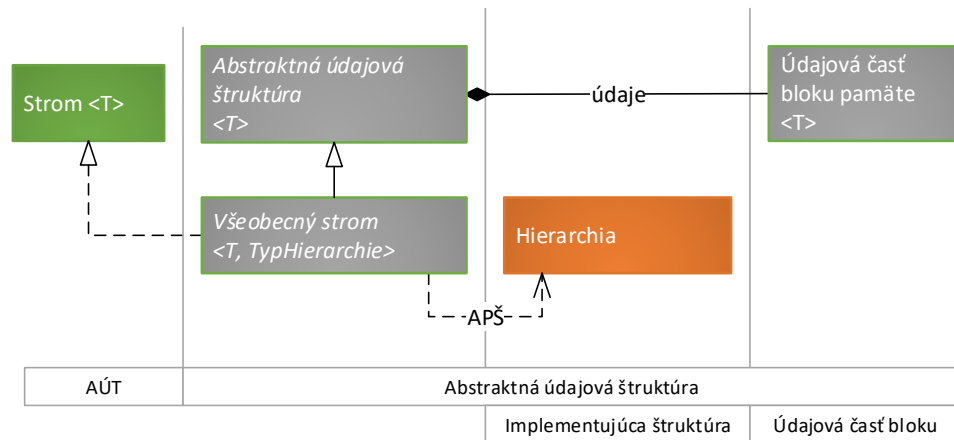


Selektory			
Názov selektora	Parametre	Návratová hodnota	Selektor sprístupní
Sprístupni koreň	-	↑Vrchol	Koreň stromu.
Sprístupni otca	↑Vrchol		Otca vrcholu odovzdaného ako parameter.
Sprístupni syna	↑Vrchol int		Syna vrcholu odovzdaného ako parameter s daným poradím.
Dotazy			
Názov dotazu	Parametre	Návratová hodnota	Dotaz vráti
Úroveň	↑Vrchol	int	Úroveň vrcholu odovzdaného ako parameter.
Stupeň	↑Vrchol		Stupeň vrcholu odovzdaného ako parameter.
Mohutnosť	↑Vrchol		Počet vrcholov v podstrome, ktorého koreň je odovzdaný ako parameter.
Je koreň	↑Vrchol	bool	Príznak, či je vrchol koreňom stromu.
Je N-tý syn	↑Vrchol int		Príznak, či je vrchol n-tým synom svojho otca.
Je list	↑Vrchol		Príznak, či je vrchol listom.
Má N-tého syna	↑Vrchol int		Príznak, či má vrchol n-tého syna.
Modifikátory			
Názov modifikátora	Parametre	Návratová hodnota	Modifikátor vykoná
Vlož koreň	-	↑Vrchol	Vloží nový koreň do stromu. Pôvodný koreň nedealokuje.
Zmeň koreň	↑Vrchol	-	Ustanoví nový koreň stromu. Pôvodný koreň nedealokuje.
Vlož syna	↑Vrchol int	↑Vrchol	Vloží nového syna na miesto s daným poradím (syn bude mať u otca požadované poradie)
Zmeň syna	↑Vrchol int ↑Vrchol	-	Zmení syna s daným poradím. Pôvodného syna nedealokuje.
Zruš syna	↑Vrchol int	-	Zruší syna s daným poradím.

AUS všeobecný strom

- Podobnosť rozhrania `Strom<T>` s rozhraním `Hierarchia<BlokPamäte>`.
- **Stromy sú hierarchie.**
- Je možné vytvoriť všeobecný strom pracujúci so všeobecnou hierarchiou - AUS `VšeobecnýStrom<T, TypHierarchie>`.
- V závislosti na voľbe hierarchie (existuje mnoho kombinácií implicitnej/explicitnej organizácie vrcholov v pamäti, organizácie množiny synov vrcholu vo vrchole a maximálneho stupňa vrcholu) získa strom dané vlastnosti a obmedzenia – úloha potomkov doplniť `TypHierarchie`.
- (takmer všetky) Operácie stromu je možné implementovať jednoducho zavolaním príslušnej operácie z hierarchie.

AUS všeobecný strom



zvolení potomkovia

AUS všeobecný strom – základné operácie – pseudokódy

Definícia konštruktora $VšeobecnýStrom<T, TypHierarchie>()$

```
1. konštruktor VšeobecnýStrom<T, TypHierarchie>() {  
2.   inicializuj predka AUS<T>(vytvor TypHierarchie())  
3. }
```



Definícia konštruktora

$VšeobecnýStrom<T, TypHierarchie>(\uparrow VšeobecnýStrom<T, TypHierarchie>)$

```
1. konštruktor VšeobecnýStrom<T, TypHierarchie>(  
   iný:  $\uparrow VšeobecnýStrom<T, TypHierarchie>$   
) {  
2.   inicializuj predka AUS<T>(vytvor TypHierarchie(), iný)  
3. }
```

AUS všeobecný strom – selektory

- Je ich možné napísať univerzálne pomocou príslušného selektoru `hierarchie`.
- Ak má používateľ (programátor) v prípade explicitných implementácií znalosť o organizácii vrcholu `hierarchie`, nemusí iné vrcholy vo vzťahu k vrcholu získavať volaním príslušnej operácie, ale priamym prístupom k danej vlastnosti vrcholu, napr.:
 - namiesto `strom.sprístupniOtca(vrchol)` volať `vrchol.otec`,
 - namiesto `strom.sprístupniSyna(vrchol, 0)` volať `vrchol.najstaršíSyn`
- Toto je možné realizovať, pretože stromy sprístupňujú celý blok pamäte, nie iba jeho údajovú časť.
- Ak však chceme jednotný prístup pre implicitné aj pre explicitné implementácie (napr. pre potreby univerzálnych algoritmov), potom je vhodné na prístup využiť nižšie uvedené selektory.

AUS všeobecný strom – selektory – pseudokódy

Definícia operácie

$VšeobecnýStrom<T, TypHierarchie>.sprístupniKoreň(): \uparrow Vrchol$

1. **operácia** $VšeobecnýStrom<T, TypHierarchie>.sprístupniKoreň()$: $\uparrow Vrchol$ {
2. $pamäťováŠtruktúra \rightarrow sprístupniKoreň()$
3. }

Definícia operácie

$VšeobecnýStrom<T, TypHierarchie>.sprístupniOtca(\uparrow Vrchol): \uparrow Vrchol$

1. **operácia** $VšeobecnýStrom<T, TypHierarchie>.sprístupniOtca(vrchol: \uparrow Vrchol)$: $\uparrow Vrchol$ {
2. $pamäťováŠtruktúra \rightarrow sprístupniOtca(vrchol)$
3. }

Definícia operácie

$VšeobecnýStrom<T, TypHierarchie>.sprístupniSyna(\uparrow Vrchol, int): \uparrow Vrchol$

1. **operácia** $VšeobecnýStrom<T, TypHierarchie>.sprístupniSyna(vrchol: \uparrow Vrchol, poradieSyna: int)$: $\uparrow Vrchol$ {
2. $pamäťováŠtruktúra \rightarrow sprístupniSyna(vrchol, poradieSyna)$
3. }

AUS všeobecný strom – dotazy

- Je ich možné napísať univerzálne pomocou príslušného dotazu `hierarchie`.
- Odpovede na dotazy, ktoré ako parameter preberajú `vrchol`, môžeme v prípade explicitných implementácií stromov opäť získať s využitím informácie o pamäťovej organizácii ich vrcholov , napr.:
 - namiesto `strom.jeKoreň(vrchol)` volať `vrchol.otec != NULL`,
 - namiesto `strom.máSyna(vrchol, 0)` volať `vrchol.najstaršíSyn != NULL`
- Podobne ako v prípade selektorov je prístup cez operácie stromu viac univerzálny a umožňujúci navrhnúť algoritmy bez ohľadu na pamäťovú organizáciu stromu či jeho vrcholov.

AUS všeobecný strom – dotazy – pseudokódy

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.jeKoreň(↑Vrchol): bool

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.jeKoreň(vrchol: ↑Vrchol): bool* {
2. **Vráť** *pamäťováŠtruktúra*→*jeKoreň(vrchol)*
3. }

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.jeNTýSyn(↑Vrchol, int): bool

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.jeNTýSyn(vrchol: ↑Vrchol, poradieSyna: int): bool* {
2. **Vráť** *pamäťováŠtruktúra*→*jeNTýSyn(vrchol, poradieSyna)*
3. }

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.stupeň(↑Vrchol): int

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.stupeň(vrchol: ↑Vrchol): int* {
2. **Vráť** *pamäťováŠtruktúra*→*stupeň(vrchol)*
3. }

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.jeList(↑Vrchol): bool

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.jeList(vrchol: ↑Vrchol): bool* {
2. **Vráť** *pamäťováŠtruktúra*→*jeList(vrchol)*
3. }

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.máNTéhoSyna(↑Vrchol, int): bool

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.máNTéhoSyna(vrchol: ↑Vrchol, poradieSyna: int): bool* {
2. **Vráť** *pamäťováŠtruktúra*→*máNTéhoSyna(vrchol, poradieSyna)*
3. }

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.úroveň(↑Vrchol): int

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.úroveň(vrchol: ↑Vrchol): int* {
2. **Vráť** *pamäťováŠtruktúra*→*úroveň(vrchol)*
3. }

Definícia operácie

VšeobecnýStrom<T, TypHierarchie>.mohutnosť(↑Vrchol): int

1. **operácia** *VšeobecnýStrom<T, TypHierarchie>.mohutnosť(vrchol: ↑Vrchol): int* {
2. **Vráť** *pamäťováŠtruktúra*→*mohutnosť(vrchol)*
3. }

AUS všeobecný strom – modifikátory

- Modifikátory `stromov` pracujúce so synmi potrebujú vždy ako parameter `vrchol`.
- Na rozdiel od `zoznamov`, kde bolo možné adresovať prvky na základe ich indexu (napr. operácie `vložPrvý`, `vlož`, `zruš`, `zrušPosledný`), v prípade `stromov` nie je možné efektívne adresovať iný prvok, ako je *koreň*.
- To, kde sa má modifikácia štruktúry vykonať, je odovzdané priamo prostredníctvom parametra.

AUS všeobecný strom – modifikátory – pseudokódy

Definícia operácie

`VšeobecnýStrom<T, TypHierarchie>.vložKoreň(): ↑Vrchol`

1. **operácia** `VšeobecnýStrom<T, TypHierarchie>.vložKoreň(): ↑Vrchol` {
2. **Vráť** `pamäťováŠtruktúra` → `vložKoreň()`
3. }

Definícia operácie

`VšeobecnýStrom<T, TypHierarchie>.zmeňKoreň(↑Vrchol)`

1. **operácia** `VšeobecnýStrom<T, TypHierarchie>.zmeňKoreň(novýKoreň: ↑Vrchol)` {
2. `pamäťováŠtruktúra` → `zmeňKoreň(novýKoreň)`
3. }

Definícia operácie

`VšeobecnýStrom<T, TypHierarchie>.vložSyna(↑Vrchol, int): ↑Vrchol`

1. **operácia** `VšeobecnýStrom<T, TypHierarchie>.vložSyna(otec: ↑Vrchol, poradieSyna: int): ↑Vrchol` {
2. `pamäťováŠtruktúra` → `vložSyna(otec, poradieSyna)`
3. }

Definícia operácie

`VšeobecnýStrom<T, TypHierarchie>.zmeňSyna(↑Vrchol, int, ↑Vrchol)`

1. **operácia** `VšeobecnýStrom<T, TypHierarchie>.zmeňSyna(otec: ↑Vrchol, poradieSyna: int, novýSyn: ↑Vrchol)` {
2. `pamäťováŠtruktúra` → `zmeňSyna(otec, poradieSyna, novýSyn)`
3. }

Definícia operácie

`VšeobecnýStrom<T, TypHierarchie>.zrušSyna(↑Vrchol, int)`

1. **operácia** `VšeobecnýStrom<T, TypHierarchie>.zrušSyna(otec: ↑Vrchol, poradieSyna: int)` {
2. `pamäťováŠtruktúra` → `zrušSyna(otec, poradieSyna)`
3. }

AUS strom – zvolení potomkovia

Definícia triedy *ViaccestnýStrom*<T>

```
1. Trieda ViaccestnýStrom<T>  
2.   rozširuje VšeobecnýStrom<T, ViaccestnáEH<T>> {  
3. }
```

Definícia triedy *ImplicitnýKCestnýStrom*<K, T>

```
1. Trieda ImplicitnýKCestnýStrom<T, K>  
2.   rozširuje VšeobecnýStrom<T, ImplicitnáHierarchia<T, K>> {  
3. }
```

Definícia triedy *ExplicitnýKCestnýStrom*<K, T>

```
1. Trieda ExplicitnýKCestnýStrom<T, K>  
2.   rozširuje VšeobecnýStrom<T, KCestnáEH<T, K>> {  
3. }
```

Definícia triedy *ImplicitnýBinárnyStrom*<T>

```
1. Trieda ImplicitnýBinárnyStrom<T>  
2.   rozširuje VšeobecnýStrom<T, ImplicitnáHierarchia<T, 2>> {  
3. }
```

Definícia triedy *ExplicitnýBinárnyStrom*<T>

```
1. Trieda ExplicitnýBinárnyStrom<T>  
2.   rozširuje VšeobecnýStrom<T, BinárnaEH<T>> {  
3. }
```

AUS strom - zložitosti

- Zložitosti operácií v stromoch vychádzajú zo zložitostí operácií použitej hierarchie.
- Budeme predpokladať, že vrcholy obsahujú referenciu na otca.

Operácie stromu						Parametre
Operácia	Zložitosť operácie podľa použitej hierarchie					
	Implicitná K-cestná	Explicitná K-cestná, synovia sú:			Viac cestná	
		vymenovaní	uložení v sekvencii			
			implicitnej	explicitnej		
Sprístupni koreň	O(1)	O(1)	O(1)	O(1)	O(1)	n: počet prvkov v strome s: stupeň vrcholu b: [s/B] B: veľkosť buffra v správcovi pamäte implicitnej sekvencie (koľko prvkov sa do neho zmestí) m: mohutnosť podstromu K: maximálny stupeň vrchola K-cestnej hierarchie h: hĺbka stromu
Sprístupni otca	O(1)	O(1)	O(1)	O(1)	O(1)	
Sprístupni syna	O(1)	O(1)	O(1)	O(s)	O(s)	
Úroveň	O(1)	O(h)	O(h)	O(h)	O(h)	
Stupeň	O(1)	O(K)	O(K)	O(K)	O(1)	
Mohutnosť	O(m)	O(m)	O(m)	O(m)	O(m)	
Je koreň	O(1)	O(1)	O(1)	O(1)	O(1)	
Je n-tý syn	O(1)	O(1)	O(1)	O(s) (s je stupeň otca)	O(s) (s je stupeň otca)	
Je list	O(1)	O(K)	O(K)	O(K)	O(1)	
Má n-tého syna	O(1)	O(1)	O(1)	O(s)	O(s)	
Vlož koreň	-	O(1)	O(1)	O(1)	O(1)	
Zmeň koreň	-	O(1)	O(1)	O(1)	O(1)	
Vlož syna	O(b) (len ako posledný list)	-	-	-	O(s)	
Zmeň syna	-	O(1)	O(1)	O(s)	O(s)	
Zruš syna	O(1) (len ako posledný list)	O(m)	O(m)	O(s) + O(m)	O(s) + O(m)	

Zhrnutie prednášky

- Abstraktné údajové štruktúry a multištruktúry
- Zoznamy
 - Implicitné (cyklické)
 - Jednostranne a obojstranne zreťazené (cyklické)
- Stromy
 - Viaccestné
 - K-cestné implicitné a explicitné
 - Binárne implicitné a explicitné

Plán na cvičenie

- Hierarchie.
 - univerzálne dotazy.
 - univerzálne prehliadky.
 - iterátory hierarchií.
- Implicitná hierarchia.
 - Binárna hierarchia.

Ďakujem za pozornosť

Doplňujúce informácie?
Vaše otázky?

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu 6UI0004 Algoritmy a údajové štruktúry 1 na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline.
- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené.

Ing. Michal Varga, PhD.
Katedra informatiky
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
Michal.Varga@fri.uniza.sk